

Exécutive Master Intelligence Artificielle & Science des Données
Université Dauphine - PSL

Flight project
Report, December 2025

Weather conditions based Flight Delay Prediction Using Data Mining and Machine Learning in Big Data environment

a scalable approach on Apache Spark

Submitted by:

- Naveed AWAN
- Henri BALAMOU
- Malik CHETTIH
- Zinedine GHERARI
- Rachna HEAN

Acknowledgments

We would like to express our sincere gratitude to all those who contributed to the successful completion of this project. First and foremost, we extend our deepest appreciation to our academic supervisors at Paris-Dauphine University for providing us with the incredible opportunity to work on such an interesting project.

Indeed, our project team members have very different career backgrounds, marketing analytics, database management, computer science, ... and as such, this big project had this great advantage to combine very diverse types of challenges into one. This made it extremely interesting and relevant for all of us, leveraging everyone's preferred skills, and at the same time, offering the possibility to learn more for each team member who felt the need to do so.

Our thanks also go to the Dario Colazzo, Kodjo Klouvi, Laurence Lelievre for their unwavering support, and the teaching staff of the EMIASD Master's program, whose rigorous training in data science, machine learning, and distributed computing provided us with the foundational knowledge necessary to tackle this complex problem.

TABLE OF CONTENT

1. Introduction	8
1.1 Business Context and Motivation	8
1.2 The Reference Paper: Belcastro et al. (2016)	8
1.3 Identified data Challenges	9
1.4 Project Objectives	11
1.5 Methodological approach – Pipeline architecture	11
2. Datasets	12
2.1 Three datasets available	12
2.1.1 The airline on-time performance (AOTP) dataset	12
2.1.2 The quality controlled local climatological data (QCLCD)	13
2.2 Problem identified in available data	14
3. Data Processing	14
3.1 Preprocessing pipeline architecture	15
3.2 Phase 1: cleaning and data synchronization to prepare for the join	15
3.3 Phase 2: Preliminary Feature Engineering	18
3.3.1 Features derived from light Data	18
3.3.2 Features derived from Weather data	20
4. Feature Engineering	21
4.1 Methodological Philosophy: the “proxy for forecast” approach	21
4.2 Phase 1: Morphological Transformation and decoupling	22
4.2.1 WMO Code sanitization and type mapping	22
4.2.2 Decoupling Weather Type from Intensity	22
4.2.3 One-hot encoding for distributed systems (?)	23
4.3 Phase 2: Domain-specific “snapshot” features	23
4.3.1 Flight rules category (LIFR to VFR)	23
4.3.2 Thermodynamic Stability: Temp-Dewpoint spread	24
4.3.3 Airframe icing risk	24
4.3.4 Barometric Momentum (pressure tendency)	25
4.3.5 Crosswind component	25
4.4 Phase 3: Temporal Features (“momentum” and “consequence”)	26
4.4.1 The “consequence” features: rolling aggregates	26
4.4.2 The “Momentum” features: trend deltas	26
4.5 Weather feature extraction from METAR codes	27
4.6 Conclusion on feature engineering	27
5. “Join” the datasets	28
5.1 Theory and foundations: the repartition join	28

5.2 “MapReduce Repartition Join” principle	28
5.3 Implementation in Spark/Scala: detailed analysis of the FlightWeatherDataJoiner code	30
5.3.1 Weather Data Preprocessing and aggregation: a fundamental Optimization	30
5.3.2 Explicit Partitioning strategy for more efficiency	31
5.3.3 Join execution – Step 1: origin airport.....	32
5.3.4 Join Execution – Step 2: Destination airport	33
5.3.5 Intermediate state management: the strategic use of .persist()	33
5.4. Summary table: comparison between Pseudocode and Scala implementation	33
6. Machine Learning	34
6.1 Introduction to the ML pipeline.....	34
6.1.1 What is a Machine Learning Pipeline?	34
6.1.2 The Hold-out test set strategy	35
6.1.3 Overview of our implementation	35
6.2 Feature Extraction Pipeline.....	36
6.2.1 What is Feature Extraction?.....	36
6.2.2 Data leakage: the silent killer of ML models.....	36
6.2.3 Feature Transformation Techniques	37
6.2.4 PCA Dimensionality reduction	38
6.3 Model Factory Design Pattern	39
6.3.1 What is the Factory Design?.....	39
6.3.2 Benefits for ML Systems	40
6.3.3 Implementation Details.....	40
6.4 Machine Learning Models	41
6.4.1 Random Forest Classifier.....	41
6.4.2 Gradient Boosted Trees (GBT).....	43
6.4.3 Logistic regression	43
6.4.4 Model Comparison.....	43
6.5 Training Process.....	44
6.5.1 What happens during training?	44
6.5.2 The Trainer Component.....	44
6.5.3 Hyperparameter configuration	45
6.6 Cross-validation	46
6.6.1 Why simple train/test split is not enough.....	46
6.6.2 K-Fold cross-validation explained	46
6.6.3 Implementation in our system.....	47
6.7 Grid search for hyperparameter tuning	48
6.7.1 What are Hyperparameters?.....	48

6.7.2 The grid search algorithm	48
6.7.3 Combining grid search with cross-validation	49
6.8 Model Evaluation.....	49
6.8.1 Understanding Classification Metrics.....	50
6.8.2 The confusion matrix	50
6.8.3 ROC Curve and AUC	51
6.8.4 Domain-specific metrics (RECd, RECo).....	51
6.9 Complete pipeline orchestration	52
6.9.1 The ML Pipeline object	52
6.9.2 Step-by-Step Execution	52
6.9.3 MLFlow Integration.....	53
6.10 Advanced Topics	54
6.10.1 Memory Management in Spark	54
6.10.2 Model Persistence	55
6.10.3 Visualization and Reporting	55
6.11 Conclusion on Machine Learning.....	55
7. Environments - Implementation Details	56
7.1 Local Docker Cluster	56
7.1.1 General architecture and components	56
7.1.2 Detailed analysis of spark components.....	58
7.1.3 Development and monitoring ecosystem.....	60
7.1.4 Analysis of applied best practices.....	61
7.1.5 Conclusion on local docker cluster	63
7.2 LAMSADE High-Performance Cluster.....	63
7.2.1 Introduction.....	63
7.2.2 LAMSADE Cluster Context.....	63
7.2.3 System architecture.....	66
7.2.3 Data Flow.....	69
7.2.4 Code and scripts development	70
7.2.5 Automated deployment.....	70
7.2.6 Usage and monitoring	72
7.2.7 Spark Configurations for LAMSADE	74
7.2.8 Results and performance.....	77
7.2.9 Conclusion and Perspectives.....	78
7.3 Google Cloud DATAPROC	78
7.3.1 Introduction.....	78
7.3.2 Context of our Project.....	79

7.3.3 General System Architecture	80
7.3.4 Code and Scripts development.....	81
7.3.5 Integration with Google Cloud Dataproc.....	82
7.3.6 Automated deployment.....	83
7.3.7 Usage and Monitoring.....	85
7.3.8 Optimal Spark Executor Sizing: Comparative Study Thin/Fat/Optimized.....	86
8. Experiments.....	90
8.1 Executive Summary on Experiments.....	90
8.1 Executive Summary on Experiments.....	91
8.1.1 Experimental Design and Methodology	91
8.1.2 Data sources and files analyzed	92
8.1.3 Structure of the experimentations section.....	92
8.1.4 Analytical strategy	93
8.1.5 Summary on experimentation approach	93
8.2 Local-D2-60-0-x Experience set.....	93
8.2.1 Global Synthesis of experimental results.....	94
8.2.2 Analysis of the destination weather experiment family (Experience-local-D2-60-0-X)....	95
8.2.3 Feature importance deep analysis	98
8.2.4 The Shift in Feature Dominance with Weather Integration.....	98
8.3 Local-D2-60-X-0 Experience set.....	99
8.3.1 Global synthesis of experiments results.....	100
8.3.2 Analysis of the origin weather Experiment Family (Experience-local-D2-60-X-0)	101
8.3.3 Feature importance deep analysis	104
8.3.4 Comparative Analysis: Origin vs. Destination weather impact.....	105
8.3.5 Feature Dominance and Predictive Power	106
8.4 Local-D2-60-X-X Experience set: symmetric weather integration analysis	106
8.4.1 Global synthesis of experimental results	106
8.4.2 Analysis of the symmetric weather experiment family (Experience-local-D2-60-X-X)..	107
8.4.3 Feature Importance Deep analysis	109
8.4.4 Comparative analysis: symmetric vs. single-source weather impact.....	110
8.5 Optimized-local-D2-60-X-X Model capacity study	111
8.5.1 Global synthesis of experimental results	111
8.5.2 Detailed analysis of the optimized model family	112
8.5.3 Feature Importance deep analysis.....	115
8.5.4 Comparative analysis: optimized vs. standard symmetric models	116
8.6 Comparative analysis of model architectures on the D2-60-7-7 feature set.....	117
8.6.1 Global Synthesis of experimental results.....	117

8.6.2 Detailed analysis of model performance and cost	118
8.6.3 Feature Importance deep analysis	120
8.6.4 Synthesis and strategic conclusion.....	121
8.7 Lamsade-D2-60-X-Xexperience set: symmetric weather integration on the Lamsade cluster	122
8.7.1 Global synthesis of experiments results.....	122
8.7.2 Detailed Analysis of the symmetric weather experiments Family	123
8.7.3 Feature Importance deep analysis	125
8.7.4 Comparative analysis: Lamsade cluster vs. Local environment.....	126
8.7.5 Strategic conclusion	127
8.8. Lamsade-Dx-7-7 Dataset comparison: Performance analysis on the Lamsade cluster	127
8.8.1 Global synthesis of experimental results	127
8.8.2 Detailed performance and cost analysis.....	128
8.8.3 Cross-dataset feature importance analysis	130
8.8.4 Strategic conclusion on dataset selection.....	131
8.9 Scalability experiments (Lamsade & Dataproc)	131
8.9.1 Dataproc Horizontal scalability analysis	132
8.9.2 Lamsade cluster stability and performance analysis.....	133
8.9.3 Strategic conclusion and recommendations.....	135
8.10 Synthesis and recommendations	136
8.10.1 Summary of key findings: from data to decisions	136
8.10.2 The champion model: experience-optimized-local-D2-60-9-9	137
8.10.3 Strategic recommendations	137
8.10.4 Conclusion	138
9. General Conclusion and perspectives	138
9.1 Summary of Key accomplishments	138
9.2 Conclusions derived from experimentation – lessons learnt	139
9.3 Cost-Benefit Analysis and operational efficiency	140
9.4 Limitations and future strategic perspectives	141
APPENDIX.....	142
APPENDIX 1 – Spatio-Temporal Join	142
APPENDIX 2 – Tracking and Managing Experiments with MLFlow	146
APPENDIX 3 – A Practical Guide to Testing and Debugging Spark Applications with Jupyter and Spark History Server.....	155
APPENDIX 4 – ML Configuration Experiments and Execution	163
References.....	171

1. Introduction

This project has as a starting point the following article: « Using Scalable Data Mining For Predicting Flight Delays », LORIS BELCASTRO, FABRIZIO MAROZZO, DOMENICO TALIA and PAOLO TRUNFIO, from the University of Calabria.

A good part of the articles details the deep technical work that the authors have led on the preprocessing, transformation, engineering, and joining of big and complex datasets containing information related to flight information and weather conditions. The purpose of this effort is to use this data to build a predictor of flight delays due to weather conditions leveraging different models. The results and performance that the authors were able to reach are also presented, which is a good reference point for our work, and least for benchmarking purposes.

We explain in this preamble, how we understood from the article, the project, and the challenges. This will lead us to the building principles, scoping, and steps of our “scientific” approach slightly summarized in the last paragraph of this preamble.

1.1 Business Context and Motivation

Why flight delays prediction is important and valuable

All over the world, around 20% of airline flights arrive more than 15 minutes late. Flight delays have significant operational challenges that go far beyond the immediate inconvenience experienced by travelers. Indeed, according to studies conducted by the Federal Aviation Administration (FAA), flight delays in the United States alone did cost the economy an estimated 32.9 billion dollars in 2007, with passengers bearing more than half of this financial burden through missed connections, lost productivity, and additional accommodation expenses. So the ability to predict flight delays - likelihood of arriving on time - based on weather conditions can have multiples impactful business applications such as recommendation systems on flight booking websites, or traffic management for airports, ultimately generating dollar savings.

The special impact of weather conditions and related complexity

The complexity of flight delay prediction stems from the multifaceted nature of delay causation. The Bureau of Transportation Statistics categorizes delays into five primary categories: Air Carrier delays (accounting for 26-30% of delays), which encompass maintenance issues, crew scheduling problems, and aircraft loading inefficiencies; Late-Arriving Aircraft delays (33-39%), representing the cascading effect of delays propagating through an airline's network; National Aviation System (NAS) delays (30-37%), attributed to air traffic control decisions, airport operations, and airspace congestion; Extreme Weather delays (approximately 3%), directly caused by severe meteorological conditions; and Security delays (0.2%), resulting from security checkpoint incidents or enhanced screening requirements.

Among these categories, weather-related delays present a particularly compelling target for predictive modeling due to their inherent predictability through meteorological forecasting. While extreme weather accounts for only 3% of officially categorized delays, this figure significantly underestimates weather's true impact on aviation operations. When considering indirect effects—such as NAS delays triggered by weather-induced airspace restrictions and late-arriving aircraft delays caused by weather disruptions at origin airports—weather-related factors contribute to an estimated 35-40% of all flight delays.

1.2 The Reference Paper: Belcastro et al. (2016)

Our project builds upon and extends the foundational work presented by Belcastro, Marozzo, Talia, and Trunfio in their paper called "Using Scalable Data Mining for Predicting Flight Delays," published in ACM Transactions on Intelligent Systems and Technology (TIST) in 2016. This paper established a rigorous methodological framework for large-scale flight delay prediction using distributed computing paradigms.

The authors leveraged two comprehensive datasets spanning five years of aviation operations (2009-2013):

- the Airline On-Time Performance (AOTP) dataset containing approximately 31 million domestic flight records,
- and the Quality Controlled Local Climatological Data (QCLCD) dataset comprising 233 million hourly weather observations from approximately 1,600 meteorological stations across the United States.

This scale of data integration required the development of scalable processing techniques implemented on distributed computing platforms.

For example, one of the key innovative element of Belcastro et al.'s approach lies in their spatio-temporal join methodology, which associates each flight with relevant weather observations based on both geographic proximity (matching airport locations to nearby weather stations via WBAN codes) and temporal alignment (synchronizing flight schedules with contemporaneous meteorological conditions).

Main results and learnings from the reference paper

Their experimental results demonstrated that incorporating weather information significantly improves prediction accuracy.

These results establish important baselines for our work and highlight the value of weather integration in delay prediction systems. Notably, prediction accuracy improves substantially with higher delay thresholds, reflecting the stronger correlation between significant delays and observable weather phenomena.

- Very interestingly, without any weather condition consideration incorporated into the model, the authors were still able to reach an accuracy of prediction of 69.1%. This means we already capture a pattern based on the airline, airports...
- Thresholds at different levels of delay have been set, like 15, 30, 45, 60 and 90 minutes. The higher the threshold is, the more accurate we get with the prediction: With a delay threshold of 15 min the authors managed to achieve an accuracy of 74.2% and 71.8% recall on delayed flights vs. 60 minutes the accuracy is 85.8% and the delay recall is 86.9%. Using Random Forest classification, they achieved the following results across different delay thresholds:

Delay Threshold	Accuracy	Recall (Delayed)	Recall (On-time)
15 minutes	74.2%	71.8%	76.6%
30 minutes	81.6%	80.1%	83.1%
45 minutes	84.1%	83.2%	85.0%
60 minutes	85.8%	86.9%	84.7%
90 minutes	86.6%	88.2%	85.0%

Metrics improvement with increasing delay treshold

- Several points of observations of weather conditions improve the prediction, up to 7 observations where we start to reach a limit: The article framework incorporates historical weather data extending several hours before scheduled departure and arrival times, capturing the cumulative effects of adverse weather on airport operations.
- Using both weather conditions at origin and destination airports respectively at departure and arrival time, does bring value vs. only one of the two.
- Decisions trees / Random forest delivers the best performance among the different models the authors tested.

1.3 Identified data Challenges

While the reference paper provides an excellent foundation, our implementation encountered several technical and methodological challenges that required careful consideration and innovative solutions. These challenges can be categorized into the below areas:

Big Data: Data volume and Processing Scalability

Processing millions of flight records and hundreds of millions of weather observations demands significant computational resources and careful optimization of data processing pipelines. The sheer volume of data—exceeding 100 GB when combined—renders traditional single-machine processing approaches impractical. Our

solution leverages Apache Spark's distributed computing capabilities, implementing optimized partitioning strategies and caching mechanisms to achieve acceptable processing times on both local development environments and production clusters.

Spatio-Temporal Join Complexity

The “join” operation between flight and weather data presents substantial complexity due to the need for multi-dimensional matching. Each flight must be associated with weather observations from both origin and destination airports, at appropriate times relative to scheduled departure and arrival. Additionally, our implementation incorporates multiple hours of historical weather data (configurable depth), resulting in a multiplicative expansion of the joined dataset. The computational cost of this operation, if naively implemented, would render the system impractical.

We addressed this challenge through a carefully designed join strategy employing explicit partitioning on join keys, shuffle-hash join hints to guide Spark's query optimizer, and pre-aggregation of weather data into map structures indexed by hour, enabling efficient extraction of historical observations.

Timezone Alignment

A subtle but critical challenge involves the alignment of temporal references between datasets. Flight schedules are recorded in local time zones (varying across the continental United States from Eastern to Pacific time), while weather observations are uniformly recorded in Coordinated Universal Time (UTC). Failure to properly convert between these temporal reference frames would result in systematic misalignment, associating flights with weather conditions from incorrect time periods.

Our implementation includes a comprehensive timezone conversion module that leverages airport-to-timezone mappings and accounts for daylight saving time transitions, ensuring accurate temporal alignment across all data processing stages.

Class Imbalance

The fundamental distribution of flight delays exhibits significant class imbalance, with approximately 80% of flights arriving on time (or with minimal delays below reporting thresholds) and only 20% experiencing notable delays. This imbalance poses challenges for machine learning algorithms, which may optimize for majority class prediction at the expense of minority class recall—precisely the opposite of the desired behavior for a delay prediction system.

We implement random under-sampling of the majority class to achieve balanced training datasets (50/50 split between delayed and on-time flights), ensuring that our models learn to discriminate between classes rather than defaulting to majority class predictions.

Data Leakage prevention

Machine learning pipelines are susceptible to data leakage, where information that would not be available at prediction time inadvertently influences model training. In the context of flight delay prediction, several features present leakage risks, including actual departure delays (DEP_DELAY), actual elapsed time (ACTUAL_ELAPSED_TIME), and arrival delay itself (ARR_DELAY). Using such features would artificially inflate model performance while providing no practical predictive value.

Our feature extraction pipeline will need to explicitly exclude potentially leaky features and implements strict train/test separation, with all data transformations (including categorical encoding) fitted exclusively on training data.

Delay causes Available data

Not all data that are in reference in the article were made available: not all causes of delays were provided (2 causes out of 5).

Time period scope available data

The data we have covers several months, which is much less than what is referenced in the paper, with several years of data. We already know that the results of the model would probably differ based on this

Models for classification

Among the different models tested in the article, random forest proved to be the most performing. This might leave some space to test other models.

Infrastructure Set up for deployment: in addition the set up requested in the project brief, leveraging Lamsade cluster, we propose to activate other set up like firstly, on the local machine with docker because it provides some agility/flexibility to run the experiments provided you have the right machine, as well as via the Google Cloud Platform Dataproc (GCP).

1.4 Project Objectives

The objective is to build a predictor of flight delay, based on weather conditions. In order to do so, keeping in mind the elements of context and challenges raised above, we will need to tackle the below four main topics:

First, to design and implement a scalable, reproducible machine learning pipeline for flight delay prediction, capable of processing large-scale aviation and meteorological datasets using distributed computing frameworks. This pipeline should be configurable, allowing experimentation with different model architectures, hyperparameters, and feature sets.

Second, to investigate the contribution of various weather-related features to prediction accuracy, including the impact of historical weather depth (number of hours of prior observations) and the relative importance of origin versus destination weather conditions.

Third, to document our methodology, implementation decisions, and experimental results in sufficient detail to enable reproducibility and serve as a foundation for future research in this domain.

Fourth, to develop and evaluate the system across multiple execution environments, from local development clusters to production-grade cloud infrastructure, demonstrating practical deployment feasibility.

1.5 Methodological approach – Pipeline architecture

We will use the reference paper as guide and benchmark: meaning we will firstly, as much as possible, try to reproduce the technical and methodological approach as we understood it from the paper, and use the results obtained as benchmark. We will then take each step of the pipeline as an opportunity to test and deep dive further than the strict scope of the paper, when we felt it was relevant, for example with different execution environments.

More concretely:

The complete pipeline consists of three major sequential phases: the **Data Pipeline**, responsible for loading, cleaning, and preprocessing raw data; the **Feature Pipeline**, which performs spatio-temporal joining, feature extraction, and dataset balancing; and the **Machine Learning Pipeline**, encompassing model training, hyperparameter optimization, and evaluation. Each phase produces intermediate artifacts that serve as inputs to subsequent phases, enabling incremental development and caching of expensive computations.

The entire pipeline is orchestrated by a central application class (FlightDelayPredictionApp) that coordinates execution flow, manages configuration loading, and handles **experiment iteration**. Configuration is externalized in YAML files, allowing researchers to define multiple experiments with varying parameters without modifying source code.

We will run these pipelines through **different environments**: local docker, LAMSADE and DATAPROC.

2. Datasets

Before doing anything else, the preliminary work is to analyze and scope the data we have available. We provide here an exhaustive description of the datasets used—the Airline On-Time Performance (AOTP) dataset and the Quality Controlled Local Climatological Data (QCLCD)

2.1 Three datasets available

Our system integrates three primary data sources, each presenting distinct characteristics and preprocessing requirements. The Airline On-Time Performance (AOTP) dataset, sourced from the Bureau of Transportation Statistics, contains detailed records of domestic US flights including scheduled and actual times, origin and destination airports, operating carrier information, and delay breakdowns by cause. Each record encompasses over 100 fields, though our processing focuses on a carefully selected subset relevant to delay prediction.

The Quality Controlled Local Climatological Data (QCLCD) dataset provides hourly weather observations from meteorological stations across the United States. Key fields include temperature (dry bulb and dew point), humidity, wind speed and direction, visibility, sky conditions (cloud coverage and ceiling height), atmospheric pressure, and weather type codes following METAR conventions. The WBAN (Weather Bureau Army Navy) mapping file establishes correspondence between airport identifiers and nearby weather station codes, enabling the spatial component of our join operation.

The efficacy of any predictive model is bounded by the fidelity, granularity, and temporal resolution of its training data. This project leverages two massive, high-dimensional datasets that provide a comprehensive view of historical flight operations and the concurrent meteorological environment. The integration of these disparate data sources—one transactional and one observational—requires a rigorous understanding of their internal schemas, reporting biases, and physical definitions.

2.2.2 The airline on-time performance (AOTP) dataset

The primary source of target variables and flight metadata for this research is the Airline On-Time Performance (AOTP) dataset. This dataset is curated and disseminated by the Bureau of Transportation Statistics (BTS), a division of the Research and Innovative Technology Administration (RITA), and represents the definitive historical record of domestic commercial aviation in the United States.¹

Scope, Granularity, and Volume

The AOTP dataset encompasses domestic flights operated by major U.S. air carriers. The regulatory definition of a "major" carrier has evolved; historically, it included carriers accounting for at least 1% of total domestic scheduled-service passenger revenues, a threshold that was subsequently lowered to 0.5% to capture a broader slice of the regional jet market.² For the specific scope of this replication study, the data spans a five-year window from January 2009 through December 2013, aligning with the temporal bounds of the Belcastro et al. study.¹

The volume of data involved is substantial, necessitating the use of distributed computing frameworks like Apache Spark. The dataset comprises approximately 31 million discrete flight tuples, occupying xx GB of uncompressed storage.¹ Each tuple represents a single scheduled flight operation, effectively a "transaction" in the aviation network, and is characterized by 109 distinct attributes.

These attributes can be taxonomized into three functional groups critical for modeling:

1. **Implicit Flight Information:** These are deterministic attributes known well in advance of the operation. They include the Origin Airport ($\$A_o\$$), Destination Airport ($\$A_d\$$), Scheduled Departure Time ($\$t_{sd}\$$), and Scheduled Arrival Time ($\$t_{sa}\$$). These form the "backbone" of the join keys used to associate flights with weather data.
2. **Operational Metrics:** These are realized data points generated during the execution of the flight. They include Actual Departure Time ($\$t_{ad}\$$), Actual Arrival Time ($\$t_{aa}\$$), Wheels-Off Time, and

Taxi-Out Time. While these are available in the historical dataset, they must be rigorously excluded from the feature set used for prediction to preventing data leakage, as they are not known at the time of prediction ($t_{pred} < t_{sd}$).

Performance Indicators: These serve as the ground truth for supervised learning. The primary target variable is the Arrival Delay ($AD(F)$), calculated as the difference between actual and scheduled arrival times ($F.t_{aa} - F.t_{sa}$).

Delay Taxonomy and Target Variable Definition

A critical nuance in the AOTP dataset is its regulatory definition of "delay." Following Federal Aviation Administration (FAA) standards, a flight is statistically counted as "delayed" only if it arrives at the destination gate 15 minutes or more past its scheduled arrival time.¹ This "15-minute buffer" is a crucial operational parameter; it implies that minor variances in flight time, often caused by headwinds or minor taxi queuing, are absorbed into the schedule padding and do not trigger delay reporting.

Furthermore, the BTS mandates that carriers report the *cause* of the delay for flights arriving more than 15 minutes late. These causes are bucketed into five broad categories, which are essential for filtering the dataset to isolate weather-related phenomena:

Delay Category	Description	Relevance to Weather Prediction
Air Carrier	Circumstances within the airline's control (e.g., maintenance, crew, baggage loading).	Excluded. These are stochastic operational noises unrelated to meteorology.
Extreme Weather	Significant meteorological conditions (actual or forecast) that prevent operation (e.g., tornadoes, blizzards).	Included. Direct causal link.
National Aviation System (NAS)	Delays attributable to the non-extreme weather conditions, airport operations, heavy traffic volume, and air traffic control.	Partially Included. A significant portion of NAS delays are weather-related (e.g., reduced airport acceptance rates due to low ceilings/visibility).
Late-Arriving Aircraft	Propagation delays caused by the late arrival of the previous leg.	Excluded. While often initiated by weather, these are propagation effects rather than primary weather impacts at the local airport.
Security	Delays caused by evacuation of a terminal or security breaches.	Excluded. Unrelated to weather.

5 categories for delay causes

The research focuses on isolating delays attributable to weather. However, the "Extreme Weather" flag in the AOTP dataset is notoriously conservative. Carriers often attribute delays to the "NAS" category when operations are slowed but not stopped by weather (e.g., spacing aircraft further apart due to fog). As noted in the BTS documentation and the baseline TIST paper, approximately 58.3% of NAS delays in 2013 were effectively weather-related.¹ Therefore, our target variable construction involves a robust filtering logic that aggregates "Extreme Weather" delays with "NAS" delays where the root cause can be inferred as meteorological.

2.1.2 The quality controlled local climatological data (QCLCD)

Scope, Granularity, and Complexity

The QCLCD is significantly larger and more complex than the flight data, occupying xxxx GB and containing xxx million tuples.¹ It aggregates observations from approximately 1,600 U.S. weather stations, most of which are co-located with airports (ASOS/AWOS stations).

The data possesses a high temporal resolution. While nominally "hourly," the dataset includes "Special" (SPECI) reports issued non-periodically when significant weather changes occur (e.g., a sudden wind shift or a drop in visibility below landing minimums). This irregularity necessitates rigorous temporal alignment strategies during the join phase—specifically, selecting the observation closest in time to the scheduled departure/arrival, rather than simply truncating to the nearest hour.

Each observation tuple *O* is defined as (*A*, *t*, *T*, *H*, *W_d*, *W_s*, *P*, *S*, *V*, *D*), where:

- *A*: Station/Airport Identifier (WBAN code).
- *t*: Observation timestamp.
- *T*: Dry Bulb Temperature (Celsius/Fahrenheit).
- *H*: Relative Humidity.
- *W_d*, *W_s*: Wind Direction and Speed.
- *P*: Barometric Pressure (Station and Sea Level).
- *S*: Sky Condition (Cloud cover and height).
- *V*: Visibility (Statute Miles).
- *D*: Weather Phenomena Descriptor (WMO Codes/METAR strings).

A critical challenge with the QCLCD dataset, which motivates much of our feature engineering, is the complexity of the "Weather Phenomena Descriptor" (often stored in the *ValueForWindCharacter* or *WeatherType* columns in raw extracts).⁵ These fields utilize World Meteorological Organization (WMO) codes or METAR strings to encode complex states. For example, a code might represent "+SN" (Heavy Snow) or "TSGR" (Thunderstorm with Hail). These codes are not immediately usable by numerical algorithms and serve as the primary subject of the Phase 1 feature engineering described in Section 4.

2.2 Problem identified in available data

At this stage we can already raise some challenges with the data made available and that we will need to address or at least keep in mind in the interpretation and analysis, that are specific to our datasets.

- Not all causes for delay were provided, only 2 out of 5 causes that were referenced in the paper
- Not all time periods represented
- Unbalanced dataset for the purpose of the analysis (we have more flights arriving on-time than delayed)
- Data requires alignment to prepare for the join, which will be detailed in the next section 3: data processing

Now that we have a better view on the scope of data we have available, let's start looking into it with the data processing.

3. Data Processing

The implemented methodology, as explained in the introduction, draws inspiration from the approaches described in the academic paper *TIST* ("Using Scalable Data Mining for Predicting Flight Delays") and has been adapted for a big data processing environment based on Apache Spark and Scala.

As described in the first section, the project faces several fundamental challenges. The first is managing an extensive volume of data, with millions of flight records (AOTP) and hundreds of millions of weather observations (QCLCD) originating from separate sources. The second challenge lies in the heterogeneity of these data. Finally, the most critical complexity involves ensuring rigorous spatio-temporal synchronization. This requires linking each flight to the nearest weather station (using a WBAN identifier) and harmonizing time references, notably by converting local flight departure times to Coordinated Universal Time (UTC) to align them with meteorological records.

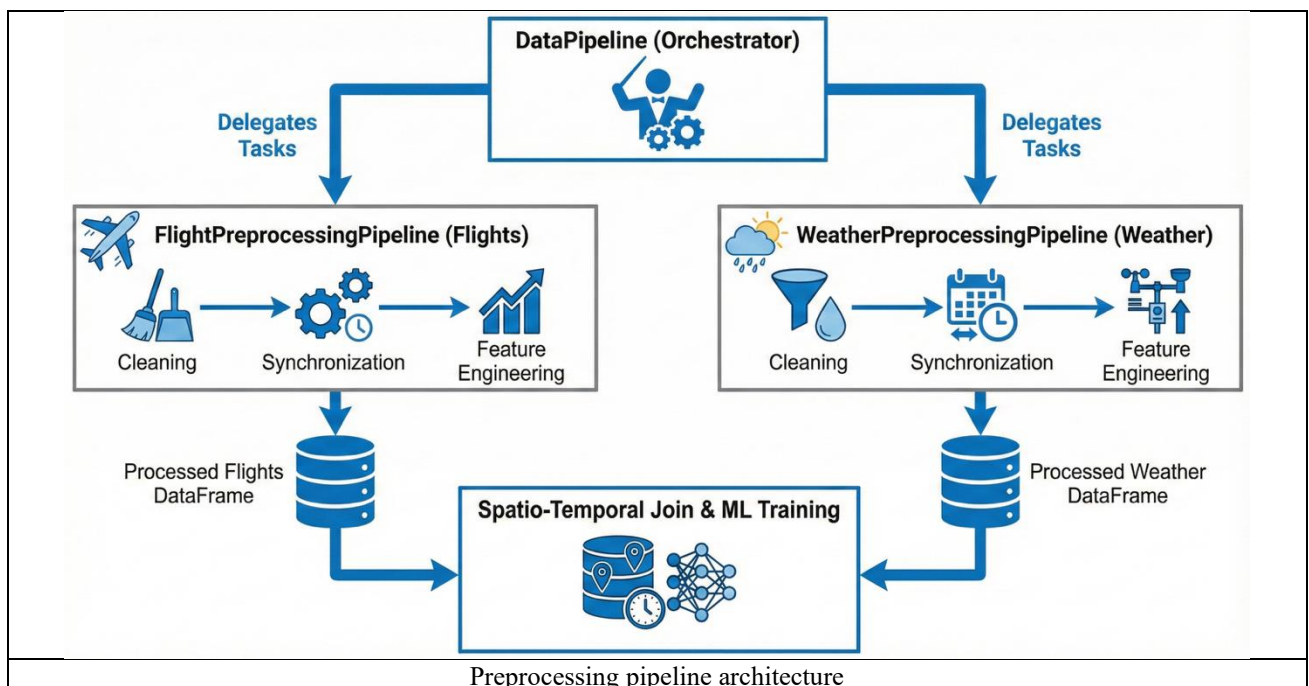
We are detailing in this chapter the architectural choices and the specific transformation steps implemented in the data pipeline. We will describe how the raw datasets—flights, weather, and airport-station mapping (WBAN)—are cleaned, synchronized, and enriched. These steps are crucial to prepare high-quality DataFrames, ready for spatio-temporal joining and, subsequently, for training machine learning models.

We start with an overview of the pipeline architecture before detailing the two main transformation phases.

3.1 Preprocessing pipeline architecture

Data processing is orchestrated by a modular and hierarchical pipeline designed to ensure clarity, maintainability, and scalability within the Apache Spark ecosystem. This architecture clearly separates the processing logic applied to each data source while maintaining coordinated execution.

The overall data flow is managed by a main orchestrator, the **DataPipeline** class. Its role is to sequence the entire preprocessing process. It loads the raw data and then delegates the specific transformation tasks to two specialized sub-pipelines: **FlightPreprocessingPipeline** for flight data and **WeatherPreprocessingPipeline** for meteorological data. This design ensures that each pipeline operates autonomously and focuses on transformations specific to its domain, thereby promoting reusability and ease of debugging.



The code structure reflects this modular organization, as shown in the package hierarchy below:

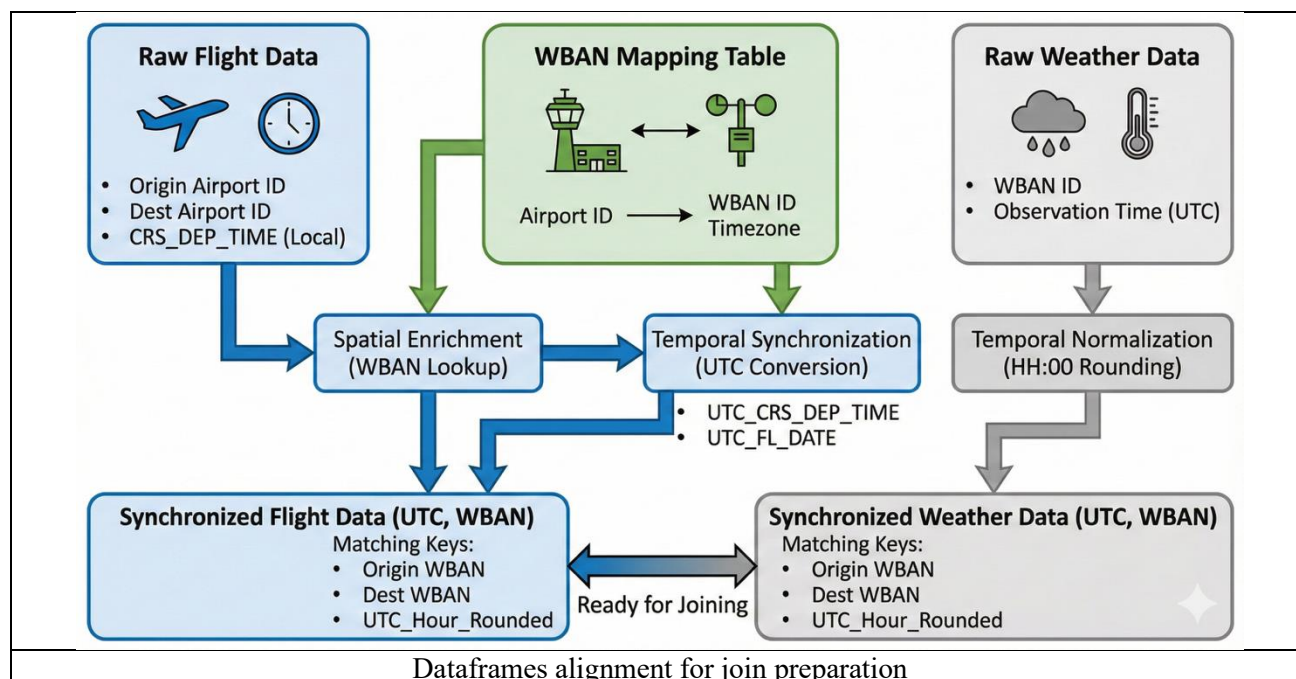
- **com.flightdelay.data**
 - **DataPipeline**: The main orchestrator that executes the entire processing flow.
 - **preprocessing**: This package contains all data transformation logic.
 - **flights**: Contains the **FlightPreprocessingPipeline** and all classes dedicated to cleaning, enriching, and feature generation for flight data.
 - **weather**: Contains the **WeatherPreprocessingPipeline** and the classes responsible for transforming raw meteorological data into usable features.

The first phase of this pipeline focuses on cleaning and synchronizing raw data - a fundamental step before any complex join operation.

3.2 Phase 1: cleaning and data synchronization to prepare for the join

This initial phase is the most critical part of the pipeline, as it ensures both the performance and accuracy of the final spatio-temporal join. The strategic objective is to resolve inconsistencies, filter out irrelevant data, and align the three main DataFrames - **flights**, **weather**, and **wban** - on common join keys before any feature engineering takes place. Without this preparation, such inconsistencies would lead to significant data skew, join

failures, and a substantial reduction in the size of the final training dataset, ultimately compromising both model performance and accuracy.



Spatial Enrichment and Temporal Synchronization

Aligning data along spatial (airport) and temporal (time) dimensions is an essential prerequisite. This process is divided into two logical steps: **spatial enrichment** and **temporal synchronization**.

Spatial Enrichment

The first component in this sequence is the **FlightWBANEnricher** class. Its role is to enrich the flight data by adding the necessary spatial context. Through a double join with the *wban* mapping table, it associates each flight with the corresponding meteorological station identifiers (**ORIGIN_WBAN**, **DEST_WBAN**) and timezone metadata (**ORIGIN_TIMEZONE**, **DEST_TIMEZONE**) for both origin and destination airports. This step provides the geographical information required for the subsequent temporal synchronization.

Temporal Synchronization

Once the timezone information is available, a crucial temporal transformation is performed: converting the scheduled local departure time (**CRS_DEP_TIME**) into UTC time. This step is essential for establishing a common temporal reference with the meteorological data, which are standardized. The calculation of **UTC_CRS_DEP_TIME** and the corresponding date **UTC_FL_DATE** correctly handles cases where the timezone offset results in a day change.

Scala

```

// Convert CRS_DEP_TIME from local time to UTC using ORIGIN_TIMEZONE
val enrichedDf = withUTCMinutes
// Calculate day offset (-1, 0, or +1)
.withColumn("_utc_day_offset",
  floor(col("_utc_total_minutes") / lit(1440)).cast(IntegerType)
)
// Convert UTC minutes to HHMM format
.withColumn("UTC_CRS_DEP_TIME",
  format_string("%04d",
    (col("_utc_minutes_in_day") / lit(60)).cast(IntegerType) * lit(100) +
    (col("_utc_minutes_in_day") % lit(60)).cast(IntegerType)
  )
)

```

```
// Calculate UTC date by adding day offset to FL_DATE
.withColumn("UTC_FL_DATE",
  date_add(col("FL_DATE"), col("_utc_day_offset"))
)
```

Temporal synchronisation

Finally, to simplify the temporal join key, all departure and arrival times (local and UTC) are rounded to the nearest hour in HH00 format (e.g., feature_utc_departure_hour_rounded and feature_utc_arrival_hour_rounded)³⁹. This normalization is a key design decision that allows for a direct and high-performance match with meteorological observations⁴⁰.

Flight Data Cleaning (FlightDataCleaner)

The cleaning of flight data, performed by FlightDataCleaner, follows a rigorous approach to ensure data quality and relevance. The initial steps involve removing duplicates and records with null values in critical columns such as date or airport identifiers. In accordance with the methodology described in the *TIST* paper, invalid flights—those that were canceled (CANCELLED) or diverted (DIVERTED)—are then filtered out, as they do not provide exploitable information for predicting arrival delays.

A major performance optimization is then implemented through an early filtering strategy (pre-join filtering). The functions filterFlightsByExistingWeatherStations and filterFlightsByCoveredMonths use left_semi joins to retain only flights where:

1. Have origin and destination airports associated with a meteorological station (**WBAN**) present in the weather dataset.
2. Occur in months covered by the available weather data.

By performing these aggressive filters before the main spatio-temporal join, the pipeline drastically reduces the volume of data redistributed across the network (**shuffle**), which is the most expensive operation in a Spark job. This architectural decision eliminates millions of records that would otherwise have no match, significantly improving the overall efficiency of the pipeline.

Weather Data Cleaning (WeatherDataCleaner)

Weather data cleaning applies a symmetrical logic for maximum optimization. The filterWeatherByFlightWBANs function acts as the counterpart to filterFlightsByExistingWeatherStations, performing a left_semi join to retain only the observations from WBAN stations that are actually referenced in the flight dataset.

Together, these two operations form a powerful bidirectional filter (**mutual pre-filtering**), ensuring that both DataFrames are minimized to contain only records with potential matches.

The most critical step is temporal normalization, performed by the normalizeWeatherTime function. For each station (WBAN) and each hour, only the observation closest to the exact hour (e.g., 14:00) is retained. This transformation is a key design pattern that converts what would otherwise be a complex and inefficient temporal range join (e.g., finding weather between 14:00 and 14:59) into a highly efficient **equijoin** on a simplified key (**HH00**).

Scala

```
// normalizeWeatherTime keeps only the closest record to HH:00 for each hour
val window = Window.partitionBy("WBAN", "Date", "hour").orderBy(col("distance_to_hour"))

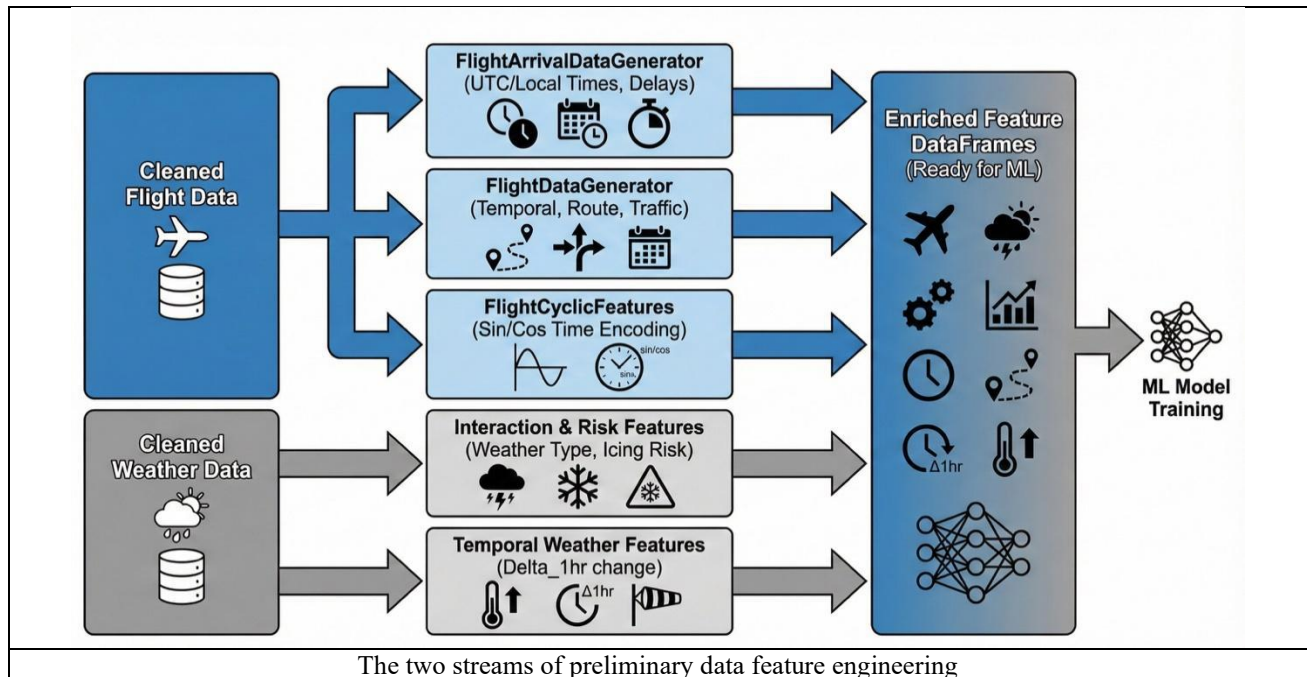
val result = dfWithDistance
  .withColumn("rank", row_number().over(window))
  .filter(col("rank") === 1)
  .withColumn("Time", format_string("%04d", col("hour") * 100))
  .drop("hour", "distance_to_hour", "rank")
```

Weather data cleaning

Once the data is cleaned and synchronized, the next step consists of enriching the datasets through feature engineering:

3.3 Phase 2: Preliminary Feature Engineering

After cleaning and synchronizing the data, the next step is to enrich the datasets by creating new high-value features. This process here is already somehow a preliminary feature engineering that will be detailed in the next section 4. This is essential for providing machine learning models with more explicit and relevant information. It is divided into two parallel streams: one for flight-specific features and the other for features derived from weather conditions.



3.3.1 Features derived from light Data

The generation of features for flight data is managed by several specialized Scala classes, including *FlightArrivalDataGenerator*, *FlightDataGenerator*, and *FlightCyclicFeatures*, each with a specific role.

Features of *FlightArrivalDataGenerator* - This class is dedicated to the precise calculation of scheduled arrival times, both in local time and UTC, as well as the creation of derived features. It takes into account flight duration and time zone offsets to provide a comprehensive view of the arrival.

Feature Name	Source Column(s)	Transformation Description
UTC_ARR_TIME	UTC_CRS_DEP_TIME, CRS_ELAPSED_TIME	Scheduled UTC arrival time, calculated by adding flight duration to UTC departure.
UTC_ARR_DATE	UTC_FL_DATE, _utc_arr_day_offset	UTC arrival date, adjusted if the flight crosses midnight.
CRS_ARR_TIME	UTC_ARR_TIME, DEST_TIMEZONE	Scheduled local arrival time at the destination.
CRS_ARR_DATE	UTC_ARR_DATE, _local_arr_day_offset	Local arrival date, adjusted according to the destination time zone.
feature_arrival_hour	CRS_ARR_TIME	Local arrival hour (0-23).

feature_utc_arrival_hour_rounded	feature_utc_arrival_hour	UTC arrival hour rounded to HH00 format, optimized for the weather join.
feature_arrival_time_period	CRS_ARR_TIME	Period of the day (e.g., "Morning", "Evening") based on local arrival time.
feature_crosses_midnight_local	CRS_ARR_DATE, FL_DATE	Binary indicator (1/0) if the flight crosses midnight in local time.
feature_timezone_difference	DEST_TIMEZONE, ORIGIN_TIMEZONE	Difference in hours between destination and origin time zones.
Features of FlightArrivalDataGenerator		

Features of FlightDataGenerator - This is the main feature-generation engine, producing a wide range of temporal, flight-specific, and aggregated information.

Feature Name	Source Column(s)	Transformation Description
feature_flight_month	FL_DATE	Month of the flight (1-12).
feature_flight_day_of_week	FL_DATE	Day of the week (1=Sunday, 7=Saturday).
feature_flight_week_of_year	FL_DATE	Week of the year (1-52).
feature_departure_hour_rounded	CRS_DEP_TIME	Local departure hour rounded to the nearest hour.
feature_utc_departure_hour_rounded	UTC_CRS_DEP_TIME	Rounded UTC departure hour, key for the weather join.
feature_departure_time_period	CRS_DEP_TIME	Period of the day (e.g., "Late_Night", "Early_Afternoon") for the departure.
feature_distance_category	CRS_ELAPSED_TIME	Categorizes the flight as "short", "medium", "long" based on duration.
feature_route_id	ORIGIN_AIRPORT_ID, DEST_AIRPORT_ID	Unique and directionally agnostic identifier for the route (e.g., '10397_12266'). Created by concatenating numerically sorted airport IDs, ensuring that route A->B has the same identifier as B->A.
feature_is_weekend	feature_flight_day_of_week	Binary indicator if the flight takes place on a weekend.
feature_origin_airport_traffic	ORIGIN_AIRPORT_ID	Total number of flights departing from the origin airport (traffic proxy).
Features of FlightDataGenerator		

Features of FlightCyclicFeatures - This class transforms temporal features (such as hour or week) into a cyclic representation using sine and cosine functions. This approach enables models to understand the periodic nature of these variables (for example, that 11 p.m. is temporally close to 12 a.m., or that week 52 is close to week 1).

Feature Name	Source Column(s)	Transformation Description
--------------	------------------	----------------------------

feature_departure_hour_rounded_sin	feature_departure_hour_rounded	Sine component of the departure hour (24h period).
feature_departure_hour_rounded_cos	feature_departure_hour_rounded	Cosine component of the departure hour (24h period).
feature_flight_week_of_year_sin	feature_flight_week_of_year	Sine component of the week of the year (52-week period).
feature_flight_week_of_year_cos	feature_flight_week_of_year	Cosine component of the week of the year (52-week period).
<u>Features of FlightCyclicFeatures</u>		

3.3.2 Features derived from Weather data

The processing of weather data aims to transform raw codes and measurements into interpretable and relevant features for delay risk assessment. This work is distributed across several classes, such as

SkyConditionFeatures, *VisibilityFeatures*, *WeatherInteractionFeatures*, *WeatherPressureFeatures*, *WeatherTemporalFeatures*, and *WeatherTypeFeatureGenerator*.

Interaction and Risk Features -These classes create high-level features by combining multiple raw weather measurements to evaluate complex conditions.

Feature Name	Source Column(s)	Transformation Description
feature_weather_type	ValueForWindCharacter	Weather type (e.g., "Rain", "Snow", "Fog_Mist") derived from a numeric code.
feature_weather_intensity	ValueForWindCharacter	Weather intensity (e.g., "Light", "Moderate", "Heavy") derived from a code.
feature_flight_category ordinal	feature_flight_category	Ordinal flight category (0=VFR, 1=MVFR, 2=IFR, 3=LIFR) based on visibility and cloud ceiling.
feature_visibility_category	feature_visibility_miles	Categorizes visibility according to aviation standards (VFR, IFR, etc.).
feature_icing_risk_level	DryBulbCelsius, has_freezing_precip, etc.	Icing risk score (0-3) based on temperature and visible humidity.
feature_icing_risk_flag	DryBulbCelsius, has_freezing_precip, etc.	Binary indicator (1/0) of icing risk.
feature_is_pressure_dropping_fast	press_change_raw	Binary indicator if pressure is dropping rapidly (≤ -2.0 hPa), signaling an approaching storm.
feature_temp_dewpoint_spread	DryBulbCelsius, DewPointCelsius	Spread between temperature and dew point. A low value indicates a risk of fog.
<u>Interaction and Risk Features</u>		

Weather Temporal Features (WeatherTemporalFeatures)- This class focuses on the dynamics of weather conditions by calculating the rate of change of key variables over a one-hour period. This makes it possible to capture the “dynamics” of the weather (for example, whether visibility is deteriorating rapidly).

	Feature Name	Source Column(s)	Transformation Description	
--	--------------	------------------	----------------------------	--

	Temp_Delta_1hr	DryBulbCelsius	Change in temperature during the last hour.
	Visibility_Delta_1hr	Visibility	Change in visibility during the last hour.
	Humidity_Delta_1hr	RelativeHumidity	Change in relative humidity during the last hour.
	WindSpeed_Delta_1hr	WindSpeed	Change in wind speed during the last hour.
Weather Temporal Features (WeatherTemporalFeatures)			

Thanks to these last two phases, the datasets are not only aligned and clean, they are also enriched with dozens of new features, ready to be leveraged.

Let's deep dive now into the proper Feature Engineering phase.

4. Feature Engineering

After the deep phase of cleaning and first level of feature engineering made in the previous section 3, we can now start the proper feature engineering. The central hypothesis driving our enhanced feature engineering strategy is that raw weather data—such as a single wind speed reading or a visibility metric—is insufficient for accurate delay classification when treated in isolation by decision-tree ensembles (e.g., Random Forest or XGBoost). These models lack inherent "physics awareness." They do not intuitively understand that a temperature of -2°C combined with visible moisture creates a severe icing hazard, nor do they inherently grasp that a rapid drop in barometric pressure signals an approaching storm front. Therefore, the burden of "teaching physics" to the model shifts to the feature engineering phase.

This part details the three-phased feature engineering protocol we developed for this project. This protocol moves beyond simple statistical aggregation to capture the causal mechanisms of flight delay, leveraging the distributed computing capabilities of Apache Spark in Scala to handle the massive volume of historical data.

4.1 Methodological Philosophy: the "proxy for forecast" approach

Before detailing the specific transformations applied to the data, it is essential to articulate the philosophical constraint governing the feature engineering process. This constraint addresses the fundamental difference between "training time" and "inference time" in a production environment.

In a real-time operational setting, a flight delay prediction model runs hours or days before the flight (t_{pred}), relying on *forecasted* weather data. However, the training process utilizes *historical* (actual) weather data. This discrepancy introduces a risk of data leakage—using future knowledge (actual weather at t_{sd}) to predict a delay at t_{sd} without accounting for the uncertainty or the specific derivative nature of a forecast.

To mitigate this while maintaining model robustness, this research adopts the **"Actuals as a Proxy for Forecast"** methodology.¹ The rationale is twofold:

1. **Learning Consequences, Not Forecasts:** The objective of the machine learning model is not to act as a meteorological model (predicting *if* it will snow), but rather as an operational impact model (predicting *if* snow leads to delay). By training the model on actual historical weather, we teach it the precise *consequences* of specific weather states. For example, the model learns the causal link "Heavy Snow $\rightarrow$ High Delay Probability." When deployed, the model is fed *forecasted* heavy snow. If the forecast is accurate, the model correctly predicts the delay. This decouples the error of the weather forecast from the error of the delay model.
2. **Simulating Forecaster Information:** A human dispatcher does not look at a single static number. They look at trends, rates of change, and accumulations. To simulate the information available to a forecaster, our feature engineering emphasizes **Trends** (Momentum) and **Accumulations** (Consequence).¹ By

engineering features like "Pressure Change over 3 Hours" or "Snow Accumulation," we provide the model with the same derivative signals that human dispatchers use to assess risk.

4.2 Phase 1: Morphological Transformation and decoupling

The raw QCLCD data contains alphanumeric codes and mixed-type fields that effectively hide information from numerical regression or classification algorithms. Phase 1 of the feature engineering pipeline focuses on **Decoupling**—breaking complex, multi-dimensional codes into independent, orthogonal feature vectors. This process was implemented using Scala on Apache Spark to ensure scalability across the 233 million weather records.

4.2.1 WMO Code sanitization and type mapping

The ValueForWindCharacter column in the QCLCD dataset serves as a repository for World Meteorological Organization (WMO) Table 4677 codes.⁷ These are integer codes representing specific weather states (e.g., 21 for Rain, 22 for Snow). However, the raw data is frequently contaminated with non-numeric artifacts such as 'M' (Missing), empty strings, or whitespace flags utilized by legacy Fortran systems.

Rationale: A tree-based model treating ValueForWindCharacter as a raw string or a continuous integer would fail. As a string, the cardinality is too high and sparse. As an integer, the ordinality is false; Code 71 (Snow) is not "greater than" Code 21 (Rain) in any mathematical sense relevant to delay (though both imply precipitation). Therefore, the data must first be sanitized and then mapped to categorical concepts.

Scala Implementation Strategy:

The cleaning process employs a rigorous casting strategy involving a User Defined Function (UDF) or Spark SQL column expressions. We explicitly handle the legacy flags ('M', ' ', '') by coercing them to nulls before casting to integers.

Scala

```
// Scala code for WMO Code Sanitization
// Transforming raw string column into nullable integer
import org.apache.spark.sql.functions._
import org.apache.spark.sql.types.IntegerType

val dfCleaned = dfRaw.withColumn("Cleaned_WMO_Code",
  when(col("ValueForWindCharacter").isin("M", " ", "", "NULL"), lit(null).cast(IntegerType))
  .otherwise(col("ValueForWindCharacter").cast(IntegerType))
)
```

Following sanitization, the integer WMO codes are mapped to human-readable categories. This mapping is not 1-to-1 but rather Many-to-One, grouping physically similar phenomena. Based on WMO Table 4677⁷, the mapping logic is as follows:

- **Rain:** Codes 20, 21, 60, 61, 62, 63, 80, 81, 82.
- **Snow:** Codes 22, 70, 71, 72, 73, 85, 86.
- **Freezing Precipitation:** Codes 24, 66, 67 (Freezing Rain/Drizzle).
- **Thunderstorm:** Codes 29, 90-99.
- **Fog/Mist:** Codes 10, 11, 12, 40-49.

4.2.2 Decoupling Weather Type from Intensity

A central tenet of the feature engineering strategy is the decoupling of "Type" from "Intensity". In standard WMO/METAR coding, the type (Rain) and intensity (Heavy) are often fused into a single code (e.g., Code 63 = Heavy Rain, Code 61 = Light Rain).

Rationale: Fused features force the model to learn the impact of "Heavy Rain" separately from "Light Rain," potentially missing the underlying correlation that all rain reduces friction and visibility. By splitting these into two separate feature sets—Weather_Type (Rain, Snow, Fog) and Weather_Intensity (Light, Moderate, Heavy)—the model can independently weight the presence of the phenomenon and its severity. This allows the model to generalize better: it learns that "Heavy" implies a higher delay probability regardless of whether the precipitation is rain or snow.

Scala Implementation Strategy: We utilize Spark's map function (or when/otherwise chains) to apply two distinct dictionaries to the Cleaned_WMO_Code.

```
Scala

// Scala code for Decoupling Type and Intensity
val dfDecoupled = dfCleaned
.withColumn("Weather_Type",
  when(col("Cleaned_WMO_Code").isin(20, 21, 60, 61, 62, 63), "Rain")
  .when(col("Cleaned_WMO_Code").isin(22, 70, 71, 72, 73), "Snow")
  .when(col("Cleaned_WMO_Code").isin(24, 66, 67), "Freezing_Precip")
  .when(col("Cleaned_WMO_Code").isin(29, 90, 91, 92, 93, 94, 95), "Thunderstorm")
  .otherwise("Clear_or_Other")
)
.withColumn("Weather_Intensity",
  when(col("Cleaned_WMO_Code").isin(61, 71), "Light")
  .when(col("Cleaned_WMO_Code").isin(63, 73, 67, 95), "Heavy")
  .when(col("Cleaned_WMO_Code").isin(62, 72), "Moderate")
  .otherwise("Not_Applicable")
)
```

4.2.3 One-hot encoding for distributed systems (?)

Once the Weather_Type and Weather_Intensity categorical columns are generated, they must be converted into a numerical format suitable for XGBoost or Random Forest classifiers running on Spark.

Rationale: While some implementations of decision trees can handle categorical data directly, one-hot encoding (creation of dummy variables) is preferred in the Spark environment to ensure consistent vectorization across distributed partitions. This creates binary features such as Weather_Type_Snow (0/1) and Intensity_Heavy (0/1). This binary representation allows the tree to create clean splits (e.g., IF Weather_Type_Snow == 1) without imposing false ordinal relationships (e.g., Snow > Rain) that might confuse the gradient descent or splitting criteria.

4.3 Phase 2: Domain-specific “snapshot” features

Phase 2 moves beyond simple data cleaning to the generation of "Snapshot" features—derived variables calculated at time t that translate raw meteorological data into aviation-specific operational metrics. These features encapsulate the "physics of delay," creating proxies for the regulatory and physical constraints that pilots and air traffic controllers face.

4.3.1 Flight rules category (LIFR to VFR)

Aviation operations are governed by flight rules dictated by visibility and cloud ceiling. These rules—Visual Flight Rules (VFR) and Instrument Flight Rules (IFR)—determine the separation required between aircraft and the acceptance rate of runways.

Rationale: A linear feature like "Visibility = 2 miles" captures only part of the operational constraint. The FAA defines specific categories that trigger massive changes in airport capacity. For example, dropping from VFR to IFR often halves the arrival rate. A model trained only on linear visibility might underestimate the step-

function change in delay probability that occurs at the 3-mile visibility threshold. The model must explicitly "know" these regulatory thresholds.

Feature Logic: The feature `Flight_Rules_Category` is an ordinal variable constructed by parsing the `SkyCondition` and `Visibility` columns. The `SkyCondition` column (e.g., "SCT010 BKN020") requires regex parsing to extract the altitude of the lowest "Broken" (BKN) or "Overcast" (OVC) layer, which constitutes the "ceiling".¹

The classification logic follows strict FAA definitions:

1. **LIFR (Low IFR):** Visibility < 1 mile OR Ceiling < 500 feet. (Ordinal Value: 3).
2. **IFR:** Visibility $1\text{--}3$ miles OR Ceiling $500\text{--}1000$ feet. (Ordinal Value: 2).
3. **MVFR (Marginal VFR):** Visibility $3\text{--}5$ miles OR Ceiling $1000\text{--}3000$ feet. (Ordinal Value: 1).
4. **VFR:** Visibility > 5 miles AND Ceiling > 3000 feet. (Ordinal Value: 0).

By encoding this as an ordinal integer (0 to 3), the model can monotonically associate higher values with strictly reduced airport capacity and higher delay probability.

Scala Implementation Strategy: This requires a complex when/otherwise structure after extracting the ceiling height using regular expressions.

Scala

```
// Scala code for Flight Rules Category
// Assuming 'ceiling_ft' has been extracted via Regex from SkyCondition
val dfFlightRules = dfWithCeiling.withColumn("Flight_Rules_Category",
  when((col("Visibility") < 1.0) |
    (col("ceiling_ft") < 500), lit(3)) // LIFR
    .when((col("Visibility") < 3.0) |
    (col("ceiling_ft") < 1000), lit(2)) // IFR
    .when((col("Visibility") <= 5.0) |
    (col("ceiling_ft") <= 3000), lit(1)) // MVFR
    .otherwise(lit(0)) // VFR
)
```

4.3.2 Thermodynamic Stability: Temp-Dewpoint spread

Fog and low clouds are among the most disruptive weather phenomena. They form when the air becomes saturated with moisture.

Rationale: The raw variables `DryBulbCelsius` (Air Temperature) and `DewPointCelsius` are less predictive in isolation. A temperature of 10°C is benign if the dewpoint is 0°C (dry air), but causes dense fog if the dewpoint is also 10°C . The predictive signal lies in the difference between them, known as the depression.

Feature Logic:

The feature `Temp_Dewpoint_Spread` is calculated as:

$\Delta T_{\text{spread}} = T_{\text{air}} - T_{\text{dew}}$

A small spread ($< 3^{\circ}\text{C}$) serves as a robust proxy for near-saturation conditions.¹ This feature acts as a leading indicator for fog formation (radiation fog) even before visibility sensors report a drop, providing the model with a "predictive" capacity regarding visibility deterioration.

4.3.3 Airframe icing risk

Icing is a severe safety hazard that requires aircraft to undergo de-icing procedures at the gate, causing significant departure delays.

Rationale: De-icing is mandatory when supercooled water droplets exist or when frozen precipitation accumulates on wings. This phenomenon occurs within a specific thermodynamic envelope. A machine learning model might struggle to find the specific non-linear interaction between temperature, moisture, and precipitation type that defines this risk without explicit engineering.

Feature Logic: The `Is_Icing_Risk` binary feature encapsulates the physics of supercooled water.¹⁵ It is triggered (set to 1) if either of the following conditions is met 1:

1. Explicit Trigger: `Weather_Type` is "Freezing Precipitation" (Freezing Rain/Drizzle).
2. Implicit Trigger:
 - Temperature is in the supercooled danger zone: $-15^{\circ}\text{C} \leq T_{\text{air}} \leq 2^{\circ}\text{C}$.
 - AND Visible Moisture is present: `Temp_Dewpoint_Spread` $\leq 3^{\circ}\text{C}$ OR `Precip Type` is Rain/Snow/Fog.

This heuristic feature explicitly flags conditions requiring chemical de-icing, directly correlating with ground processing delays.

Scala

```
// Scala code for Icing Risk
val dfIcing = dfDecoupled.withColumn("Is_Icing_Risk",
  when(col("Weather_Type") === "Freezing_Precip", lit(1))
    .when(
      (col("DryBulbCelsius") <= 2) &&
      (col("DryBulbCelsius") >= -15) &&
      (
        (col("Temp_Dewpoint_Spread") <= 3) |
        col("Weather_Type").isin("Rain", "Snow", "Fog")
      ), lit(1)
    )
    .otherwise(lit(0))
)
```

4.3.4 Barometric Momentum (pressure tendency)

Atmospheric pressure patterns drive weather systems. A rapidly falling barometer indicates an approaching low-pressure system, which is associated with storms, wind shear, and instability.

Rationale: While the QCLCD dataset contains a `PressureChange` column, explicitly flagging high-velocity changes helps the model identify the onset of severe weather. The `"Is_Pressure_Dropping_Fast"` feature captures this momentum.

Feature Logic: A binary flag is created based on a threshold derived from the distribution of pressure changes (e.g., dropping more than 2.0 hPa in the reporting period).¹ This acts as a storm warning signal, often preceding precipitation and wind events.

4.3.5 Crosswind component

Runways have fixed magnetic headings. Aircraft have strict limits on the amount of wind they can accept perpendicular to their direction of travel (crosswind).

Rationale: Wind speed alone is insufficient. A 30-knot wind aligned with the runway is manageable; a 30-knot wind perpendicular to it may close the runway.

Feature Logic: Calculating this requires joining an external lookup table of runway headings for each airport (WBAN). The crosswind component (V_{cross}) is derived trigonometrically:

$$V_{\text{cross}} = |W_s| \times \sin(W_d - H_{\text{runway}})$$

where W_s is wind speed, W_d is wind direction, and H_{runway} is the runway heading.¹ This feature represents the actual physical force acting against the aircraft's lateral stability.

4.4 Phase 3: Temporal Features (“momentum” and “consequence”)

The final and most critical phase of feature engineering addresses the temporal limitations of decision trees. Standard Random Forests treat each row as an independent event. However, weather impacts are cumulative and sequential. Phase 3 creates features that restore this temporal context.

4.4.1 The “consequence” features: rolling aggregates

Weather impacts often lag the event itself. For example, a runway does not lose friction immediately upon the first snowflake; it loses friction after *accumulation*.

Rationale: To capture these "Consequence" effects, we engineer rolling window aggregates. These features summarize the history of the weather over the preceding N hours (typically 3 hours).¹

Feature Logic: Using Spark's Window functions, defined by partitioning by Airport ID (WBAN) and ordering by Timestamp, we calculate:

- `Sum_HourlyPrecip_3hr`: Captures accumulation (snow depth or standing water).¹
- `Max_WindSpeed_3hr`: Captures the persistence of gust fronts.
- `Min_Visibility_3hr`: Identifies if the airport has recently suffered a "visibility blackout," which often causes a backlog of holding aircraft that persists even after the sky clears.

These features allow the model to understand that a flight departing at 10:00 AM is still at risk if there was heavy snow at 8:00 AM, even if the 10:00 AM observation is clear.

Scala Implementation Strategy: We define a Spark WindowSpec to look back 3 hours. Note that because the data is hourly, a "range between" 3 hours is roughly equivalent to looking at the previous 3 rows if the data is dense. However, a range-based window on the timestamp is more robust to missing rows.

Scala

```
// Scala code for Temporal Features (Rolling Window)
import org.apache.spark.sql.expressions.Window

// Define window: Partition by Station, Order by Time, Look back 3 hours (seconds)
val window3h = Window.partitionBy("WBAN")
  .orderBy(col("Timestamp").cast("long"))
  .rangeBetween(-3 * 3600, 0)

val dfTemporal = dfIceing
  .withColumn("Sum_HourlyPrecip_3hr", sum("HourlyPrecip").over(window3h))
  .withColumn("Max_WindSpeed_3hr", max("WindSpeed").over(window3h))
  .withColumn("Min_Visibility_3hr", min("Visibility").over(window3h))
```

4.4.2 The “Momentum” features: trend deltas

While "Consequence" features look backward to assess damage, "Momentum" features measure the rate of change to predict the future.

Rationale: A "forecast" is essentially a vector of change. By calculating the first derivative of key meteorological variables, we provide the model with the "velocity" of the weather.

Feature Logic: Using a lag function (e.g., `lag(col, 1)` in Spark), we calculate the 1-hour delta:

- `Visibility_Delta_1hr`: Positive values indicate clearing; negative values indicate deterioration.
- `Temp_Delta_1hr`: Rapid drops often signify cold front passage.¹

This provides the model with a dynamic view of the atmosphere, allowing it to differentiate between a stable fog bank and a lifting fog bank.

4.5 Weather feature extraction from METAR codes

The WeatherType field in QCLCD data follows METAR (Meteorological Aerodrome Report) conventions, encoding weather phenomena using standardized two-letter codes with optional intensity prefixes. Our feature engineering pipeline parses these codes to extract binary and categorical features indicating the presence of specific weather conditions:

Additionally, we compute derived features such as `weather_hazard_level` (an ordinal score from 0-3 indicating overall severity) and `feature_precipitation_intensity` (an ordinal encoding of intensity: -1 for light, 0 for moderate, +1 for heavy).

4.6 Conclusion on feature engineering

The feature engineering strategy we presented, transforms raw, disjointed data into a rich, physics-aware feature space suitable for high-performance machine learning. By cleaning and mapping WMO codes, we render qualitative descriptions quantitative. By deriving "Snapshot" features like Icing Risk and Flight Rules, we inject domain knowledge regarding aviation regulations and thermodynamics. Finally, by engineering "Momentum" and "Consequence" features, we bridge the gap between static decision trees and the dynamic, temporal nature of weather systems. This comprehensive approach, implemented scalably in Spark, ensures that the predictive model is not merely finding statistical correlations, but is learning the causal drivers of flight delays.

Phase	Feature Name	Type	Description	Operational Rationale
I	Weather_Type_X	Categorical (One-Hot)	Binary flag for Rain, Snow, etc.	Decouples phenomenon from intensity.
I	Weather_Intensity_X	Categorical (One-Hot)	Binary flag for Light, Heavy.	Captures severity independent of type.
II	Flight_Rules_Category	Ordinal (0-3)	VFR, MVFR, IFR, LIFR classification.	Maps to FAA capacity regulations.
II	Temp_Dewpoint_Spread	Continuous	$T_{\text{air}} - T_{\text{dew}}$	Proxy for saturation/fog risk.
II	Is_Icing_Risk	Binary	Heuristic based on T, Moisture, Precip.	Identifies need for ground de-icing.
II	Is_Pressure_Drop	Binary	High negative pressure tendency.	Signal for approaching storm fronts.
III	Sum_Precip_3hr	Continuous	Rolling sum of precipitation.	Captures accumulation/runway friction.

III	Vis_Delta_1hr	Continuous	Change in visibility over 1 hr.	Indicates clearing or worsening trend.
Summary of Key Engineered Features				

Now that we have built all the features we've thought relevant to feed our model, we go to the next very specific phase of this project, the join of the flight and weather datasets, that was highly complex. We will explain why, and how we did address this.

5. "Join" the datasets

As raised in the introduction of this report, one key challenge of the project is the ability to join the flight and weather datasets in the most relevant manner, according to our modelling objectives. That's why we dedicated this section 5, to detail our efforts to make the join happen successfully, knowing that even the data preprocessing work had been made in the perspective of the join as well.

5.1 Theory and foundations: the repartition join

In a Big Data context, where data volumes exceed the memory capacity of a single compute node, distributed join algorithms are indispensable. The *repartition join* (or join by repartitioning) is a fundamental technique that enables the joining of two very large tables by partitioning and distributing them across a cluster.

This section breaks down the join algorithm proposed in the academic paper (*Algorithm 1*), which represents an enhanced version of the *repartition join*, in order to establish a theoretical baseline for our comparative analysis.

5.2 "MapReduce Repartition Join" principle

ALGORITHM 1: MapReduce pseudo-code for the first join step.

Map(K : null, V : a tuple from a split of either *OT* or *FT*)

```

if  $V$  is a tuple from OT then
   $join\_key \leftarrow \langle V.A, Date(V.t) \rangle$ 
   $table\_tag \leftarrow "OT"$ 
   $tagged\_tuple \leftarrow$  add a tag "OT" to  $V$ 
   $composite\_key \leftarrow \langle join\_key, table\_tag \rangle$ 
   $emit(composite\_key, tagged\_tuple)$ 
else
   $join\_key \leftarrow \langle V.A_o, Date(V.t_{sd}) \rangle$ 
   $table\_tag \leftarrow "FT"$ 
   $tagged\_tuple \leftarrow$  add a tag "FT" to  $V$ 
   $composite\_key \leftarrow \langle join\_key, table\_tag \rangle$ 
   $emit(composite\_key, tagged\_tuple)$ 
  if  $Date(V.t_{sd}).plusHours(12)$  is  $Date(V.t_{sd}).plusDays(1)$  then
     $join\_key \leftarrow \langle V.A_o, Date(V.t_{sd}).plusDays(1) \rangle$ 
     $composite\_key \leftarrow \langle join\_key, table\_tag \rangle$ 
     $emit(composite\_key, tagged\_tuple)$ 
  end
end

```

Partition(K' : a composite key)

```

 $hashcode \leftarrow hash\_function(K'.join\_key)$ 
return  $hashcode \bmod \#reducers$ 

```

Reduce(K' : a composite key, $LIST_V'$: a list of tagged tuples for K' first from *OT* then *FT*)

```

create an array of observations  $A_o$  ordered by time
create a temporary array of observations  $A_T$ 
for each OT tuple  $o$  in  $LIST\_V'$  do
  put  $o$  in  $A_o$ 
end
for each FT tuple  $f$  in  $LIST\_V'$  do
   $A_T \leftarrow get\_hourly\_observations(A_o, f.t_{sd})$ 
   $emit(null, merge(f, A_T))$ 
end

```

MapReduce pseudo-code

The “improved repartition join” algorithm is structured around the three canonical phases of the MapReduce paradigm: Map, Partition, and Reduce. Its purpose is to join the flight table (FT) with the weather observations table (OT) using a composite key that combines the airport and the date.

“Map” Phase

Each record (tuple) from the input tables is processed individually by a Map task. The mapper generates a composite key in the form <join key, table tag>.

- The join key is defined as <Airport, Date>.
- The table tag is a simple label ("FT" for a flight, "OT" for weather) used by the reducer to identify the origin of each tuple.
- A boundary condition logic is explicitly mentioned: `` if Date(V.tsd).plusHours(12) == Date(V.tsd).plusDays(1)

This condition handles the case where the 12-hour window of required weather observations for a flight overlaps two consecutive days.

The goal is not to alter the flight data itself but to ensure that the same flight record is sent to two different reducers: one for the departure date (**Date(V.tsd)**) and another for the following day (**Date(V.tsd).plusDays(1)**).

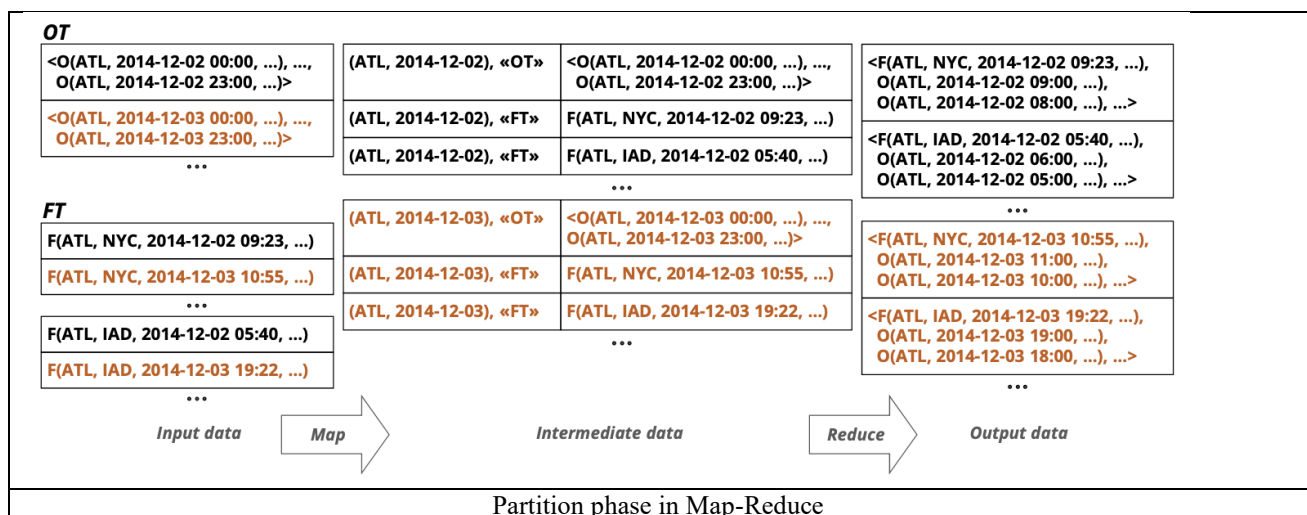
To achieve this, the mapper emits the flight tuple a second time, using the join key for the next day.

This **key duplication** ensures that the flight can collect the relevant weather data corresponding to both dates.

“Partition” Phase

This phase, often implicit but critical, is managed by the partitioner. Its role is to apply a hashing function on the join key portion of the composite key. This mechanism guarantees that all tuples sharing the same join key—for example, all flights departing from Atlanta Airport on December 2, 2014, and all weather observations for that same location and date—are routed to the same Reduce task.

This data co-location is the cornerstone of distributed join processing.



“Reduce” Phase

Each Reduce task receives a list of tuples corresponding to a given join key. Thanks to secondary sorting (implicitly done on the *table tag*), the tuples from the weather table (OT) arrive before those from the flight table (FT).

- The reducer first iterates through all the weather tuples (OT) and stores them in memory within a data structure, such as an array sorted by hour (AO).
- It then processes the flight tuples (FT). For each flight, it searches the array of weather observations (AO) to find the relevant records (for example, the 12 hourly observations preceding the departure time) and performs the join.

This theoretical approach is robust but can be further optimized. The next section examines how these concepts are implemented and enhanced in a modern computing environment such as Apache Spark.

5.3 Implementation in Spark/Scala: detailed analysis of the FlightWeatherDataJoiner code

This section represents the core of our analysis. We will demonstrate, step by step, how the Scala code provided below implements and optimizes the theoretical join algorithm.

The implementation leverages the declarative computation model and built-in optimization mechanisms of Apache Spark to go far beyond the basic pseudocode-adopting architectural design choices that ensure large-scale performance and efficiency.

5.3.1 Weather Data Preprocessing and aggregation: a fundamental Optimization

The first part of the Scala code prepares the weatherDF DataFrame and represents a major optimization compared to the traditional MapReduce approach.

Handling Daily Boundaries

In the pseudocode, date overlap is handled by duplicating flight records (FT). The Spark implementation adopts a much more efficient strategy. It creates two DataFrames - meteoSameDay and meteoNextDay (where the date is incremented by one day and the hour adjusted by -24) - and then combines them using unionByName.

This approach was chosen to minimize the volume of data to be processed: the logic is applied upstream to the weather data table, which is typically smaller than the flights table.

By doing so, it avoids costly flight data redundancy and simplifies the join logic by pre-conditioning the weather data in advance.

```
// Préparation de WeatherDF
val weather = weatherDF.select(
  col("Date").cast(DateType).as("WDATE"),
  col("Time").as("WTIME_HHMM"),
  col("*")
).where(col("WBAN").isNotNull && length(col("WBAN")) > 0 && col("WDATE").isNotNull)

// -----
// Préparation météo avec relHour + duplication J/J+1 (Map)
// -----
val weatherWithHour = weather
  .withColumn("hour", hmmmHourCol(col("WTIME_HHMM")))
  .na.fill(Map("hour" -> -1))

val meteoSameDay = weatherWithHour
  .withColumn("relHour", col("hour"))
  .withColumn("DATE", col("WDATE"))

val meteoNextDay = weatherWithHour
  .withColumn("relHour", col("hour") - lit(24))
  .withColumn("DATE", date_add(col("WDATE"), 1))

val weatherRel = meteoSameDay.unionByName(meteoNextDay)
  .filter(col("relHour").between(-24, 23))
```

Preparation of Weather dataframe

“Pre-aggregation” with groupBy and map_from_entries

```
// 1) Colonnes fixes dans la struct
val staticCols = Seq(
  col("relHour").as("hour"),
  col("WBAN"),
  col("WDATE"),
  col("WTIME_HHMM")
)
```

```
// 2) Colonnes dynamiques venant de la config
val staticNames = Set("relHour", "WBAN", "WDATE", "WTIME_HHMM")
val allWeatherCols: Seq[String] =
  selectedWeatherColumns.getOrElse(Seq.empty[String])
val dynamicFeatureCols =
  allWeatherCols
    .filterNot(staticNames.contains)
    .map(c => col(c).alias(c))

// 3) Struct finale : [hour, WBAN, WDATE, WTIME_HHMM, <features>...]
val weatherStruct =
  struct((staticCols ++ dynamicFeatureCols): _*)

// 4) Agrégation en map relHour -> struct(...)
val weatherByKey: DataFrame =
  weatherRel
    .groupBy(col("WBAN"), col("DATE"))
    .agg(
      map_from_entries(
        collect_list(struct(col("relHour"), weatherStruct))
      ).as("wmap")
    )
)
```

The code then creates the **weatherByKey** DataFrame. This step represents the most significant optimization compared to the pseudocode.

While the MapReduce reducer must store a raw list of observations (**AO**), the Spark implementation **pre-aggregates** the weather data using : `groupBy("WBAN", "DATE").agg(map_from_entries(...))`

This operation transforms the list of hourly observations into a single **Map structure (wmap)** per join key.

From an architectural perspective, the impact is twofold:

1. It drastically reduces the volume of data shuffled during the main join-the most common source of latency in Spark jobs.
2. It minimizes memory pressure on executors, which now receive a single row containing one Map per key (<Airport, Date>) instead of a potentially massive list of individual observations to iterate over. This conversion transforms an expensive $O(n)$ lookup operation into an efficient $O(1)$ key-value access.

5.3.2 Explicit Partitioning strategy for more efficiency

The Spark implementation makes the Partition phase of the pseudocode explicit and controllable-an essential element for ensuring both performance and stability.

Determining the Number of Partitions - The utility function `pickParts` and the configuration parameter `spark.sql.shuffle.partitions` are used to dynamically define the number of partitions. This provides fine-grained control over the degree of parallelism, allowing the execution to be adapted to the available cluster resources.

```
// Paramètres de partitions "reducers"
val cores = spark.sparkContext.defaultParallelism

val numPartsOrigin = pickParts(3.3, 32, 128, cores) // ≈ 40
val numPartsDest = pickParts(5.2, 48, 192, cores) // ≈ 64
spark.conf.set("spark.sql.shuffle.partitions", numPartsDest) // borne haute DF
```

Hash Partitioning - The call to `repartition(numPartsOrigin, col("ORIGIN_WBAN"), col("UTC_FL_DATE"))` is the direct implementation of the Partition phase. This instruction forces Spark to redistribute the data before the join operation. This explicit use of repartitioning is crucial for preventing data skew and ensuring an even distribution of workload across executors-a common performance issue that Spark's default partitioning strategy may not handle optimally.

```

val originPre = flightsDep
  .repartition(numPartsOrigin, col("ORIGIN_WBAN"), col("UTC_FL_DATE")) // <-- Hash partition explicite
  .join(
    weatherByKey.hint("shuffle_hash"),
    col("ORIGIN_WBAN") === weatherByKey("WBAN") &&
    col("UTC_FL_DATE") === weatherByKey("DATE"),
    "left"
  )
  .drop(weatherByKey("WBAN")).drop(weatherByKey("DATE"))

```

5.3.3 Join execution – Step 1: origin airport

```

val withOriginWeather = if (weatherOriginDepthHours >= 0) {
  // -----
  // JOIN #1 — ORIGIN (Partition = hash(ORIGIN_WBAN, UTC_FL_DATE))
  // -----
  val flightsDep = flightDF
    .withColumn("depHour", coalesce(hmmmHourCol(col("UTC_CRS_DEP_TIME")), lit(0)))

  val originPre = flightsDep
    // <-- Hash partition explicite
    .repartition(numPartsOrigin, col("ORIGIN_WBAN"), col("UTC_FL_DATE"))
    .join(
      weatherByKey.hint("shuffle_hash"),
      col("ORIGIN_WBAN") === weatherByKey("WBAN") &&
      col("UTC_FL_DATE") === weatherByKey("DATE"),
      "left"
    )
    .drop(weatherByKey("WBAN")).drop(weatherByKey("DATE"))

  val originWithWoArr = originPre
    .withColumn("Wo", expr(s"transform(sequence(1, ${weatherOriginDepthHours}), i -> element_at(wmap, depHour - i))"))
    .drop("wmap")

  val woCols = (0 until weatherOriginDepthHours).map(i => col("Wo").getItem(i).as(s"Wo_h${i+1}"))
  val originDF = originWithWoArr
    .select(col("*") +: woCols: _*)
    .drop("Wo")
    .persist()

  originDF
}

```

Join execution phase 1 – origin airport

The code block that creates withOriginWeather corresponds to the **Reduce phase** in the pseudocode, but it fully leverages the power of the Spark SQL execution engine.

1. **Join Operation:**

The statement `.join(weatherByKey, ...)` performs the join.

The condition `col("ORIGIN_WBAN") === weatherByKey("WBAN") && col("UTC_FL_DATE") === weatherByKey("DATE")` matches rows based on the {Airport, Date} key-exactly as defined by the algorithm.

2. **Temporal Data Extraction:** The expression `transform(sequence(1, ${weatherOriginDepthHours}), i -> element_at(wmap, depHour - i))`

is the modern and highly efficient equivalent of the *get hourly observations* loop.

This function performs two main operations:

- `sequence(...)` generates an array of integers representing the past hours to retrieve (e.g., [1, 2, ..., 12]).
- `transform` iterates through this array, and for each hour *i*, calculates the required absolute hour (`depHour - i`) and uses `element_at` to extract the corresponding weather observation from the `wmap`. The use of native Spark SQL functions, rather than user-defined functions (UDFs), is a fundamental design principle: it allows Spark's Catalyst optimizer to understand the

entire query plan and generate highly optimized Tungsten bytecode, thereby avoiding the serialization/deserialization overhead between the JVM and the execution environment that UDFs typically introduce.

5.3.4 Join Execution – Step 2: Destination airport

```
// Deuxième jointure : aéroport de destination (seulement si >= 0)
val withBothWeather = if (weatherDestinationDepthHours >= 0) {
  val flightsArr = withOriginWeather
    .withColumn("arrHour", coalesce(hhmmHourCol(col("UTC_ARR_TIME")), lit(0)))

  val destPre = flightsArr
    .repartition(numPartsDest, col("DEST_WBAN"), col("UTC_ARR_DATE")) // <-- Hash partition explicite
    .join(
      weatherByKey.hint("shuffle_hash"),
      col("DEST_WBAN") === weatherByKey("WBAN") &&
      col("UTC_ARR_DATE") === weatherByKey("DATE"),
      "left"
    )
    .drop(weatherByKey("WBAN")).drop(weatherByKey("DATE"))

  val destWithWdArr = destPre
    .withColumn("Wd", expr(s"transform(sequence(1, ${weatherDestinationDepthHours}), i -> element_at(wmap, arrHour - i))"))
    .drop("wmap")

  val wdCols = (0 until weatherDestinationDepthHours).map(i => col("Wd")).getItem(i).as(s"Wd_h${i+1}")

  val baseCols: Seq[org.apache.spark.sql.Column] =
    withOriginWeather.columns.map(col).toSeq

  val joinedDF = destWithWdArr
    .select( (baseCols ++ wdCols):_* )
    .drop("Wd")
    .persist()

  joinedDF
}

Join execution phase 2 (destination airport)
```

The join process for the **destination airport**, which creates the withBothWeather DataFrame, follows exactly the same logic as for the origin airport. This second step uses a different join key (DEST_WBAN, UTC_ARR_DATE) and potentially a different number of partitions (numPartsDest), demonstrating the flexibility of the implementation.

The execution of these two successive joins aligns perfectly with the overall methodology described on page 8 of the academic paper.

5.3.5 Intermediate state management: the strategic use of .persist()

A crucial architectural decision-absent from the pseudocode but essential in the Spark implementation-is the use of .persist() after each join (on originDF and joinedDF).

This operation caches the intermediate DataFrame in memory (and/or on disk), effectively “breaking” Spark’s computation lineage. In a multi-stage pipeline such as this one, the action prevents Spark from re-executing the entire computational graph of the first (origin) join when performing the second (destination) join.

This design choice delivers a massive performance gain and represents a standard architectural pattern for complex ETL pipelines in Spark, while also serving as a checkpoint for fault tolerance.

5.4. Summary table: comparison between Pseudocode and Scala implementation

Algorithmic Concept (Pseudocode)	Scala/Spark Implementation	Notes and Optimizations
----------------------------------	----------------------------	-------------------------

Map Phase: Creation of the join key (Airport, Date) and tuple tagging	<code>.repartition(..., col("..._WBAN"), col("..._DATE"))</code> implicitly defines the join key. Tagging is handled by DataFrame semantics.	The implementation is declarative. Spark infers “tags” through the DataFrame schema.
Map Phase: Handling date overlap (<code>plusDays(1)</code>)	Creation of <code>meteoSameDay</code> and <code>meteoNextDay</code> with relative hours (<code>relHour</code>), then combined using <code>unionByName</code> .	Major optimization: logic applied upstream on the smaller table (weather), avoiding flight data duplication.
Partition Phase: Hashing of the join key for co-location	Explicit call to <code>.repartition(numParts, joinKeyCol1, joinKeyCol2)</code> .	Direct and explicit control enables fine performance tuning and prevents data skew.
Reduce Phase: Storing weather observations (OT) in memory	<code>weatherRel.groupBy(...).agg(map_from_entries(...))</code> to create <code>weatherByKey</code> .	Most significant optimization: transforms a list into a Map (hash table) for O(1) lookup and drastically reduces shuffle size.
Reduce Phase: Iteration over flights (FT) and retrieval of weather data (<i>get hourly observations</i>)	<code>.join(weatherByKey, ...)</code> followed by <code>.withColumn("Wo", expr("transform(...)))</code> .	Operation is vectorized and highly efficient, leveraging the pre-aggregated wmap and native Spark SQL functions optimized by Catalyst.
Process Management: Sequential execution of join stages	Use of <code>.persist()</code> to cache intermediate results (<code>originDF</code>).	Key architectural pattern for multi-stage pipelines-prevents costly Spark lineage re-execution and provides fault tolerance.
<u>Mapping Between the Theoretical Algorithm and the Spark Implementation</u>		

Conclusion on the join

This analysis demonstrates that the Scala and Spark implementation of the spatio-temporal join faithfully adheres to the fundamental principles of the *repartition join* algorithm proposed in the academic paper. However, it goes far beyond a simple translation. The implementation intelligently leverages Spark’s capabilities to introduce critical architectural optimizations.

The **pre-aggregation of meteorological data into a Map structure**, **explicit partitioning to mitigate data skew**, **use of native functions to take full advantage of the Catalyst optimizer**, and **intermediate state management using `.persist()`** are all design choices essential for achieving high performance at real-world data scales.

This analysis perfectly illustrates the successful transition from a theoretical algorithmic concept to a **robust, scalable, and high-performing data engineering solution**, fully aligned with the demands of modern Big Data platforms.

We can now, start the machine learning pipeline. That’s the new section 6.

6. Machine Learning

6.1 Introduction to the ML pipeline

6.1.1 What is a Machine Learning Pipeline?

Definition: Machine Learning Pipeline

A Machine Learning Pipeline is an automated sequence of data processing steps that transforms raw input data into predictions. It encapsulates the entire workflow from data preprocessing, feature engineering, model training, to evaluation and deployment.

Think of a pipeline like an assembly line in a factory. Raw materials (data) enter at one end, go through various processing stations (transformations), and finished products (predictions) emerge at the other end. Each station has a specific job, and the output of one station becomes the input of the next.

A typical ML pipeline consists of:

Data Ingestion: Loading raw data from various sources (databases, files, APIs)

Data Preprocessing: Cleaning, handling missing values, removing duplicates

Feature Engineering: Creating new features, encoding categorical variables

Feature Selection: Choosing the most relevant features for the model

Model Training: Teaching the algorithm to recognize patterns in data

Model Evaluation: Measuring how well the model performs on unseen data

Model Deployment: Making the model available for real-world predictions

Why is this important? Pipelines ensure reproducibility, maintainability, and scalability. Without a structured pipeline, ML projects often become 'spaghetti code' - difficult to debug, impossible to reproduce, and prone to subtle bugs like data leakage.

Implementation Perspective - In our Flight Delay Prediction system, the pipeline is implemented across several Scala objects: DataPipeline for data processing, FeatureExtractor for feature engineering, and MLPipeline for model training and evaluation. The main orchestration happens in MLPipeline.scala, which coordinates all steps in a single train() method.

6.1.2 The Hold-out test set strategy

Definition: Hold-out Test Set

A hold-out test set is a portion of the data that is completely set aside and never used during training or hyperparameter tuning. It serves as the final, unbiased evaluation of the model's performance on truly unseen data.

Imagine you're a teacher creating an exam. If students practice with the exact same questions they'll see on the test, their scores won't reflect true understanding. Similarly, if we evaluate our model on data it has already seen, we get an overly optimistic estimate of its real-world performance.

Our system uses an 80/20 split strategy:

80% Development Set: Used for training and cross-validation. The model learns from this data and we tune hyperparameters based on performance within this set.

20% Hold-out Test Set: Completely untouched until final evaluation. This gives us an honest estimate of how the model will perform on new flights it has never seen.

The split is performed BEFORE any data preprocessing that could leak information. This is critical: if we normalize the entire dataset first and then split, the test set statistics (mean, standard deviation) would influence the training set, causing data leakage.

Implementation Perspective - In our implementation, the split is performed in FeaturePipeline BEFORE the join operation and feature extraction. This ensures that the test set remains completely independent. The MLPipeline.train() method receives pre-split devDataRow and testDataRaw DataFrames, and the feature extraction pipeline is fit only on devDataRow.

6.1.3 Overview of our implementation

Our ML pipeline follows a rigorous, industry-standard methodology designed to produce reliable and generalizable predictions. The complete workflow is:

Step 1: Receive pre-split, balanced datasets (75% dev, 25%)

Step 2: Extract features from dev set (fit transformers)

Step 3: Transform test set using pre-fitted transformers

Step 4: Perform K-Fold Cross-Validation on dev set

Step 5: Optionally run Grid Search for hyperparameter optimization

Step 6: Train final model on full dev set with best hyperparameters

Step 7: Evaluate on hold-out test set for unbiased performance estimate

Step 8: Save model, metrics, and artifacts for reproducibility

Component	File	Responsibility
MLPipeline	MLPipeline.scala	Main orchestrator - coordinates all ML steps
FeatureExtractor	FeatureExtractor.scala	Feature extraction with leakage protection
ModelFactory	ModelFactory.scala	Creates appropriate ML model instances
CrossValidator	CrossValidator.scala	K-Fold CV and Grid Search
Trainer	Trainer.scala	Model training with hyperparameters
ModelEvaluator	ModelEvaluator.scala	Computes classification metrics
MLFlowTracker	MLFlowTracker.scala	Experiment tracking and logging
Main Components and responsibility		

6.2 Feature Extraction Pipeline

6.2.1 What is Feature Extraction?

Definition: Feature Extraction

Feature extraction is the process of transforming raw data into a numerical format that machine learning algorithms can process. It involves selecting, transforming, and combining raw attributes into features that capture the essential patterns in the data.

Machine learning algorithms, especially those in Spark MLlib, require numerical input in the form of vectors. Raw data, however, often contains:

Categorical variables: Airline names ('Delta', 'United'), airport codes ('LAX', 'JFK')

Text data: Weather descriptions, flight status messages

Date/Time: Departure times, arrival dates

Missing values: Null entries that need to be handled

Different scales: Temperature (0-100) vs. distance (0-5000)

Feature extraction transforms all of this into a consistent numerical format. For example, the categorical variable 'Airline' with values ['Delta', 'United', 'American'] might become numerical indices [0, 1, 2].

Why is this important? Good feature extraction is often more important than model selection. A simple model with excellent features will outperform a complex model with poor features. Features should capture the underlying patterns that predict flight delays: weather severity, airport congestion, historical delay patterns, etc.

Implementation Perspective - Our FeatureExtractor.scala provides two main methods: extract() for training data (fits transformers and transforms data) and transform() for test data (uses pre-fitted transformers). The extraction pipeline uses Spark ML Pipeline stages including StringIndexer, VectorAssembler, and StandardScaler.

6.2.2 Data leakage: the silent killer of ML models

Definition: Data Leakage

Data leakage occurs when information from outside the training dataset is used to create the model. This includes: (1) using test data information during training, (2) using future information to predict the past, or (3) including the target variable or its proxies as features.

Data leakage is called the 'silent killer' because it doesn't cause errors or warnings - the model trains successfully and appears to perform exceptionally well. But when deployed to production, performance mysteriously drops because the leaked information isn't available.

Common sources of data leakage in flight delay prediction:

Target Leakage: Including 'ACTUAL_ARRIVAL_TIME' as a feature when predicting delays

Feature Leakage: Using 'TAXI_OUT_TIME' which is only known after departure

Temporal Leakage: Using future weather data to predict past delays

Preprocessing Leakage: Computing mean/std on full dataset before train/test split

Our implementation prevents data leakage through several mechanisms:

DataLeakageProtection.clean() removes columns that could leak target information

Feature transformers (StringIndexer, Scaler) are fit ONLY on training data

The FeatureExtractionModels case class stores fitted models for reuse on test data

The transform() method uses pre-fitted models without refitting

```
// FeatureExtractionModels stores all fitted components
case class FeatureExtractionModels(
  basePipelineModel: PipelineModel, // Fitted StringIndexers, VectorAssembler, Scaler
  pcaModel: Option[PCAModel],       // Fitted PCA (if enabled)
  featureNames: Array[String],      // Feature names for interpretation
  varianceAnalysis: Option[VarianceAnalysis] // PCA variance info
)

// Extract: Fits on training data
def extract(data: DataFrame, experiment: ExperimentConfig)
  : (DataFrame, FeatureExtractionModels) = {

  val cleanData = DataLeakageProtection.clean(data, target) // Remove leaky columns
  val (features, names, pipelineModel) = fitTransformPipeline(cleanData)
  // Returns transformed data AND fitted models
}

// Transform: Uses pre-fitted models on test data
def transform(data: DataFrame, models: FeatureExtractionModels, experiment: ExperimentConfig)
  : DataFrame = {

  val cleanData = DataLeakageProtection.clean(data, target)
  // NO FITTING - just transform with pre-fitted models
  models.basePipelineModel.transform(cleanData)
}
```

6.2.3 Feature Transformation Techniques

Our pipeline employs several transformation techniques to convert raw data into ML-ready features. Each technique serves a specific purposes:

StringIndexer for Categorical Variables

Definition: StringIndexer

StringIndexer converts a column of string labels to a column of label indices. The indices are assigned based on frequency, with the most frequent label getting index 0. This is Spark MLlib's primary method for encoding categorical variables.

For example, if we have an 'AIRLINE' column with values ['Delta', 'United', 'Delta', 'American', 'Delta'], StringIndexer would map: Delta -> 0 (most frequent), United -> 1, American -> 2.

Important considerations:

handleInvalid='keep' assigns a special index to unseen categories in test data

The mapping is learned from training data and applied consistently to test data

Indices are numerical but represent categories - not ordinal relationships

VectorAssembler for Feature Combination

Definition: VectorAssembler

VectorAssembler combines multiple columns (numerical and indexed categorical) into a single feature vector column. This is required because Spark ML models expect a single 'features' column containing all predictors as a dense or sparse vector.

After indexing categorical variables and identifying numerical columns, VectorAssembler concatenates them into a single vector. For example, if we have indexed_airline (0, 1, or 2), temperature (72.5), and humidity (0.65), the assembled vector would be [0.0, 72.5, 0.65].

StandardScaler for Normalization

Definition: StandardScaler

StandardScaler transforms features to have zero mean and unit variance (z-score normalization). For each feature, it computes: $z = (x - \text{mean}) / \text{std}$. This is essential for algorithms sensitive to feature scale, including PCA and regularized models.

Why scaling matters: If temperature ranges from 0-100 and distance from 0-5000, distance would dominate the model simply due to its larger scale. Scaling puts all features on equal footing.

Why is this important? StandardScaler is particularly important when using PCA because PCA finds directions of maximum variance. Without scaling, features with larger scales would dominate the principal components, potentially hiding important patterns in smaller-scale features.

```
// Feature extraction pipeline configuration
val enhancedPipeline = new EnhancedDataFeatureExtractorPipeline(
  textCols = allTextCols,      // Categorical columns for StringIndexer
  numericCols = allNumericCols, // Numerical columns (direct)
  booleanCols = allBooleanCols, // Boolean columns
  dateCols = allDateCols,      // Date columns (extract year, month, day, etc.)
  target = target,              // Target column name
  maxCat = 280,                 // Max categories per column
  handleInvalid = "keep",       // How to handle unseen categories
  scalerType = Some("standard") // StandardScaler for normalization
)
```

Feature extraction pipeline configuration

6.2.4 PCA Dimensionality reduction

Definition: Principal Component Analysis (PCA)

PCA is a dimensionality reduction technique that transforms data into a new coordinate system where the axes (principal components) are ordered by the amount of variance they explain. The first component captures the most variance, the second captures the next most (orthogonal to the first), and so on.

Imagine you have a dataset with 200 features. Many of these features are likely correlated - for example, temperature and humidity often move together. PCA finds these patterns and creates new 'super features' (principal components) that summarize the information more efficiently.

Benefits of PCA:

Reduced Dimensionality: Fewer features mean faster training and lower memory usage

Noise Reduction: Minor components often capture noise; removing them can improve generalization

Decorrelation: Principal components are orthogonal, eliminating multicollinearity

Visualization: First 2-3 components can be plotted to visualize data structure

Our implementation uses variance-based component selection. Instead of specifying a fixed number of components, we specify a variance threshold (e.g., 70%). The algorithm automatically selects enough components to explain at least that much variance.

Implementation Perspective - PCA is optional and configured in the experiment YAML. When enabled, the PCAFeatureExtractor fits on training data and the fitted PCAModel is stored in FeatureExtractionModels for reuse on test data. Variance analysis metrics are saved for visualization.

```
// PCA configuration in experiment
featureExtraction:
  featureType: "pca"
  pcaVarianceThreshold: 0.70 # Keep 70% of variance

// PCA application in FeatureExtractor
def applyPCA(data: DataFrame, featureNames: Array[String], experiment: ExperimentConfig)
  : (DataFrame, PCAModel, VarianceAnalysis) = {

  val pca = PCAFeatureExtractor.varianceBased(
    threshold = experiment.featureExtraction.pcaVarianceThreshold,
    outputCol = "pcaFeatures"
  )

  val (pcaModel, pcaData, analysis) = pca.fitTransform(data)

  // Log results
  println(f"Original features:  ${analysis.originalDimension}")
  println(f"PCA components:    ${analysis.numComponents}")
  println(f"Variance explained:  ${analysis.totalVarianceExplained * 100}%{f"%.2f"}%")

  (pcaData, pcaModel, analysis)
}
```

PCA experimentation in experiments

6.3 Model Factory Design Pattern

6.3.1 What is the Factory Design?

Definition: Factory Pattern

The Factory Pattern is a creational design pattern that provides an interface for creating objects without specifying their concrete classes. A factory method takes a specification (e.g., model type) and returns the appropriate object instance.

In everyday terms, think of a car factory. You don't build the car yourself - you tell the factory 'I want a sedan' and it handles all the complexity of creating the right vehicle. Similarly, our ModelFactory takes a model type ('randomforest') and returns a properly configured Random Forest classifier.

The Factory Pattern provides several advantages in software design:

Encapsulation: Object creation logic is centralized in one place

Extensibility: Adding new types requires changes only to the factory

Consistency: All objects are created with the same initialization patterns

Decoupling: Client code doesn't need to know concrete class names

6.3.2 Benefits for ML Systems

ML systems particularly benefit from the Factory Pattern because:

Experiments can easily switch between model types via configuration

Hyperparameters are consistently passed to all model types

Adding a new model (e.g., XGBoost) requires only extending the factory

Grid Search can iterate over model types if needed

Testing is simplified - mock models can be injected

Why is this important? In research and experimentation, we frequently want to compare different algorithms. The Factory Pattern allows us to run the same experiment with 'randomforest', 'gbt', and 'logisticregression' just by changing a configuration parameter, without touching any code.

6.3.3 Implementation Details

Our ModelFactory implementation:

```
object ModelFactory {  
  
  /**  
   * Create appropriate ML model based on experiment configuration  
   *  
   * @param experiment Configuration containing model type and hyperparameters  
   * @return MLModel instance ready for training  
   */  
  def create(experiment: ExperimentConfig): MLModel = {  
    val modelType = experiment.model.modelType.toLowerCase  
  
    println(s"[ModelFactory] Creating model: $modelType")  
  
    modelType match {  
      case "randomforest" | "rf" =>  
        println(" -> Random Forest Classifier")  
        new RandomForestModel(experiment)  
  
      case "gbt" | "gradientboostedtrees" =>  
        println(" -> Gradient Boosted Trees Classifier")  
        new GradientBoostedTreesModel(experiment)  
  
      case "logisticregression" | "lr" =>  
        println(" -> Logistic Regression Classifier")  
        new LogisticRegressionModel(experiment)  
  
      case unknown =>  
        throw new IllegalArgumentException(  
          s"Unknown model type: '$unknown'. " +  
          s"Available: randomforest, gbt, logisticregression"  
        )  
    }  
  }  
}
```

```

    }

    def supportedModels: Seq[String] = Seq(
      "randomforest", "gbt", "logisticregression"
    )
  }

```

Model Factory Implementation

All models implement the common MLModel trait, ensuring a consistent interface:

```

trait MLModel {
  /**
   * Train the model on provided data
   * @param data DataFrame with 'features' and 'label' columns
   * @return Trained model as a Transformer
   */
  def train(data: DataFrame): Transformer

  /**
   * Make predictions on new data
   */
  def predict(model: Transformer, data: DataFrame): DataFrame = {
    model.transform(data)
  }

  /**
   * Save model to disk (HDFS-compatible)
   */
  def saveModel(model: Transformer, path: String): Unit = {
    model.asInstanceOf[PipelineModel].write.overwrite().save(path)
  }

  /**
   * Load model from disk
   */
  def loadModel(path: String): Transformer = {
    PipelineModel.load(path)
  }
}

```

Trait MLModel

6.4 Machine Learning Models

Our system implements three classification algorithms, each with distinct characteristics suitable for different scenarios. Understanding these algorithms helps in choosing the right model for specific use cases.

6.4.1 Random Forest Classifier

Definition: Random Forest

Random Forest is an ensemble learning method that constructs multiple decision trees during training. Each tree is trained on a random subset of the data (bootstrap sample) and considers only a random subset of features at each split. The final prediction is the majority vote of all trees (for classification).

The 'randomness' in Random Forest comes from two sources:

Bootstrap Aggregating (Bagging): Each tree is trained on a random sample of the data drawn with replacement. Typically, about 63% of data points appear in each sample.

Random Feature Selection: At each split in a tree, only a random subset of features is considered. This decorrelates the trees, reducing variance.

Why Random Forest works well for flight delay prediction:

Handles non-linear relationships between weather conditions and delays

Robust to outliers (extreme weather events don't break the model)

Provides feature importance scores to understand what drives delays

Naturally handles class imbalance through bootstrap sampling

Resistant to overfitting due to ensemble averaging

Hyperparameters Explained

Parameter	What it controls	Typical range
numTrees	Number of trees in the forest. More trees = more stable but slower	25 -100
maxDepth	Maximum depth of each tree. Deeper = more complex patterns	15-50
maxBins	How finely continuous features are discretized	280
minInstancesPerNode	Minimum samples required at leaf. Higher = less overfitting	5-15
subsamplingRate	Fraction of data used per tree	0.8-1
featureSubsetStrategy	How many features to consider per split	sqrt, log2, auto, all
Impurity	Split quality measure	gini

Implementation Perspective

Our RandomForestModel includes critical optimizations for Spark: `setCacheNodeIds(true)` enables node ID caching for faster training, `setCheckpointInterval(5)` checkpoints every 5 trees to prevent lineage issues, and `setMaxMemoryInMB(2048)` allocates adequate memory for histogram aggregation.

```
class RandomForestModel(experiment: ExperimentConfig) extends MLModel {
```

```
  def train(data: DataFrame): Transformer = {  
    val hp = experiment.model.hyperparameters
```

```
    val rf = new RandomForestClassifier()  
      .setLabelCol("label")  
      .setFeaturesCol("features")  
      .setNumTrees(hp.numTrees.getOrElse(Seq(100)).head)  
      .setMaxDepth(hp.maxDepth.getOrElse(Seq(5)).head)  
      .setMaxBins(hp.maxBins.getOrElse(Seq(32)).head)  
      .setImpurity(hp.impurity.getOrElse("gini"))  
    // CRITICAL OPTIMIZATIONS for Spark  
      .setCacheNodeIds(true)    // Cache node IDs for faster training  
      .setCheckpointInterval(5) // Checkpoint every 5 trees  
      .setMaxMemoryInMB(2048)  // 2GB for histograms
```

```
    val pipeline = new Pipeline().setStages(Array(rf))  
    pipeline.fit(data)
```

```
  }  
}
```

6.4.2 Gradient Boosted Trees (GBT)

Definition: Gradient Boosted Trees

GBT is an ensemble method that builds trees sequentially. Unlike Random Forest where trees are independent, in GBT each tree tries to correct the errors of the previous ensemble. The final prediction is a weighted sum of all trees.

The key idea of boosting: start with a simple prediction, then iteratively add trees that focus on the mistakes. Each new tree is trained on the residual errors (the difference between predictions and actual values).

How GBT differs from Random Forest:

Sequential vs Parallel: GBT builds trees one after another; RF builds them independently

Error Correction vs Averaging: GBT corrects errors; RF averages predictions

More prone to overfitting: GBT can memorize training data if not regularized

Often higher accuracy: GBT typically achieves better accuracy with proper tuning

Slower training: Sequential nature limits parallelization

Key hyperparameters specific to GBT:

maxIter: Number of boosting iterations (trees). More = more complex model

stepSize: Learning rate - how much each tree contributes. Lower = more stable but slower

subsamplingRate: Fraction of data per tree. Lower helps prevent overfitting

Implementation Perspective - GBT in Spark MLlib supports binary classification natively. For probability outputs, it uses logistic transformation of raw predictions. The implementation mirrors RandomForestModel but with GBT-specific hyperparameters like stepSize (learning rate).

6.4.3 Logistic regression

Definition: Logistic Regression

Logistic Regression is a linear model that predicts the probability of a binary outcome. It uses the logistic (sigmoid) function to transform a linear combination of features into a probability between 0 and 1.

Despite its name, Logistic Regression is a classification algorithm, not regression. It models: $P(\text{delay}) = \text{sigmoid}(w_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + \dots + w_n \cdot x_n)$, where $\text{sigmoid}(z) = 1/(1+e^{-z})$

Advantages of Logistic Regression:

Fast training: Linear algorithms scale well to large datasets

Interpretable: Coefficients directly indicate feature influence

Probability calibration: Output probabilities are well-calibrated

Regularization support: L1 (Lasso), L2 (Ridge), ElasticNet prevent overfitting

Baseline model: Good starting point to compare against complex models

Regularization types:

L2 (Ridge): elasticNetParam=0.0: Shrinks coefficients, keeps all features

L1 (Lasso): elasticNetParam=1.0: Drives some coefficients to zero (feature selection)

ElasticNet: elasticNetParam=0.5: Combines L1 and L2 benefits

6.4.4 Model Comparison

Aspect	Random Forest	GBT	Logistic Regression
Training Speed	Fast (parallel)	Slow (sequential)	Very Fast
Accuracy	Good	Often Best	Moderate
Overfitting Risk	Low	Higher	Low
Interpretability	Feature Importance	Feature Importance	Coefficients

Model Comparison: Accuracy vs Training Time

6.5 Training Process

6.5.1 What happens during training?

Definition: Model Training

Training is the process of finding optimal model parameters by learning patterns from labeled training data. The algorithm iteratively adjusts parameters to minimize the difference between predictions and actual labels (the loss function).

For different algorithms, training means different things:

Random Forest: Building multiple decision trees on bootstrap samples, each with random feature selection. Each tree learns split points that best separate classes.

GBT: Sequentially building trees where each new tree fits the residual errors of the current ensemble. Parameters include split points and tree weights.

Logistic Regression: Finding coefficients (weights) that maximize the likelihood of observed labels. Uses iterative optimization (L-BFGS in Spark).

In distributed Spark, training is parallelized across the cluster. For Random Forest, different trees can be trained on different executors. For GBT and LR, data is partitioned and gradients are computed in parallel, then aggregated.

6.5.2 The Trainer Component

The Trainer object provides two methods for training models in different contexts:

trainFinal(): Used after cross-validation to train the final model on the full development set using the best hyperparameters found during Grid Search.

trainWithParams(): Used internally by CrossValidator to train models with specific hyperparameter configurations during Grid Search.

```
object Trainer {

  /**
   * Train final model on full development set with best hyperparameters
   */
  def trainFinal(
    devData: DataFrame,
    experiment: ExperimentConfig,
    bestHyperparameters: Map[String, Any]
  ): Transformer = {

    println(s"[Trainer] Training final model")
    println(s" - Samples: ${devData.count()}")

    if (bestHyperparameters.nonEmpty) {
      println(" - Best hyperparameters from Grid Search:")
      bestHyperparameters.foreach { case (k, v) => println(s"  $k: $v") }
    }
  }
}
```

```

    }

    val model = ModelFactory.create(experiment)
    model.train(devData)
  }

  /**
   * Train with specific hyperparameters (used during Grid Search)
   */
  def trainWithParams(
    data: DataFrame,
    experiment: ExperimentConfig,
    hyperparameters: Map[String, Any]
  ): Transformer = {

    val classifier = experiment.model.modelType match {
      case "randomforest" =>
        new RandomForestClassifier()
          .setNumTrees(hyperparameters("numTrees").asInstanceOf[Int])
          .setMaxDepth(hyperparameters("maxDepth").asInstanceOf[Int])
          // ... other hyperparameters
      case "gbt" => // ...
      case "logisticregression" => // ...
    }

    new Pipeline().setStages(Array(classifier)).fit(data)
  }
}

```

The two methods for train object

6.5.3 Hyperparameter configuration

Definition: Hyperparameters

Hyperparameters are configuration settings that control the learning process, as opposed to model parameters which are learned from data. Hyperparameters must be set before training and significantly impact model performance.

In our system, hyperparameters are configured through YAML files. For Grid Search, arrays of values are specified, and all combinations are tested:

```

# Example experiment configuration (YAML)
experiments:
- name: "rf_grid_search"
  description: "Random Forest with Grid Search optimization"

  model:
    modelType: "randomforest"
    hyperparameters:
      numTrees: [100, 200, 300]    # 3 values
      maxDepth: [10, 15, 20]      # 3 values
      maxBins: [32, 64]           # 2 values
      # Total: 3 x 3 x 2 = 18 combinations
      minInstancesPerNode: [1]
      subsamplingRate: [0.8]
      featureSubsetStrategy: ["sqrt"]
      impurity: "gini"

```

```

train:
  trainRatio: 0.8      # 80% dev, 20% test
  gridSearch:
    enabled: true
    evaluationMetric: "f1" # Optimize for F1-Score
  crossValidation:
    numFolds: 5        # 5-fold CV

```

Example experiment configuration (YAML)

Why is this important? YAML configuration allows experiments to be versioned, shared, and reproduced without code changes. A researcher can run different experiments just by modifying configuration files, making it easy to track what settings produced which results.

6.6 Cross-validation

6.6.1 Why simple train/test split is not enough

A single train/test split has a fundamental problem: the performance estimate depends heavily on which data points ended up in which set. Different splits can yield very different accuracy numbers.

Consider this scenario:

You split data 80/20 and get 85% accuracy on the test set

Your colleague uses a different random seed, same algorithm, gets 79%

Who has the better model? Neither - the difference is due to the split!

The 'lucky' split might have easier examples in the test set

Cross-validation solves this by evaluating on multiple different splits and averaging the results. This gives a more stable and reliable estimate of model performance.

Why is this important? Cross-validation answers two questions: (1) How well will this model perform on new data? (2) How stable is this performance estimate? A model with 82% +/- 1% is more reliable than one with 82% +/- 5%, even if the average is the same.

6.6.2 K-Fold cross-validation explained

Definition: K-Fold Cross-Validation

K-Fold CV divides the training data into K equal-sized 'folds'. The model is trained K times, each time using K-1 folds for training and 1 fold for validation. Each data point serves as validation exactly once. Final metrics are averaged across all folds.

How K-Fold works with K=5:

Divide data into 5 equal folds: F1, F2, F3, F4, F5

Iteration 1: Train on F2+F3+F4+F5, validate on F1

Iteration 2: Train on F1+F3+F4+F5, validate on F2

Iteration 3: Train on F1+F2+F4+F5, validate on F3

Iteration 4: Train on F1+F2+F3+F5, validate on F4

Iteration 5: Train on F1+F2+F3+F4, validate on F5

Final: Average metrics from all 5 iterations

Choosing K:

K=5 or K=10: Most common choices. Good balance of bias and variance

Large K (e.g., 20): Lower bias but higher variance, computationally expensive
K=N (Leave-One-Out): Nearly unbiased but very high variance, very slow
Small K (e.g., 3): Faster but higher bias, less reliable estimate

6.6.3 Implementation in our system

```
object CrossValidator {

  case class CVResult(
    avgMetrics: EvaluationMetrics, // Mean across folds
    stdMetrics: EvaluationMetrics, // Standard deviation
    foldMetrics: Seq[EvaluationMetrics], // Per-fold results
    bestHyperparameters: Map[String, Any],
    numFolds: Int
  )

  def validate(devData: DataFrame, experiment: ExperimentConfig): CVResult = {
    val numFolds = experiment.train.crossValidation.numFolds

    // Assign each row to a fold (random, reproducible)
    val dataWithFold = devData.withColumn(
      "fold",
      (rand(seed) * numFolds).cast("int")
    )

    // Train and evaluate on each fold
    val foldMetrics = (0 until numFolds).map { foldIdx =>

      // Split: train on all except current fold
      val trainFold = dataWithFold.filter(col("fold") != foldIdx)
      val valFold = dataWithFold.filter(col("fold") == foldIdx)

      // Train model
      val model = ModelFactory.create(experiment)
      val trainedModel = model.train(trainFold)

      // IMPORTANT: Save and reload to avoid Spark broadcast OOM
      trainedModel.write.overwrite().save(tempPath)
      val reloadedModel = PipelineModel.load(tempPath)

      // Evaluate on validation fold
      val predictions = reloadedModel.transform(valFold)
      ModelEvaluator.evaluate(predictions)
    }

    // Aggregate statistics
    val (avgMetrics, stdMetrics) = calculateStatistics(foldMetrics)

    CVResult(avgMetrics, stdMetrics, foldMetrics, Map.empty, numFolds)
  }
}
```

Cross-validation

Implementation Perspective

A critical implementation detail: after training each fold's model, we save it to disk and reload before evaluation. This prevents Spark broadcast out-of-memory errors with large models like Random Forest with 300 trees. The temporary models are cleaned up after each fold.

6.7 Grid search for hyperparameter tuning

6.7.1 What are Hyperparameters?

To understand hyperparameters, it helps to contrast them with regular model parameters:

Aspect	Model Parameters	Hyperparameters
Set by	Learned from data during training	Specified before training
Example (RF)	Tree split points, feature thresholds	numTrees, maxDepth
Example (LR)	Feature coefficients (weights)	regParam, maxIter
How to optimize	Automatic (optimization algorithm)	Manual tuning or Grid Search

Hyperparameters have profound effects on model behavior. For example, in Random Forest:
numTrees=10 vs numTrees=500: More trees = more stable predictions but slower
maxDepth=3 vs maxDepth=20: Deeper trees capture more complexity but risk overfitting
minInstancesPerNode=1 vs minInstancesPerNode=100: Higher = simpler model, less overfitting

6.7.2 The grid search algorithm

Definition: Grid Search

Grid Search is an exhaustive hyperparameter optimization technique. It defines a 'grid' of possible values for each hyperparameter and evaluates all possible combinations, selecting the combination that achieves the best performance metric.

Example grid and search process:

```
# Hyperparameter grid
numTrees: [100, 200, 300] # 3 values
maxDepth: [10, 15] # 2 values
subsamplingRate: [0.8, 1.0] # 2 values

# Total combinations: 3 x 2 x 2 = 12

# Grid Search evaluates all 12:
1. numTrees=100, maxDepth=10, subsamplingRate=0.8 -> F1=0.78
2. numTrees=100, maxDepth=10, subsamplingRate=1.0 -> F1=0.77
3. numTrees=100, maxDepth=15, subsamplingRate=0.8 -> F1=0.79
4. numTrees=100, maxDepth=15, subsamplingRate=1.0 -> F1=0.78
5. numTrees=200, maxDepth=10, subsamplingRate=0.8 -> F1=0.80
6. numTrees=200, maxDepth=10, subsamplingRate=1.0 -> F1=0.79
7. numTrees=200, maxDepth=15, subsamplingRate=0.8 -> F1=0.82 <-- BEST
8. numTrees=200, maxDepth=15, subsamplingRate=1.0 -> F1=0.81
9. numTrees=300, maxDepth=10, subsamplingRate=0.8 -> F1=0.80
10. numTrees=300, maxDepth=10, subsamplingRate=1.0 -> F1=0.79
11. numTrees=300, maxDepth=15, subsamplingRate=0.8 -> F1=0.81
12. numTrees=300, maxDepth=15, subsamplingRate=1.0 -> F1=0.80

# Winner: combination 7 with F1=0.82
```

Example of GridSearch process

Why is this important? Grid Search guarantees finding the best combination within the specified grid. However, it doesn't guarantee finding the global optimum - that depends on choosing good value ranges. It's also computationally expensive: 10 values for each of 4 hyperparameters means 10,000 combinations!

6.7.3 Combining grid search with cross-validation

Grid Search alone has a problem: evaluating on a single validation set can be misleading due to variance in the split. The solution is to combine Grid Search with K-Fold CV:

For each hyperparameter combination:

Run K-Fold Cross-Validation (e.g., 5 folds)

Compute mean metric across all folds

Use mean metric to compare combinations

Standard deviation indicates stability of that configuration

This gives a much more reliable comparison. A configuration with $F1=0.80 \pm 0.01$ is preferable to one with $F1=0.81 \pm 0.05$, even though the latter has a higher mean.

```
private def validateWithGridSearch(...): CVResult = {  
  
    // Build parameter grid from configuration  
    val paramGrid = buildParameterGrid(experiment)  
    println(s"Total combinations: ${paramGrid.size}")  
  
    // Evaluate each combination with full K-Fold CV  
    val gridResults = paramGrid.map { params =>  
        println(s"Testing: $params")  
  
        // Run K-Fold CV with these hyperparameters  
        val cvResult = validateWithParams(devData, experiment, params, numFolds)  
  
        // Get the optimization metric (e.g., F1)  
        val metricValue = getMetricValue(  
            cvResult.avgMetrics,  
            experiment.train.gridSearch.evaluationMetric // "f1", "accuracy", "auc"  
        )  
  
        println(f" -> Mean ${experiment.train.gridSearch.evaluationMetric}: $metricValue%.4f")  
  
        (params, cvResult, metricValue)  
    }  
  
    // Select best combination  
    val (bestParams, bestCVResult, _) = gridResults.maxBy(_._3)  
  
    println("=" * 80)  
    println("[Grid Search] Best hyperparameters found:")  
    bestParams.foreach { case (k, v) => println(s" $k: $v") }  
  
    bestCVResult.copy(bestHyperparameters = bestParams)  
}
```

6.8 Model Evaluation

6.8.1 Understanding Classification Metrics

For binary classification (delayed vs on-time), several metrics help us understand different aspects of model performance. No single metric tells the whole story.

Accuracy

Definition: Accuracy

Accuracy = (Correct Predictions) / (Total Predictions) = $(TP + TN) / (TP + TN + FP + FN)$. It measures the overall correctness of the model.

Accuracy is intuitive but can be misleading with imbalanced classes. If 90% of flights are on-time, a model that always predicts 'on-time' achieves 90% accuracy while being completely useless for detecting delays.

Precision

Definition: Precision

Precision = $TP / (TP + FP)$. Of all flights predicted as delayed, what fraction actually was delayed? High precision means few false alarms.

Precision is important when false positives are costly. Sending delay notifications for flights that are actually on-time annoys passengers and wastes resources.

Recall

Definition: Recall

Recall = $TP / (TP + FN)$. Of all actually delayed flights, what fraction did we correctly predict? High recall means we catch most delays.

Recall is important when missing positives is costly. Failing to predict a delay means passengers aren't warned and can't make alternative arrangements.

F1-Score

Definition: F1-Score

$F1 = 2 * (Precision * Recall) / (Precision + Recall)$. The harmonic mean of precision and recall, providing a single metric that balances both concerns.

F1-Score is useful when you need a single number to compare models and care about both precision and recall. It's our primary optimization metric during Grid Search.

6.8.2 The confusion matrix

The confusion matrix is a 2x2 table that shows the four possible prediction outcomes:

	Predicted: On-Time	Predicted: Delayed
Actual: On-Time	TN (Correct)	FP (Type I Error)
Actual: Delayed	FN (Type II Error)	TP (Correct)

Confusion Matrix

Understanding each cell of the matrix:

True Negative (TN): On-time flight correctly predicted as on-time

False Positive (FP): On-time flight incorrectly predicted as delayed (false alarm)

False Negative (FN): Delayed flight incorrectly predicted as on-time (missed delay)

True Positive (TP): Delayed flight correctly predicted as delayed

Implementation Perspective - Our ModelEvaluator computes the confusion matrix directly from predictions using Spark aggregations, then derives all metrics from these four counts. This is more efficient than using separate evaluators for each metric.

```
def evaluate(predictions: DataFrame): EvaluationMetrics = {  
  
  // Compute confusion matrix via groupBy  
  val confusionMatrix = predictions  
    .groupBy("label", "prediction")  
    .count()  
    .collect()  
    .map(row => ((row.getDouble(0), row.getDouble(1)), row.getLong(2)))  
    .toMap  
  
  val tp = confusionMatrix.getOrElse((1.0, 1.0), 0L)  
  val tn = confusionMatrix.getOrElse((0.0, 0.0), 0L)  
  val fp = confusionMatrix.getOrElse((0.0, 1.0), 0L)  
  val fn = confusionMatrix.getOrElse((1.0, 0.0), 0L)  
  
  // Derive metrics  
  val precision = if (tp + fp > 0) tp.toDouble / (tp + fp) else 0.0  
  val recall = if (tp + fn > 0) tp.toDouble / (tp + fn) else 0.0  
  val f1 = if (precision + recall > 0)  
    2.0 * (precision * recall) / (precision + recall) else 0.0  
  
  // ...  
}
```

Confusion matrix implementation

6.8.3 ROC Curve and AUC

Definition: ROC Curve

The Receiver Operating Characteristic (ROC) curve plots the True Positive Rate (Recall) against the False Positive Rate at various classification thresholds. It shows the trade-off between catching positives and avoiding false alarms.

Most classifiers output a probability (e.g., $P(\text{delay}) = 0.73$), which is then converted to a class using a threshold (default 0.5). The ROC curve shows performance across all possible thresholds.

Definition: AUC-ROC

Area Under the ROC Curve. Ranges from 0 to 1, with 0.5 being random guessing and 1.0 being perfect classification. AUC can be interpreted as: the probability that a randomly chosen positive example ranks higher than a randomly chosen negative example.

Interpreting AUC values:

0.90-1.00: Excellent discrimination

0.80-0.90: Good discrimination

0.70-0.80: Acceptable discrimination

0.60-0.70: Poor discrimination

0.50-0.60: No discrimination (random)

6.8.4 Domain-specific metrics (RECD, RECO)

For flight delay prediction, we define two domain-specific metrics that directly relate to business requirements:

- **RECD (Recall for Delayed flights):** The proportion of actual delayed flights that we correctly predict. This is the same as standard Recall. Formula: $TP / (TP + FN)$
- **RECO (Recall for On-time flights):** The proportion of actual on-time flights that we correctly predict. This is also known as Specificity. Formula: $TN / (TN + FP)$

The balance between RECD and RECO reflects business priorities:

High RECD, Lower RECO: Catch most delays but more false alarms (annoying but safe)

High RECO, Lower RECD: Few false alarms but miss some delays (risky for passengers)

Balanced: Trade-off depends on costs of false positives vs false negatives

Implementation Perspective - RECD and RECO are computed in the EvaluationMetrics case class as derived properties from the confusion matrix counts. They are logged to MLFlow and included in all evaluation reports.

6.9 Complete pipeline orchestration

6.9.1 The ML Pipeline object

MLPipeline is the main entry point that orchestrates the entire machine learning workflow. It coordinates feature extraction, cross-validation, training, evaluation, and artifact persistence.

```
object MLPipeline {

  /**
   * Complete result of ML pipeline execution
   */
  case class MLResult(
    experiment: ExperimentConfig,
    model: Transformer,      // Trained model
    cvMetrics: CVMetrics,    // Cross-validation metrics (mean +/- std)
    holdOutMetrics: EvaluationMetrics, // Final test set metrics
    bestHyperparameters: Map[String, Any],
    trainingTimeSeconds: Double
  )

  /**
   * Main entry point for training and evaluation
   */
  def train(
    devDataRow: DataFrame, // 80% development set
    testDataRow: DataFrame, // 20% hold-out test set
    experiment: ExperimentConfig
  ): MLResult = {
    // Step 1-7 orchestration...
  }
}
```

ML Pipeline

6.9.2 Step-by-Step Execution

The pipeline executes in 7 well-defined steps:

Step 1: Receive Datasets

The pipeline receives pre-split, balanced datasets. The split was done earlier in FeaturePipeline to ensure no data leakage.

Step 2: Feature Extraction (Dev)

FeatureExtractor.extract() fits all transformers (StringIndexer, Scaler, PCA) on the development set and returns transformed data plus fitted models.

Step 3: Feature Extraction (Test)

FeatureExtractor.transform() applies pre-fitted models to the test set WITHOUT refitting, preventing data leakage.

Step 4: Cross-Validation

CrossValidator.validate() runs K-Fold CV on the development set. If Grid Search is enabled, all hyperparameter combinations are evaluated.

Step 5: Train Final Model

Trainer.trainFinal() trains the final model on the FULL development set using the best hyperparameters from Grid Search.

Step 6: Hold-out Evaluation

The final model is evaluated on the completely unseen test set, giving an unbiased performance estimate. Model is saved/reloaded to avoid broadcast OOM.

Step 7: Save Artifacts

Metrics, model files, configuration, and visualizations are saved for reproducibility and logged to MLFlow.

6.9.3 MLFlow Integration

Definition: MLFlow

MLFlow is an open-source platform for managing the ML lifecycle. It provides: (1) Experiment tracking - log parameters, metrics, artifacts; (2) Model registry - version and stage models; (3) Reproducibility - record all inputs and outputs.

Our integration logs the following to MLFlow:

Parameters: experiment_name, model_type, train_ratio, cv_folds, feature_type, hyperparameters

Metrics: cv_mean_accuracy, cv_std_accuracy, test_accuracy, test_f1, test_auc, RECD, RECo

Artifacts: Trained model files, configuration YAML, metrics CSVs, visualization plots

Tags: experiment_description, environment (dev/prod)

Implementation Perspective - MLFlowTracker is initialized at the start of the pipeline. Each run gets a unique ID, and all metrics/artifacts are logged throughout execution. The run is closed at the end with a final status.

```
// MLFlow tracking throughout pipeline
MLFlowTracker.initialize(trackingUri, enabled)
val experimentId = MLFlowTracker.getOrCreateExperiment()
val runId = MLFlowTracker.startRun(experimentId, experiment.name, description)

// Log parameters
runId.foreach { rid =>
  MLFlowTracker.logParams(rid, Map(
    "model_type" -> experiment.model.modelType,
    "train_ratio" -> experiment.train.trainRatio,
    // ...
  ))
}

// Log metrics after CV
MLFlowTracker.logMetrics(rid, Map(
  "cv_mean_f1" -> cvResult.avgMetrics.f1Score,
  "cv_std_f1" -> cvResult.stdMetrics.f1Score,
  // ...
})
```

```

))

// Log metrics after hold-out evaluation
MLFlowTracker.logMetrics(rid, Map(
  "test_accuracy" -> holdOutMetrics.accuracy,
  "test_f1" -> holdOutMetrics.f1Score,
  // ...
))

// Log artifacts
MLFlowTracker.logArtifactWithPath(rid, modelPath, "models")
MLFlowTracker.logArtifactWithPath(rid, metricsPath, "metrics")

// End run
MLFlowTracker.endRun(rid)

```

MLFlow tracking implementation through the pipeline

6.10 Advanced Topics

6.10.1 Memory Management in Spark

Distributed systems like Spark have unique memory challenges. Our implementation addresses several common issues:

Broadcast Out-of-Memory Prevention

When Spark transforms data using a model (e.g., Random Forest with 300 trees), it broadcasts the model to all executors. Large models can exceed broadcast limits, causing OOM errors.

Solution: Save and reload the model before transformation:

```

// Problem: Large model causes broadcast OOM
val predictions = largeModel.transform(testData) // OOM!

// Solution: Save to disk, reload, then transform
largeModel.asInstanceOf[PipelineModel].write.overwrite().save(modelPath)
val reloadedModel = PipelineModel.load(modelPath)
val predictions = reloadedModel.transform(testData) // Works!

```

Strategic DataFrame Caching

Spark evaluates DataFrames lazily. Without caching, the same computation might run multiple times. We cache strategically at key points:

After feature extraction (before multiple CV folds use the data)

After joining datasets (expensive operation)

Predictions DataFrame before computing multiple metrics

```

// Cache after expensive operations
val features = extractFeatures(data)
val cachedFeatures = features.cache()
cachedFeatures.count() // Force materialization

// Use cached data multiple times
val cvResult = CrossValidator.validate(cachedFeatures, ...)
val finalModel = Trainer.trainFinal(cachedFeatures, ...)

```

```
// Unpersist when done
cachedFeatures.unpersist()
```

6.10.2 Model Persistence

Trained models need to be saved for deployment and reproducibility. Spark ML provides native serialization that works with both local filesystem and HDFS.

```
// Saving a trained model
val model: PipelineModel = pipeline.fit(data)
model.write.overwrite().save("/output/experiment_1/models/rf_final")

// Loading a saved model
val loadedModel = PipelineModel.load("/output/experiment_1/models/rf_final")
val predictions = loadedModel.transform(newData)

// For Random Forest, extract the classifier to get feature importances
val rfModel = loadedModel.stages(0).asInstanceOf[RandomForestClassificationModel]
val importances = rfModel.featureImportances
```

Model saving

Implementation Perspective

All paths use Hadoop FileSystem API, making the code compatible with both local development and HDFS cluster deployment. The same code works regardless of whether paths start with 'file:/// or 'hdfs:///.

6.10.3 Visualization and Reporting

The pipeline generates comprehensive outputs for analysis and reporting:

cv_fold_metrics.csv: Per-fold metrics for analyzing CV stability

cv_summary.csv: Mean and standard deviation of CV metrics

holdout_test_metrics.csv: Final test set performance

best_hyperparameters.csv: Selected hyperparameters from Grid Search

holdout_roc_data.csv: Data points for plotting ROC curve

training_summary.txt: Human-readable summary of the entire experiment

feature_importances.csv: Feature importance rankings

A Python visualization script generates plots from these CSV files:

```
# Visualize ML pipeline results
```

```
python work/scripts/visualize_ml_pipeline.py /output/experiment_1/metrics
```

```
# Generated plots:
```

```
# - cv_fold_metrics.png: Bar chart of metrics per fold
```

```
# - cv_vs_holdout.png: Comparison of CV and test performance
```

```
# - confusion_matrix.png: Visual confusion matrix
```

```
# - roc_curve.png: ROC curve with AUC
```

```
# - metrics_radar.png: Radar chart of all metrics
```

6.11 Conclusion on Machine Learning

This document has provided a comprehensive, pedagogical overview of the Machine Learning implementation for the Flight Delay Prediction system. The key takeaways are:

Rigorous Methodology: Hold-out test set + K-Fold CV ensures reliable and unbiased performance estimates

Data Leakage Prevention: Strict fit/transform separation prevents information leakage from test data to training

Modular Architecture: Factory pattern and clear component separation enable extensibility and maintainability

Hyperparameter Optimization: Grid Search with CV finds optimal configurations systematically

Comprehensive Evaluation: Multiple metrics (Accuracy, F1, AUC, RECD, RECO) provide complete performance picture

Reproducibility: MLFlow tracking and artifact persistence ensure experiments can be reproduced and compared

Production-Ready: Memory management, HDFS compatibility, and robust error handling support real-world deployment

The implementation demonstrates best practices in distributed machine learning with Apache Spark MLlib, providing a solid foundation for flight delay prediction at scale. The principles and patterns described here are applicable to many other classification problems in industry.

7. Environments - Implementation Details

Our system is designed for deployment across multiple execution environments, from local development setups to production-grade distributed clusters. This flexibility enables iterative development with rapid feedback cycles on local machines, followed by full-scale evaluation on high-performance infrastructure. We describe three primary environments utilized during this project: Local docker cluster, LAMSADE Cluster, and Google cloud DATAPROC.

7.1 Local Docker Cluster

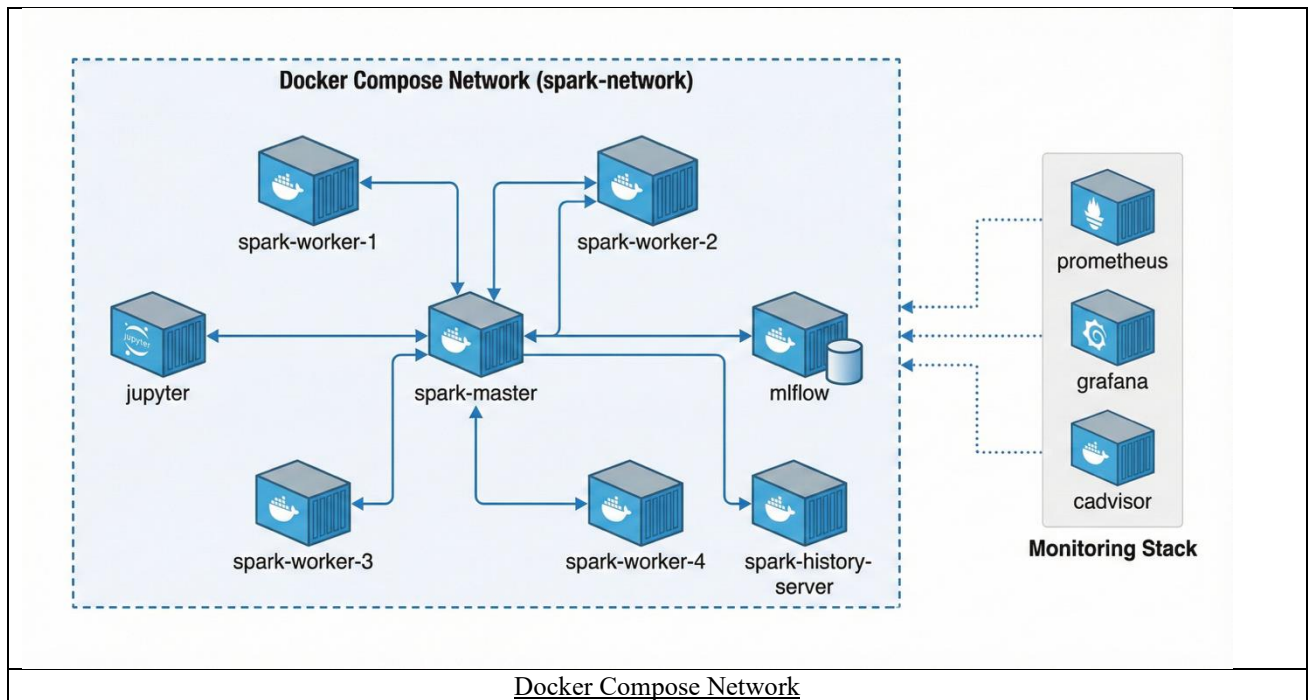
We document and analyze here the architecture of a complete **Apache Spark cluster** deployed locally using **Docker Compose**. Specifically designed for the development, experimentation, and monitoring of data science and machine learning solutions, this cluster provides an integrated and high-performance ecosystem.

It is important to note that the host environment is a **Mac Pro M4** equipped with **14 cores** and **48 GB of RAM**. This hardware context directly influenced and justified the resource allocation decisions within the cluster, allowing for a generous and realistic configuration suited to complex workloads.

7.1.1 General architecture and components

Service Overview

The cluster is defined by a set of interconnected services described in a **docker-compose.yml** file, where each container fulfills a specific and essential role within the overall ecosystem. This modular approach ensures **clarity**, **maintainability**, and **reproducibility** of the infrastructure.



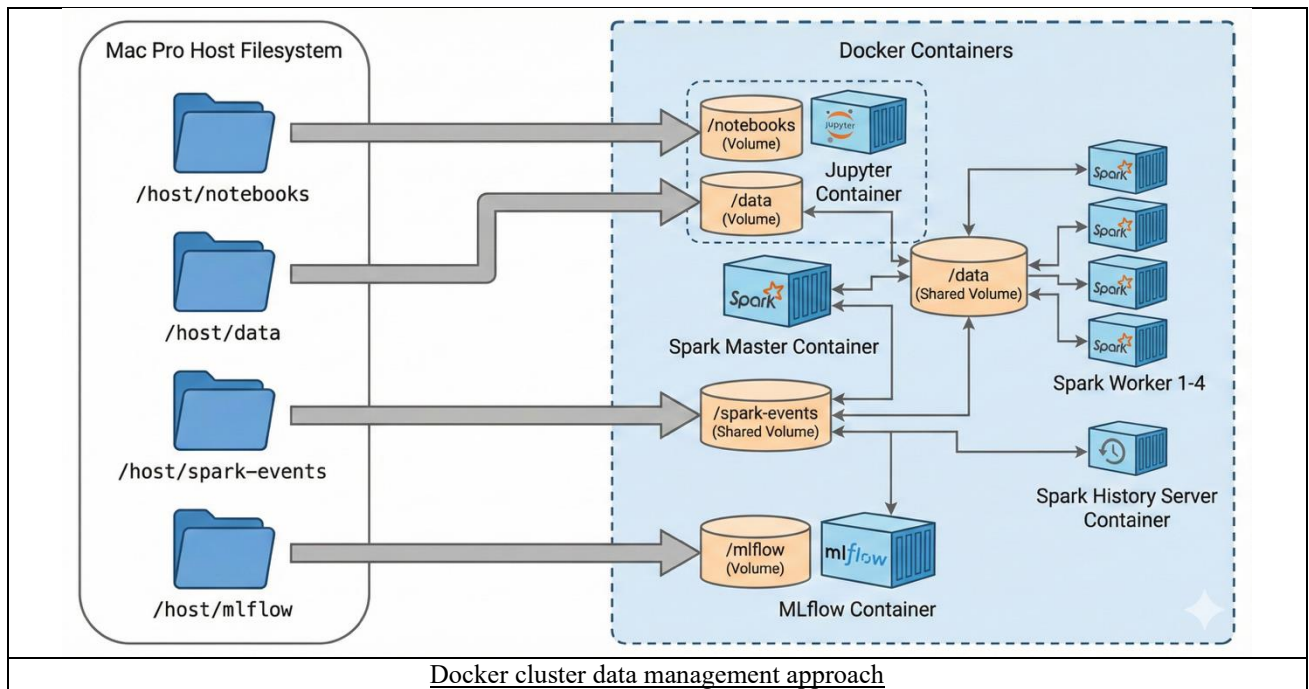
The below table lists each of the deployed services, and respective role in the architecture:

Service	Primary Role
spark-master	Cluster orchestration, resource management, and application scheduling.
spark-worker-1 to 4	Execution of Spark application tasks; these are the compute nodes.
spark-submit	Utility container for submitting Spark batch applications to the cluster.
jupyter	Interactive development environment (JupyterLab) for Python and Scala code.
mlflow	Centralized tracking server for machine learning experiments.
spark-history-server	Web interface for post-mortem analysis of completed Spark applications.
prometheus	Collection and storage of performance metrics as time series data.
grafana	Visualization platform for creating dashboards from Prometheus metrics.
cadvisor	Collection of low-level Docker container metrics (CPU, memory, network).

Service roles overview

Network and data persistence strategy

In this Docker cluster, we adopt a network and data management strategy that is both simple and robust, perfectly suited to a development environment.



- **Network:** All containers communicate over a single bridge-type network named spark-network. This configuration ensures complete isolation of the cluster from the host network while allowing seamless communication and simplified name resolution between services (for example, spark-worker-1 can reach spark-master using its service name).
- **Data Persistence:** Data, code, and configuration persistence are provided through the use of Docker volumes. This approach maps directories from the host file system to directories inside the containers. The main mapped volumes include:
 - **/notebooks, /data, /apps:** Ensure the persistence of code, datasets, and compiled applications, making them accessible across restarts and shared among relevant services (Jupyter, Master, Workers).
 - **/spark-events:** A critical directory where Spark applications write their event logs, later read by the Spark History Server.
 - **/mlflow:** Contains the SQLite database and artifacts (models, charts) generated by MLFlow experiment tracking, ensuring durability over time.

This unified approach to network and data management not only simplifies service discovery, but also establishes a foundational model for environment replication and disaster recovery-ensuring that the entire development stack can be reliably reconstructed.

The following analysis will focus on the Spark components, the core of this architecture.

7.1.2 Detailed analysis of spark components

The Master Node (spark-master)

The **spark-master** service is the brain of the cluster. It serves as the central coordinator, responsible for resource allocation, communication with worker nodes, and scheduling of submitted applications. Its configuration is optimized for a local development environment.

- **Image Used:** bitnamilegacy/spark:3.5.3 - a stable and widely used image for Spark deployments.
- **Memory Configuration:** SPARK_DAEMON_MEMORY=2g allocates 2 GB of memory to the master daemon process, which is sufficient to manage the state of a cluster of this size.
- **Key Environment Variables:**
 - SPARK_MODE=master configures the container to operate as the master node.

- The options `SPARK_RPC_...` and `SPARK_SSL_...` are explicitly disabled (no). This decision significantly simplifies configuration for local use by removing the complexity of certificate and authentication management, which is unnecessary in this context.
- **Port Exposure:**
 - **8080:** Exposes the Spark Master Web UI, allowing users to view the cluster status, worker nodes, and running applications.
 - **7077:** The main RPC communication port. This is the URL (`spark://spark-master:7077`) used by workers and client applications to connect to the master.
 - **9999:** Exposes JMX metrics through a JMX Exporter agent, making them available for collection by Prometheus.

Worker nodes (spark-workers)

The spark-worker containers are the backbone of the cluster. They provide the computational power by executing the distributed tasks assigned by the master. The architecture deploys four such nodes, all configured identically.

- **Number of Workers and Total Resources:** The cluster is composed of **4 workers**. Combined, they offer a total computing capacity of **12 CPU cores** (4 workers × 3 cores) and **40 GB of RAM** (4 workers × 10 GB).
- **Individual Configuration:** Each worker is configured as follows:
 - **Memory and Cores:**
`PARK_WORKER_MEMORY=10G` and `SPARK_WORKER_CORES=3` define the resources each worker contributes to the cluster for Spark executors.
 - **Connection to the Master:**
The variable `SPARK_MASTER_URL` is set to `spark://spark-master:7077`, enabling each worker to register with the master coordinator.
 - **JMX Configuration:**
The variable `SPARK_WORKER_OPTS` is used to attach the JMX Exporter agent, allowing Prometheus to collect detailed metrics on JVM performance and executor activity for each worker.
- **Custom Docker Image:**
The workers use the image `custom-spark-mlflow:3.5.3`, which is not a standard distribution. This image is built locally to include specific dependencies, as detailed in the following section.

Custom Docker Images

The use of custom Docker images is a best practice that enables the creation of reproducible runtime environments and the integration of specific libraries or tools not included in the base images.

Image: custom-spark-mlflow

This image, built from the `Dockerfile.spark-mlflow`, is designed for executing machine learning tasks.

- **Base Image:** It extends the `bitnamilegacy/spark:3.5.3` image, ensuring full compatibility with the master node.
- **Purpose:** Its primary goal is to enrich the Spark runtime environment with essential Python libraries for machine learning and experiment tracking. This includes MLflow, scikit-learn, pandas, matplotlib, and seaborn. Crucially, all Python libraries are installed with pinned versions, ensuring a deterministic and reproducible runtime environment for all machine learning tasks across the cluster.
- **Usage:** The image is used by all worker nodes as well as the `spark-submit` container. This uniformity ensures that code executed anywhere in the cluster has access to the same dependencies, regardless of the node on which a task is scheduled-thereby preventing environment-related errors.

Development Image – Dockerfile.jupyter

This image, defined by `Dockerfile.jupyter`, creates a complete and preconfigured development environment.

- **Base Image:** It is built on `quay.io/jupyter/all-spark-notebook:spark-3.5.3`, a well-equipped community Jupyter image.

- **Polyglot Environment:** This image establishes a polyglot development environment by extending the Python-centric base with a full Scala toolchain and JDK 11. The integration of a complete Spark distribution, rather than just a client library, combined with the inclusion of native Delta Lake JARs, enables developers to build and test end-to-end data pipelines that leverage both PySpark and Scala APIs on modern transactional data formats.
- **Kernel Configuration:** The kernel.json file for the Scala kernel is modified-a critical step that injects a predefined set of Spark properties directly into every session launched from Jupyter. This ensures that all interactive workloads automatically benefit from optimized parallelism (spark.sql.shuffle.partitions=128), adaptive query execution, and consistent event logging to /spark-events, which is essential for the Spark History Server.

These custom images form the backbone of the ecosystem, which is complemented by additional development and monitoring services.

7.1.3 Development and monitoring ecosystem

Integrated development environment (Jupyter)

The Jupyter service provides a JupyterLab web interface, a powerful interactive development environment for writing, executing, and debugging Spark code in Python and Scala. Its configuration is finely tuned to ensure seamless integration with the cluster.

- **Cluster Connection:**
The variable SPARK_MASTER_URL is set so that all Spark sessions initiated from Jupyter automatically connect to the cluster master.
- **Resource Allocation:**
The variables SPARK_DRIVER_MEMORY=8g and SPARK_EXECUTOR_MEMORY=7g are critical. They define the default resources allocated to the driver (the main program running in the Jupyter container) and the executors (the processes running on the workers) for each launched application.
- **Event Logging Activation:**
The configuration --conf spark.eventLog.enabled=true, passed through the SPARK_OPTS variable, is essential.
It instructs every Spark application to log its execution events in a shared directory-a prerequisite for the Spark History Server to reconstruct and display detailed execution information.

Experiment Tracking (MLflow)

The mlflow service acts as a centralized tracking server for machine learning experiments. It provides a web interface and API to log, in a structured manner, the parameters, metrics, and artifacts (such as trained models or performance charts) for each model execution.

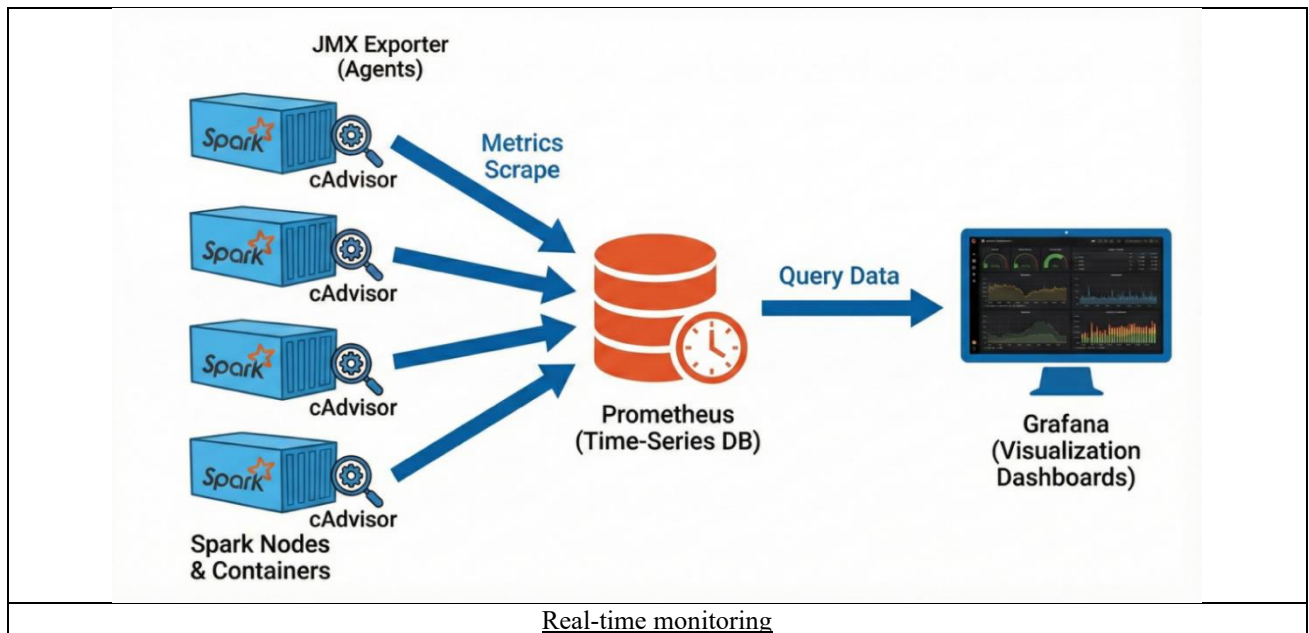
In this architecture, it relies on a SQLite database for metadata storage and the local file system for artifact storage-both made persistent through Docker volumes.

Post-Mortem Analysis (Spark History Server)

The spark-history-server service provides a web interface that allows users to review the execution details of completed Spark applications. It is essential for debugging and performance optimization, as it enables the analysis of execution plans (DAGs), access to task logs, and inspection of detailed performance metrics. Its operation depends entirely on the shared /spark-events volume, where applications-such as those launched from Jupyter-write their event logs.

Real-Time Monitoring Stack

A comprehensive real-time monitoring stack is integrated to observe the cluster's health and performance continuously. Its architecture is composed of four complementary services:



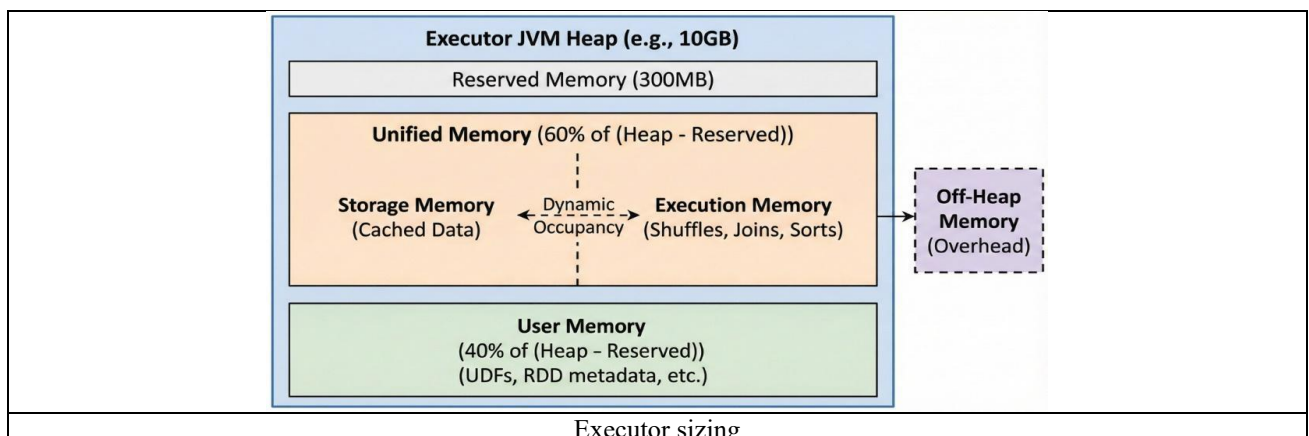
- **cAdvisor:** Collects low-level metrics directly from the host's Docker daemon. It provides information on resource utilization (CPU, memory, network, disk I/O) for each running container.
- **JMX Exporter:** Operates as a Java agent within each Spark JVM (spark-master, spark-worker-*). It extracts internal JMX MBeans and translates them into a Prometheus-compatible format, exposed through an HTTP endpoint.
- **Prometheus:** Acts as the core engine of the monitoring stack. It periodically scrapes the endpoints exposed by cAdvisor and the various JMX Exporters, then stores the collected metrics in a time-series database optimized for query performance and alerting.
- **Grafana:** Serves as the visualization layer. It connects to Prometheus as a data source and enables the creation of rich, interactive dashboards. These dashboards allow for a fine-grained, visual analysis of cluster performance, Spark applications, and individual container metrics.

This architecture is not only fully functional but also implements several recognized best practices within the Spark ecosystem.

7.1.4 Analysis of applied best practices

Executor Sizing and Resource Allocation

Proper sizing of Spark executors is one of the most critical factors in optimizing job performance. Incorrect sizing can lead to underutilization of resources, memory errors, or excessive Garbage Collector (GC) overhead.



- **Number of Cores per Executor:** The decision to configure each worker with `SPARK_WORKER_CORES=3` is well-founded. A value in the range of 3 to 5 cores per executor is often an optimal compromise. Assigning too many cores (>5) can lead to performance degradation due to two distinct factors: limited HDFS bandwidth and excessive Garbage Collector (GC) activity. Conversely, assigning too few cores results in inefficient resource utilization. With three cores, this configuration promotes good task parallelism while keeping overhead under control.
- **Memory Allocation:** The configuration `SPARK_WORKER_MEMORY=10G` allocates a substantial amount of memory to each worker. This memory is not a single monolithic block; it is divided into several regions managed by Spark: execution memory (for shuffles, joins, and sorts), storage memory (for caching), user memory (for user-defined objects and data structures), and overhead memory. The allocation of 10 GB represents a well-balanced compromise between a “fat executor” approach (large memory with longer GC cycles) and a “thin executor” approach (smaller memory with frequent disk spills).

Partitioning and Parallelism Management

The way data is partitioned directly impacts the level of parallelism and, consequently, query performance. The parameters defined in the `kernel.json` file for Jupyter sessions reflect a proactive partitioning strategy.

- **`spark.sql.shuffle.partitions=128` and `spark.default.parallelism=128`:** These parameters control the number of partitions created after a shuffle operation (such as a `groupBy` or `join`). This number defines the degree of parallelism for subsequent job stages. Setting this value to 128 is a reasonable starting point for a local cluster with 12 total CPU cores. It ensures that there are enough tasks to fully utilize available cores without causing excessive overhead from managing too many small partitions.
- **`spark.sql.files.maxPartitionBytes=64m`:** As described in “*How Partitioning Works*”, this parameter controls the maximum size of a partition when reading files from a data source. Setting this limit to 64 MB ensures that Spark does not create overly large initial partitions. Large partitions can cause out-of-memory errors on individual tasks before computation even begins. This value represents a best practice for balancing file size and the number of initial tasks.

Adaptive Query Execution (AQE)

Adaptive Query Execution (AQE), introduced in Spark 3.x, is one of the most significant performance optimizations in recent years. It allows Spark to dynamically adjust the query execution plan at runtime, using real-time statistics from completed stages—a major evolution beyond the static planning used in earlier versions.

The

parameters `spark.sql.adaptive.enabled=true` and `spark.sql.adaptive.coalescePartitions.enabled=true` in `kernel.json` enable this key feature. This configuration provides substantial performance benefits:

1. **Partition Coalescing:**
AQE can detect when partitions become too small after a filtering or shuffle operation. Using `coalescePartitions`, it automatically merges these into more reasonably sized partitions. This drastically reduces scheduling overhead and the management of thousands of small tasks and files—a common performance issue in Spark jobs.
2. **Data Skew Handling:**
Data skew occurs when one or more partitions are much larger than others, creating bottlenecks. AQE can detect these skewed partitions during the shuffle stage and dynamically split them into smaller sub-partitions. This technique ensures a more balanced workload distribution across executors and can significantly reduce the execution time of jobs affected by skew.

This proactive configuration demonstrates a deep understanding of Spark’s modern optimization mechanisms, aligning with advanced best practices for high-performance big data processing.

7.1.5 Conclusion on local docker cluster

In summary, the architecture of this Apache Spark cluster, deployed via Docker Compose, constitutes a complete, powerful, and remarkably well-configured local development ecosystem. Its strength lies not only in the deployment of the core Spark components but in the thoughtful integration of complementary tools that collectively support the entire data science project lifecycle.

The inclusion of services for interactive development (Jupyter), rigorous experiment tracking (MLflow), and real-time performance monitoring (Prometheus/Grafana) creates an integrated and efficient work environment.

Moreover, the cluster configuration reflects a meticulous application of Spark optimization best practices—particularly regarding executor sizing, proactive partition management, and adaptive query execution (AQE).

This combination of features and optimizations makes the platform an exemplary environment for prototyping, developing, and fine-tuning complex data processing and machine learning solutions.

7.2 LAMSADE High-Performance Cluster

7.2.1 Introduction

After the local approach, we are describing now, the complete integration of the Flight Delay Prediction application on the Hadoop/Spark cluster of Paris-Dauphine University (LAMSADE). The objective is to demonstrate how we adapted the locally developed Spark application to run efficiently on a shared cluster, covering all aspects of the DevOps cycle: development, integration, deployment, and usage.

The main interest in automating our deployment cycle (CI/CD) in this Data Science/Machine Learning context is manifold:

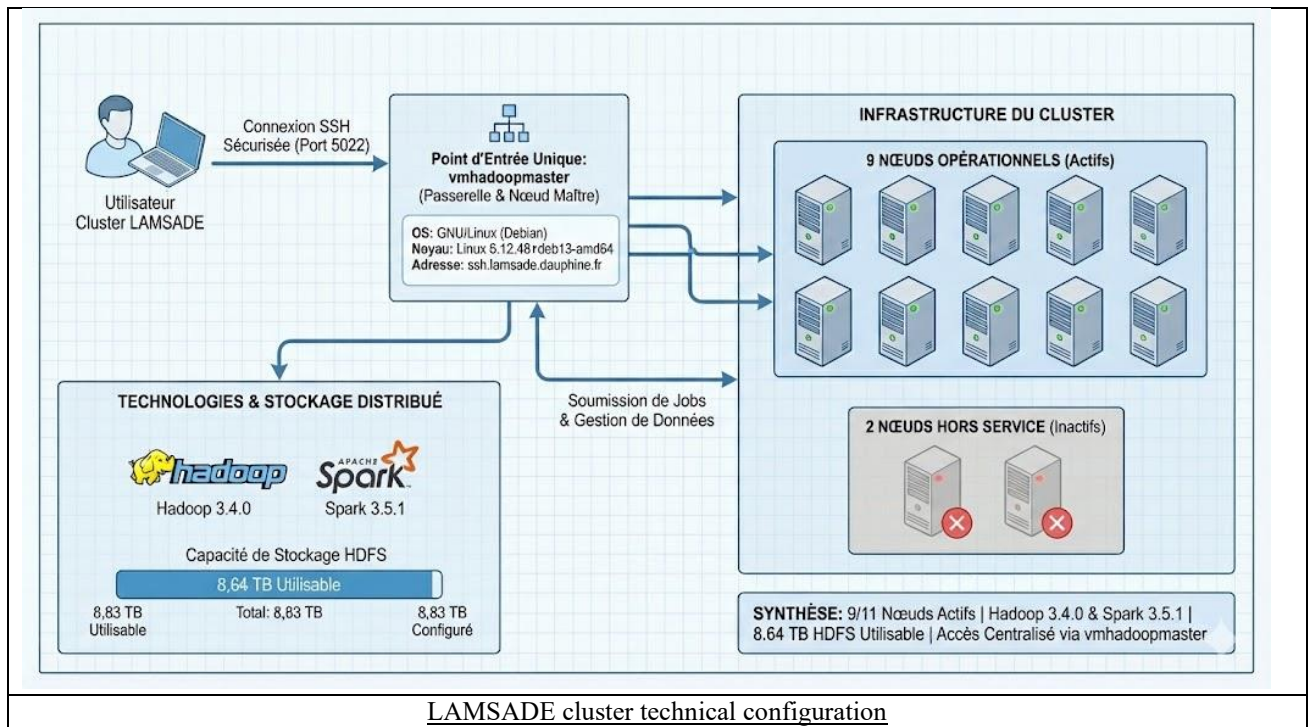
1. **Focus on Added Value (Analysis):** By eliminating manual and repetitive tasks of configuring and submitting Spark jobs, we can dedicate ourselves entirely to analyzing our results, improving our models, and designing our experiments, which is our core business.
2. **Scientific Rigor and Comparability:** Automation ensures that every ML experiment is executed in an environment and with a configuration that is strictly identical (reproducibility). This is fundamental for reliably comparing the performance of different models or hyperparameters we test.
3. **Reliability and Error Reduction:** Eliminating manual interventions drastically minimizes our risk of human errors (wrong dependency version, cluster configuration error, etc.), making our process more robust.
4. **Efficiency and Resource Savings:** Our approach allows us to optimize cluster usage (dynamic release or adjustment of resources after each execution), an essential point for our costly and shared environments.

In summary, for us, it's about moving from a craft and time-consuming mode to an industrial, reproducible, and optimized process, thereby accelerating our cycle of innovation and experimentation in Machine Learning.

7.2.2 LAMSADE Cluster Context

LAMSADE Infrastructure

The LAMSADE cluster constitutes a shared computing environment, whose technical configuration is detailed below:



Cluster Summary:

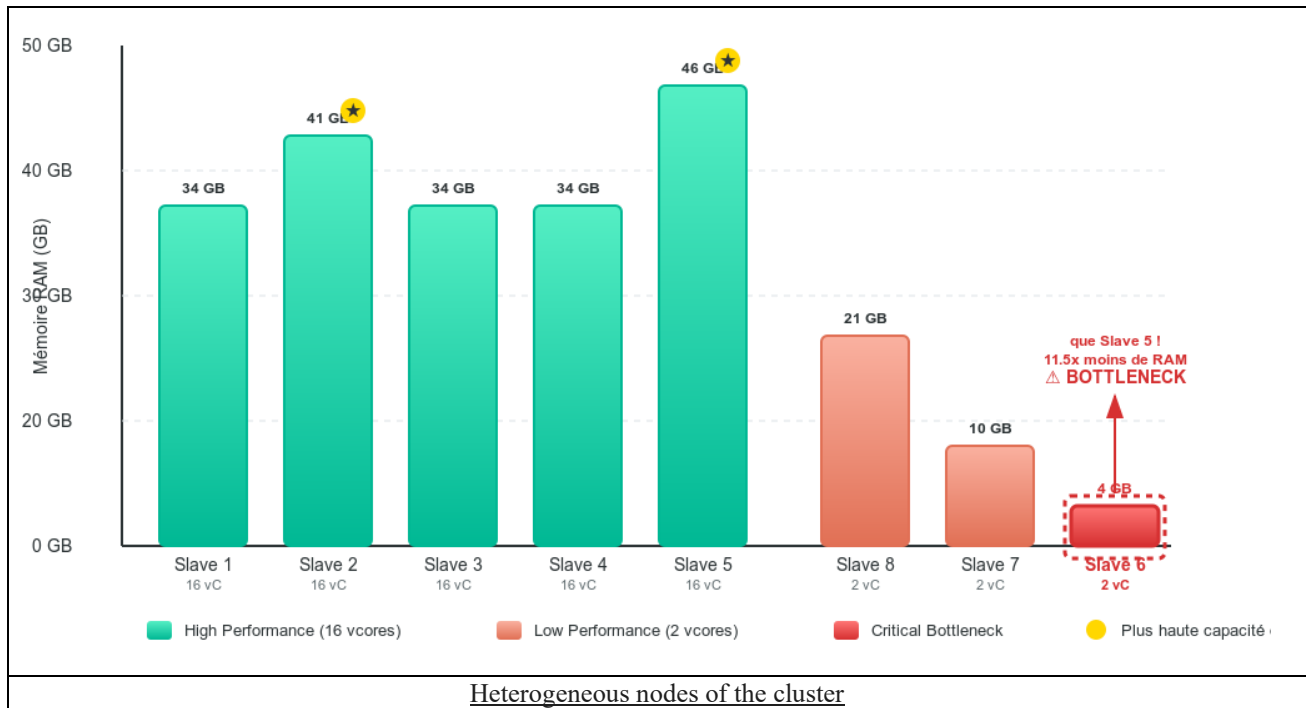
- **Operational Nodes:** Nine (9) nodes are currently active, out of a total of eleven (11) configured nodes (two nodes being temporarily out of service).
- **Technologies Implemented:** The cluster is based on the following versions of data processing frameworks:
 - Hadoop 3.4.0
 - Spark 3.5.1
- **HDFS Storage Capacity:** Usable storage space amounts to 8.64 Terabytes (TB), out of a total configured capacity of 8.83 TB.

Access Methods and Architecture: Access to the LAMSADE cluster is centralized and secured, being exclusively established via the **SSH** protocol.

- **Single Entry Point:** The initial connection must be directed to the network address `ssh.lamsade.dauphine.fr` using port 5022.
- **Gateway and Master Node:** All computing and storage resources are accessible via the `vmhadoopmaster` machine.
 - This machine simultaneously performs the functions of master node (in client mode deployment) and access gateway.
 - It operates under a GNU/Linux operating system (Debian distribution), as evidenced by the kernel version: `Linux vmhadoopmaster 6.12.48+deb13-amd64`. This technological foundation is known to guarantee robustness and stability within distributed computing environments.

Usage for Researchers/Students: Once we have established the SSH session to `vmhadoopmaster`, we have the ability to interact with cluster services, submit our jobs (via available frameworks such as Hadoop and Spark), and manage our data, while benefiting from the security and authentication framework offered by the SSH connection.

Available resources: Inspection reveals the following overall characteristics for the cluster. It is composed of heterogeneous nodes, detailed below:



Heterogeneity Management and "Least Common Denominator"

Spark Task Optimization: Neutralizing Slow Nodes - To ensure the stability and speed of our data processing (Spark tasks), we had to manage a problem: the overall performance of the cluster was limited by its weakest element (the "Least Common Denominator"), in this case, servers like vmhadoopslave6.

The Problem: The Weak Link - This server is a major bottleneck because it has very few resources: 2 cores and less than 4 GB of random access memory (RAM).

Slow Server	RAM (GB)	vCores
vmhadoopslave6 (example)	< 4	2

The Technical Dilemma : If we included this slow server in the general execution pool, we would be forced to significantly reduce the amount of resources allocated to each worker process (executor) so that the system could uniformly schedule tasks on *all* nodes (about 2 GB of RAM and 1 vCore per executor).

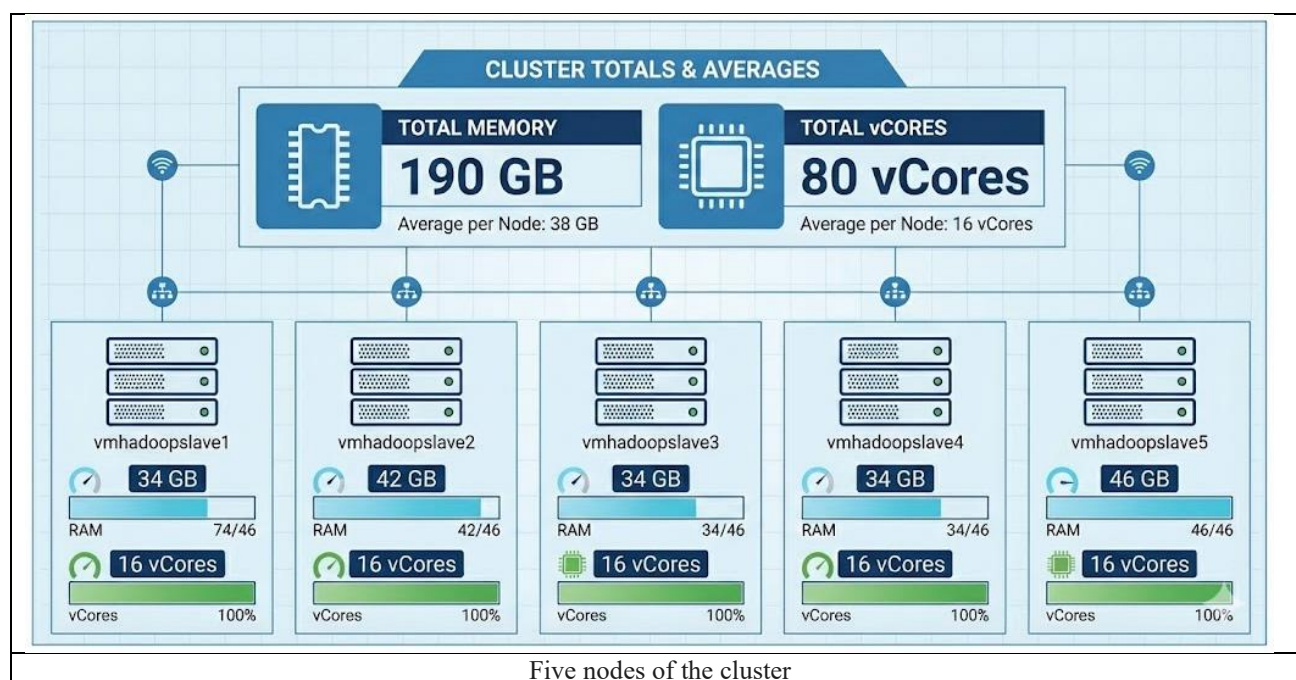
The Negative Consequence : This reduction would throttle the powerful servers. For example, on a 46 GB RAM node, about 90% of the memory would be unused, severely penalizing the general performance of the processing.

Our Solution - Prioritizing Speed : We chose to maximize performance by voluntarily excluding this pool of slow servers (which represents about 11% of the total processing capacity).

The strategy implemented involved increasing the memory dedicated to each worker process on the most powerful nodes, escalating it from 2 GB to 8 GB—a fourfold increase. While this means some servers are unused, it results in significantly faster and more stable execution for our memory-intensive Big Data application.

Conclusion: The cluster consists of five heterogeneous nodes. To maximize computational efficiency and optimize resource allocation, high-intensity tasks are strategically processed on the five most performant nodes.

This targeted approach leverages the full capacity of the highest-specification nodes. Collectively, our five-node cluster (illustrated below) currently offers a Total Memory of 190 GB and 80 vCores.



Specific constraints

Here are the constraints of the environment:

- Limited and Shared Resources: The computing environment has restricted resources that are shared among users.
- Restricted Privileges: Users do not have administrative rights; therefore, operations must utilize user-specific paths.
- Remote Access: Connection to the environment is established via an SSH tunnel.
- Storage Limitations: HDFS has user-specific quotas, limiting available storage.

7.2.3 System architecture

The LAMSADE laboratory manages a strategic, on-premise high-performance computing (HPC) cluster located on our campus. This infrastructure, overseen by laboratory teams, guarantees us privileged access to intensive computing resources.

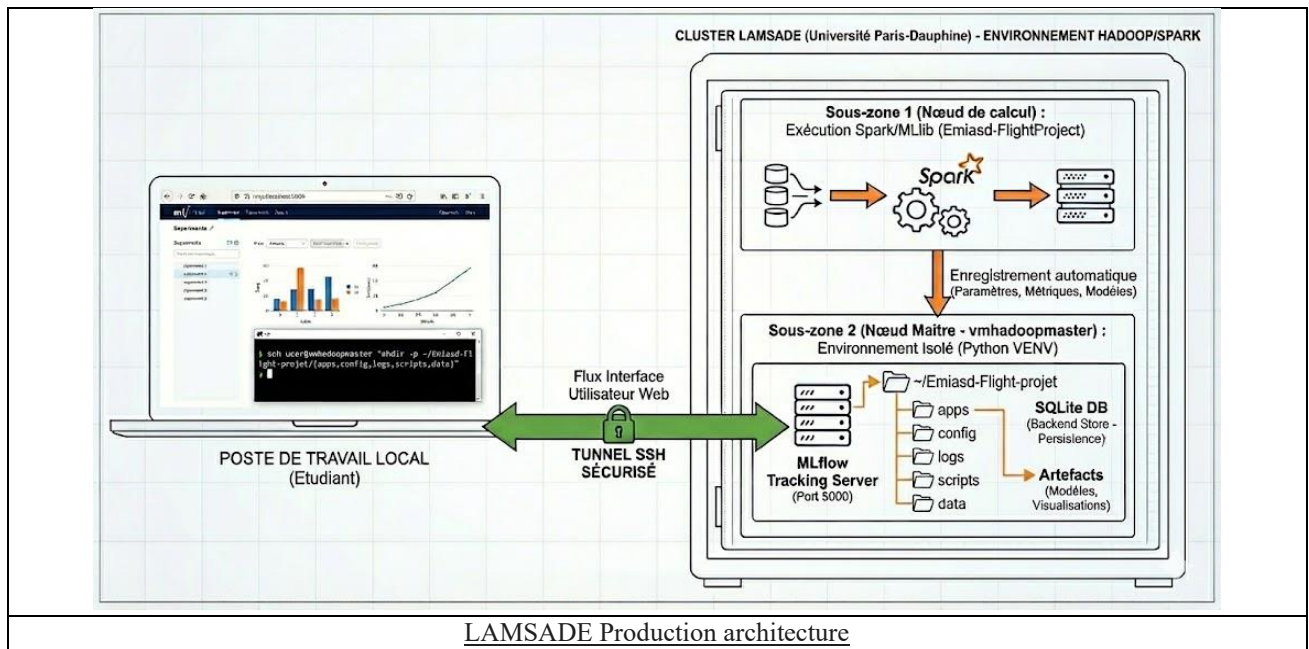
The major advantages of this internal hosting model are:

1. Total Control: Complete mastery for performance optimization adapted to specific needs.
2. Security/Confidentiality: Sensitive data remains secured within the environment.
3. Low Latency: Very fast internal network connection, crucial for large volumes of data.
4. Adaptation: Ability to evolve the cluster (hardware, storage) based on projects and funding.

This cluster provides LAMSADE with a robust, secure, and customized computing platform. It is a critical component for supporting scientific innovation within the laboratory.

Production architecture

The LAMSADE production architecture is structured as follows:



The "Emiasd-FlightProject" solution operates on the LAMSADE Hadoop/Spark cluster. This technical architecture diagram illustrates the project's operational environment.

In summary: The remote cluster is accessed securely from a local workstation using an SSH tunnel. This setup facilitates the execution of distributed calculations (Spark/MLlib) on the cluster. The results, including metrics and models, are automatically logged to an MLflow server located on the master node, enabling us to view them through a web interface on their workstation.

Environment preparation on vmhadoopmaster

Unlike platforms such as GCP, Lamsade lacks a native storage service. Consequently, a separate solution must be configured to support the architecture, handling elements like JAR files, data, and logs.

- **Remote Workspace Tree**

The primary working directory for the project is located at `~/Emiasd-Flight-projet` within the user's path on our workspace. This directory is accessible from the master node (`vmhadoopmaster`) and is designed to clearly separate the various application components.

Directory	Objective	Main Content
~/Emiasd-Flight-projet/apps/	Executable Storage	Compiled application JAR, MLflow JARs (mlflow-client, mlflow-spark).
~/Emiasd-Flight-projet/config/	Configuration Files	YAML configuration file (prodlamsade-config.yml) for Spark execution.
~/Emiasd-Flight-projet/data/	Local Raw and Intermediate Data	Temporary data files before HDFS upload, upload/cleanup scripts.
~/Emiasd-Flight-projet/logs/	Execution Logs	Detailed logs of each Spark <i>run</i> , symbolic links (latest.log).

~/Emiasd-Flight-projet/scripts/	Automation Scripts	Bash scripts, utility Python scripts.
~/Emiasd-Flight-projet/.venv/	Isolated Python Environment	Virtual environment containing all necessary Python dependencies (MLflow, PySpark, scikit-learn).
<u>Remote workspace tree</u>		

- **Isolated Python Environment Preparation (venv)**

Given the shared environment and the lack of administrator rights, using a Python virtual environment (venv) is essential to manage dependencies in isolation.

The installation of the production architecture on the LAMSADE cluster focuses on creating a structured working tree and setting up an isolated Python environment, ensuring reproducibility and project organization. The following steps detail the installation of the necessary environment, performed by SSH connection on the vmhadoopmaster virtual machine:

1. **Creation of the virtual environment (venv):** Execute the command to create the virtual environment.
2. **Activation of the virtual environment:** Activate the created virtual environment to isolate project dependencies.
3. **Installation of dependencies (libraries):** Once the environment is activated, install the required libraries for executing Spark tasks (PySpark), tracking experiments (MLflow), as well as for data analysis and visualization.

Key Note: The activation of this venv, performed after its installation, was maintained throughout the project.

- **HDFS Storage Paths**

In addition to the local working tree, a dedicated storage space is necessary for massive data on the Hadoop Distributed File System (HDFS). The required HDFS paths are:

- **Raw Data:** /students/p6emiasd2025/\$USER/data/
Downloading data to these HDFS locations is done via option 5 of the LAMSADE deployment scripts (Production) and should only be done once.
- **Results:** /students/p6emiasd2025/\$USER/output/
The output directory contains the following subdirectories:

Directory	Description	Typical Content
Experience-lamsade-1h	MLflow/Tracking: Data and artifacts of a <i>run</i>	Models (RandomForest), metrics (F1, AUC), PNG graphs.
Common	Intermediate/Shared Outputs: Transformed data, pre-calculated <i>features</i> , reusable aggregated results.	<i>Feature-engineered</i> datasets, intermediate tables.
spark-checkpoints	Spark Checkpointing: Ensures fault tolerance and optimizes complex calculations (ML, DAG), allowing recovery after failure.	Directories/metadata generated by SparkContext.setCheckpointDir().
<u>HDFS storage path</u>		

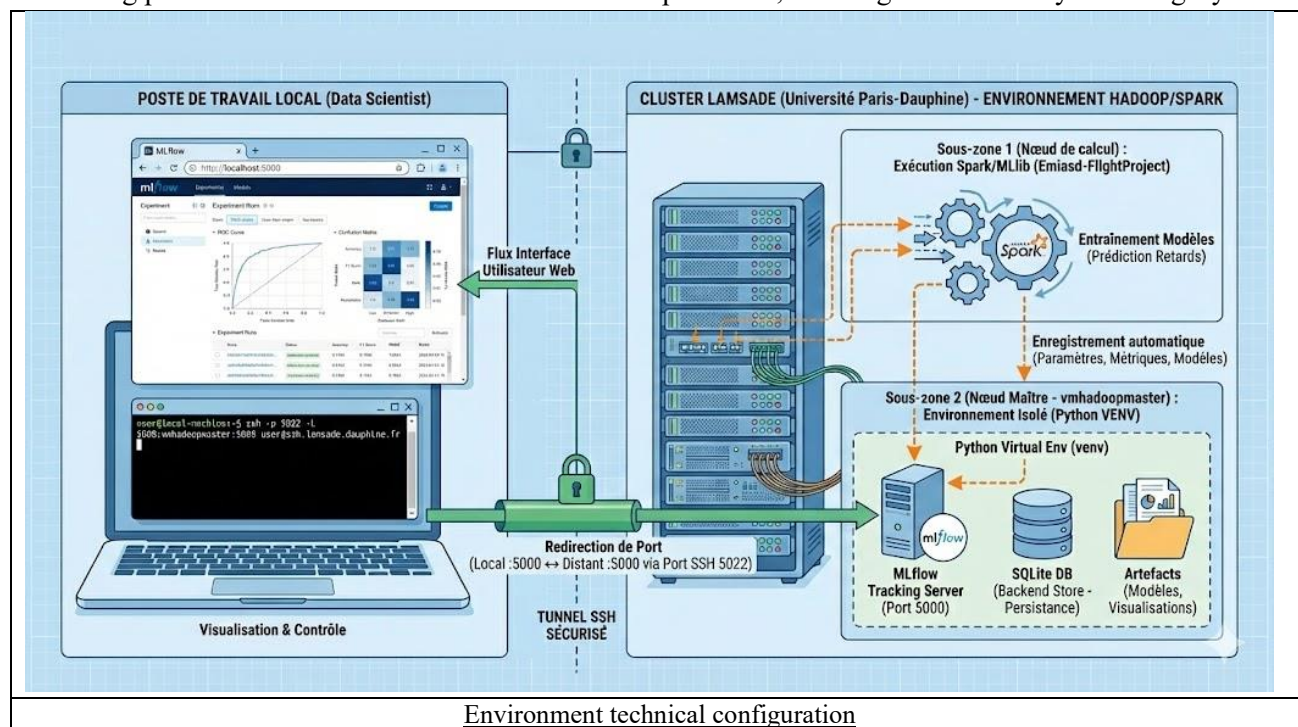
- **Setting up MLflow**

The Emiasd-FlightProject (flight delay prediction) relies critically on integrating MLflow with the LAMSADE cluster. This integration is vital for ensuring the traceability, reproducibility, and strict monitoring of ML experiments, including the logging of hyperparameters, metrics, models, and artifacts.

A prerequisite for this deployment is obtaining the necessary JAR files: **mlflow-client-3.4.0.jar** and **mlflow-spark_2.13-3.4.0.jar**. These files are transmitted to the cluster using option 4 of the LAMSADE deployment scripts.

- **Architecture and Security**

It is installed in the isolated environment. Access to the MLflow interface is secured by an SSH tunnel, redirecting port 5000 of the LAMSADE server to local port 5000, ensuring confidentiality and integrity.



Technical Configuration

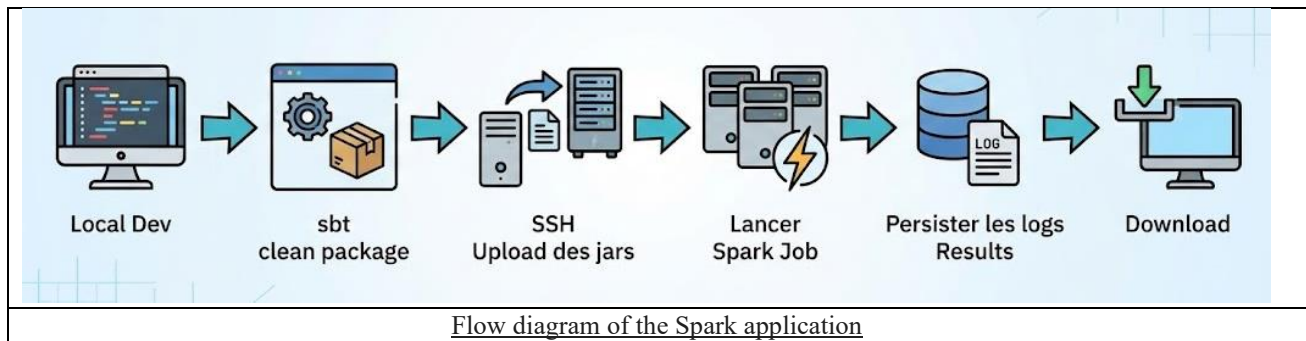
1. **Isolated Environment (venv) and Dependencies:** Installation of mlflow, visualization libraries (matplotlib, seaborn) to generate critical artifacts (ROC curves, confusion matrices), and project-specific libraries (pyspark, scikit-learn).
2. **Starting the MLflow Server:** The server is launched to persist data. The `--backend-store-uri sqlite:///mlflow1.db` option ensures the storage of metadata (runs, metrics, artifact paths). The server listens on port 5000 (`--host 0.0.0.0`) and automatically records Spark/MLlib experiments.
3. **SSH Tunnel for Secure Remote Access:** A *Port Forwarding* mechanism via SSH (`-L 5000:vmhadoopmaster:5000`) is used with private key authentication (`-i /hbalamou.key`) to access the interface without public exposure.

Exploitation

The MLflow interface is accessible locally via **http://localhost:5000**. It allows viewing the complete history of experiments, examining the details of the *runs* (e.g., RandomForest, Gradient Boosting), comparing model performances (F1-score, AUC), and downloading models and visualizations (artifacts)

7.2.3 Data Flow

The following image is a flow diagram illustrating the key steps in the life cycle of our Spark application, from development to result retrieval. Simplified process description:



1. **Local Dev:** Initial application development on the local machine.
2. **sbt clean package:** Compilation and packaging of the application (JAR creation) via sbt.
3. **SSH Upload of jars:** Secure deployment of JARs to the remote environment (server/cluster) via SSH.
4. **Launch Spark Job:** Execution of distributed data processing on the Spark cluster.
5. **Persist logs Results:** Saving of processing results and execution logs in a database.
6. **Download:** Retrieval of results or logs by the user on their local machine.

This workflow summarizes the typical cycle of development, deployment, and execution of Big Data applications with Spark.

7.2.4 Code and scripts development

The **Scala code structure** is detailed in the Github repository:

<https://github.com/MalikChettihIA/Emiasd-FlightProject>

The **YAML file** is the central element of the configuration for the flight delay prediction application. It brings together all the parameters necessary for the execution of the machine learning pipeline in the production environment, in this case, the LAMSADE cluster.

Key Principle: This YAML file allows modifying experiments without needing to touch the code; a simple update of the file is sufficient.

Deployment scripts have two levels of automation developed:

- Bash Scripts: `lamsade_fulldeployment.sh` (interactive menu)
- GitHub Actions: Automated CI/CD

7.2.5 Automated deployment

Interactive Bash Script (`Lamsade_fulldeployment.sh`)

Executing the script opens a complete interactive menu, offering the following seven options:

1. **Full deployment:** Compile + deploy + run
2. **Compile only:** SBT clean package
3. **Deploy JAR:** Upload to cluster
4. **Deploy config:** Upload configuration
5. **Upload data to HDFS:** Data to HDFS
6. **Run on cluster:** Spark execution
7. **Download logs:** Log retrieval

In practice, options 2, 3, and 4 are the most frequently used, generally in combination with the `submit` execution on the server.

GitHub Actions CI/CD

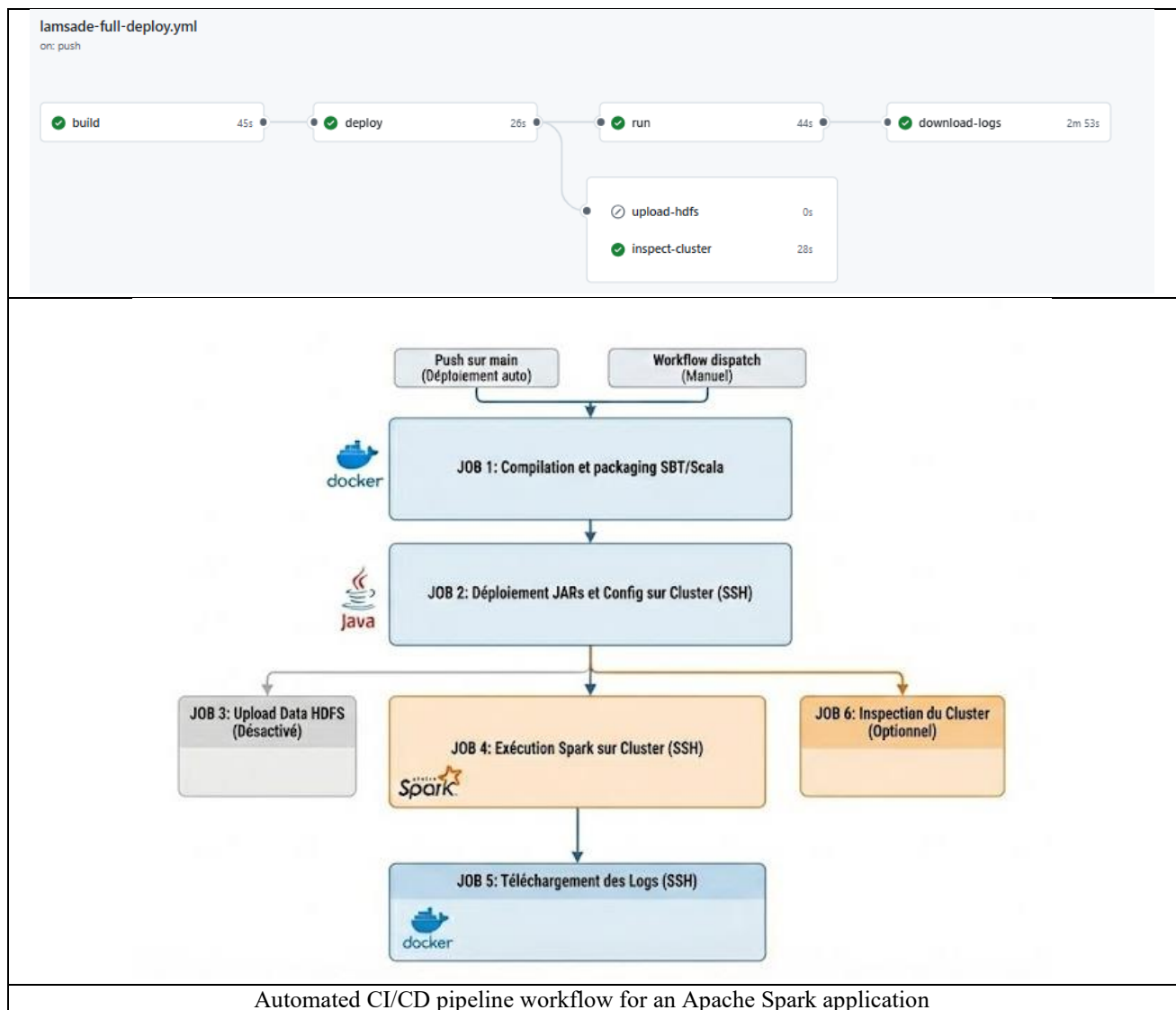
The integration of **GitHub Actions** has been adopted for several major benefits:

- **Simplified Integration and Deployment (CI/CD):** The solution is **natively integrated with GitHub**, which greatly simplifies the setup of the continuous integration and deployment (CI/CD) chain. This

eliminates the need for external configurations for triggers and artifacts.

- **Centralized and Versioned Workflow:** The workflow is defined in YAML files, versioned directly with the source code. This approach ensures that automation is always synchronized with the current state of the repository.
- **Reliability and Isolated Environment:** Task execution is ensured in a **dedicated, clean, and reliable environment** (the GitHub runners). This minimizes potential errors related to the user's working environment configuration.
- **Traceability and Detailed History:** Each execution is automatically **logged**, thus providing complete **traceability** (execution logs, status) far superior to using a simple script.

This diagram illustrates the workflow of an automated CI/CD pipeline for an Apache Spark application:



The pipeline is initiated automatically by a *Push on Main* (code update) or manually via *Workflow Dispatch*. It then proceeds sequentially:

1. **Build & Package (Job 1):** Compilation and packaging of the Scala/SBT code in a Docker container.
2. **Deployment (Job 2):** Transfer of artifacts (JARs and configuration files) to the remote cluster via SSH.
3. **Execution (Job 4):** Launch of the main Spark job on the cluster.
4. **Logs (Job 5):** Retrieval of execution logs once processing is complete.

7.2.6 Usage and monitoring

Launching the Deployment

The complete deployment of the application on Lamsade can be carried out in several ways, offering flexibility and different levels of automation and control.

1. Execution via Local Interactive Script

The most direct approach, often preferred for local development or testing, or when the operator wants direct control over the process, is the use of a dedicated automation script.

- **Execution command:** `./work/scripts/lamsade_fulldeployment.sh`

2. Automation via GitHub Actions (CI/CD)

For robust, reliable, and reproducible continuous integration and deployment (CI/CD), the use of a cloud-based or server-based automation platform is essential.

- **Technology:** GitHub Actions.
- **Trigger Method:** Via the web interface with `workflow_dispatch` (manual triggering) or automatically during a push to the configured branch, depending on the desired action.
- **Description:**
 - **Dedicated Workflow:** A specific workflow file is configured in the `.github/workflows/` directory. This workflow encapsulates the steps of the complete "Lamsade" deployment, similar to or more sophisticated than those of the local script.
 - **Manual Triggering (`workflow_dispatch`):** The use of `workflow_dispatch` allows a user with appropriate rights to manually trigger the execution of the workflow directly from the GitHub web interface (under the "Actions" tab).

Real-Time Monitoring

Real-time monitoring of a Spark application submitted to the LAMSADE cluster via `spark-submit` is performed using the YARN ResourceManager web interface (port 8088). YARN is essential for managing resource allocation (CPU, memory) and for tracking the status of jobs, containers, logs, and overall cluster metrics.

To access it securely from a local workstation, an SSH tunnel is necessary, mapping local port 8088 to `vmhadoopmaster`'s port 8088.

Access:

Once the tunnel is established, the interface is accessible via `http://localhost:8088`. It provides the status of Spark applications (e.g., flight prediction): list of jobs, resource usage, detailed container logs (debugging), and performance metrics. This monitoring complements the MLflow tool by offering a system view of the cluster.

- **YARN UI:** <http://vmhadoopmaster:8088>
- Spark UI ;
- **Real-time logs:** locally on the cluster via `tail -F ~/workspace/logs/latest.log`

Result retrieval

Method	Command/Instructions	Notes
Via Script	<code>./work/scripts/lamsade_fulldeployment.sh</code> (Option 7: Download logs)	Uses a local full deployment script.
Via Ansible	<code>ansible-playbook download-logs.yml</code>	Executes an Ansible playbook specifically for log downloading.
Via Direct SCP	<code>scp -P 5022 user@ssh.lamsade.dauphine.fr:~/workspace/logs/*.log ./logs/</code>	Direct secure copy, requiring the specified port and user credentials to retrieve all <code>.log</code>

		files to the local <code>./logs/</code> directory.
<u>Methods for Downloading Logs</u>		

Application Management

Application management is accessible via the command line. Below is the list of available commands.

- Monitoring Commands**

Command	Description
yarn top	Displays real-time resources (YARN equivalent of htop)
yarn node -list	Lists cluster nodes
yarn node -list -showDetails	Lists nodes with details (memory, vcores)
yarn node -list -all	Lists all nodes (including inactive ones)
yarn node -status <node_id>	Displays the detailed status of a specific node
<u>Monitoring commands</u>	

- Application Management**

Command	Description
yarn application -list	Lists currently running applications
yarn application -list -appStates RUNNING	Lists only RUNNING applications
yarn application -status <application_id>	Displays the status of an application
yarn application -kill <application_id>	Stops/kills an application
<u>Application Management</u>	

- Log Consultation**

Command	Description
yarn logs -applicationId <application_id>	Retrieves complete logs of an application
yarn logs -applicationId <app_id> > app_logs.txt	Saves logs to a file
<u>Log Consultation</u>	

- ** View shuffle metrics ****

• Command	Description
yarn logs -applicationId <application_id> > app_logs.txt	Retrieves complete application logs and saves them to a local file for analysis.
`grep -i "shuffle" app_logs.txt   head -30`	
grep -i "shuffle.*bytes shuffle.*records" app_logs.txt	Specifically searches for detailed <i>shuffle</i> metrics (number of bytes and records transferred), essential for network I/O optimization.

yarn logs command	Description
yarn logs -applicationId application_xxx grep -i "error exception"	Search for errors
yarn logs -applicationId application_xxx grep -i "OutOfMemory"	Check for OutOfMemory
yarn logs -applicationId application_xxx grep -i "failed"	View failed tasks

[View shuffle metrics](#)

• **** Advanced Log Consultation ****

Command	Description
yarn logs -applicationId application_1234567890123_0001 grep -i "shuffle"	Extract shuffle metrics
yarn logs -applicationId application_1234567890123_0001 grep -E "Shuffle Read Shuffle Write"	View shuffle read and write

[Advanced log consultation](#)

7.2.7 Spark Configurations for LAMSADE

Comparison of Spark Configuration architectures: Compact (thin) , Voluminous (Fat), Optimized

The efficiency, robustness, and optimal resource utilization of a Spark environment are closely dependent on rigorous task configuration. This analysis compares three distinct configuration approaches:

- Compact (Thin)
- Voluminous (Fat)
- Optimized

to evaluate their impact on overall performance. The main objective of this study is to demonstrate the relevance of choosing the **OPTIMIZED** configuration, identified as the most performing and stable, for the LAMSADE production environment. The Three Configuration Strategies

1. **Compact (Thin):** Favors maximum parallelism (multiplication of small *executors*), but leads to a

- degradation of network efficiency and Input/Output (I/O) operations.
2. **Voluminous (Fat):** Aims to reduce network *overhead*, but introduces critical issues related to *Garbage Collection* (GC) and system instability.
 3. **Optimized (Optimized):** Relies on industry standards and best practices (Cloudera/Databricks) to establish an ideal compromise between I/O efficiency, memory management, and network performance.

Detailed Analysis of Configurations on the LAMSADE Cluster

The experimentation relies on the five most performing nodes of the LAMSADE cluster. To ensure uniformity of results, the allocated resources are aligned with those of the most constrained node: **34 GB of RAM and 16 vCores** (i.e., 33 GB of memory and 15 vCores available after deducting operating system resources).

Here are the four key rules for cluster resource sizing and management, aimed at performance/stability optimization:

- **Reserve the essential (Guaranteed Stability):** Systematically allocate **1 GB of RAM and 1 CPU core** per node for the OS and YARN to prevent saturation and ensure stability.
- **Size uniformly (Global Allocation):** Adapt the size of executors (memory and cores) to the resources of the weakest node to guarantee container allocation on all nodes by YARN.
- **Optimize cores (Equilibrium Point):** Use **5 cores per executor** to maximize HDFS throughput while controlling I/O contention and Garbage Collection.
- **Control memory (Overshoot Prevention):** Ensure that the sum of *heap* and *overhead* of executors does not exceed the available RAM of the most limited node, even if a slight overshoot is sometimes tolerated.

1. Compact Configuration (Thin): Extreme Parallelism

Parameter	Value	Justification
Executors	35	7 executors per node x 5 nodes
Cores/exec	2	Minimum operational requirement
Memory/exec	5G	Derived from the 33 GB available per node (including 1 GB overhead)
Total YARN/exec	6G	5 GB (Memory) + 1 GB (Overhead)

Evaluation: Although parallelism reaches its maximum (70 simultaneous tasks), the observed instability is critical. The network overhead is prohibitive (with a potential for 1,225 *shuffle* connections), and the low number of cores per *executor* (2) induces low HDFS I/O throughput (100 MB/s). This approach is deemed unviable for data-intensive processing (machine learning).

2. Voluminous Configuration (Fat): Minimizing Network Overhead

Parameter	Value	Justification
Executors	5	1 executor per node x 5 nodes
Cores/exec	15	Use of all available cores per node

Memory/exec	28G	Derived from the 33 GB available per node (including high overhead of 5 GB)
Total YARN/exec	33G	28 GB (Memory) + 5 GB (Overhead)

Evaluation: Network overhead is minimized (only 25 *shuffle* connections), but the substantial size of the JVM (28 GB) causes insoluble *Garbage Collection* (GC) problems. GC pauses, lasting 2 to 10 seconds, cause *timeouts* and major instability. Moreover, the concentration of 15 cores on a single *executor* degrades HDFS I/O throughput. Conclusion: Rejected due to persistent instability related to GC.

3. Optimized Configuration (Optimized): The Balance (Recommended Practice)

This methodology exploits the "sweet spot" of 5 cores per *executor*, a configuration recognized for offering the best HDFS I/O throughput while ensuring fast and efficient GC.

Parameter	Value	Justification
Executors	15	3 <i>executors</i> per node x 5 nodes
Cores/exec	5	HDFS equilibrium point (350 MB/s) and efficient GC
Memory/exec	10G	Derived from the 33 GB available per node (including 1.5 GB overhead)
Total YARN/exec	11.5G	10 GB (Memory) + 1.5 GB (Overhead)

Evaluation: This configuration establishes an optimal balance.

- Optimal HDFS Throughput: The allocation of 5 cores per *executor* allows reaching 350 MB/s.
- Efficient GC: The 10 GB JVM ensures fast GC pauses (about 150 ms), guaranteeing system stability.
- Balanced Parallelism: Allows simultaneous execution of 75 tasks.

Conclusion on optimized: This is the retained configuration. Indeed, it maximizes I/O performance and memory stability, two essential factors for our data-intensive application.

Criterion	Compact (Thin)	Voluminous (Fat)	Optimized ★
Executors	35	5	15
Cores/exec	2	15	5
Memory/exec	5G	28G	10G
HDFS Throughput	⚠ 100 MB/s	✗ 200 MB/s	☑ 350 MB/s
GC Pauses	☑ 50 ms	✗ 2-10 s	☑ 150 ms
Shuffle Connections	✗ 1,225	☑ 25	☑ 225
Stability	⚠ Average	⚠ OOM/GC Risk	☑ Excellent
Overall Performance	✗ Insufficient	✗ Very Insufficient	☑ Optimal

Comparative Synthetic Table

Conclusion - The *Compact* and *Voluminous* configurations were quickly ruled out due to the major bottlenecks they generate (network saturation and low I/O for the Compact; GC instability for the Voluminous). The **Optimized strategy represents the most favorable pragmatic compromise, perfectly adapted to the heterogeneity constraints of the LAMSADE cluster. It guarantees maximum performance while maintaining critical stability, essential for massive data processing and machine learning operations.**

Parameter Justification

Driver Resources

- **32G RAM, 8 cores:** Sufficient for coordination without overloading
- **maxResultSize=8g:** Handling voluminous results

Executor Resources

- **8G RAM/executor:** Adapted to 34-46 GB nodes
- **6 executors:** 3 per node on the 5 main ones (avoids weak slave6-8)
- **No cores specified:** Leaves YARN to decide (generally 1-2 per executor)

Spark Optimizations

- **400 partitions:** Adapted to available parallelism
- **Kryo serializer:** Optimal performance
- **Off-heap 2G:** Native memory for shuffle
- **Extended timeouts:** Stability on shared network

Aspect	Configuration Used	Optimal Theoretical	Justification
Executors	6	15 (5×3)	Avoids weak nodes (slave6-8)
Executor Memory	8G	10G	Conservative for stability
Driver Memory	32G	16G	Surplus for security
Partitions	400	300	Adapted to resources

Comparison with Theoretical Configurations

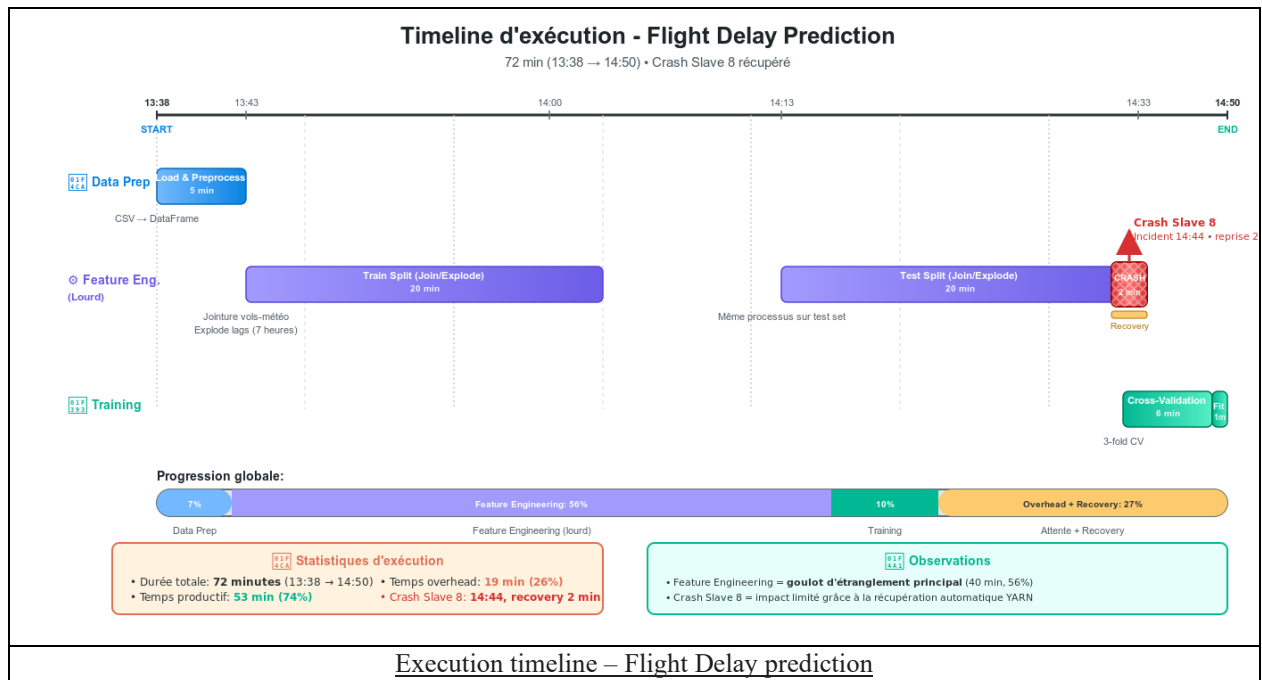
7.2.8 Results and performance

Performance metrics

- **Execution time:** 2-3 hours for complete pipeline
- **Cluster utilization:** 6 executors on 9 available nodes
- **Stability:** Successful executions without memory crash
- **Network:** Correct management of SSH latency

ML Metrics

- **Accuracy:** ~0.85 on cross-validation
- **Features:** 44 after PCA and feature engineering
- **Model:** Random Forest with optimized hyperparameters
- **Data:** 142,000+ flights processed



7.2.9 Conclusion and Perspectives

Big Data Production Deployment on LAMSADE Cluster is a DevOps Success

The integration of the project on the LAMSADE cluster has been a key factor, transforming the initiative into a production-ready Big Data application. The DevOps approach, characterized by multiple levels of automation (Bash, Ansible, GitHub Actions), ensures robust and fully reproducible deployment, even in a shared university environment.

Major Technical Achievements

- **Complete Automation:** The continuous integration and deployment (CI/CD) chain is fully automated, from the initial *build* to the final deployment.
- **Resilience and Robustness:** The system integrates error and *timeout* management to ensure stability.
- **Efficient Supervision:** Implementation of a *monitoring* system with real-time logs and retrieval functionalities.
- **Resource Optimization:** The configuration has been specifically adjusted to maximize cluster resource utilization.

Roadmap and future evolutions

- **Multi-cluster Extension:** Preparation for supporting heterogeneous environments (e.g., GCP Dataproc in addition to LAMSADE).
- **Container Orchestration:** Integration of Kubernetes for more advanced infrastructure management.
- **Extended CI/CD Pipeline:** Addition of automated tests directly on the cluster.
- **Advanced Monitoring:** Deployment of metrics with Prometheus and Grafana tools.

7.3 Google Cloud DATAPROC

7.3.1 Introduction

We are now describing the comprehensive integration of Google Cloud Dataproc for our project. Our goal was to transform the original local Spark application into an automated, scalable, and cloud-native solution. The

following content will demonstrate this transformation, covering all facets of the DevOps lifecycle: development, integration, deployment, and operation.

7.3.2 Context of our Project

Business objective - The project aims to predict flight delays by analyzing flight and weather data. The application processes:

- **142,000+ flights** with temporal data
- **44 features** after feature engineering
- **Datasets of several GB** after transformation

Technical Architecture and Key Components for Data Processing and Machine Learning

DataProc : Key Components and Services for Data Processing

The **Google Cloud Dataproc** solution is a managed computing platform designed for distributed data processing and machine learning (ML) experiments. It relies on a set of fundamental technological components.

- **Apache Spark 3.5.3:** This is the distributed data processing engine par excellence. Spark is renowned for its speed (thanks to in-memory execution) and versatility, allowing for batch processing, real-time *streaming*, as well as interactive queries. Version 3.5.3 was specifically selected to target a precise machine, thus ensuring perfect alignment with the one used by LAMSADE.
- **Scala 2.12:** This programming language is often favored for developing Spark applications. Scala combines object-oriented and functional paradigms, offering great conciseness and performance for writing distributed data processing code. The choice of this version ensures uniformity with the LAMSADE environment. Scala, which combines object-oriented and functional programming paradigms, was selected for its conciseness and performance in distributed data processing

ML Monitoring and Management Tool

MLflow: This tool is of paramount importance for monitoring and managing the lifecycle of machine learning experiments. MLflow notably offers the following functionalities:

- **Tracking:** Allows recording of parameters, metrics, and artifacts associated with models.
- **Projects:** Ensures the encapsulation of ML code in a format guaranteeing reproducibility.
- **Models:** Facilitates the management and deployment of machine learning models.
- **Model Registry:** This central element centralizes the referencing, version management, and transition of models to a production environment.

Platform and Storage Service

- **Google Cloud Dataproc:** It is the managed computing platform solution provided by Google Cloud. Dataproc simplifies the deployment, management, and scaling of Apache Spark, Hadoop, Flink, and other Big Data tools clusters. It allows users to focus on data analysis rather than infrastructure administration.
- **Google Cloud Storage (GCS):** This service acts as the storage service for data and artifacts. GCS is a highly available, durable, and scalable object storage service, ideal for:
 - Storing raw and processed datasets used by Spark.
 - Backing up models and artifacts generated by MLflow.
 - Serving as a high-performance distributed file system decoupled from Dataproc clusters.

ML Pipeline

1. **Data Pipeline:** Data ingestion and cleaning
2. **Feature Extraction:** Feature generation
3. **Training:** Random Forest model training
4. **Evaluation:** Performance metrics and logging

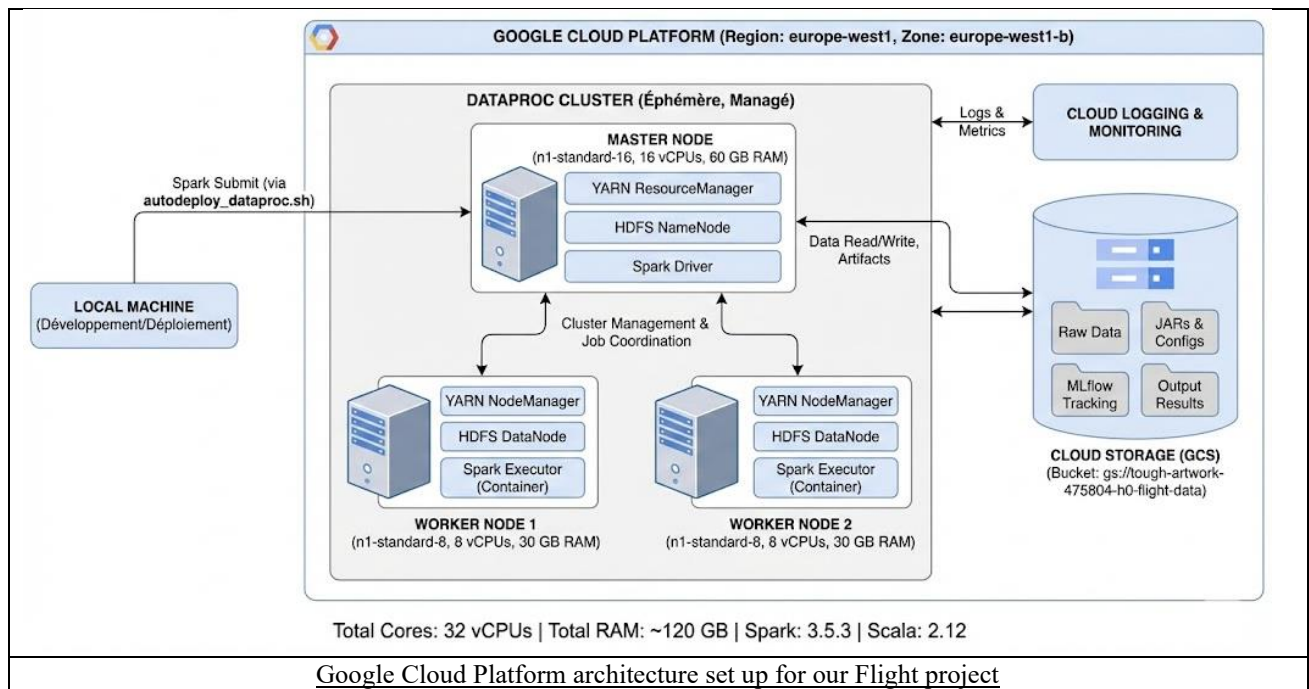
7.3.3 General System Architecture

Local Architecture (development)

Cloud Architecture (Production)

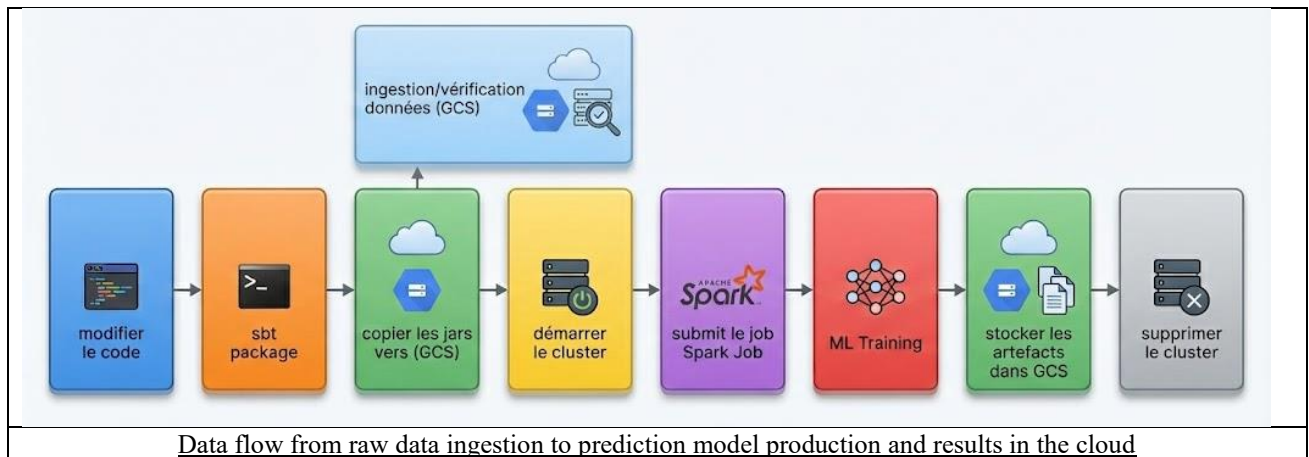
The **Google Cloud Platform (GCP)** architecture set up for the *flight* project is robust, focused on availability, *scalability*, and performance. It is structured around four pillars:

- **Cloud Storage (GCS):** Used to store data, JARs, configurations, and outputs.
- **Dataprocc Workflow:** A managed ephemeral cluster composed of one *Master Node* (n1-standard-16) and two *Worker Nodes* (n1-standard-8 × 2).
- **MLflow Tracking:** Whose tracking data is stored in GCS.
- **Cloud Monitoring & Cloud Logging:** Performance monitoring, log centralization, and operational alerts.



Data Flow

The chart below illustrates the flow.



The data flow describes the sequence of operations from the ingestion of raw data to the production of the prediction model and its results in the cloud.

1. **Data Source:** Raw datasets (flights, weather) are stored in a dedicated Google Cloud Storage (GCS) bucket.
2. **Processing (Dataproc):** The ephemeral Dataproc cluster is created and executes the Spark/Scala application. It reads the data from GCS, performs cleaning and *feature engineering*.
3. **ML Training:** The *Training* step uses Spark MLLib's distributed Random Forest model on the processed data.
4. **Tracking and Artifacts (MLflow):** Parameters, metrics, and the serialized model are recorded by MLflow and stored in a dedicated subdirectory of GCS.
5. **Results:** Predictions and final model artifacts are written to an output GCS bucket, ready for consumption or deployment.

This architecture was designed so that the data is not subject to any persistent storage on the Dataproc nodes. The cluster is assigned an exclusively computational role, which consequently optimizes both performance and cost management.

7.3.4 Code and Scripts development

In this project, we opted for complete process automation via the command line (with `gcloud` and `gsutil`), prioritizing this approach over using the GCP web interface. This strategic decision presents several major advantages:

- **Guaranteed Reproducibility:** The use of a single execution script ensures the uniformity of results, regardless of the deployment environment.
- **Optimal Traceability:** Direct integration of commands into the source code facilitates monitoring (via versioning) and guarantees flawless traceability.
- **Efficiency Gain:** Cluster management is fast and automated, eliminating time-consuming manual operations.
- **Ideal for CI/CD:** This method is perfectly suited for deploying the infrastructure in our automated continuous integration pipelines.

This approach requires a preliminary environment preparation step, both for development and deployment.

Environment Prepared for DEvelopment and Deployment on Dataproc

For the deployment of the application on Dataproc, we carried out the following preparation steps on our workstation:

- **Setting up the Execution and Compilation Environment:**
 - We needed and installed the Java Development Kit (JDK), version 8 or higher, for running JVM applications.
 - We configured the specific version of Scala required by the project (verified in build.sbt).
 - We installed and configured SBT (Simple Build Tool) to manage Scala dependencies and compile the project efficiently.
- **Configuration of Project Management and Version Tools:**
 - We set up Git (version control system) to clone the source code repository and ensure version management.
- **Configuration of Google Cloud Access:**
 - We needed and installed and configured the Google Cloud SDK (gcloud CLI) to interact with Google Cloud Platform (GCP).
 - We secured access by authenticating via the `gcloud auth login` command.
 - We defined the target GCP project as the default project with the command: `gcloud config set project tough-artwork-475804-h0`
- **Choice and Configuration of the Development Environment:**
 - We selected and configured a Code Editor / IDE (such as IntelliJ IDEA or VS Code) supporting Scala and SBT to facilitate code writing and testing.

Scala code structure

The code architecture is detailed either on the GitHub repository:

<https://github.com/MalikChettihIA/Emiasd-FlightProject>

YAML Configuration

This YAML file is the masterpiece of configuration for the flight delay prediction application. It contains all the parameters required for executing the machine learning pipeline in the production environment (here GCP).

Fundamental Philosophy: The experiment can be modified without altering the code; simply update this YAML file.

Build and development scripts

- `autodeploy_dataproc.sh`: Main deployment script.
- `configure_dataproc.sh`: Script for interactive configuration.

7.3.5 Integration with Google Cloud Dataproc

Choice of Dataproc

The choice of Dataproc is justified by several major assets:

- **Managed Service:** Eliminates the burden of infrastructure management.
- **Cost-Effectiveness:** Use of ephemeral clusters billed on a pay-per-use basis.
- **Optimal Integration:** Native compatibility with key tools like Spark, GCS, and MLflow.
- **Guaranteed Reproducibility:** Ensures that any evaluator can recreate the exact environment to validate the results.

GCP Project configuration

This section aims to provide a comprehensive overview of the crucial parameters configuring the DataProc environment used for data processing. This information is essential for any maintenance operation, workload deployment, or troubleshooting.

Project Identifiers and Location- The DataProc environment is hosted within a specifically dedicated Google Cloud Platform (GCP) project.

Parameter	Label (EN)	Value	Detailed Description
ID Project	Project ID	<code>tough-artwork-475804-h0</code>	Unique and global identifier for the GCP project. All interactions with APIs or resources must reference this ID.
Region	Region	<code>eu-west-1</code>	The main region where DataProc computing resources are deployed.
Zone	Zone	<code>eu-west-1-b</code>	The specific availability zone within the region.

Data storage Infrastructure (Google Cloud Storage – GCS) - The DataProc cluster uses a GCS *bucket* as the persistent storage layer for input data, intermediate results, job logs, and final results.

Parameter	GCS Reference	Value	Role in the Architecture
GCS Location	GCS Bucket URI	<code>gs://tough-artwork-475804-h0-flight-data</code>	This <i>bucket</i> is the main repository for flight data (raw and processed). It is virtually mounted on the DataProc cluster for low-latency access (Spark/Hadoop) and serves as the default location for cluster logs/events history.

Precise knowledge of these parameters is vital for:

- **Job Submission:** Ensuring that `gcloud dataproc jobs submit` commands correctly specify the `--region` (`eu-west-1`) and the `--project` (`tough-artwork-475804-h0`).
- **Access Management:** Verifying that the service accounts associated with the cluster have the necessary IAM permissions to read/write to the *bucket* `gs://tough-artwork-475804-h0-flight-data`.
- **Monitoring:** Configuring dashboards and alerts in Cloud Monitoring by targeting the resources in this region and project.

MLflow Integration

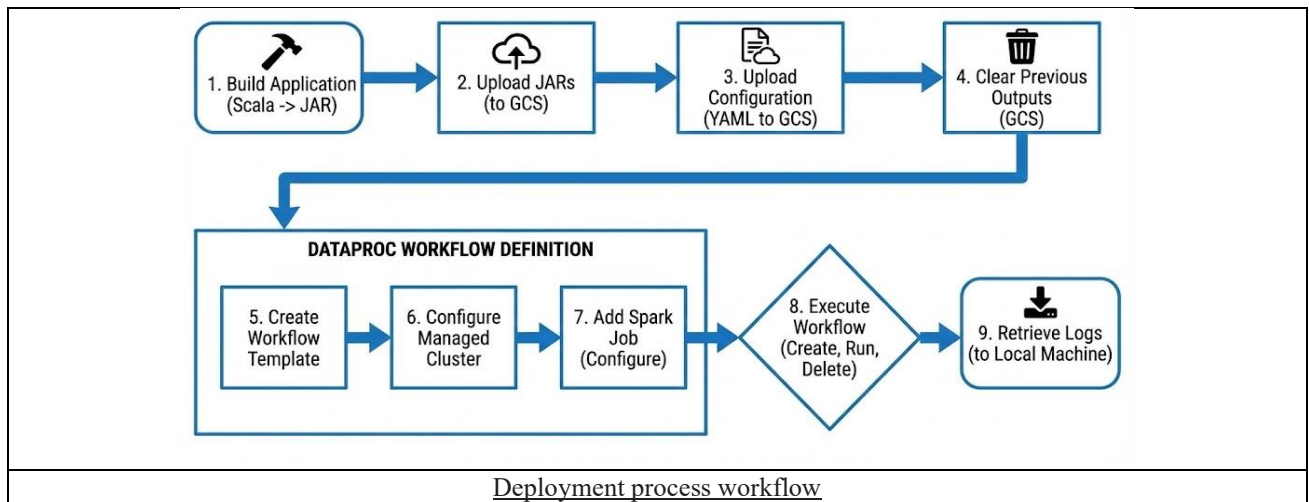
MLflow tracking is configured to use GCS:

Dependency Management

- MLflow JARs uploaded to GCS
- Configuration via `--jars` and `--files` in `spark-submit`

7.3.6 Automated deployment

The diagram below illustrates the integration of Google Cloud Platform services and the data flow for executing the flight delay prediction pipeline via Dataproc.

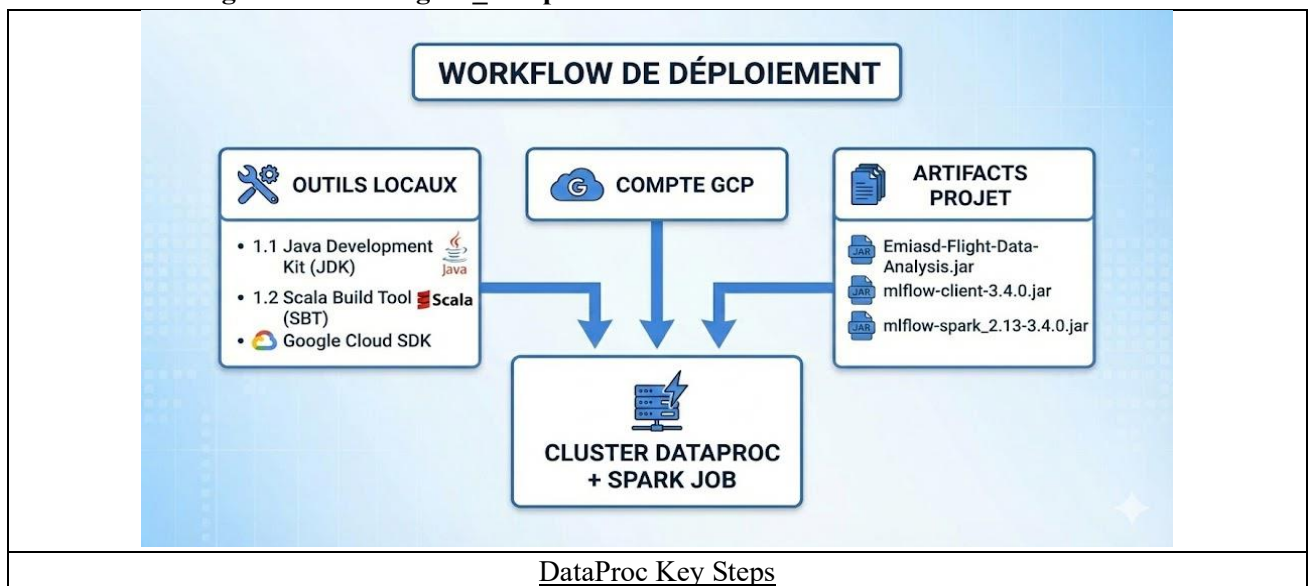


Main Script `autodeploy_dataproc.sh`

The automated script:

1. **Build:** `sbt clean package`
2. **Upload:** JARs and configs to GCS
3. **Workflow Creation:** Idempotent Dataproc Template
4. **Cluster Configuration:** Managed ephemeral cluster
5. **Job Submission:** Spark job with parameters
6. **Monitoring:** Extraction of the operation ID

Interactive Configuration : `configure_dataproc.sh`



- **GCP Authentication:** Connection to Google Cloud Platform.
- **Configuration:** Setting up the project and the storage *bucket*.
- **Resources:** Parameterizing the *cluster*.

Environment variables

The `.env.dataproc` file contains:

`GCP_PROJECT_ID=tough-artwork-475804-h0`

```
GCS_BUCKET=gs://tough-artwork-475804-h0-flight-data
WORKFLOW_NAME=flight-delay-workflow
# ... other parameters
```

7.3.7 Usage and Monitoring

Launching the deployment

- 1. Initial Configuration:** Execute the configuration script:
`./configure_dataproc.sh`
- 2. Automated Deployment: With default tasks**
`./autodeploy_dataproc.sh`
- 3. With specific tasks (e.g., *data-pipeline* and *feature-extraction*)**
`./autodeploy_dataproc.sh "data-pipeline,feature-extraction"`

Real-Time Monitoring

- **GCP Console:** <https://console.cloud.google.com/dataproc>
- **Workflow Progress:** Operation tracking via operation ID
- **Logs:** Via Cloud Logging or YARN UI

DataProc Monitoring Interfaces via SSH Tunnel

Access and Supervision of the DataProc Cluster

To ensure effective supervision of our cluster, we have set up SSH tunnels to directly access the following monitoring interfaces:

Interface	URL (via SSH tunnel)	Main Role
YARN	<code>`http://flight-temp-cluster-utcztjemtipca-m:8088`</code>	Monitoring of running jobs.
Spark History	<code>`http://flight-temp-cluster-utcztjemtipca-m:18080`</code>	Detailed history of completed Spark applications.
HDFS NameNode	<code>`http://flight-temp-cluster-utcztjemtipca-m:9870`</code>	Display of the distributed file system status.

YARN ResourceManager

- **Function:** This is the core of cluster resource management, acting as the main scheduler for YARN (Yet Another Resource Negotiator).
- **Interest for supervision:** Essential interface for monitoring active applications. It allows visualization of resource allocation (CPU, memory), checking the status of NodeManagers, and performing crucial diagnostics in case of resource assignment issues for different jobs (Spark, MapReduce).

Spark History Server

- **Function:** This server is dedicated to the persistence and presentation of the complete history of Spark applications after their completion.
- **Interest for supervision:** Essential for post-execution performance analysis. It offers a detailed view of Spark operations (stages, tasks, *shuffles*), facilitating the identification of bottlenecks, the evaluation of data distribution, and the determination of necessary optimizations for future processing.

HDFS NameNode

- **Function:** Control interface for the NameNode, the central component of HDFS (Hadoop Distributed File System) responsible for namespace management and regulating data access.
- **Interest for supervision:** Key role in verifying file system integrity. It allows monitoring the overall

storage space and its usage, ensuring the proper functioning of DataNodes, inspecting file blocks, and diagnosing anomalies related to replication or data access.

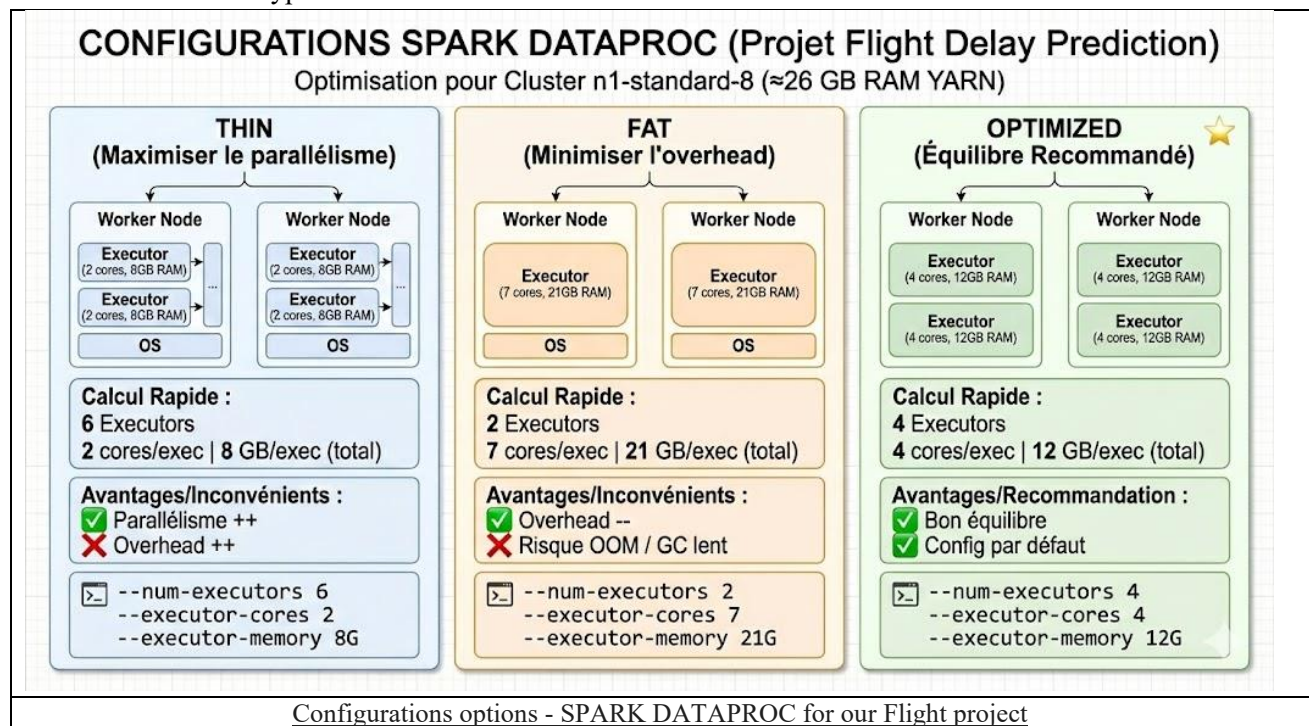
Results retrieval

Results are saved in Google Cloud Storage (GCS). Once the pipeline is complete, the data is downloaded locally.

7.3.8 Optimal Spark Executor Sizing: Comparative Study Thin/Fat/Optimized

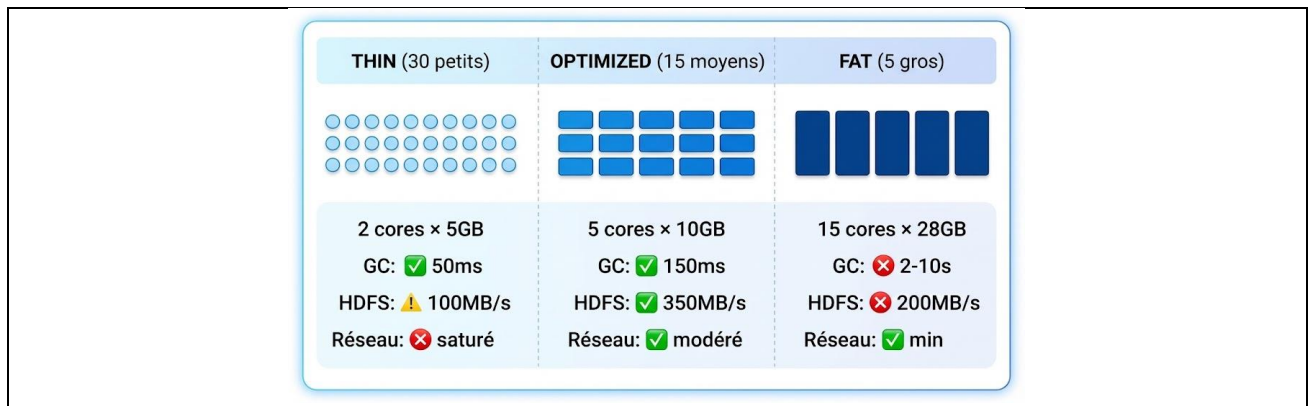
The objective of this section is to determine the optimal configuration of Spark resources (executors, cores, and memory) on the LAMSADE cluster. This is to ensure the best possible performance when executing flight delay prediction tasks.

We adjusted and compared different configurations (THIN, FAT, and Optimized) to evaluate their respective advantages and disadvantages, while considering the limits of available DataProc resources. The calculations take into account YARN constraints (notably the maximum memory allocation limit, and the specifics of the n1-standard machine types used).



Review of principles

Here is a reorganization and reformulation of the text on executor configuration:



Executor Configuration Strategies

Strategy	Description	Advantages	Disadvantages/Consequences
THIN (Narrow)	Many small executors.	Maximizes task execution parallelism.	Potential increased <i>overhead</i> related to managing a large number of instances.
FAT (Large)	Few large executors.	Minimizes network-related overhead.	Reduces the level of parallelism.
Optimized	Balance between the number and size of executors.	Achieves the best compromise between parallelism and overall processing efficiency.	Requires fine-tuning based on the specific workload.

Executor Configuration Strategies

THIN Configuration (Many small executors)

This calculation is based on the following Dataproc cluster configuration:

- **Cluster:**
 - 1 Master node (machine type: n1-standard-16)
 - 2 Worker nodes (machine type: n1-standard-8)
- **Total Cluster Capacity:**
 - **Cores:** 16 (Master) + 2 × 8 (Workers) = 32 cores
 - **RAM:** Approximately 120 GB (with a YARN limit of about 26 GB per container)

Spark/YARN Parameters (Calculator):

- **executor-cores:** 2 (Conservative/small value)
- **Executors per Worker:** (8 Worker cores - 1 reserved core) ÷ 2 cores per executor ≈ 3.5. Rounded to 3 executors.
- **Total Number of Executors:** 2 Workers × 3 executors/worker = 6 executors
- **executor-memory:** The YARN limit is ~26 GB. After deducting overhead (≈ 26 GB ÷ 1.15), the available memory is about 20 GB. A more conservative value of 8 GB is retained.

Spark-submit Command for THIN

Here is the explanation of the choice of these parameters in the `spark-submit` command, generally used to optimize the execution of a Spark application on a YARN cluster (like DataProc):

Basic Configuration (YARN/Deployment):

- `--master yarn`: Specifies that the application should be executed on the YARN resource manager.
- `--deploy-mode client`: Indicates that the Spark Driver will be executed on a Worker Node of the cluster, not on the machine where the command is launched. This is the recommended mode for production applications.

Resource Allocation (Executors and Driver):

- `--num-executors 6`: Requests 6 execution processes (Executors) for the application. The choice of the number of executors depends on the cluster size and the volume of data.
- `--executor-cores 2`: Allocates 2 CPU cores to *each* Executor. A good compromise between parallelism and contention.
- `--executor-memory 8G`: Allocates 8 GB of RAM to *each* Executor. Memory is crucial for RDD cache, shuffle, and calculations.
- `--driver-memory 12G`: Allocates 12 GB of RAM to the Driver. The Driver manages the application, stores the state, and can collect results; it may require more memory if many `collect()` actions are performed or if the execution plan is very large.
- `--driver-cores 4`: Allocates 4 CPU cores to the Driver.

Advanced Memory and Parallelism Configuration:

- `--conf spark.yarn.executor.memoryOverhead=2G`: This is additional memory (off-heap) allocated by YARN for internal JVM needs, metadata, and native code, *in addition* to `--executor-memory`. **Total memory per Executor (according to YARN) = 8G + 2G = 10G**. This is crucial to avoid YARN container shutdown errors (Container killed by YARN for exceeding memory limits).
- `--conf spark.memory.offHeap.enabled=false`: Disables the use of off-heap memory by Spark itself (distinct from YARN overhead).
- `--conf spark.sql.shuffle.partitions=100`: Defines the number of partitions used during *shuffle* operations (grouping, join, etc.) by Spark SQL. 100 is a common value; if the data is very large, this number should be increased to ensure that each partition is not too large (a target partition size of 100-200 MB is often recommended).
- `--conf spark.default.parallelism=100`: Defines the number of tasks that will be created for RDDs during transformation operations. Keeping this number aligned with `spark.sql.shuffle.partitions` (or a multiple of the total available cores: 6 executors * 2 cores/executor = 12 cores; 100 is a much higher value to ensure good task granularity).

Application Specification:

- `--class com.flightdelay.app.FlightDelayPredictionApp`: Indicates the main class (containing the main method) of your Spark application.
- `--jars gs://bucket/jars/*.jar`: Includes additional Java/Scala libraries required for the application execution, stored on Google Cloud Storage (GCS).
- `--files gs://bucket/config/prod-config.yml`: Makes a configuration file available to the application, which will be accessible by the Driver and the Executors.

`prod "data-pipeline,feature-extraction,train"`: These are the arguments passed to the main method of the specified class. Here, it is probably an environment profile (`prod`) and a list of *pipelines* to execute.

Advantages: High parallelism, fast GC

Disadvantages: Network overhead, limited shuffle buffers

FAT Configuration (Few large executors)

Calculation for Dataproc

executor-cores = 7 (large, leaves 1 core for OS)

Executors per worker = 1 (1 large executor per worker)

Total executors = 2 workers $\times$ 1 = 2 executors

executor-memory = 26 GB max - 5 GB overhead = 21 GB

Spark-submit Command for FAT

```
spark-submit \  
  --master yarn \  
  --deploy-mode cluster \  
  --num-executors 2 \  
  --executor-cores 7 \  
  --executor-memory 21G \  
  --driver-memory 12G \  
  --driver-cores 4 \  
  --conf spark.yarn.executor.memoryOverhead=5G \  
  --conf spark.memory.offHeap.enabled=true \  
  --conf spark.memory.offHeap.size=4G \  
  --conf spark.sql.shuffle.partitions=50 \  
  --conf spark.default.parallelism=50 \  
  --class com.flightdelay.app.FlightDelayPredictionApp \  
  --jars gs://bucket/jars/*.jar \  
  --files gs://bucket/config/prod-config.yml \  
  prod "data-pipeline,feature-extraction,train"
```

Advantages: Minimal overhead, large shuffle buffers

Disadvantages: Slow GC, OOM risk, under-parallelism

Optimized Configuration (Balance)

Calculation for Dataproc (Recommended)

executor-cores = 4 (balanced)

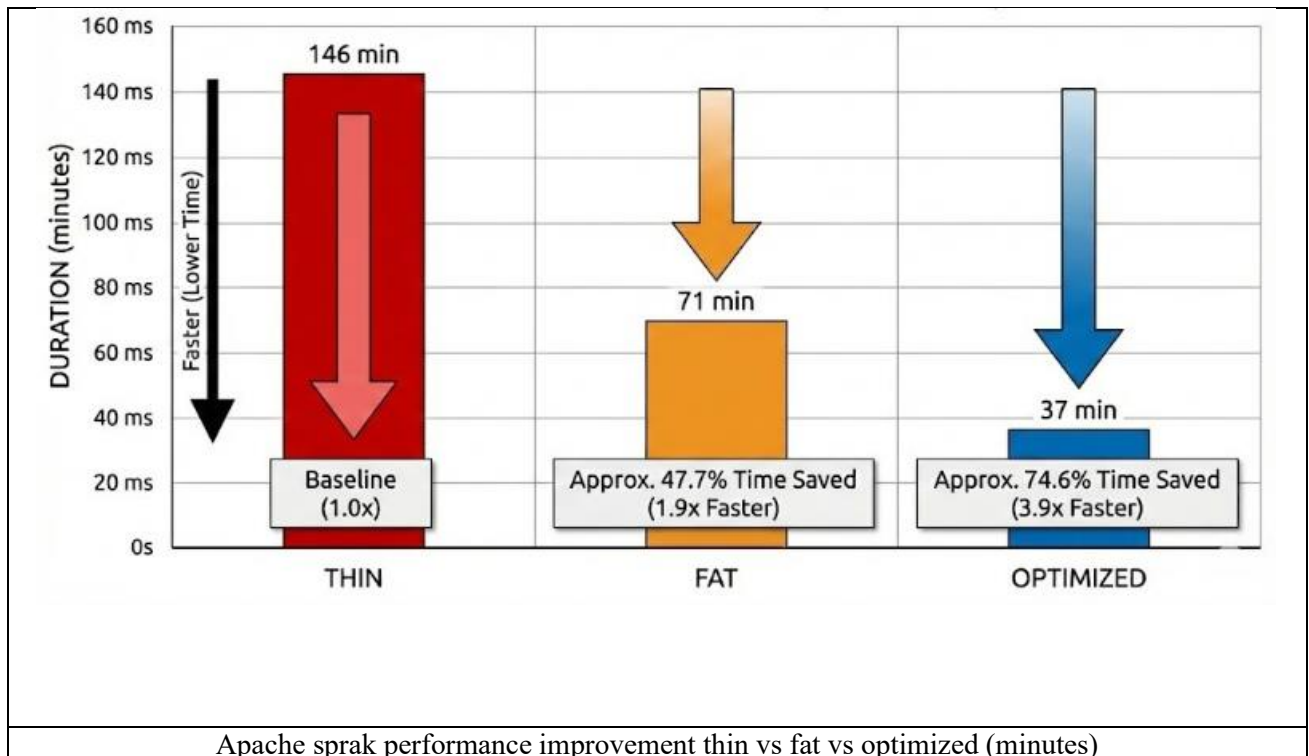
Executors per worker = $(8-1) \div 4 = 1.75 \rightarrow 1-2$ executors

Total executors = 2 workers $\times$ 2 = 4 executors (estimated)

executor-memory = 12 GB (safe for YARN)

Spark-submit Command for Optimized

```
spark-submit \  
  --master yarn \  
  --deploy-mode cluster \  
  --num-executors 4 \  
  --executor-cores 4 \  
  --executor-memory 12G \  
  --driver-memory ...
```



8. Experiments

Our system is designed for deployment across multiple execution environments, from local development setups to production-grade distributed clusters. This flexibility enables iterative development with rapid feedback cycles on local machines, followed by full-scale evaluation on high-performance infrastructure. We describe three primary environments utilized during this project.

8.1 Executive Summary on Experiments

This report presents a rigorous, data-driven analysis of a series of flight delay prediction experiments. The strategic objective is to quantify the impact of incorporating destination weather data at varying temporal depths—from 0 to 11 hours prior to departure—on model performance, computational cost, and underlying feature importance. By systematically varying this single parameter while holding all other factors constant, we can isolate its effect and derive clear, actionable insights for model deployment and future research.

However, this analysis extends far beyond the evaluation of weather depth alone. Over 100 experiments were executed across multiple computational environments, dataset configurations, model capacities, and cluster scales. Each experiment generated several text files documenting metrics, runtime behavior, feature importances, Spark configurations, and detailed execution logs. Collectively, these resources provide an exceptionally rich foundation for understanding not only *what* performs best, but *why* certain configurations succeed or fail.

To ensure the most exhaustive interpretation, this report integrates all available information—metrics, configurations, execution traces, and feature-level analyses—into a unified, structured narrative. The methodology is deliberately comprehensive: we compare all experiment families, cross-reference their results, identify recurring behavioral patterns, and examine failure modes in depth. The goal is not only to evaluate model performance, but also to understand the computational dynamics and stability challenges associated with large-scale, distributed machine learning pipelines.

8.1 Executive Summary on Experiments

8.1.1 Experimental Design and Methodology

Experimental configuration nomenclature

The Flight Experiment naming follows the format:

Experience-{environment}-{dataset}-{threshold}-{origin}-{destination}

This format defines the experiment's configuration based on the following parameters:

Parameter	Description	Valid Values/Examples
environment	Execution platform	local (Docker, no HDFS), lamsade (Lamsade distributed cluster, HDFS), dataproc (Google Cloud Dataproc)
dataset	Data version	D1, D2, D3, or D4 (as defined in the referenced article)
threshold	Delay classification boundary	Time in minutes (e.g., 60 = flights delayed 60 minutes)
origin	Origin airport weather data window	Hours of weather data <i>before</i> departure
destination	Destination airport weather data window	Hours of weather data <i>before</i> arrival

Example:

Experience-dataproc-D3-60-5-7

- **Environment:** Google Dataproc
- **Dataset:** D3
- **Threshold:** Delays ≥ 60 minutes
- **Origin:** 5 hours of origin weather data
- **Destination:** 7 hours of destination weather data

Experimental scope and objectives

The experiments analyzed in this report can be grouped into seven major families, each designed to shed light on a specific modeling or engineering dimension:

- **Weather Dependency Experiments**
 - *Local-D2-60-0-X*: Destination weather only ($X \in 0-11$ hours)

- *Local-D2-60-X-0*: Origin weather only
- *Local-D2-60-X-X*: Symmetric weather depth at both airports
- **Dataset Comparison**
 - *Lamsade-Dx-7_7*: Weather depth 7/7 across D1, D2, D3, D4 datasets
- **Model Diversity Experiments**
 - *Logistique Regression, Gradient Boosted Trees & Random Forest*
- **Model Capacity Experiments**
 - *Optimized-local-D2-60-X-X*: Larger Random Forest configurations.
- **Scalability Experiments**
 - *Lamsade-N_Workers_scalability*
 - *Dataproc-N_Workers_scalability*
- **Baseline**
 - *Dataproc-d2-60-0-0*: Model with no weather data

Together, these families explore the interactions between **data richness**, **weather granularity**, **cluster stability**, **Spark resource allocation**, **model capacity**, and **execution environment** (Local, Lamsade, Dataproc).

8.1.2 Data sources and files analyzed

Each experiment consists of four primary files:

- **config.txt**: Spark configuration, dataset used, number of workers, model hyperparameters
- **training_summary.txt**: Accuracy, Precision, Recall, F1-Score, AUC-ROC, runtime, confusion matrix
- **feature_importances_report.txt**: Feature ranking, weather contribution, temporal feature relevance
- **execution log**: Worker failures, executor crashes, memory pressure, shuffle failures, GC pauses, startup overhead

These files collectively allow for a multi-dimensional analysis integrating:

- Predictive performance trends
- Feature-level behavioral shifts
- Cluster stability and error diagnostics
- Runtime and scalability modeling
- Impact of weather temporal depth
- Influence of dataset variability

8.1.3 Structure of the experimentations section

To ensure clarity and precision, the remainder of this section on experimentation is organized as follows:

- **Local-D2-60-0-X Experience Set** – Destination weather only
- **Local-D2-60-X-0 Experience Set** – Origin weather only
- **Local-D2-60-X-X Experience Set** – Symmetric weather depth
- **Optimized-Local-D2-60-X-X Model Capacity Study**
- **Comparative Analysis of Model Architectures on the D2-60-7-7 Feature Set**
- **Lamsade-Dx-7_7 Dataset Comparison**
- **Scalability Experiments (Lamsade & Dataproc)**
- **Synthesis and Recommendations**

Each section provides:

- Complete metric comparison tables
- Interpretations of model behavior
- Runtime and cluster-performance analysis
- Cross-experiment trend synthesis
- Identification of diminishing returns and optimal configurations

8.1.4 Analytical strategy

Throughout the report, the analysis goes beyond raw metrics to answer deeper engineering questions, including:

- Does increasing weather depth meaningfully improve prediction accuracy?
- Is destination weather more informative than origin weather?
- Which datasets (D1–D4) generate the most robust models?
- How does cluster stability impact final model performance?
- What is the optimal number of Spark workers?
- Do larger Random Forest models offer real benefits?
- Which features consistently emerge as most predictive?

The overarching goal is to produce insights that are not only descriptive but operationally actionable.

8.1.5 Summary on experimentation approach

In sum, this report provides a complete, structured, and deeply technical examination of all 100+ experiments. It synthesizes performance outcomes, computational phenomena, and feature-level dynamics into a unified narrative, establishing a solid foundation for future optimization and production deployment of the flight delay prediction system.

8.2 Local-D2-60-0-x Experience set

The analysis yielded several critical findings that inform our strategic direction:

- **Optimal Performance Threshold:** The highest predictive accuracy, as measured by the F1-Score (73.50%), was achieved in experiment Experience-local-D2-60-0-3, which incorporated 3 hours of historical destination weather data. This configuration represents the most effective balance of signal and noise for this key metric.
- **Law of Diminishing Returns:** While increasing weather data beyond 3 hours led to a decline in the F1-Score, the model's underlying discriminatory power (AUC-ROC) continued to improve. This indicates a trade-off between achieving the optimal classification threshold, to which F1-Score is sensitive, and improving overall class separation.
- **Significant Computational Cost:** The inclusion of any destination weather data introduces a substantial runtime overhead, primarily during the data joining and preparation stages. The total execution time increased sharply from approximately 10 minutes for the baseline model (no weather data) to over 28 minutes for the model incorporating 11 hours of weather history.
- **Predictive Feature Dominance:** Upon their introduction, destination weather features immediately supplant prior top predictors, becoming the most dominant features by a significant margin. Variables related to atmospheric moisture, specifically average relative humidity (RelativeHumidity_Avg) and the minimum temperature-dewpoint spread (temp_dewpoint_spread_Min), consistently displaced geographical and temporal features as the most influential factors.

Based on these findings, the primary recommendation is to select the model from **Experience-local-D2-60-0-3** for **operational deployment**. This configuration represents the optimal trade-off between peak predictive power, as measured by the F1-Score, and computational efficiency.

The following sections provide a detailed breakdown of the experiments, a deep dive into performance and cost metrics, and an analysis of the feature importances that drive these outcomes.

8.2.1 Global Synthesis of experimental results

This section provides a high-level, comparative overview of the seven experiments conducted. The goal is to establish a baseline understanding of the global trends in model performance and computational cost as a function of weather data depth, before proceeding to a more granular analysis.

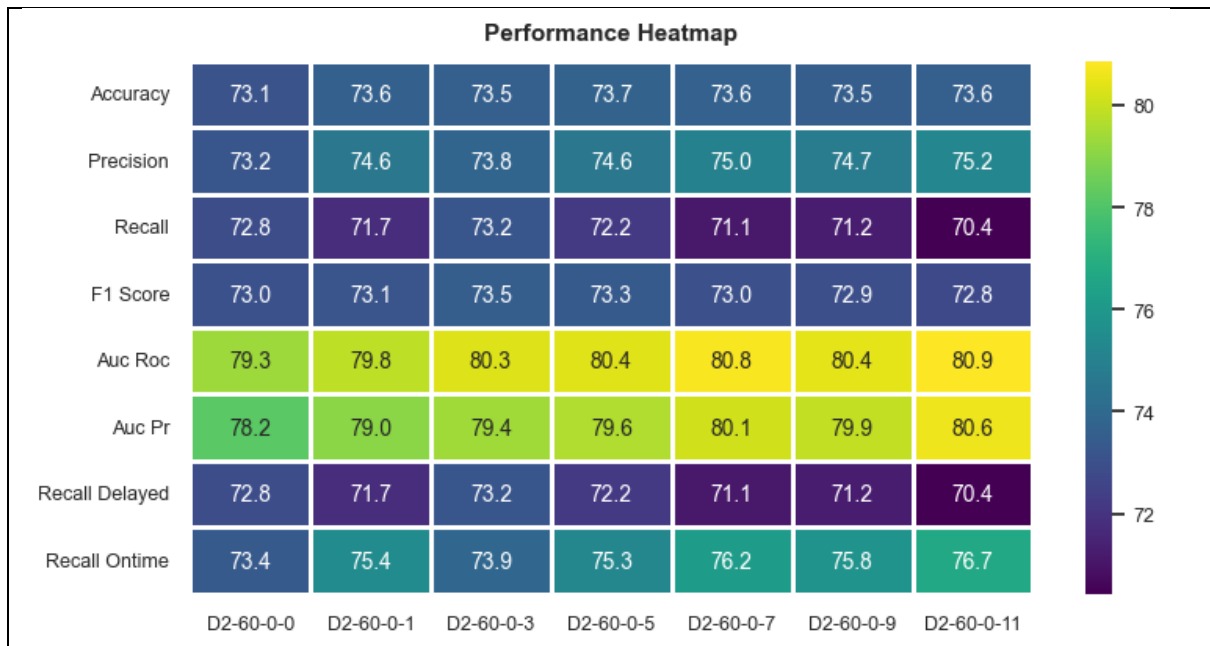
Note: For brevity, experiments named Experience-local-D2-60-0-X are referred to as D2-60-0-X in all tables and figures.

The table below summarizes the key outcomes for each experiment, from the baseline with no weather data (D2-60-0-0) to the most feature-rich model with 11 hours of weather history (D2-60-0-11).

Experiment Name	Weather Depth (Hours)	Hold-out F1-Score (%)	Hold-out AUC-ROC	Total Execution Time (min)
D2-60-0-0	0	73.03%	0.7931	9.92
D2-60-0-1	1	73.12%	0.7983	22.34
D2-60-0-3	3	73.50%	0.8031	23.73
D2-60-0-5	5	73.35%	0.8035	24.91
D2-60-0-7	7	72.99%	0.8075	25.98
D2-60-0-9	9	72.95%	0.8045	26.84
D2-60-0-11	11	72.75%	0.8086	28.12

The data reveals two distinct and important trends. First, there is a non-linear relationship between weather depth and the F1-Score, which peaks at the 3-hour mark (D2-60-0-3) before gradually declining. This suggests that while near-term weather is highly predictive, more distant weather data may introduce noise that slightly degrades the model's precision-recall balance. Second, there is a clear positive correlation between weather depth and both AUC-ROC and Total Execution Time. While increasing weather depth from 3 to 11 hours improved AUC-ROC from 0.8031 to 0.8086, it also increased runtime from 23.73 to 28.12 minutes and decreased the F1-Score.

This complex relationship is further visualized in the Performance Heatmap. The heatmap, which displays results across eight different metrics, shows a distinct "hot" band for AUC-ROC and AUC PR. These metrics consistently improve (indicated by brighter colors) as weather depth increases, signifying a steadily improving ability to distinguish between delayed and on-time flights. This contrasts sharply with the more varied results for metrics like Recall and F1-Score, which do not show a monotonic improvement, reinforcing the finding of an optimal performance point at a shallow weather depth.



This global overview establishes the critical trade-offs at play. The next section will dissect these results in greater detail to understand the underlying drivers of performance, cost, and stability.

8.2.2 Analysis of the destination weather experiment family (Experience-local-D2-60-0-X)

This section provides a detailed examination of the experiment family designed to isolate and measure the effect of incorporating historical destination weather data into the flight delay prediction model. By maintaining a controlled environment, we can attribute observed changes in performance and cost directly to the single variable being tested: the depth of weather history.

Methodology and controlled variables

To ensure a fair and direct comparison, all experiments in this family shared a common, controlled configuration. The only independent variable was the `weatherDestinationDepthHours` parameter, which was systematically varied from 0 (baseline) to 1, 3, 5, 7, 9, and 11 hours.

All other critical parameters were held constant across the seven runs:

- **Model:** RandomForest (modelType: "randomforest")
- **Model Hyperparameters:** Key hyperparameters were fixed to ensure model architecture did not influence the results, including numTrees: 15, maxDepth: 7, and maxBins: 280.
- **Dataset:** D2_60, where a delay is defined as ≥ 60 minutes.
- **Execution Environment:** All experiments were run in a local environment.

Predictive performance analysis

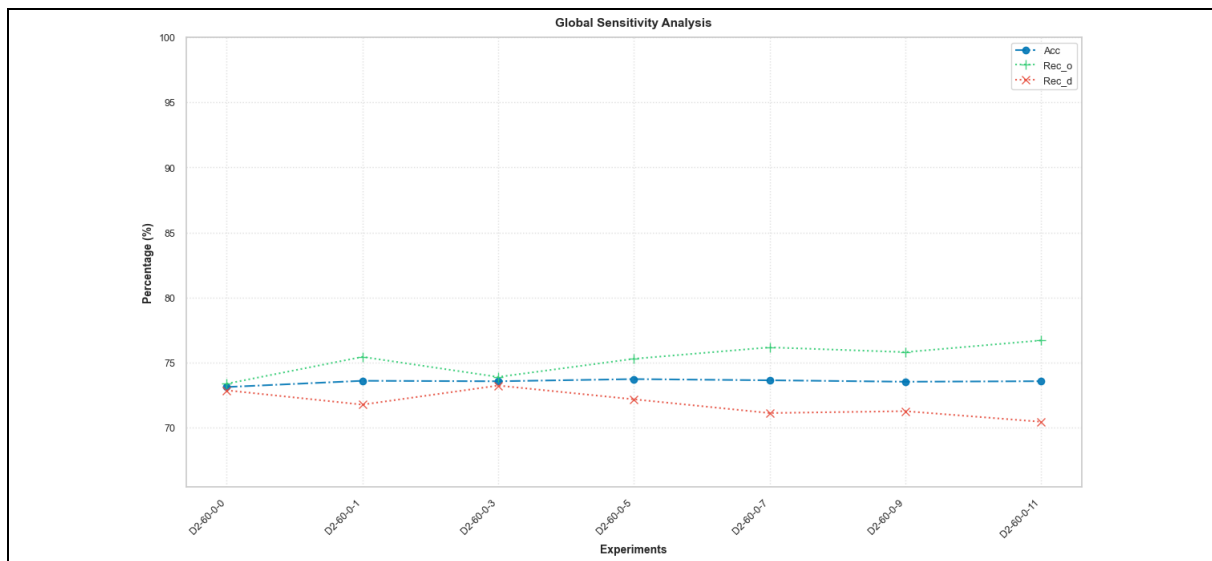
This subsection dissects the model's predictive capabilities using a comprehensive set of metrics calculated on the hold-out test set, which the model did not see during training or cross-validation.

The table below presents a full comparison of the final test set results for each experiment.

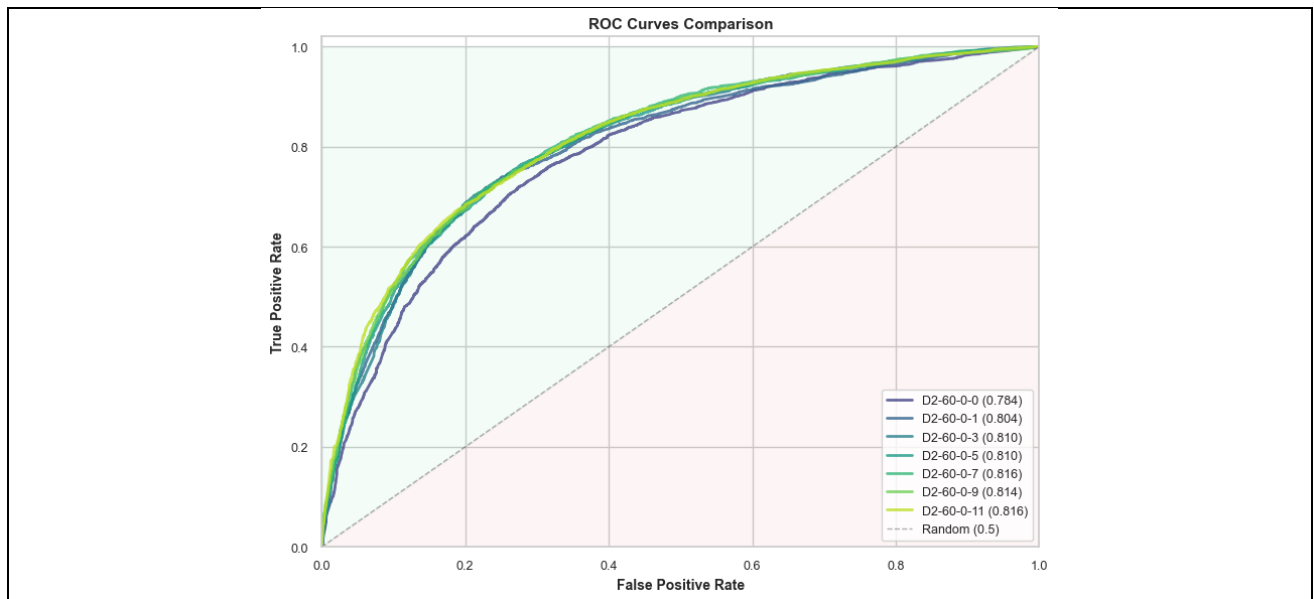
Experiment	Accuracy	Precision	Recall (Delayed)	F1-Score	AUC-ROC
------------	----------	-----------	------------------	----------	---------

D2-60-0-0	73.10%	73.22%	72.85%	73.03%	0.7931
D2-60-0-1	73.58%	74.55%	71.75%	73.12%	0.7983
D2-60-0-3	73.55%	73.78%	73.22%	73.50%	0.8031
D2-60-0-5	73.71%	74.58%	72.16%	73.35%	0.8035
D2-60-0-7	73.62%	74.97%	71.11%	72.99%	0.8075
D2-60-0-9	73.51%	74.73%	71.24%	72.95%	0.8045
D2-60-0-11	73.55%	75.23%	70.43%	72.75%	0.8086

Interpreting these metrics reveals two key behavioral patterns. The **F1-Score**, which balances precision and recall, peaks decisively for D2-60-0-3 at 73.50% and then begins a gradual decline. This suggests that the 3-hour window provides the optimal signal-to-noise ratio for this model configuration. Additional, more distant weather data may introduce more correlational noise than predictive signal, slightly degrading the model's ability to make classifications at the optimal threshold.



In contrast, the **AUC-ROC** shows a clear monotonic increase, rising from 0.7931 in the baseline to a high of 0.8086 in the 11-hour experiment. This signifies that adding more weather features consistently improves the model's fundamental ability to distinguish between the delayed and on-time classes. This consistent improvement is visually confirmed in the ROC Curves Comparison plot, where the curve for D2-60-0-11 (AUC: 0.816 in CV) is consistently positioned further to the top-left than the baseline D2-60-0-0 (AUC: 0.784 in CV), demonstrating superior performance across all thresholds.



Further analysis of recall dynamics reveals an important trade-off. As weather depth increases, the model's ability to correctly identify on-time flights (Recall On-time, or RECo) generally improves (from 73.36% to 76.69%). However, this comes at the expense of its ability to correctly identify true delayed flights (Recall Delayed, or RECd), which trends downward (from 72.85% to 70.43%). This suggests that with more features, the model becomes slightly more conservative in predicting the positive (delayed) class.

Cross-Validation stability analysis

A stable model exhibits low variance in performance across different subsets of the training data. This is a critical indicator of generalizability. We measured stability by calculating the standard deviation of key metrics across the 4 cross-validation folds.

Experiment	Recall Std Dev (%)	F1-Score Std Dev (%)	AUC-ROC Std Dev (%)
D2-60-0-0	0.55%	0.26%	0.30%
D2-60-0-1	0.51%	0.34%	0.20%
D2-60-0-3	1.06%	0.44%	0.17%
D2-60-0-5	0.79%	0.16%	0.23%
D2-60-0-7	0.66%	0.29%	0.23%
D2-60-0-9	0.90%	0.29%	0.05%
D2-60-0-11	1.42%	0.46%	0.31%

The results show a notable increase in the instability of the Recall metric as weather depth increases, rising from a low of 0.51% to a high of 1.42% for the 11-hour model. This indicates that while adding more features can improve average performance (like AUC-ROC), it can also make the model less robust. The model's recall on the positive class becomes less consistent across data folds, suggesting a higher sensitivity to the specific composition of the training data.

Runtime and computational cost analysis

Predictive performance must always be weighed against its computational cost, especially for operational systems. The following table provides a detailed breakdown of the execution time for each major stage of the pipeline.

Experiment	Data Processing (min)	Balancing (min)	Join (min)	Explode (min)	ML Pipeline (min)	Total Time (min)
D2-60-0-0	0.14	3.42	0.00	0.00	6.37	9.92
D2-60-0-1	0.14	3.37	12.25	0.15	6.41	22.34
D2-60-0-3	0.14	3.40	12.50	0.29	7.38	23.73
D2-60-0-5	0.14	3.41	12.66	0.47	8.20	24.91
D2-60-0-7	0.15	3.43	12.52	0.62	9.24	25.98
D2-60-0-9	0.14	3.45	12.65	0.85	9.71	26.84
D2-60-0-11	0.15	3.50	12.89	1.01	10.55	28.12

Two key insights emerge from this data:

1. **The Initial Cost of Weather:** The transition from the baseline (D2-60-0-0) to the first weather-enabled model (D2-60-0-1) causes a dramatic cost increase of over 12 minutes. This overhead is driven almost entirely by the introduction of the Join and Explode stages, which are required to integrate the time-series weather data with the flight data and collectively jump from 0 to 12.4 minutes.
2. **Shifting Bottleneck:** As weather depth increases from 1 to 11 hours, the Join cost remains relatively stable, but the primary cost driver shifts to the ML Pipeline stage. This stage's runtime increases steadily from 6.41 minutes to 10.55 minutes, indicating that the model training process itself becomes significantly more expensive as the number of input features grows.

This detailed analysis confirms a clear trade-off between performance gains and their associated computational and stability costs. The following section will explore *why* these changes occur by examining the features the model finds most important.

8.2.3 Feature importance deep analysis

To understand *why* the model's behavior changes so significantly with the addition of weather data, this section investigates the underlying feature importances. Analyzing which data points the model considers most predictive provides direct insight into its decision-making process.

Baseline feature profile: Experience-local-D2-60-0-0

The baseline experiment, which did not include any weather data, serves as a crucial reference point. The top 5 most important features for this model were:

1. indexed_DEST_AIRPORT_ID: 27.67%
2. imputed_feature_flight_week_of_year_cos: 24.94%
3. indexed_ORIGIN_AIRPORT_ID: 23.61%
4. imputed_CRS_DEP_TIME: 10.67%
5. imputed_feature_flight_week_of_year_sin: 7.02%

In the absence of meteorological data, the model relies heavily on a combination of geographical features (origin and destination airport IDs, which collectively account for over 51% of importance) and high-level temporal features (week of year and scheduled departure time).

8.2.4 The Shift in Feature Dominance with Weather Integration

The shift in feature dominance with weather integration

The introduction of just one hour of destination weather data in experiment D2-60-0-1 caused a radical and immediate shift in feature importance. The previous top feature, indexed_DEST_AIRPORT_ID, was supplanted by

dest_temp_dewpoint_spread_Min, which became the single most important predictor with an importance score of 26.44%.

The table below tracks the evolution of the single most important feature as more weather data was added.

Experiment	Top Feature Name*	Importance (%)
D2-60-0-0	DEST_AIRPORT_ID	27.67%
D2-60-0-1	dest_temp_dewpoint_spread_Min	26.44%
D2-60-0-3	dest_RelativeHumidity_Avg	24.01%
D2-60-0-5	dest_RelativeHumidity_Avg	26.75%
D2-60-0-7	dest_RelativeHumidity_Avg	25.96%
D2-60-0-9	dest_RelativeHumidity_Avg	24.07%
D2-60-0-11	dest_RelativeHumidity_Avg	22.00%
<i>Feature names are shortened for clarity. The dest_ prefix indicates a destination weather feature.</i>		

This evolution is telling. After the initial introduction of weather data, the model consistently identifies features related to atmospheric moisture at the destination—average relative humidity and the spread between temperature and dewpoint—as the primary predictors of flight delays. These meteorological variables are demonstrably more influential than any single geographical, carrier, or temporal feature.

Aggregate Importance of weather features

To quantify the total contribution of weather, we summed the importance scores of all destination weather-related features for each experiment. This aggregate value was calculated by summing the Gini importance of all features prefixed with destination_weather_ in the feature importance report for each experiment.

Weather Depth (Hours)	Total Weather Feature Importance (%)
1	28.00%
3	28.41%
5	28.71%
7	28.30%
9	26.39%
11	24.73%

The analysis reveals that the total contribution of weather features peaks at the 5-hour mark and then begins to decline. This trend strongly corroborates the "diminishing returns" observed in the F1-Score. The model appears to gain the most value from the first few hours of weather data. This suggests that beyond the 5-hour mark, the additional, more temporally distant weather features begin to introduce more noise than signal, diluting the predictive power of the core meteorological variables and leading to the observed decay in both F1-Score and aggregate feature importance.

This deep dive into feature importance provides a clear explanation for the observed performance trends, confirming that destination weather is a powerful but nuanced predictor.

8.3 Local-D2-60-X-0 Experience set

This set of experiments is designed to isolate and quantify the impact of incorporating **origin weather data** at varying temporal depths on model performance, computational cost, and feature importance. The primary objective is to determine the optimal depth of historical origin weather and to directly compare its predictive power against the destination weather experiments analyzed in the previous section. By systematically varying

only the origin weather depth from 1 to 11 hours, we can attribute all observed changes in model behavior to this single factor.

The analysis reveals a clear and compelling case for the superiority of origin weather data. Key findings include:

- **Optimal Performance:** The model's F1-Score, a balanced measure of precision and recall, peaks at an impressive **75.85%** when incorporating 9 hours of origin weather history.
- **Sustained Discriminatory Power:** Unlike the F1-Score, the model's fundamental ability to distinguish between delayed and on-time flights (AUC-ROC) improves monotonically with increased weather depth, reaching a maximum of 0.8286.
- **Significant Performance Lift:** The introduction of origin weather provides a substantial and immediate uplift in predictive performance over the baseline model, though it also incurs a notable increase in computational cost, primarily due to data joining operations.
- **Complementary Predictive Signal:** Origin weather features consistently rank among the most important predictors. However, unlike destination weather, they do not completely displace core geographical and temporal features, suggesting they provide a powerful complementary signal rather than a single dominant one.

This global synthesis establishes that integrating origin weather is a highly effective strategy for improving model accuracy, a conclusion that will be explored in detail in the following subsections.

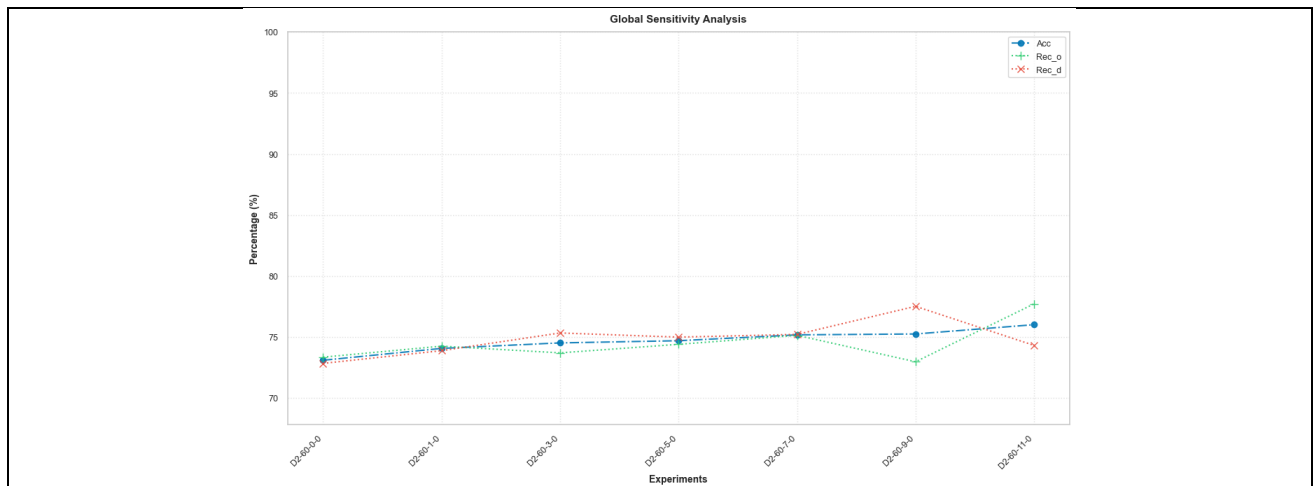
8.3.1 Global synthesis of experiments results

This subsection provides a high-level, comparative overview of the seven experiments conducted in this family. The goal is to establish a baseline understanding of the global trends in model performance and computational cost as a function of origin weather data depth, from the baseline with no weather data to the model incorporating 11 hours of history.

The table below summarizes the key outcomes for each experiment.

Experiment Name	Weather Depth (Hours)	Hold-out F1-Score (%)	Hold-out AUC-ROC	Total Execution Time (min)
D2-60-0-0	0	73.03%	0.7931	9.92
D2-60-1-0	1	74.08%	0.8068	22.78
D2-60-3-0	3	74.78%	0.8077	23.27
D2-60-5-0	5	74.83%	0.8131	24.24
D2-60-7-0	7	75.24%	0.8189	25.30
D2-60-9-0	9	75.85%	0.8215	26.30
D2-60-11-0	11	75.64%	0.8286	27.06

The data reveals a compelling narrative of trade-offs. We observe a non-linear relationship for the F1-Score, which steadily improves with added weather data before peaking at D2-60-9-0 and then slightly declining. This suggests that the 9-hour window achieves the optimal balance of precision and recall. In contrast, both the AUC-ROC and Total Execution Time show a monotonic increase. This indicates that while the model's underlying discriminatory power continues to improve with more data, this comes at a direct computational cost and does not guarantee an improvement in the classification metric that balances false positives and false negatives.



This high-level overview demonstrates the significant value of origin weather data. The following detailed analysis will unpack the specific drivers behind these performance and cost trends.

8.3.2 Analysis of the origin weather Experiment Family (Experience-local-D2-60-X-0)

A granular analysis of the origin weather experiments reveals a direct causal link between increasing weather depth, improved model performance, and escalating computational costs. By holding all other variables constant, any observed changes can be attributed solely to the depth of origin weather history, allowing for a precise evaluation of its impact.

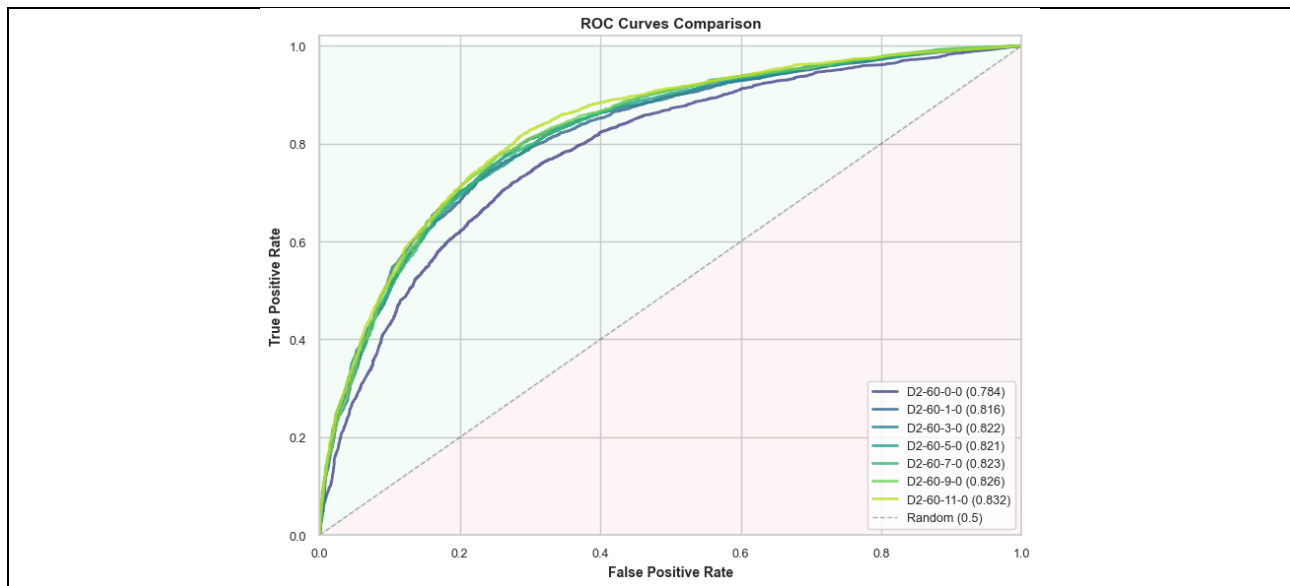
Methodology and controlled variables

The experimental setup was rigorously controlled to isolate the effect of a single independent variable: `weatherOriginDepthHours`. This parameter was systematically varied across six experiments (1, 3, 5, 7, 9, and 11 hours) and compared against the D2-60-0-0 baseline, which contained no weather data. All critical parameters were held constant across all runs:

- **Model:** RandomForest (modelType: "randomforest")
- **Model Hyperparameters:** Key settings were fixed to ensure architectural consistency: numTrees (15), maxDepth (7), and maxBins (280).
- **Dataset:** D2_60 (flights with a delay greater than or equal to 60 minutes are labeled as positive).
- **Execution Environment:** All experiments were executed in a local environment.

Predictive performance analysis

A detailed performance analysis reveals a crucial trade-off: while the model's fundamental discriminatory power (AUC-ROC) improves with all available data, the optimal precision-recall balance (F1-Score) is achieved at a finite 9-hour weather depth. This subsection dissects the model's predictive capabilities using a comprehensive set of metrics calculated on the hold-out test set for an unbiased assessment of generalization performance.



The table below presents a full comparison of the final test results for each experiment in the D2-60-X-0 series.

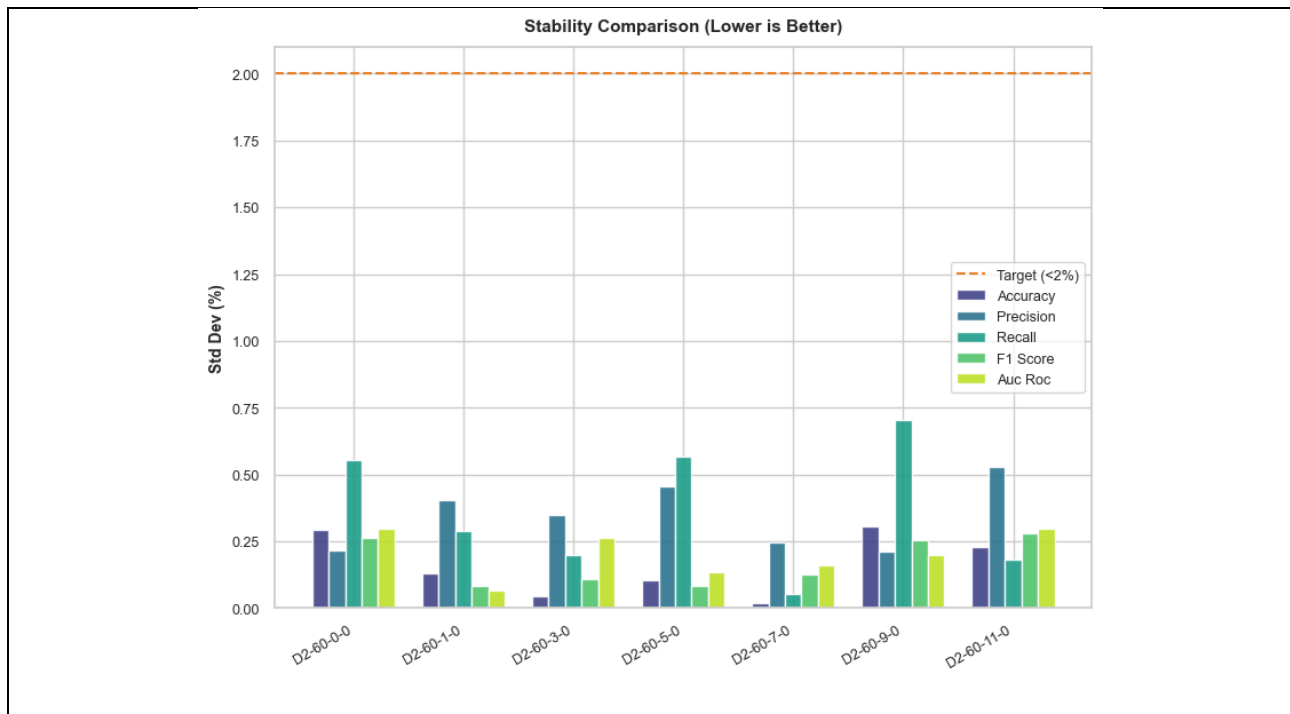
Experiment	Accuracy	Precision	Recall (Delayed)	F1-Score	AUC-ROC
D2-60-1-0	74.07%	74.27%	73.89%	74.08%	0.8068
D2-60-3-0	74.53%	74.23%	75.34%	74.78%	0.8077
D2-60-5-0	74.70%	74.65%	75.00%	74.83%	0.8131
D2-60-7-0	75.18%	75.25%	75.22%	75.24%	0.8189
D2-60-9-0	75.26%	74.24%	77.52%	75.85%	0.8215
D2-60-11-0	76.02%	77.01%	74.32%	75.64%	0.8286

The performance trends reveal a nuanced relationship between data depth and predictive power.

- F1-Score Analysis:** The F1-Score shows a clear pattern of improvement, rising steadily as more origin weather data is added. It reaches a distinct peak of **75.85%** for the D2-60-9-0 experiment before a slight dip in the 11-hour model. This indicates that the 9-hour window provides the optimal signal-to-noise ratio for balancing precision and recall.
- AUC-ROC Analysis:** In contrast, the **AUC-ROC exhibits a monotonically increasing trend**, climbing from 0.8068 (D2-60-1-0) to a high of 0.8286 (D2-60-11-0). This signifies a consistent improvement in the model's fundamental ability to distinguish between delayed and on-time flights as more features are added. The visual superiority of the ROC curve for later models in the comparison chart demonstrates better class separation, a conclusion directly corroborated by the rising hold-out AUC scores presented in the table.
- Recall Dynamics:** An analysis of the trade-off between recall for delayed flights (RECD) and on-time flights (RECO) reveals a shifting model bias. RECD generally improves up to the 9-hour mark, peaking at an excellent 77.52%, before falling in the final experiment. Conversely, RECO trends in the opposite direction, bottoming out at 72.98% for the 9-hour model before rising again. This demonstrates how the model adjusts its classification bias as the feature space expands.

Cross-validation stability analysis

Model stability, defined as low variance in performance across different training data subsets, is crucial for reliable production deployment. We measure this using the standard deviation of metrics over the 4 cross-validation folds.



Experiment	Recall Std Dev (%)	F1-Score Std Dev (%)	AUC-ROC Std Dev (%)
D2-60-1-0	0.29%	0.08%	0.07%
D2-60-3-0	0.20%	0.11%	0.26%
D2-60-5-0	0.57%	0.08%	0.13%
D2-60-7-0	0.05%	0.12%	0.16%
D2-60-9-0	0.70%	0.25%	0.20%
D2-60-11-0	0.18%	0.28%	0.30%

While all experiments remain well below the 2% instability target, the data does not show a simple, clear trend. Notably, D2-60-9-0, which achieved the best F1-Score, also exhibits the highest instability for the Recall metric (0.70%). This suggests a potential "overfitting" to specific characteristics of the training distribution for that metric. While 9 hours is optimal for the F1-score on this particular hold-out set, a slightly more stable model like D2-60-7-0 might generalize better in production, presenting a clear performance-versus-stability trade-off for the engineering team.

Runtime and computational cost analysis

The analysis of computational cost reveals that performance gains from origin weather come with a significant, but predictable, increase in resource consumption. Weighing predictive performance against this cost is essential for building efficient and scalable systems.

The table below breaks down the execution time for each major stage of the data and machine learning pipeline.

Experiment	Data Processing (min)	Balancing (min)	Join (min)	Explode (min)	ML Pipeline (min)	Total Time (min)
D2-60-1-0	0.14	3.43	12.46	0.15	6.57	22.78
D2-60-3-0	0.14	3.38	12.41	0.28	7.04	23.27
D2-60-5-0	0.14	3.38	12.37	0.44	7.89	24.24
D2-60-7-0	0.14	3.38	12.46	0.60	8.70	25.30
D2-60-9-0	0.14	3.38	12.43	0.79	9.53	26.30
D2-60-11-0	0.14	3.37	12.42	0.96	10.15	27.06

The data reveals two primary drivers of computational cost:

- **Fixed, One-Time Cost:** Introducing origin weather features incurs a substantial initial overhead of over 12 minutes compared to the baseline. This is a fixed, one-time cost for enabling weather features, almost entirely attributable to the computationally expensive Join stage (12.46 min), which merges flight records with their corresponding time-series weather data.
- **Variable Cost Scaling with Dimensionality:** As weather depth increases, the primary cost driver shifts from the fixed Join stage to the ML Pipeline stage. This is a variable cost that scales linearly with feature dimensionality. The ML Pipeline runtime grows steadily from 6.57 minutes (D2-60-1-0) to 10.15 minutes (D2-60-11-0), directly reflecting the increased computational burden of training a RandomForest model on a wider feature set.

To understand *why* these performance and cost changes occur, the next section examines which features the model finds most predictive.

8.3.3 Feature importance deep analysis

This section investigates the underlying feature importances to understand *why* the model's behavior and performance change with the addition of origin weather data. By analyzing what the model learns, we can connect the input data to the predictive outcomes.

Baseline Feature Profile: Experience-local-D2-60-0-0

As a point of reference, the baseline model (D2-60-0-0), which lacked any weather data, relied heavily on geographical and temporal predictors. The most influential features were `indexed_DEST_AIRPORT_ID`, `indexed_ORIGIN_AIRPORT_ID`, and the cosine transformation of the week of the year (`imputed_feature_flight_week_of_year_cos`), which together accounted for the vast majority of the model's predictive power.

The shift in Feature Dominance with weather Integration

The introduction of origin weather data significantly alters the feature importance landscape. The table below tracks the top four most important features and their relative importance for each experiment in the series.

Experiment	Rank 1 Feature	Rank 2 Feature	Rank 3 Feature	Rank 4 Feature
D2-60-1-0	<code>indexed_DEST_AIRPORT_ID</code> (29.36%)	<code>imputed_origin_weather_feature_temp_dewpoint_spread_Min</code> (24.77%)	<code>imputed_feature_flight_week_of_year_cos</code> (19.06%)	<code>indexed_ORIGIN_AIRPORT_ID</code> (12.73%)
D2-60-3-0	<code>indexed_DEST_AIRPORT_ID</code> (29.23%)	<code>imputed_origin_weather_RelativeHumidity_Avg</code> (25.43%)	<code>imputed_feature_flight_week_of_year_cos</code> (18.48%)	<code>indexed_ORIGIN_AIRPORT_ID</code> (12.98%)
D2-60-5-0	<code>indexed_DEST_AIRPORT_ID</code> (29.51%)	<code>imputed_origin_weather_RelativeHumidity_Avg</code> (22.90%)	<code>imputed_feature_flight_week_of_year_cos</code> (16.85%)	<code>indexed_ORIGIN_AIRPORT_ID</code> (12.15%)
D2-60-7-0	<code>indexed_DEST_AIRPORT_ID</code> (27.44%)	<code>imputed_origin_weather_RelativeHumidity_Avg</code> (21.25%)	<code>imputed_feature_flight_week_of_year_cos</code> (18.06%)	<code>indexed_ORIGIN_AIRPORT_ID</code> (12.93%)
D2-60-9-0	<code>indexed_DEST_AIRPORT_ID</code> (27.45%)	<code>imputed_origin_weather_RelativeHumidity_Avg</code> (18.17%)	<code>imputed_feature_flight_week_of_year_cos</code> (16.91%)	<code>indexed_ORIGIN_AIRPORT_ID</code> (12.34%)
D2-60-11-0	<code>imputed_feature_flight_week_of_year_cos</code> (24.11%)	<code>indexed_DEST_AIRPORT_ID</code> (22.59%)	<code>indexed_ORIGIN_AIRPORT_ID</code> (14.38%)	<code>imputed_CRS_DEP_TIME</code> (6.22%)

This evolution reveals a fascinating pattern. Unlike the destination weather experiments where weather features immediately seized the top rank, origin weather features act as powerful co-predictors. Core geographical features like `indexed_DEST_AIRPORT_ID` and `indexed_ORIGIN_AIRPORT_ID`, along with the seasonal `imputed_feature_flight_week_of_year_cos`, remain highly dominant. However, key origin weather features, particularly `imputed_origin_weather_RelativeHumidity_Avg` and `imputed_origin_weather_feature_temp_dewpoint_spread_Min`, consistently appear in the top 5, demonstrating their significant predictive value. The final experiment, D2-60-

11-0, shows an interesting shift where the seasonality feature overtakes airport IDs, indicating a complex interplay between predictors as the feature space grows.

Aggregate Importance of weather Features

To quantify the total contribution of origin weather to the model's decision-making, the importance scores of all origin weather-related features were summed for each experiment.

Weather Depth (Hours)	Total Origin Weather Feature Importance (%)
1	27.98%
3	28.13%
5	27.10%
7	25.19%
9	22.39%
11	23.47%

The analysis reveals a critical divergence: the aggregate importance of origin weather features peaks early at the 3-hour mark, while the F1-Score peaks much later at 9 hours. This suggests that the initial weather features provide the most potent, unique signal to the model. Later features (from hours 4-9) likely offer smaller, incremental gains that, while having lower individual Gini importance or some informational redundancy, are crucial for refining the model's precision-recall balance and pushing the F1-Score to its optimal point.

8.3.4 Comparative Analysis: Origin vs. Destination weather impact

This section conducts a direct comparison between the Local-D2-60-0-X (destination weather) and Local-D2-60-X-0 (origin weather) experiment sets. The goal is to provide a definitive, data-driven answer to which weather source—at origin or destination—is more predictive, cost-effective, and ultimately more valuable for the flight delay prediction task.

Head-to-head Performance comparison

The evidence is conclusive: the Local-D2-60-X-0 experiment set, which incorporates **origin weather data, demonstrates definitively superior predictive performance.**

The table below compares the best-performing model from each family, selected based on the highest achieved F1-Score on the hold-out test set.

Metric	Best Destination Model (D2-60-0-3)	Best Origin Model (D2-60-9-0)
Hold-out F1-Score	73.50%	75.85%
Hold-out AUC-ROC	0.8031	0.8215
Total Execution Time (min)	23.73	26.30

The best origin model provides a significant performance lift across the board. It achieves an F1-Score of **75.85%** compared to **73.50%** for the best destination model—a substantial improvement of over 2.3 percentage points. This superiority is systemic; the top three models in the origin weather family all achieve an F1-Score above 75%, a threshold no destination weather model reached.

8.3.5 Feature Dominance and Predictive Power

The feature importance analyses from both experiment sets reveal fundamentally different model behaviors.

- **Destination Weather:** When introduced, destination weather features (e.g., `imputed_destination_weather_RelativeHumidity_Avg`) immediately became the single most dominant predictors, supplanting all other geographical and temporal features.
- **Origin Weather:** In contrast, origin weather features, while highly influential and consistently ranked in the top 5, did not displace the core geographical (`indexed_DEST_AIRPORT_ID`) and temporal (`imputed_feature_flight_week_of_year_cos`) features with the same decisiveness. They acted as powerful complementary predictors.

This suggests that while destination weather introduces a new and dominant signal about conditions upon arrival, origin weather provides a rich, complementary signal that enhances the model's understanding of conditions at the point of departure without overshadowing existing high-value features.

Cost-Benefit Analysis

A crucial part of the comparison is evaluating the computational trade-offs required to achieve peak performance.

- **Destination:** Peak F1-Score (73.50%) was achieved with D2-60-0-3 at a computational cost of **23.73 minutes**.
- **Origin:** Peak F1-Score (75.85%) was achieved with D2-60-9-0 at a computational cost of **26.30 minutes**.

The conclusion is clear: the origin weather experiments deliver a substantial performance gain of **+2.35 percentage points on F1-Score** for only a marginal increase in computational cost of **+2.57 minutes**. This represents a highly efficient investment of resources for a significant improvement in predictive accuracy.

Strategic Conclusion

Based on a comprehensive review of performance, cost, and feature dynamics, the verdict is unequivocal: **incorporating origin weather data is strategically superior to using destination weather data for this modeling task**. This conclusion is firmly supported by its significantly higher predictive accuracy (both F1-Score and AUC-ROC) and its excellent cost-benefit profile.

Therefore, it is recommended that future model development and optimization efforts should prioritize the use and refinement of origin weather features as the primary meteorological input for the flight delay prediction system.

8.4 Local-D2-60-X-X Experience set: symmetric weather integration analysis

This set of experiments evaluates the impact of integrating both origin and destination weather data simultaneously, using symmetric temporal depths (e.g., 3 hours of origin data and 3 hours of destination data). The primary objective is to determine if this combined, feature-rich approach yields a significant performance improvement over using a single weather source. A secondary goal is to analyze the associated trade-offs in computational cost and model complexity that arise from doubling the weather-related feature space.

8.4.1 Global synthesis of experimental results

This section provides a high-level, comparative overview of the symmetric weather experiments. The goal is to identify broad trends in model performance and computational cost as a function of weather data depth. This global view will help locate the peak-performing model configuration and establish the context for the more granular analysis that follows.

High-Level Performance and Cost Metrics for Symmetric Weather Models

Experiment Name	Symmetric Weather Depth (Hours)	Hold-out F1-Score (%)	Hold-out AUC-ROC	Total Execution Time (min)
-----------------	---------------------------------	-----------------------	------------------	----------------------------

D2-60-1-1	1	74.69%	0.8182	32.57
D2-60-3-3	3	75.45%	0.8252	34.28
D2-60-5-5	5	76.48%	0.8345	36.34
D-60-7-7	7	82.11%	0.8906	132.28
D2-60-9-9	9	76.74%	0.8349	39.74
D2-60-11-11	11	76.45%	0.8401	42.88

The trends revealed in the table are multifaceted. For the standard models, there is a clear non-linear relationship between weather depth and F1-Score, which steadily improves before peaking at **76.74%** for the D2-60-9-9 experiment and then slightly declining. This suggests an optimal signal-to-noise ratio at the 9-hour mark. In contrast, the AUC-ROC shows a more consistent upward trend, indicating that adding more data continues to improve the model's fundamental discriminatory power even after the F1-Score has plateaued. Finally, the computational cost escalates significantly with increased feature dimensionality.

Crucially, the D-60-7-7 experiment (note the different naming convention for this high-capacity model) is a significant outlier. Its superior performance (F1-Score of 82.11%) and extreme runtime (132.28 minutes) are due to a different, higher-capacity hyperparameter configuration (numTrees: 50, maxDepth: 30, and featureSubsetStrategy: "sqrt"). This classifies it as a more complex model that is not a direct like-for-like comparison with the other standard-capacity experiments in this family.

This global overview establishes the critical trade-offs at play, which will be dissected in greater detail in the following sections.

8.4.2 Analysis of the symmetric weather experiment family (Experience-local-D2-60-X-X)

This section provides a granular examination of the symmetric weather experiments. The following subsections will cover the precise methodology, a comprehensive breakdown of predictive performance, an assessment of model stability, and an analysis of the computational costs incurred, thereby building a complete picture of their behavior.

Methodology and controlled variables

The experimental setup for this family was designed to isolate the impact of increasing symmetric weather depth. The single independent variable was the temporal depth, applied equally to both weatherOriginDepthHours and weatherDestinationDepthHours and varied across the experiments. All other critical parameters were held constant to ensure a direct and fair comparison.

- **Model:** RandomForest
- **Dataset:** D2_60
- **Execution Environment:** Local
- **Standard Model Hyperparameters:** numTrees: 15, maxDepth: 7, maxBins: 280

It is essential to note that the D-60-7-7 experiment deviated from this standard. It utilized a higher-capacity configuration with numTrees: 50, maxDepth: 30, and featureSubsetStrategy: "sqrt". This architectural difference must be considered when interpreting its performance and cost results relative to the other models in this set.

Predictive Performance analysis

This subsection dissects the models' predictive capabilities on the hold-out test set. This provides an unbiased assessment of their ability to generalize to unseen data, allowing for a clear understanding of the relationship between increased feature dimensionality and performance on key classification metrics.

Hold-out Test Set Performance Comparison

Experiment	Accuracy	Precision	Recall (Delayed)	F1-Score	AUC-ROC
D2-60-1-1	74.84%	75.24%	74.14%	74.69%	0.8182
D2-60-3-3	75.40%	75.39%	75.50%	75.45%	0.8252
D2-60-5-5	76.38%	76.23%	76.73%	76.48%	0.8345
D-60-7-7	81.56%	79.81%	84.56%	82.11%	0.8906
D2-60-9-9	76.65%	76.53%	76.95%	76.74%	0.8349
D2-60-11-11	76.89%	78.01%	74.96%	76.45%	0.8401

An analysis of these performance metrics reveals several key dynamics:

1. **F1-Score Analysis:** For the standard models, the F1-Score exhibits a clear upward trend that peaks at **76.74%** for the D2-60-9-9 experiment. Beyond this point, the score plateaus and slightly declines, indicating that the 9-hour symmetric depth represents the optimal signal-to-noise ratio for this balanced metric.
2. **AUC-ROC Analysis:** In contrast, the AUC-ROC shows a near-monotonic increase as weather depth increases, culminating at 0.8401 for the D2-60-11-11 model. This signifies a continuous improvement in the models' fundamental discriminatory power, even after the F1-Score stagnates.
3. **Outlier Performance:** The high-capacity D-60-7-7 model demonstrates superior performance across all metrics, achieving an F1-Score of 82.11%. This significant lift is directly attributable to its more complex and powerful model architecture.

Cross-Validation Stability analysis

Model stability, measured by the standard deviation of metrics across cross-validation folds, is a critical indicator of a model's robustness and its ability to generalize to new, unseen data. A model with low variance is considered more reliable for production deployment.

Cross-Validation Metric Stability (Standard Deviation)

Experiment	Recall Std Dev (%)	F1-Score Std Dev (%)	AUC-ROC Std Dev
D2-60-1-1	0.50%	0.36%	0.0041
D2-60-3-3	0.46%	0.08%	0.0018
D2-60-5-5	0.28%	0.23%	0.0043
D-60-7-7	0.00%	0.00%	0.0000
D2-60-9-9	0.22%	0.17%	0.0028
D2-60-11-11	0.59%	0.14%	0.0012

The stability results indicate that all standard models exhibit good stability with low variance across key metrics, well within acceptable limits. However, it is critical to note that the D-60-7-7 experiment's training summary reports Number of Folds: 1. Therefore, its reported standard deviation of 0.00% is an artifact of not performing a full cross-validation and should not be interpreted as a measure of superior stability.

Runtime and computational cost analysis

This subsection evaluates the computational cost associated with the performance gains from integrating symmetric weather data. Understanding the resource consumption of each pipeline stage is essential for assessing the operational feasibility and scalability of each model configuration.

Execution Time Breakdown by Pipeline Stage (minutes)

Experiment	Data Processing (min)	Balancing (min)	Join (min)	Explode (min)	ML Pipeline (min)	Total Time (min)
D2-60-1-1	0.15	3.42	21.12	0.32	7.50	32.57
D2-60-3-3	0.14	3.36	20.93	0.85	8.94	34.28
D2-60-5-5	0.14	3.40	21.10	1.34	10.29	36.34
D-60-7-7	2.91	3.36	21.13	1.95	102.86	132.28
D2-60-9-9	0.14	3.40	20.99	2.65	12.47	39.74
D2-60-11-11	0.15	3.39	21.07	3.46	14.71	42.88

An analysis of the runtime data reveals several key cost drivers:

1. **Escalating Cost:** There is a significant increase in total execution time as the symmetric weather depth increases for the standard models, rising from ~32 minutes to ~43 minutes. This highlights the substantial computational burden of processing a larger feature set.
2. **Shifting Bottlenecks:** While the Join stage introduces a large, relatively fixed cost of around 21 minutes, the Explode and ML Pipeline stages emerge as the primary variable bottlenecks. Their runtimes scale directly with the number of features, with the Explode stage growing more than tenfold from 0.32 to 3.46 minutes.
3. **Outlier Cost:** The massive total runtime of D-60-7-7 (132.28 minutes) is driven almost entirely by its ML Pipeline stage, which took 102.86 minutes. This directly connects the higher-capacity model configuration to its extreme computational expense.

To understand what the model is learning from this increasingly costly data, we turn next to a feature importance analysis.

8.4.3 Feature Importance Deep analysis

To understand *why* performance changes with the integration of symmetric weather data, this section analyzes which features the model finds most predictive. By examining the Gini importance assigned to each feature, we can gain insight into the model's decision-making process. This analysis reveals the interplay between core non-weather features and the newly introduced weather variables from both origin and destination.

Evolution of Top 5 Feature Importances

D2-60-1-1	D2-60-5-5	D2-60-9-9	D-60-7-7
1. indexed_DEST_AIRPORT_ID (25.95%)	1. indexed_DEST_AIRPORT_ID (21.34%)	1. indexed_DEST_AIRPORT_ID (19.29%)	1. indexed_DEST_AIRPORT_ID (13.06%)
2. imputed_origin_weather_feature_temp_dewpoint_spread_Min (22.78%)	2. imputed_destination_weather_RelativeHumidity_Avg (19.40%)	2. imputed_destination_weather_RelativeHumidity_Avg (19.19%)	2. indexed_ORIGIN_AIRPORT_ID (9.99%)
3. imputed_feature_flight_week_of_year_cos (18.89%)	3. imputed_feature_flight_week_of_year_cos (13.83%)	3. imputed_feature_flight_week_of_year_cos (14.13%)	3. imputed_feature_flight_week_of_year_cos (6.20%)
4. indexed_ORIGIN_AIRPORT_ID (11.43%)	4. indexed_ORIGIN_AIRPORT_ID (13.22%)	4. indexed_ORIGIN_AIRPORT_ID (12.43%)	4. imputed_feature_flight_week_of_year_sin (5.96%)

5. imputed_destination_weather_feature_temp_dewpoint_spread_Min (7.54%)	5. imputed_origin_weather_RelativeHumidity_Avg (12.44%)	5. imputed_origin_weather_RelativeHumidity_Avg (10.80%)	5. imputed_destination_weather_feature_temp_dewpoint_spread_Min (3.99%)
--	--	--	--

Analyzing the dynamics of feature importance across these experiments reveals a key pattern of **blended influence**. Unlike some single-source experiments where one data type could dominate, the top-ranked features here are consistently a mix of core non-weather features (like `indexed_DEST_AIRPORT_ID` and `indexed_ORIGIN_AIRPORT_ID`) and high-importance features derived from both origin (`imputed_origin_weather_RelativeHumidity_Avg`) and destination (`imputed_destination_weather_RelativeHumidity_Avg`) weather sources.

This consistent blending demonstrates that neither origin nor destination weather data completely overshadows the other. Instead, the model effectively leverages signals from both sources simultaneously in a complementary manner. This synergy is the likely driver behind the superior performance of the symmetric models, as they can capture more complex interaction effects between conditions at departure and arrival.

8.4.4 Comparative analysis: symmetric vs. single-source weather impact

This final section provides a strategic analysis to deliver a definitive, data-driven verdict on the value of symmetric weather integration. It achieves this by directly comparing the best-performing standard symmetric model against the champions from the single-source (origin-only and destination-only) experiment families, weighing the marginal performance gain against the increase in computational cost.

Head-to-head performance comparison

The following table pits the best standard symmetric model against the best models from the origin-only and destination-only experiment sets, selected based on their peak F1-Scores.

Peak Performance Comparison Across Experiment Families

Metric	Best Destination Model (D2-60-0-3)	Best Origin Model (D2-60-9-0)	Best Standard Symmetric Model (D2-60-9-9)
Hold-out F1-Score	73.50%	75.85%	76.74%
Hold-out AUC-ROC	0.8031	0.8215	0.8349

The data shows that the standard symmetric model (D2-60-9-9) achieves the highest absolute performance on both F1-Score and AUC-ROC. However, while it significantly outperforms the destination-only model, the improvement over the strong origin-only model is marginal.

Cost-Benefit analysis

A direct cost-benefit analysis reveals a clear case of diminishing returns when moving from the best single-source model to the symmetric configuration.

1. **Quantify the Gain:** The standard symmetric model (D2-60-9-9) provides a performance lift of **+0.89 percentage points** on the F1-Score compared to the best origin-only model (D2-60-9-0).
2. **Quantify the Cost:** This marginal gain comes at a substantial computational cost. The total execution time increases from **26.30 minutes** for the origin-only model to **39.74 minutes** for the symmetric model, an additional cost of 13.44 minutes, representing a **~51% increase in runtime**.

This analysis reveals a clear case of diminishing returns.

Strategic conclusion

Based on the preceding analysis, a two-part strategic conclusion can be drawn.

First, for the **standard models**, while using both weather sources yields the highest absolute performance, the substantial increase in computational cost is not justified by the marginal gain in accuracy. For operational efficiency, the **origin-only model represents the optimal strategic choice** among standard configurations, providing a near-peak level of performance at a much more favorable cost.

Second, the result of the **D-60-7-7 high-capacity model** offers a critical final insight. Its F1-Score of **82.11%** dramatically surpasses the standard symmetric champion's score of 76.74%. This demonstrates that **increasing model capacity provides a much larger performance improvement** than simply adding symmetric weather data to a standard model. This finding strongly suggests that future optimization efforts should prioritize model architecture and complexity over merely expanding the feature set with more of the same type of data.

8.5 Optimized-local-D2-60-X-X Model capacity study

This set of experiments was strategically designed to evaluate the impact of significantly increasing the model's capacity. Building on the insights from previous sections, this study utilizes a larger, more complex RandomForest model with hyperparameters derived from prior grid search analyses. The core objective is to determine if this more computationally expensive, optimized model yields a justifiable improvement in predictive performance when compared to the standard symmetric models analyzed in Section 4. This analysis will proceed with a global synthesis of the experimental family, followed by a detailed breakdown of performance and cost, a deep dive into feature importance, and a direct comparative analysis against the baseline symmetric models to provide a definitive verdict on its value.

8.5.1 Global synthesis of experimental results

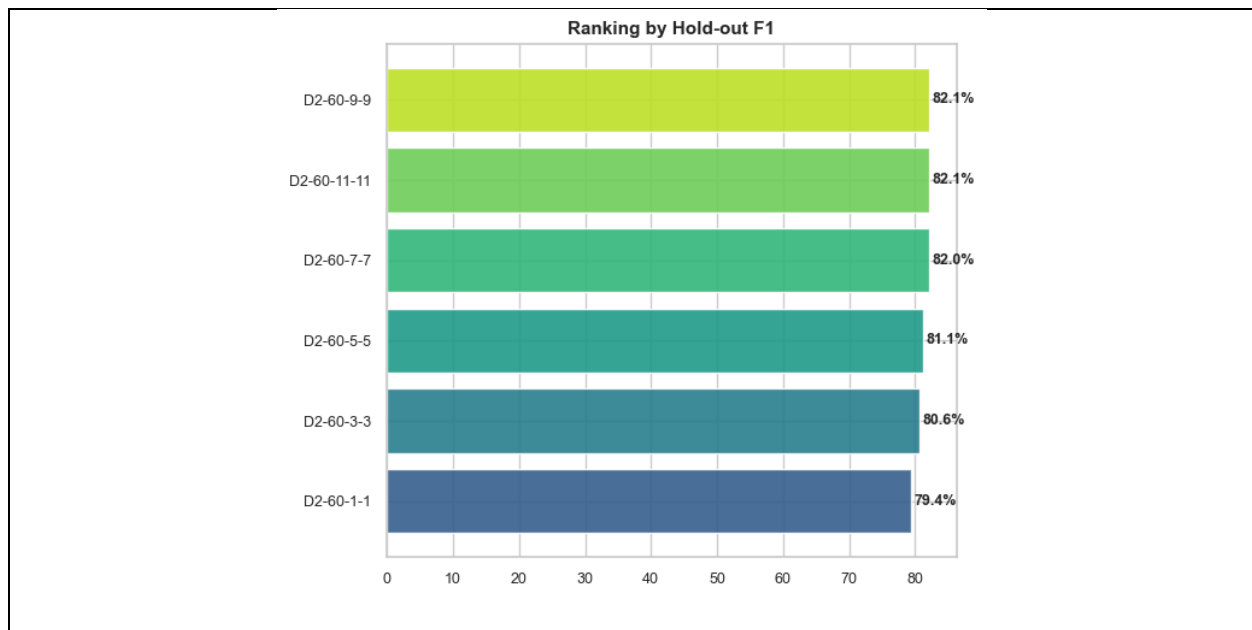
This subsection provides a high-level, comparative overview of the six experiments conducted in the optimized model family. The goal is to identify global trends in model performance and computational cost as symmetric weather depth increases, allowing us to understand the overall behavior of this higher-capacity model before a more granular examination.

The table below summarizes the key outcomes for each experiment, from a 1-hour symmetric weather window to the most feature-rich model with an 11-hour window.

Experiment Name	Symmetric Weather Depth (Hours)	Hold-out F1-Score (%)	Hold-out AUC-ROC	Total Execution Time (min)
Experience-optimized-local-D2-60-1-1	1	79.42%	0.8675	48.22
Experience-optimized-local-D2-60-3-3	3	80.64%	0.8796	58.91
Experience-optimized-local-D2-60-5-5	5	81.14%	0.8840	67.61
Experience-optimized-local-D2-60-7-7	7	82.04%	0.8923	79.32
Experience-optimized-local-D2-60-9-9	9	82.13%	0.8925	85.42
Experience-optimized-local-D2-60-11-11	11	82.06%	0.8925	94.31

The results reveal a clear but complex relationship between weather depth, performance, and cost. There is a non-linear relationship for the F1-Score, which improves steadily before peaking decisively with the Experience-

optimized-local-D2-60-9-9 model at **82.13%**. Beyond this point, performance plateaus and then slightly declines, indicating a point of diminishing returns. This contrasts with the near-monotonic improvement in AUC-ROC, which signifies a consistent enhancement of the model's underlying ability to distinguish between classes, even after the optimal F1-score is reached.



The bar chart above visually confirms the ranking from the comparison report, clearly positioning the D2-60-9-9 and D2-60-11-11 configurations at the top with F1-Scores of 82.1%. This global overview establishes the critical trade-offs at play. The following sections will dissect these high-level results to understand the underlying drivers of performance and cost for this optimized model family.

8.5.2 Detailed analysis of the optimized model family

This section provides a granular examination of the optimized model family. By holding all variables constant except for the symmetric weather depth, any observed changes in performance can be directly attributed to the interaction between the expanding weather feature dimensionality and the higher-capacity model architecture.

Methodology and controlled variables

The experimental methodology was rigorously controlled to ensure a direct and fair comparison. The single independent variable was the symmetric weather depth, which was systematically varied from 1 to 11 hours. All other critical parameters were held constant across the six experimental runs:

- **Dataset:** D2_60
- **Execution Environment:** local
- **Model Type:** RANDOMFOREST

The key differentiator for this experiment set was the use of a significantly larger RandomForest model. The hyperparameters, detailed below, represent a substantial increase in model capacity compared to the standard models analyzed in previous sections, which used 15 trees and a max depth of 7.

Optimized RandomForest Hyperparameters

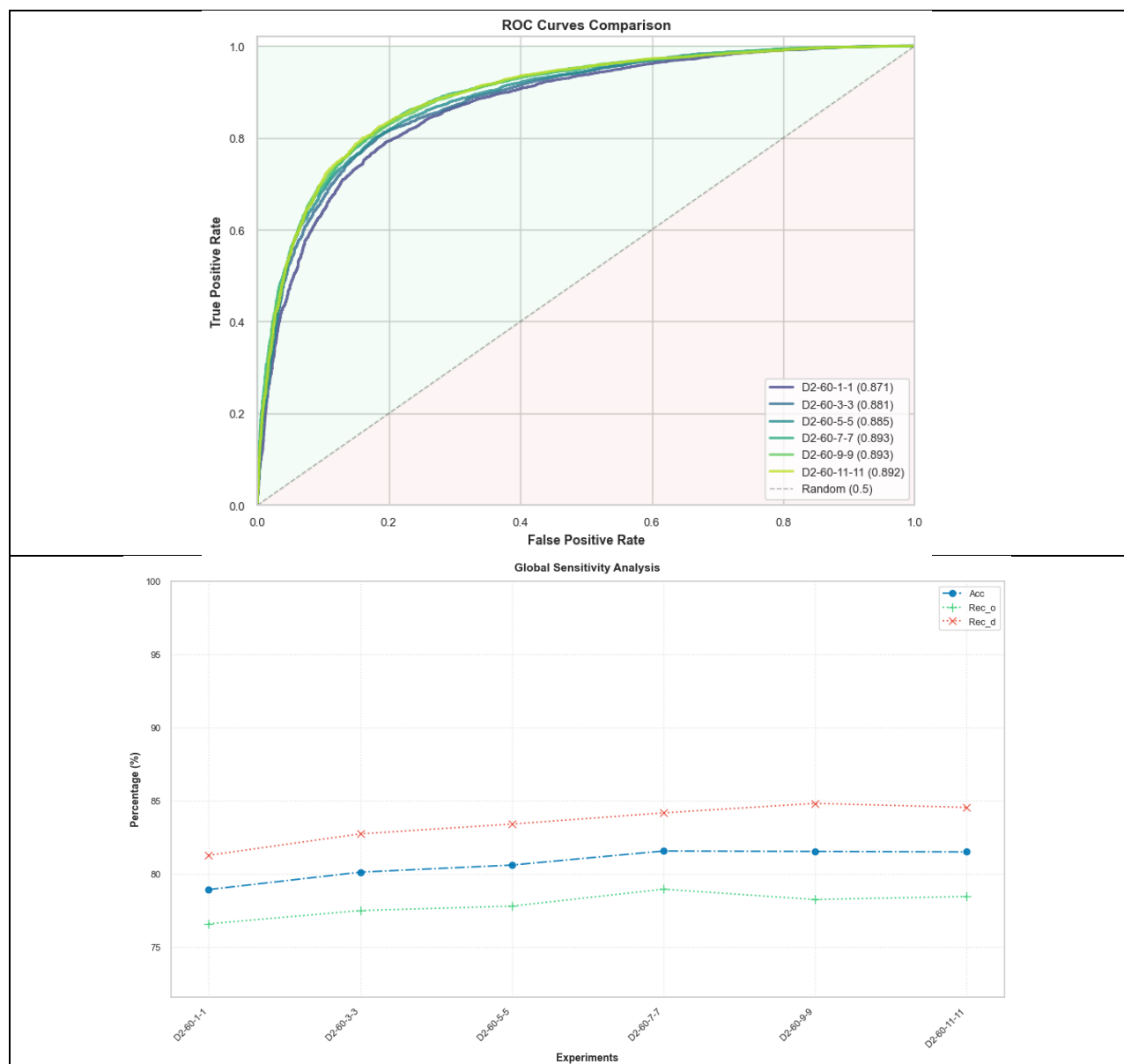
Hyperparameter	Value
numTrees	50
maxDepth	30
minInstancesPerNode	15
subsamplingRate	0.8
featureSubsetStrategy	sqrt

Predictive Performance analysis

This subsection evaluates the model's predictive capabilities on the hold-out test set, providing an unbiased assessment of its ability to generalize to unseen data.

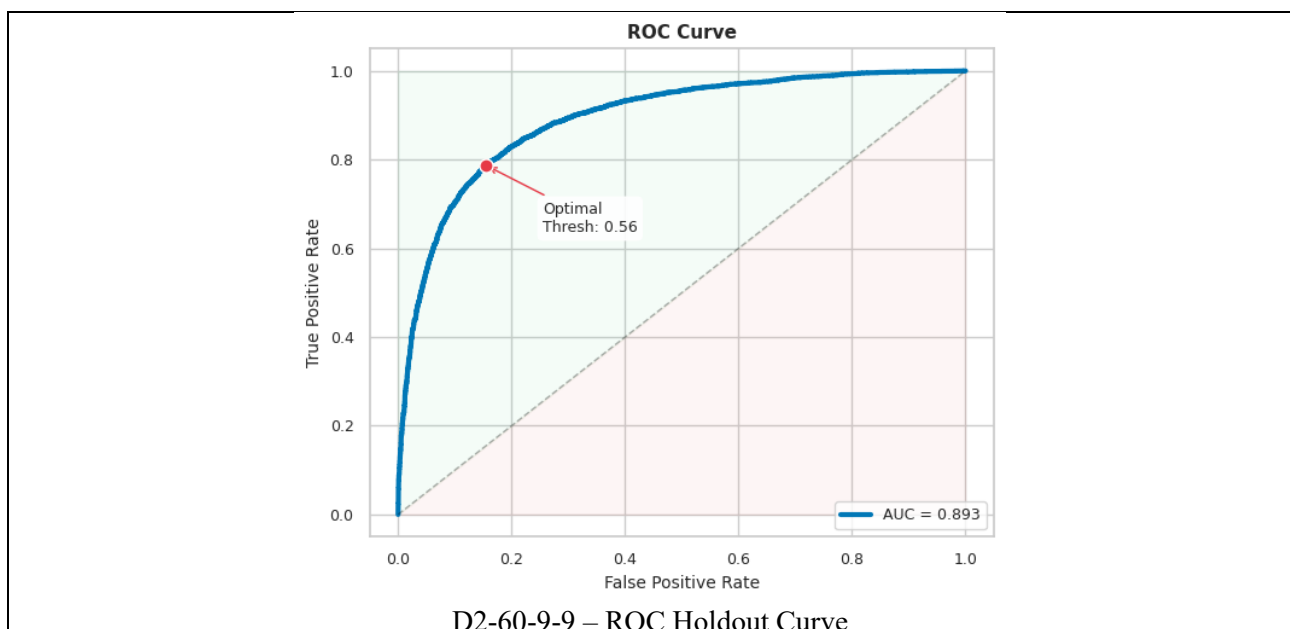
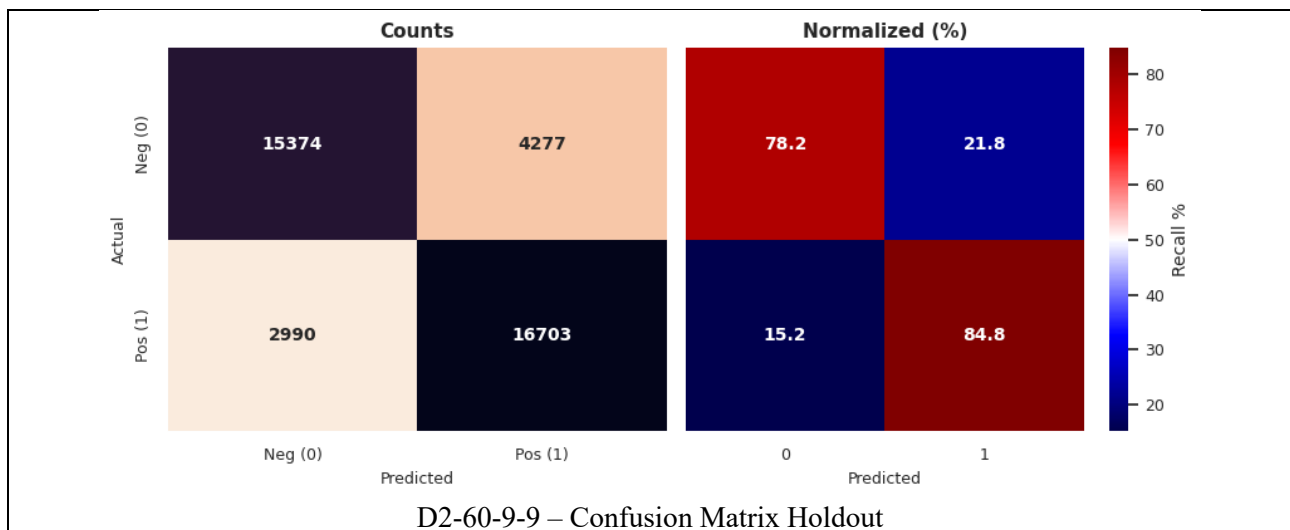
Hold-out Test Set Performance of Optimized Models

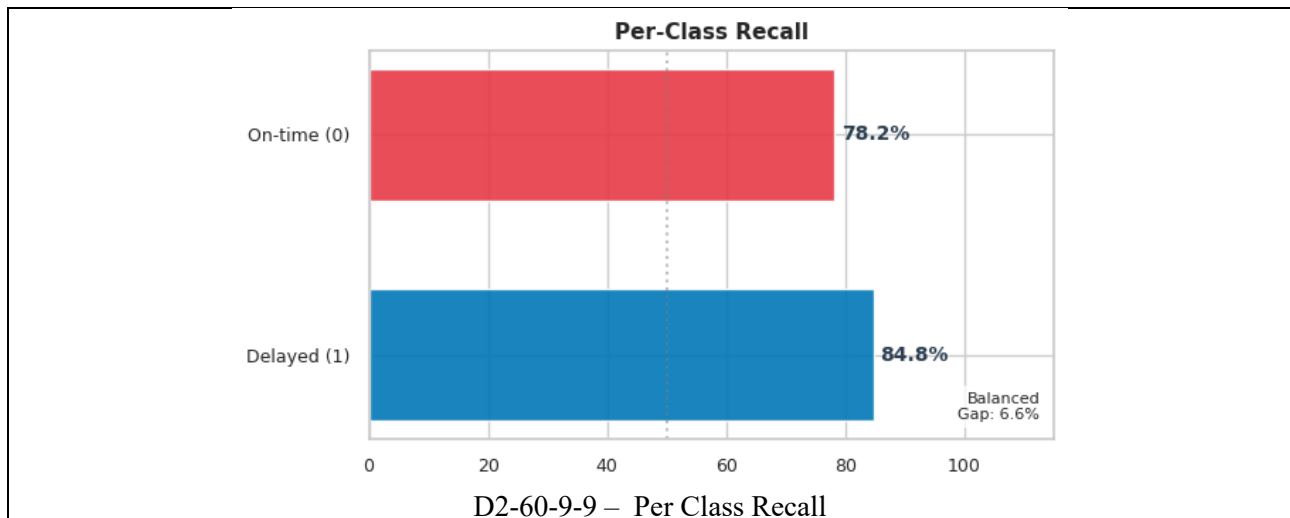
Experiment	Accuracy	Precision	Recall (Delayed)	F1-Score	AUC-ROC
D2-60-1-1	78.92%	77.66%	81.26%	79.42%	0.8675
D2-60-3-3	80.11%	78.64%	82.73%	80.64%	0.8796
D2-60-5-5	80.59%	79.00%	83.40%	81.14%	0.8840
D2-60-7-7	81.56%	80.02%	84.17%	82.04%	0.8923
D2-60-9-9	81.53%	79.61%	84.82%	82.13%	0.8925
D2-60-11-11	81.50%	79.72%	84.54%	82.06%	0.8925



An analysis of these performance metrics reveals several key dynamics:

- **F1-Score Analysis:** The data shows a clear upward trend in the F1-Score, which peaks definitively at **82.13%** for the D2-60-9-9 model. The fact that the 11-11 model's performance is nearly identical demonstrates that a significant performance ceiling has been reached, and incorporating more distant weather data offers no additional benefit to the precision-recall balance.
- **AUC-ROC Analysis:** The AUC-ROC metric shows consistent and robust improvement as weather depth increases, reaching a maximum value of **0.8925** for both the 9-9 and 11-11 models. This signifies a fundamental improvement in the model's ability to distinguish between delayed and on-time flights, confirming that the additional features are indeed providing valuable predictive signals.
- **Recall Dynamics:** The recall for delayed flights (RECD) follows a similar pattern to the F1-Score, peaking at an impressive **84.82%** for the 9-9 model. Meanwhile, the recall for on-time flights (RECO) remains relatively stable in the top-performing models, hovering around 78%. This indicates that the 9-hour configuration achieves an optimal balance in identifying both classes correctly.





Cross-validation stability analysis

Cross-Validation was not performed for this set of experiments. As noted in the training summary reports for each run, this step was SKIPPED (fast mode enabled).

The implication of this is that it is not possible to assess the stability or variance of these higher-capacity models across different subsets of the training data. This limits the ability to gauge the model's robustness to variations in data distribution, a key consideration for production deployment.

Runtime and computational cost analysis

This subsection evaluates the computational cost required to achieve the substantial performance gains observed with the higher-capacity models. Analyzing these runtimes is crucial for understanding the operational trade-offs of deploying a more complex and resource-intensive solution.

Execution Time Breakdown for Optimized Models (minutes)

Experiment	Data Processing	Balancing	Join	Explode	ML Pipeline	Total Time (min)
D2-60-1-1	0.14	3.38	21.63	0.34	22.68	48.22
D2-60-3-3	0.14	3.39	21.07	0.82	33.45	58.91
D2-60-5-5	0.14	3.40	21.14	1.35	41.52	67.61
D2-60-7-7	2.90	3.38	21.48	2.11	49.37	79.32
D2-60-9-9	0.14	3.41	21.26	2.66	57.86	85.42
D2-60-11-11	0.14	3.41	21.28	3.44	65.94	94.31

An analysis of the cost drivers reveals two primary insights:

- **Escalating Cost:** There is a significant and near-linear increase in total execution time as weather depth increases. The runtime nearly doubles, rising from **48.22** minutes for the 1-1 model to **94.31** minutes for the 11-11 model, highlighting the substantial computational burden of the optimized configuration.
- **Primary Bottleneck:** The ML Train Final Model step, which is part of the broader ML PIPELINE stage, is the dominant and scaling cost driver. The time required for this single step grows from **18.61** minutes for the 1-1 model to a massive **61.08** minutes for the 11-11 model. This is the direct and unavoidable cost of training a much larger and deeper random forest on an expanding feature set.

8.5.3 Feature Importance deep analysis

This subsection investigates the underlying feature importances to understand *how* the higher-capacity model leverages the symmetric weather data to achieve its superior performance. Analyzing which features the model prioritizes provides direct insight into its decision-making process.

Top 5 Feature Importances for Optimized Models

Rank	Top 5 Features (1-1)	Top 5 Features (3-3)	Top 5 Features (5-5)	Top 5 Features (7-7)	Top 5 Features (9-9)	Top 5 Features (11-11)
1	indexed_DEST_AIRPORT_ID (18.89%)	indexed_DEST_AIRPORT_ID (16.76%)	indexed_DEST_AIRPORT_ID (14.64%)	indexed_DEST_AIRPORT_ID (12.98%)	indexed_DEST_AIRPORT_ID (11.84%)	indexed_DEST_AIRPORT_ID (10.88%)
2	indexed_ORIGIN_AIRPORT_ID (15.12%)	indexed_ORIGIN_AIRPORT_ID (13.29%)	indexed_ORIGIN_AIRPORT_ID (11.22%)	indexed_ORIGIN_AIRPORT_ID (9.83%)	indexed_ORIGIN_AIRPORT_ID (8.86%)	indexed_ORIGIN_AIRPORT_ID (8.36%)
3	imputed_origin_weather_RelativeHumidity_Avg (9.83%)	imputed_feature_flight_week_of_year_cos (7.33%)	imputed_feature_flight_week_of_year_cos (6.77%)	imputed_feature_flight_week_of_year_cos (6.40%)	imputed_feature_flight_week_of_year_sin (5.90%)	imputed_feature_flight_week_of_year_sin (6.39%)
4	imputed_destination_weather_RelativeHumidity_Avg (8.70%)	imputed_feature_flight_week_of_year_sin (6.91%)	imputed_feature_flight_week_of_year_sin (6.39%)	imputed_feature_flight_week_of_year_sin (5.78%)	imputed_feature_flight_week_of_year_cos (5.71%)	imputed_feature_flight_week_of_year_cos (5.34%)
5	imputed_feature_flight_week_of_year_cos (7.86%)	imputed_origin_weather_feature_temp_dewpoint_spread_Min (4.73%)	imputed_origin_weather_RelativeHumidity_Avg (4.14%)	imputed_destination_weather_feature_temp_dewpoint_spread_Min (4.13%)	imputed_destination_weather_RelativeHumidity_Avg (3.32%)	imputed_destination_weather_RelativeHumidity_Avg (3.33%)

The analysis reveals a dynamic and insightful pattern that explains *why* the higher-capacity model succeeds. While the top features are a consistent blend of geographical, temporal, and weather-related variables, the critical insight lies in the changing distribution of importance. As weather depth and feature count increase, the model systematically *spreads out* its feature importance.

This is evidenced by the dilution of the top features' dominance. The Gini importance of the two primary geographical features, indexed_DEST_AIRPORT_ID and indexed_ORIGIN_AIRPORT_ID, systematically drops as the feature set expands. Concurrently, the cumulative importance of the Top 5 features decreases significantly, from over 60% in the 1-1 model to under 35% in the 11-11 model. This demonstrates that the model's increased capacity is being used effectively to capture subtle interactions and predictive signals from a much wider array of features in the long tail, rather than simply relying on a few dominant predictors. This ability to integrate weaker signals is the core mechanism driving the observed performance gains.

8.5.4 Comparative analysis: optimized vs. standard symmetric models

This section provides the crucial final analysis for this experiment set. Its objective is to deliver a definitive, data-driven verdict by directly comparing the best-performing optimized model against the best-performing standard symmetric model from Section 4. This head-to-head comparison will clarify the true value of increasing model capacity.

Head-to-head performance comparison

The evidence is conclusive: the optimized model with increased capacity delivers a substantial improvement in predictive accuracy over the standard configuration.

Peak Performance: Standard vs. Optimized Symmetric Model

	Standard Symmetric Model (D2-60-9-9)	Optimized Model (D2-60-9-9)
F1-Score	76.75%	82.13%
AUC-ROC	0.8363	0.8925

The results show that the optimized model provides a significant performance lift across both key metrics. This represents a gain of **+5.38 percentage points on the F1-Score** and **+0.0562 on AUC-ROC**, confirming that the additional model complexity is translating directly into superior predictive power.

5.4.2 Cost-Benefit Analysis

While the performance gains are significant, they must be weighed against the additional computational resources required to achieve them. This analysis quantifies that trade-off.

Cost-Benefit Trade-off

Performance Gain (F1-Score)	+5.38%
Computational Cost Increase	+31.07 minutes (+57%)

The data reveals that the significant performance improvement comes at a steep, but quantifiable, computational cost. The optimized model requires an additional 31 minutes of runtime, representing a **57% increase** over the standard model, to achieve its superior accuracy.

Strategic conclusion

The findings from this comprehensive study lead to a clear strategic conclusion. The optimized model with increased capacity is unequivocally superior in terms of predictive accuracy, proving the initial hypothesis that a more complex model could better leverage the available features. The choice of which model to deploy thus becomes an operational trade-off.

The final decision hinges on whether the business value derived from a **~5.4 point F1-score improvement**—which could translate into better resource planning, enhanced operational efficiency, and improved customer satisfaction—justifies the **~57% increase in computational cost** and runtime for the training pipeline.

Given the magnitude of the performance gain, it is recommended that the **optimized D2-60-9-9 model be considered the new champion model**. However, this recommendation comes with the critical caveat that its higher operational cost must be factored into deployment strategies and retraining schedules.

8.6 Comparative analysis of model architectures on the D2-60-7-7 feature set

This section details a crucial set of experiments designed to compare different model architectures on a fixed, optimal dataset. After prior analyses identified the **D2** dataset (60-minute delay threshold) with a 7-hour symmetric weather window as the most effective feature configuration, the next critical step was to evaluate three distinct model types: Random Forest (RF), Gradient Boosted Trees (GBT), and Logistic Regression (LR). The primary objective of this study is to identify the model that provides the optimal balance of predictive accuracy and computational efficiency—a critical trade-off for operational deployment.

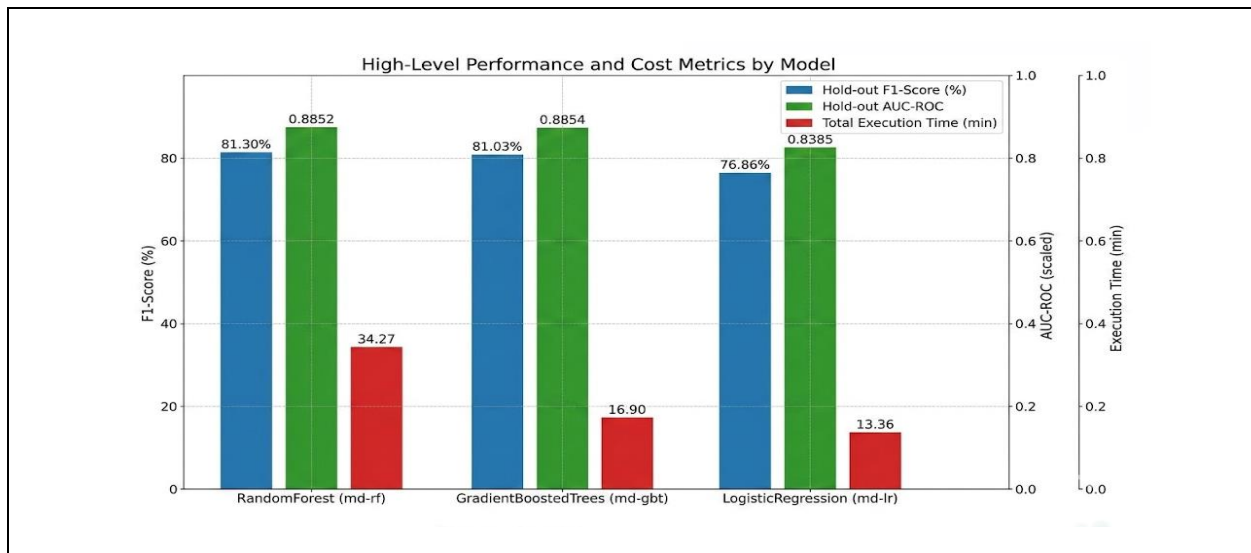
8.6.1 Global Synthesis of experimental results

This section provides a high-level, comparative overview of the three experiments conducted. The goal is to establish a baseline understanding of the global trends in model performance and computational cost as a function of the chosen algorithm. This synthesis will frame the subsequent, more granular analysis by highlighting the key performance tiers and cost disparities among the candidate models.

Overall Performance and cost summary

A summary of the key performance and cost metrics for each model immediately reveals a distinct two-tiered performance structure and a significant disparity in computational expense.

High-Level Performance and Cost Metrics



Analysis of these high-level results yields two primary findings:

- **Performance Tiering:** There is a clear performance gap between the tree-based ensemble models and the linear model. RandomForest and Gradient Boosted Trees demonstrate nearly identical, top-tier predictive power, while Logistic Regression lags significantly across both F1-Score and AUC-ROC.
- **Cost Disparity:** The similar performance of RF and GBT stands in stark contrast to their computational costs. The RandomForest model's total execution time is more than double that of the GBT model, despite offering a negligible performance advantage.

These high-level trends establish a clear trade-off between marginal performance gains and substantial computational overhead, warranting a deeper investigation into the specific performance metrics and cost drivers for each model.

8.6.2 Detailed analysis of model performance and cost

This section provides a granular examination of the experimental results. By dissecting the specific predictive metrics and runtime costs, we can precisely quantify the performance-versus-cost trade-offs offered by the RandomForest, Gradient Boosted Trees, and Logistic Regression model architectures and understand the nuanced differences in their behavior.

Methodology and controlled variables

To isolate the impact of the model algorithm itself, the experimental setup was rigorously controlled. Key data and environmental variables were held constant across all three runs, ensuring a direct and unbiased comparison.

The controlled variables were:

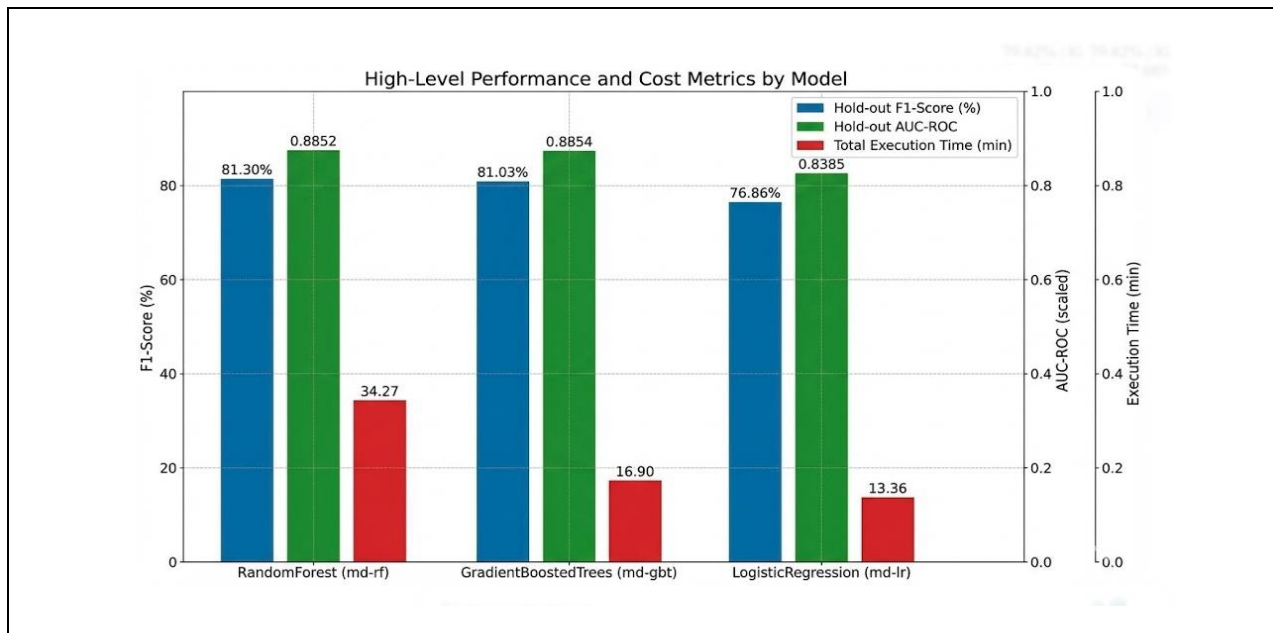
- **Dataset:** D2_60 (Delay threshold of 60 minutes)
- **Feature Set:** Symmetric weather integration with a 7-hour depth for both origin and destination.
- **Execution Environment:** local

The only independent variable in this experiment set was the `modelType`, which was systematically set to RandomForest, GradientBoostedTrees, and LogisticRegression for each respective run.

Predictive performance analysis

A detailed comparison of hold-out test set metrics confirms the top-tier performance of the ensemble methods and the substantial gap to the linear model.

Detailed Hold-out Test Set Performance Comparison



A critical evaluation of these performance metrics reveals several key insights:

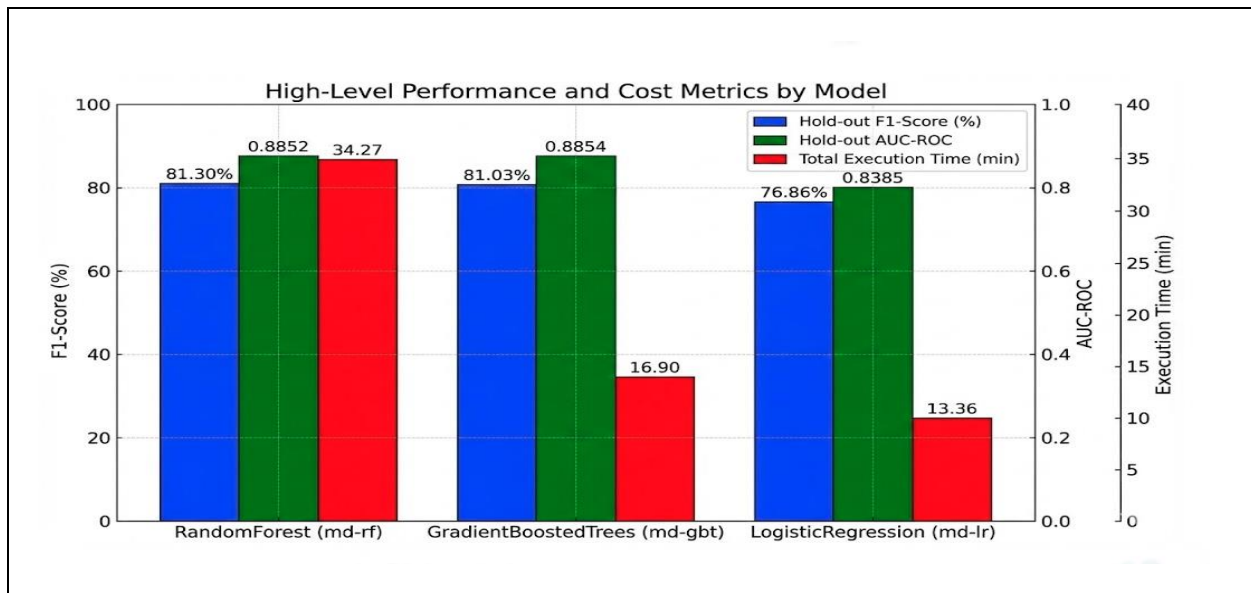
- **RandomForest** emerges as the top performer on F1-Score (81.30%) and Recall (82.89%), but its advantage over the GBT model is exceptionally narrow. This is the first indicator of a potential "point of diminishing returns," where additional computational complexity yields minimal predictive gain.
- **Gradient Boosted Trees** is the top performer on AUC-ROC (0.8854), again by a negligible margin over RF (0.8852), indicating that the two models have a virtually identical ability to distinguish between delayed and on-time flights across all classification thresholds.
- **Logistic Regression** shows a significant performance drop-off across all key metrics, confirming that a linear model is less suited for capturing the complex, non-linear relationships present in this dataset.

The **ROC Curves Comparison** plot visually corroborates these findings. The curves for GBT and RF are tightly clustered and significantly outperform the LR curve, confirming their superior and nearly indistinguishable discriminatory power.

Runtime and computational cost analysis

While the predictive performance of RF and GBT models is nearly identical, their computational costs are dramatically different. The analysis of execution time reveals GBT as the far more efficient solution.

Total Execution Time Breakdown (minutes)



An analysis of these computational costs highlights the following:

- The RandomForest model incurs a severe computational penalty, running 2.03 times slower than GBT and 2.56 times slower than LR in total execution time, making it the most resource-intensive option by a significant margin.
- The ML Pipeline stage is unequivocally the bottleneck, exposing the dramatic inefficiency in the Random Forest training process for this task. The RF model's training and evaluation process (20.98 minutes) is substantially more intensive than that of GBT (4.84 minutes) and LR (2.46 minutes).

This analysis concludes that the Gradient Boosted Trees model provides a far more efficient computational profile for achieving a performance level that is almost identical to that of the more costly RandomForest model.

8.6.3 Feature Importance deep analysis

Examining the feature importances assigned by each model provides critical insight into its underlying learning strategy. Even when top-line performance metrics are similar, as is the case with RandomForest and GBT, analyzing which features each model prioritizes can reveal fundamental differences in *how* they achieve their results. This deep analysis helps to move beyond simply what the models predict to understanding why they make those predictions.

RandomForest Feature profile

The RandomForest model distributes its predictive importance across a balanced mix of geographical, temporal, and weather-related features. This distributed approach suggests that its ensemble of independent trees captures signals from a wide array of predictive variables.

The top 5 most important features for the RandomForest model are:

1. indexed_DEST_AIRPORT_ID (13.04%)
2. indexed_ORIGIN_AIRPORT_ID (9.26%)
3. imputed_feature_flight_week_of_year_cos (7.16%)
4. imputed_feature_flight_week_of_year_sin (6.87%)
5. imputed_destination_weather_RelativeHumidity_Avg (4.92%)

Gradient Boosted Trees feature profile

The GBT model relies on a similar set of core features as the RandomForest model, but it distributes the importance very differently, concentrating its predictive power on a few key variables.

The top 5 most important features for the GBT model are:

1. `indexed_DEST_AIRPORT_ID` (22.11%)
2. `indexed_ORIGIN_AIRPORT_ID` (21.11%)
3. `imputed_feature_flight_week_of_year_cos` (10.85%)
4. `indexed_OP_CARRIER_AIRLINE_ID` (3.46%)
5. `imputed_origin_weather_press_change_abs_Max` (2.59%)

Comparative Importance Analysis

A direct comparison of the feature importance profiles for the RF and GBT models reveals a critical difference in their learning strategies.

Both models correctly identify the core predictive features, such as airport IDs, flight seasonality, and weather humidity. However, the key distinction lies in the **concentration of importance**.

- The **GBT** model concentrates over **43%** of its total predictive importance ($22.11\% + 21.11\%$) on the top two airport ID features alone.
- In contrast, the **RF** model assigns only **22.3%** ($13.04\% + 9.26\%$) of its importance to the same two features, demonstrating a much more distributed learning approach.

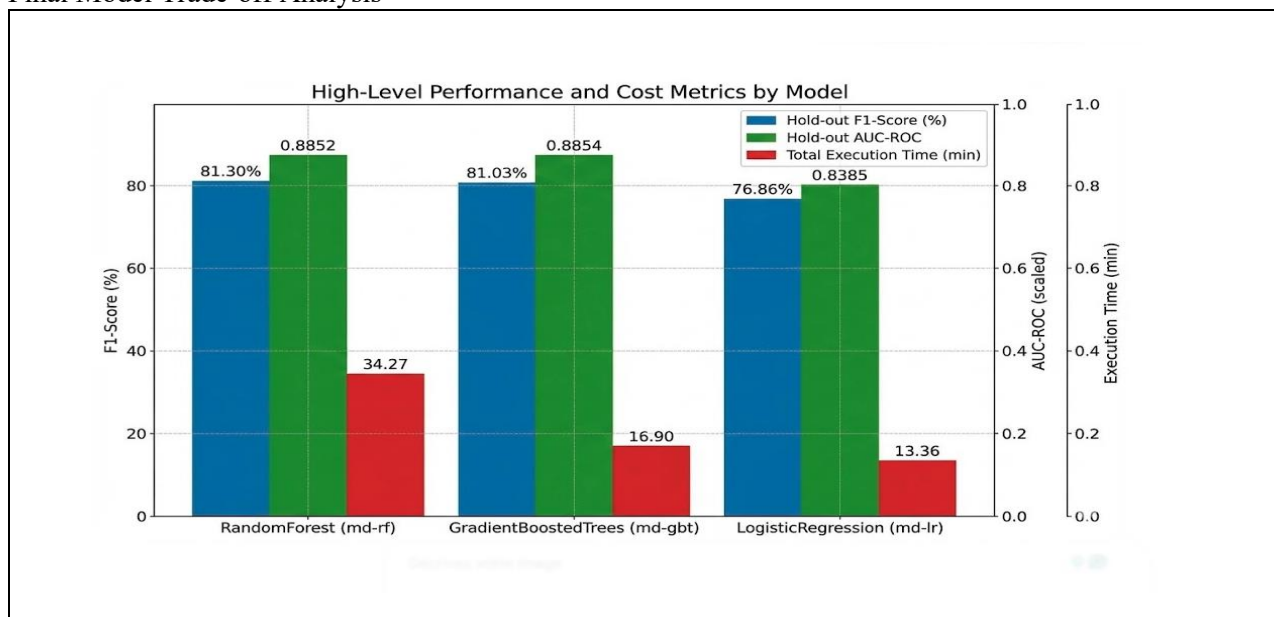
This finding can be interpreted as a direct consequence of their respective architectures. GBT's sequential boosting process builds a model that heavily relies on the strongest predictors to correct the errors of previous trees. In contrast, RF's ensemble of independent trees allows it to capture signals from a wider and more diverse array of secondary features, resulting in a more balanced distribution of importance. This architectural difference directly explains the cost disparity: GBT's sequential nature allows it to focus on the strongest predictors early on, leading to its computational efficiency. Conversely, RF's methodology of building numerous deep, independent trees is inherently more resource-intensive, which explains its **4.3x** longer ML Pipeline time (20.98 min vs. 4.84 min).

8.6.4 Synthesis and strategic conclusion

Head-to-head performance vs. cost trade-off

A final summary of the key trade-offs encapsulates the core decision-making criteria.

Final Model Trade-off Analysis



Cost-Benefit analysis

The cost-benefit analysis for selecting between the two top-performing models is unambiguous and decisive.

1. **Quantify the Gain:** The RandomForest model offers a marginal performance improvement of only **+0.27 percentage points** on the F1-Score over the GBT model.
2. **Quantify the Cost:** Achieving this negligible gain requires an additional **17.37 minutes** of runtime, which represents a **102.8% increase** in total execution time compared to GBT.
3. **The Verdict:** This cost-benefit trade-off is exceptionally poor. The substantial increase in computational resources and pipeline execution time required by the RandomForest model is not justified by its minuscule performance advantage, making it an inefficient choice for this task.

8.7 Lamsade-D2-60-X-Xexperience set: symmetric weather integration on the Lamsade cluster

This series of experiments mirrors the local symmetric weather depth study but is executed on the distributed Lamsade Spark cluster. The objectives of this analysis are twofold. First, it aims to determine the optimal symmetric weather depth for the D2 dataset in a distributed computing environment, identifying the point at which predictive performance is maximized. Second, it facilitates a critical comparison of model performance and computational cost between the Lamsade cluster and the local execution environment, providing key insights into the trade-offs of distributed machine learning for this specific task.

8.7.1 Global synthesis of experiments results

This section's purpose is to establish a high-level understanding of performance and cost trends as the symmetric weather depth is systematically increased on the Lamsade cluster. This overview will identify the peak-performing configuration and highlight the relationship between feature dimensionality and resource consumption.

The graph below, as well as the table that follows, summarize the key results of each experiment, from the baseline without weather data to the most feature-rich model.



The data reveals several important trends. A non-linear relationship exists for the F1-Score, which improves steadily before peaking at **76.75%** for the Experience-lamsade-d2-60-9-9 experiment and then slightly declining. This indicates an optimal signal-to-noise ratio at a 9-hour depth. This is contrasted with the AUC-ROC, which improves monotonically as more features are added, suggesting a continuous enhancement of the model's class separation power. Finally, the computational cost escalates significantly with weather depth, with total

execution time nearly tripling from **21.80 minutes** to **60.36 minutes**. This sets the stage for a more detailed analysis of the performance-cost trade-off.

8.7.2 Detailed Analysis of the symmetric weather experiments Family

This section provides a granular analysis of the experiments, leveraging the controlled methodology to isolate the impact of symmetric weather depth on performance, stability, and computational cost in the Lamsade cluster environment.

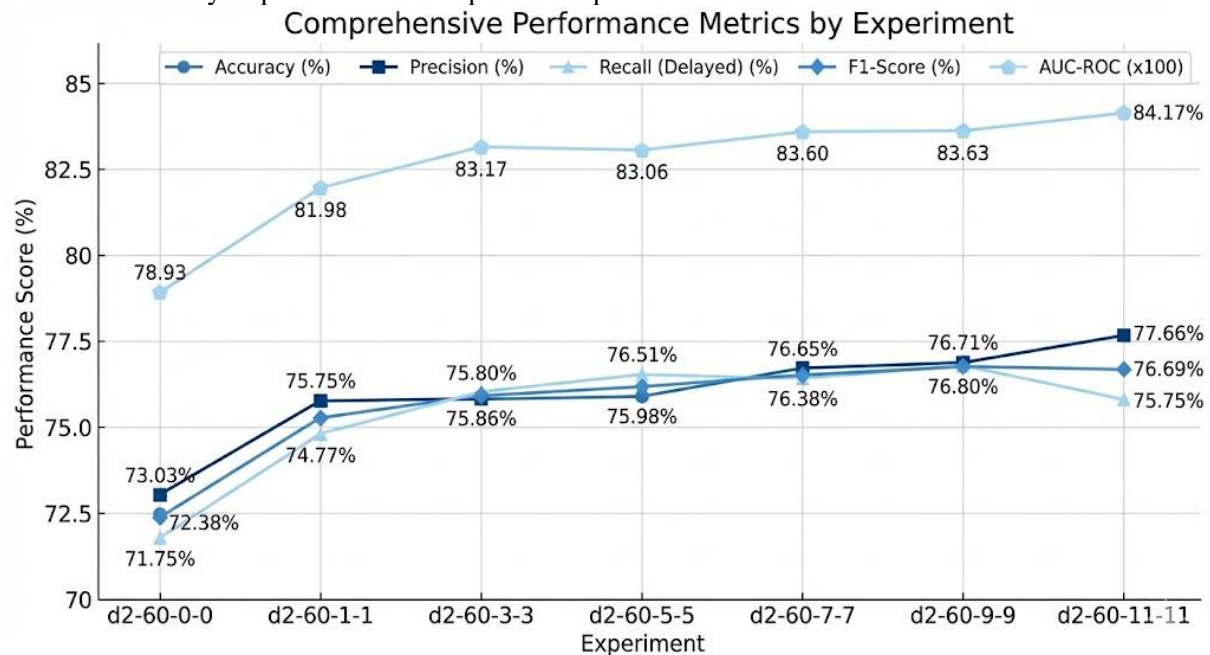
Methodology and controlled variables

The experimental design was rigorously controlled to isolate the effect of a single independent variable: symmetric weather depth. This parameter was varied from 0 hours (baseline) to 11 hours. All other critical parameters were held constant across all runs.

- **Execution Environment:** Lamsade Spark Cluster
- **Dataset:** D2
- **Model:** RandomForest
- **Model Hyperparameters:**
 - numTrees: 15
 - maxDepth: 7
 - minInstancesPerNode: 5
 - maxBins: 280

Predictive performance analysis

A comprehensive analysis of performance on the hold-out test set is essential for understanding how increasing feature dimensionality impacts the model's predictive power in a distributed environment.



A detailed analysis of these metrics reveals two distinct behavioral patterns:

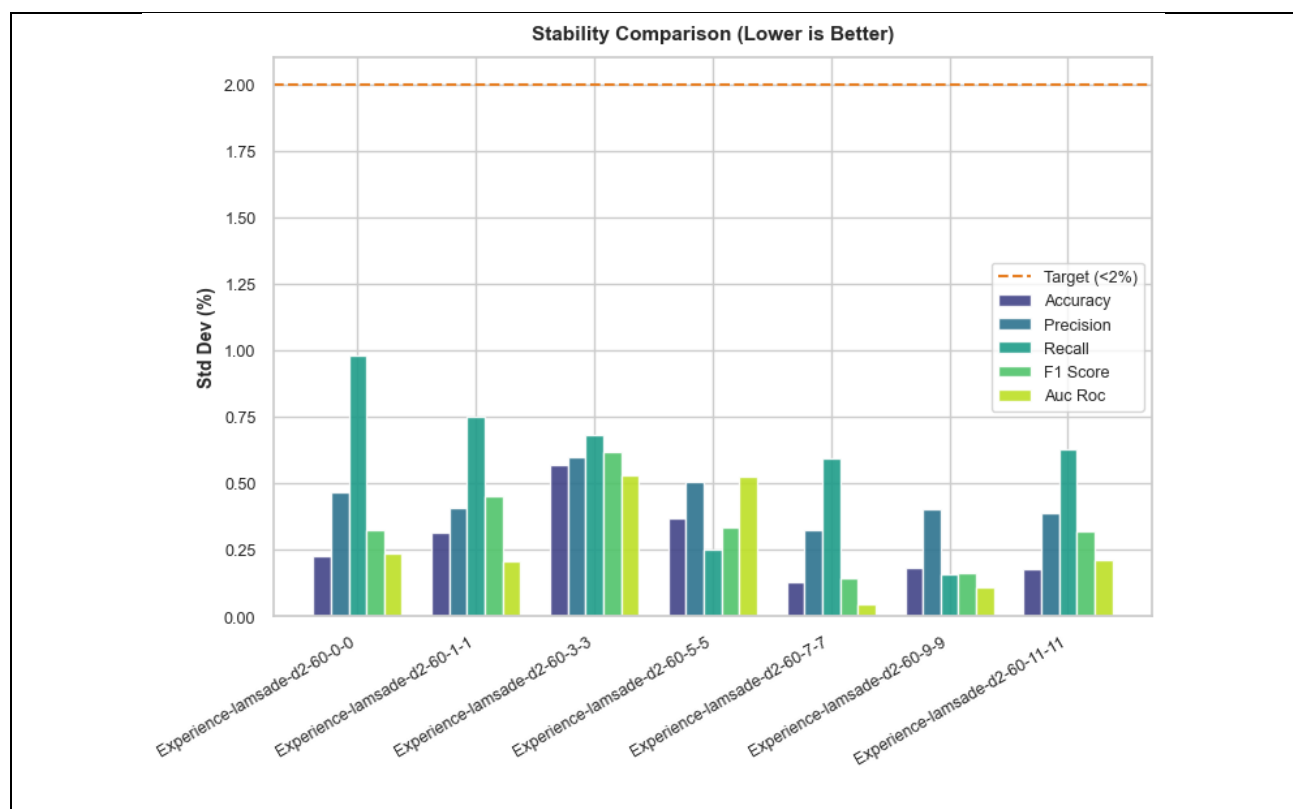
- **F1-Score:** The F1-Score, which balances precision and recall, demonstrates a clear peak performance of **76.75%** for the 9-9 experiment. This suggests that the 9-hour symmetric window provides the optimal signal-to-noise ratio. Beyond this point, adding more distant weather data does not improve, and in fact slightly degrades, the model's classification balance.
- **AUC-ROC:** In contrast, the AUC-ROC shows a consistent, monotonic improvement, rising with each increase in weather depth and reaching a maximum of **0.8417** for the 11-11 experiment. This signifies that the model's fundamental ability to distinguish between delayed and on-time flights continues to

improve even after the F1-Score has peaked, highlighting the difference between optimizing for class separation versus a specific classification threshold.

Cross-validation stability analysis

Model stability, defined as low variance across cross-validation folds, is a critical indicator of a model's robustness and its ability to generalize.

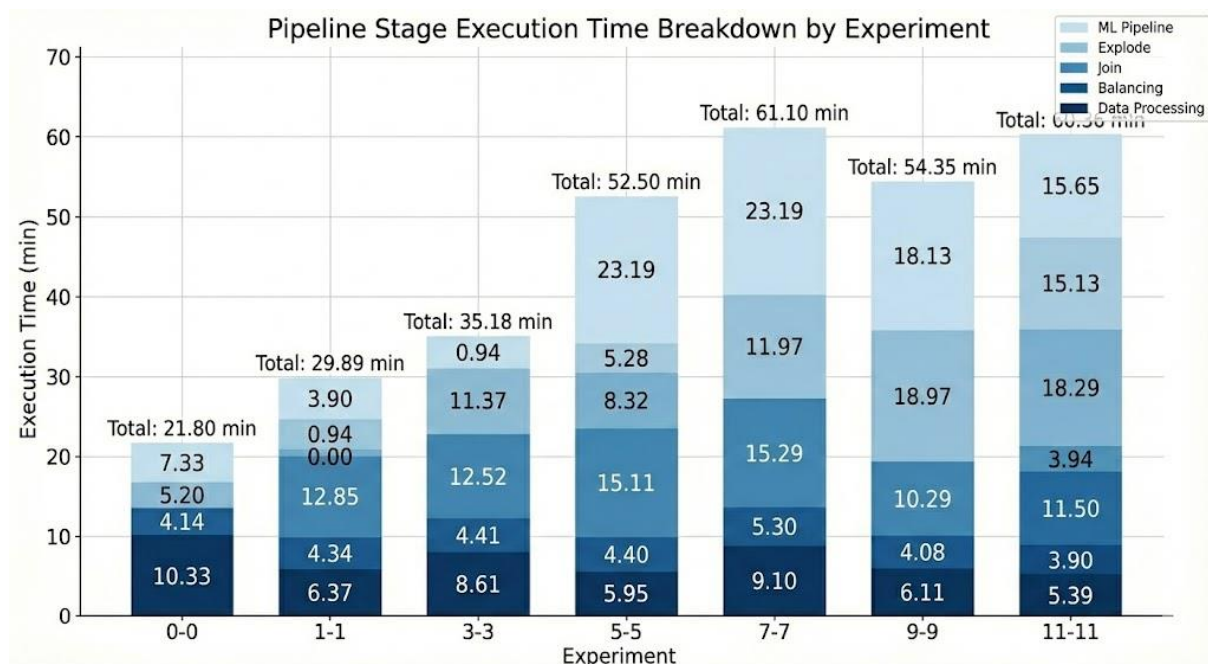
Experiment	Recall Std Dev (%)	F1-Score Std Dev (%)	AUC-ROC Std Dev (%)
d2-60-0-0	0.98%	0.32%	0.0023
d2-60-1-1	0.75%	0.45%	0.0021
d2-60-3-3	0.68%	0.62%	0.0053
d2-60-5-5	0.25%	0.34%	0.0053
d2-60-7-7	0.59%	0.14%	0.0005
d2-60-9-9	0.16%	0.16%	0.0011
d2-60-11-11	0.63%	0.32%	0.0021



The stability results indicate that all models are highly stable, with standard deviations well below the 1% threshold. This suggests that the underlying learning process is robust to variations in the training data. Critically, the top-performing d2-60-9-9 model also exhibits excellent stability, with an F1-Score standard deviation of only **0.16%**, reinforcing its standing as a strong candidate model.

Runtime and computational cost analysis

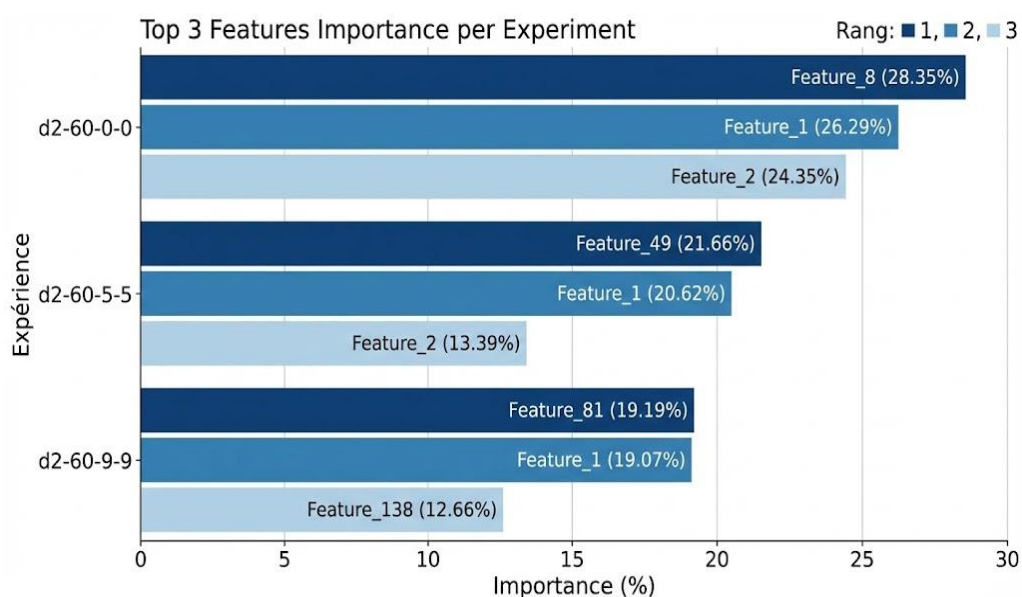
Analyzing computational costs in a distributed environment is essential for understanding scalability and operational feasibility. This analysis identifies the primary bottlenecks in the pipeline.



The cost analysis reveals a massive increase in total runtime as weather depth increases, driven primarily by the growing complexity of data processing. The Explode stage, which is responsible for unnesting the time-series weather data, emerges as the primary bottleneck. Its runtime grows exponentially from effectively zero minutes in the baseline experiment to over **23 minutes** (1391.51 s) in the 11-11 experiment, demonstrating that this operation is the most significant scalability challenge in the pipeline.

8.7.3 Feature Importance deep analysis

This analysis explores the evolution of feature importances to understand the model's learning process as the dimensionality of the feature space expands. By tracking which features the model prioritizes, we can see how it integrates new weather information.



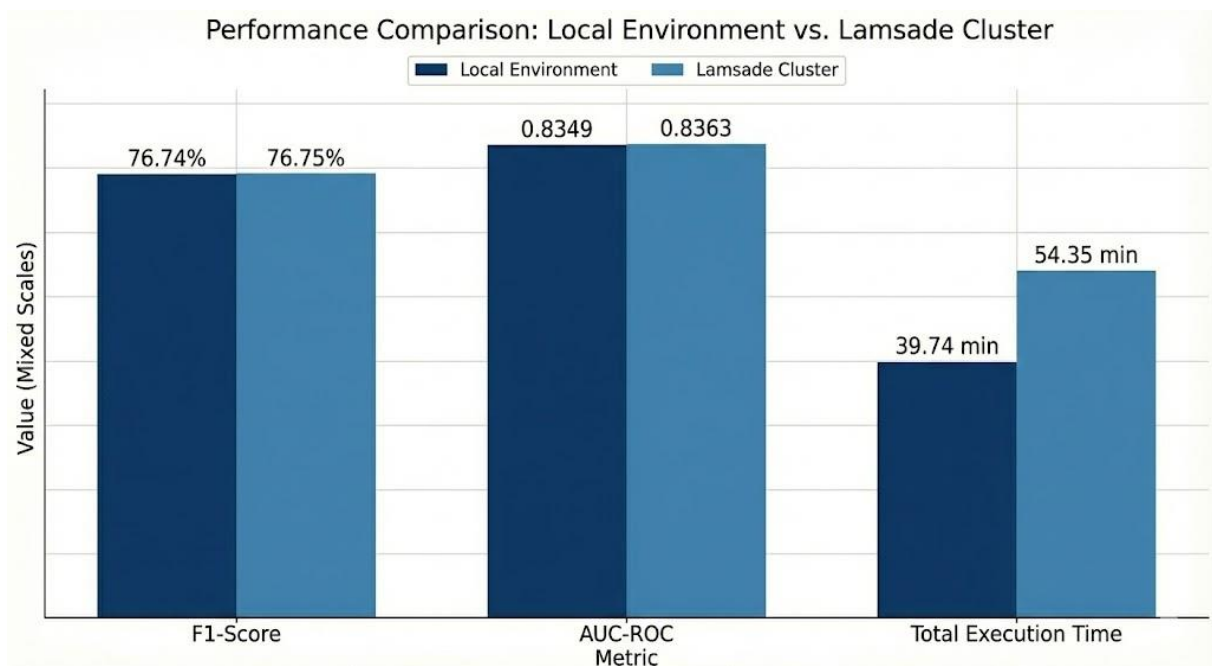
The evolution of feature importance tells a clear story of effective learning. The baseline 0-0 model relies on simple, non-weather features such as Feature_8, Feature_1, and Feature_2. As symmetric weather data is introduced, the model begins to incorporate new, highly predictive weather-derived signals. By the peak-performing 9-9 model, the top features are a sophisticated blend of foundational non-weather features (Feature_1, Feature_2) and

highly predictive weather features (Feature_81, Feature_138, and Feature_179 at rank 5). This demonstrates that the model is not merely replacing old predictors but is effectively integrating the new information to build a more nuanced and powerful predictive model.

8.7.4 Comparative analysis: Lamsade cluster vs. Local environment

This section provides a critical comparison between executing experiments on the Lamsade distributed cluster versus a local machine. By comparing the performance and runtime for the analogous D2-60-9-9 experiment—the peak performer in both environments—we can evaluate the practical trade-offs of using a distributed system for this task. The results for the local environment are drawn from the analysis in Section 4.4 of this report and are presented here alongside the Lamsade results for a direct comparison.

Performance and Cost: Lamsade vs. Local (D2-60-9-9)



Overall Interpretation:

The flight delay prediction model shows **consistent performance (F1-Score, AUC-ROC)** but **significant speed differences** between a Local Environment (Docker, no HDFS) and a Distributed Cluster (Lamsade, HDFS). **Key Facts & Analysis:**

- **Model Performance is Consistent:** F1-Score (Local: 76.74%, Cluster: 76.75%) and AUC-ROC (Local: 0.8349, Cluster: 0.8363) are virtually identical, confirming the model's reliability across architectures.
- **Execution Time Difference is Substantial:** The Local Environment (39.74 minutes) completes the task **36.8% faster** than the Lamsade Cluster (54.35 minutes).
- **Source of Delay:** The Cluster's slower time is attributed to **HDFS overhead**: network I/O, distributed coordination, data replication, and network latency.

Conclusion & Practical Implications:

Model quality is **robust** regardless of storage. However, the **Local Environment is substantially faster** due to the absence of HDFS overhead.

- **Local/Docker is best for:** Rapid development, testing, and small-to-medium workloads where speed is prioritized.

- **Lamsade/HDFS is essential for:** Large-scale deployments, fault tolerance, and datasets exceeding single-node capacity, accepting a **measurable performance cost**

Organizations must weigh the speed advantage of local storage against the scalability and reliability features of HDFS.

8.7.5 Strategic conclusion

This comprehensive analysis of the Lamsade cluster experiments yields several critical strategic insights. First, the 9-hour symmetric weather depth emerges as the optimal configuration, maximizing F1-Score performance at 76.75% while maintaining exceptional model stability. Second, the comparison between local and distributed environments demonstrates that predictive quality remains consistent regardless of infrastructure, with near-identical F1-Scores and AUC-ROC values. However, the Lamsade cluster introduces a measurable 36.8% performance penalty attributable to HDFS overhead—network I/O, replication, and distributed coordination costs absent in local execution.

These findings lead to clear operational recommendations: deploy the local Docker environment for rapid development cycles, model experimentation, and workloads fitting within single-node capacity, where the 37% speed advantage translates to faster iteration and reduced development time. Conversely, the Lamsade cluster's distributed architecture becomes essential for production systems requiring fault tolerance, high availability, or processing datasets exceeding local storage capacity, where the performance trade-off is justified by scalability and reliability guarantees. The 9-hour weather depth configuration should serve as the production standard, offering the best performance-cost ratio validated across both environments

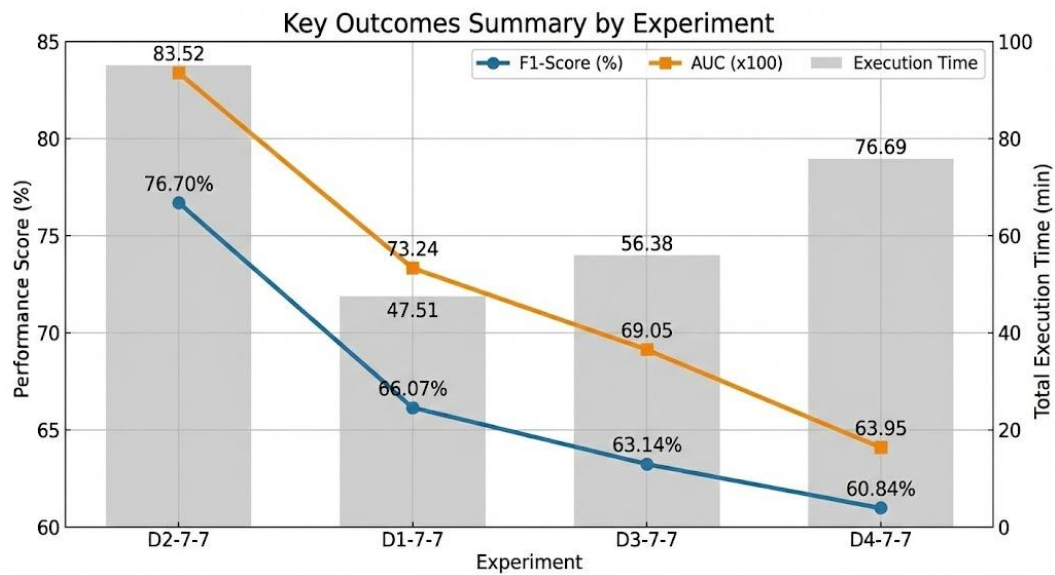
8.8. Lamsade-Dx-7-7 Dataset comparison: Performance analysis on the Lamsade cluster

This section details a series of experiments conducted on the Lamsade Spark cluster, designed to systematically evaluate how model performance, computational cost, and learning behavior change when trained on four distinct datasets: D1, D2, D3, and D4. To isolate the impact of the dataset itself, all other variables—including the model architecture (RandomForest), its hyperparameters, and the depth of weather data integration (7 hours symmetric)—were held constant. This analysis is critical for identifying and selecting the optimal dataset for production deployment, balancing predictive power against operational resource consumption.

8.8.1 Global synthesis of experimental results

This subsection provides a high-level comparative overview of the four dataset experiments. The objective is to identify broad trends in predictive performance and computational cost, establishing a foundational understanding before proceeding to a more granular analysis of the results.

The image below summarizes the key outcomes for each experiment:



The results clearly indicate the superior performance of the Experience-lamsade-D2-7-7 experiment, which surpasses all others across both F1-Score and AUC. This is in sharp contrast to the significant performance degradation observed when using the D3 and D4 datasets; while D3 and D4 lagged by over 13 and 15 percentage points on F1-Score, respectively, rendering them non-viable for production. The data also highlights a substantial variation in computational cost, with the top-performing D2-7-7 experiment requiring the longest execution time at 97.83 minutes.

The following subsections will dissect these high-level results to understand the underlying drivers of both predictive performance and computational cost.

8.8.2 Detailed performance and cost analysis

This section provides a detailed examination of the experiments, attributing the observed differences in model performance and runtime directly to the dataset, which was the sole independent variable.

Methodology and controlled variables

A rigorously controlled methodology was employed for this analysis. The single independent variable was the dataset, which was varied across four configurations: D1, D2, D3, and D4.

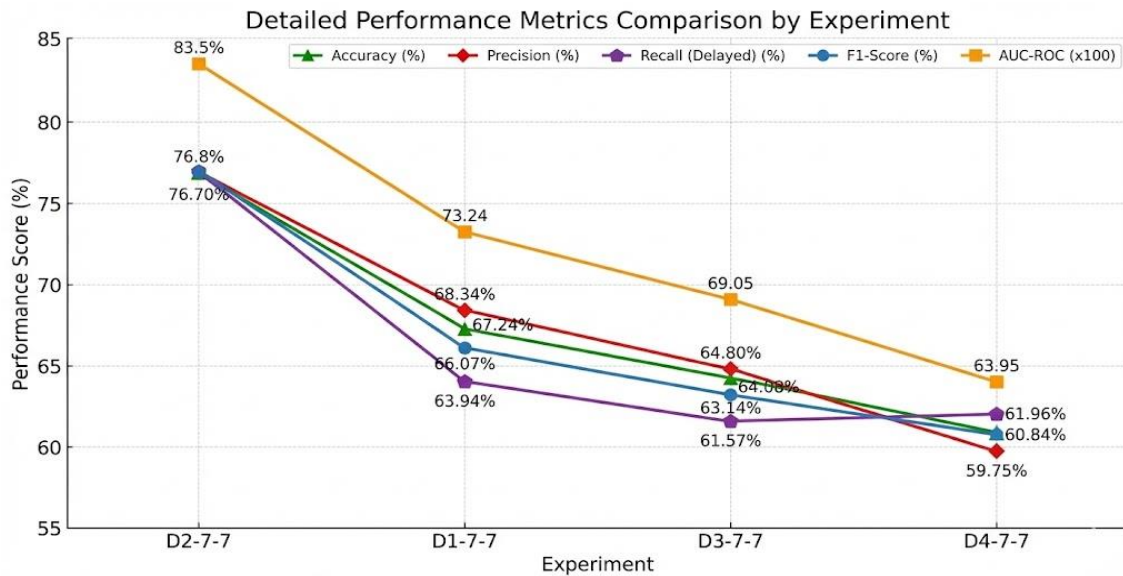
All other critical parameters were held constant across the four experimental runs:

- **Execution Environment:** Lamsade Spark Cluster
- **Model:** RandomForest
- **Model Hyperparameters:**
 - numTrees: 15
 - maxDepth: 7
 - maxBins: 280
 - minInstancesPerNode: 5

Predictive performance analysis

A comprehensive comparison of performance on the hold-out test set reveals the definitive impact of dataset selection on the model's predictive capabilities. The D2 dataset enabled the model to achieve a level of accuracy that was unattainable with the other datasets.

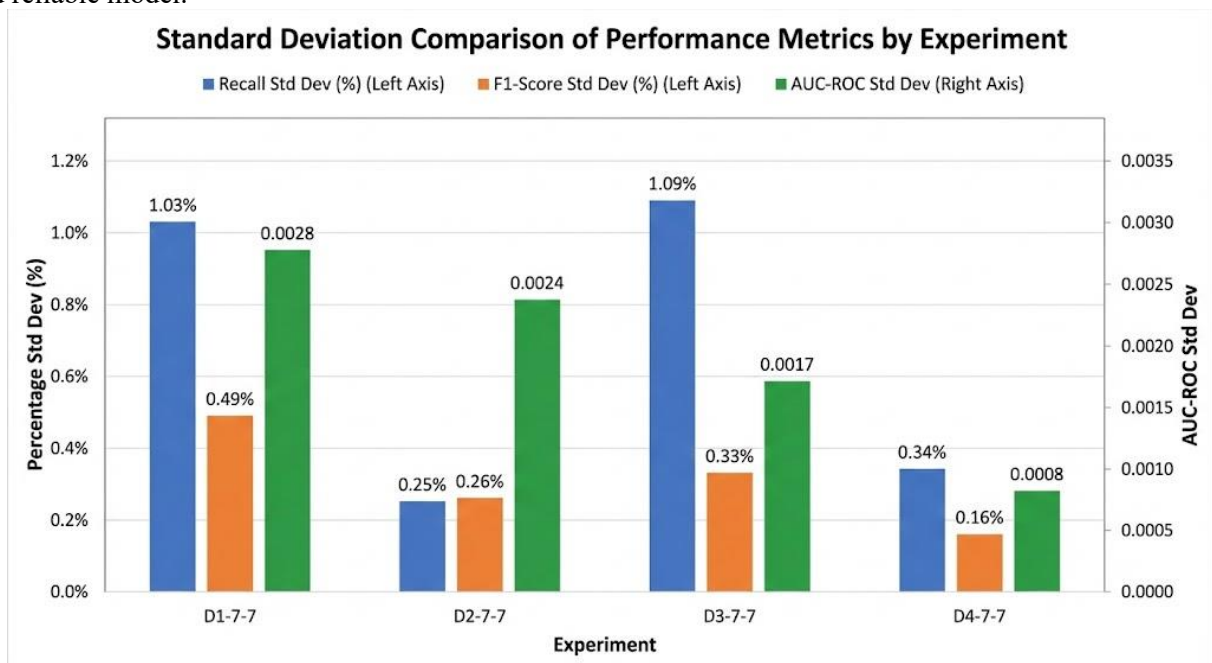
Hold-out Test Set Performance Comparison by Dataset



The analysis of these metrics highlights the clear superiority of the model trained on the D2 dataset. The D2-7-7 model achieved an F1-Score of **76.70%** and an AUC-ROC of **0.8352**, establishing a significant performance gap of over 10 percentage points in F1-Score compared to the next-best model trained on the D1 dataset (66.07%). The models trained on D3 and D4 performed even more poorly. The performance heatmap further corroborates this conclusion, visually demonstrating that the D2-7-7 experiment outperforms the others across nearly every reported metric. This performance is not merely higher; it is also more robust. As shown in the subsequent stability analysis (5.2.3), the D2 model achieves this with excellent cross-validation stability, and the feature importance analysis (5.3) reveals it leverages a more balanced set of predictive signals compared to the other datasets.

Cross-validation stability analysis

Model stability, measured by the standard deviation of performance metrics across cross-validation folds, is a key indicator of a model's ability to generalize to unseen data. A lower standard deviation suggests a more robust and reliable model.

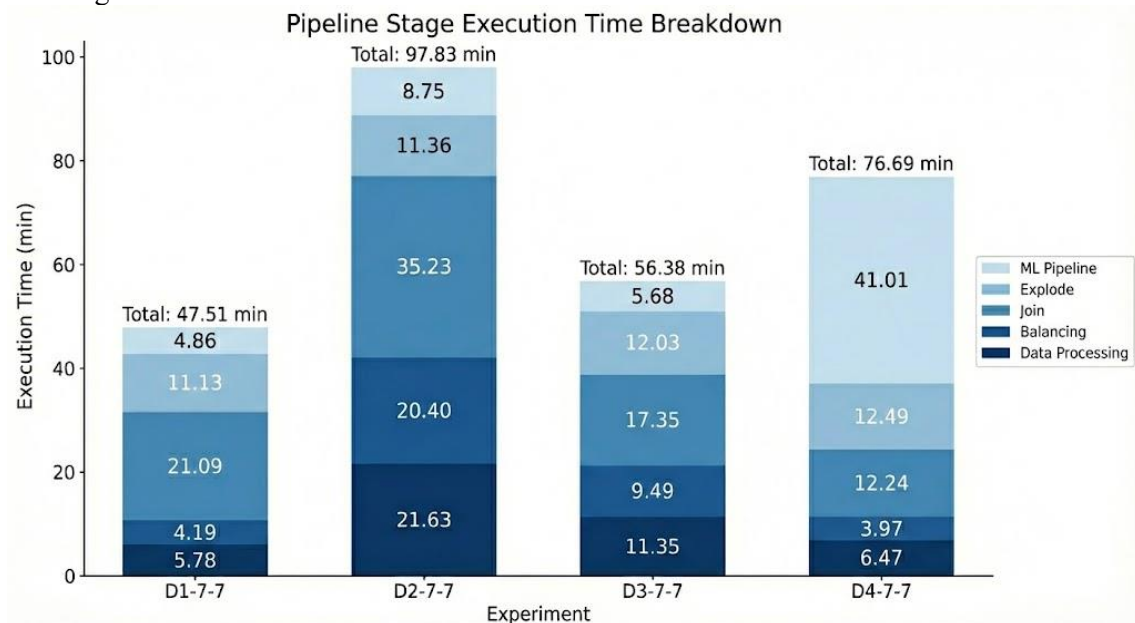


The stability analysis reveals that all models exhibit low variance, with standard deviations for all percentage-based metrics well below the 1.10% mark, indicating good stability and generalization potential. Critically, the

top-performing D2-7-7 model also maintains excellent stability, with an F1-Score standard deviation of just **0.26%**. This confirms that its superior performance is not an artifact of a particular data split but is a consistent and robust outcome.

Runtime and computational cost analysis

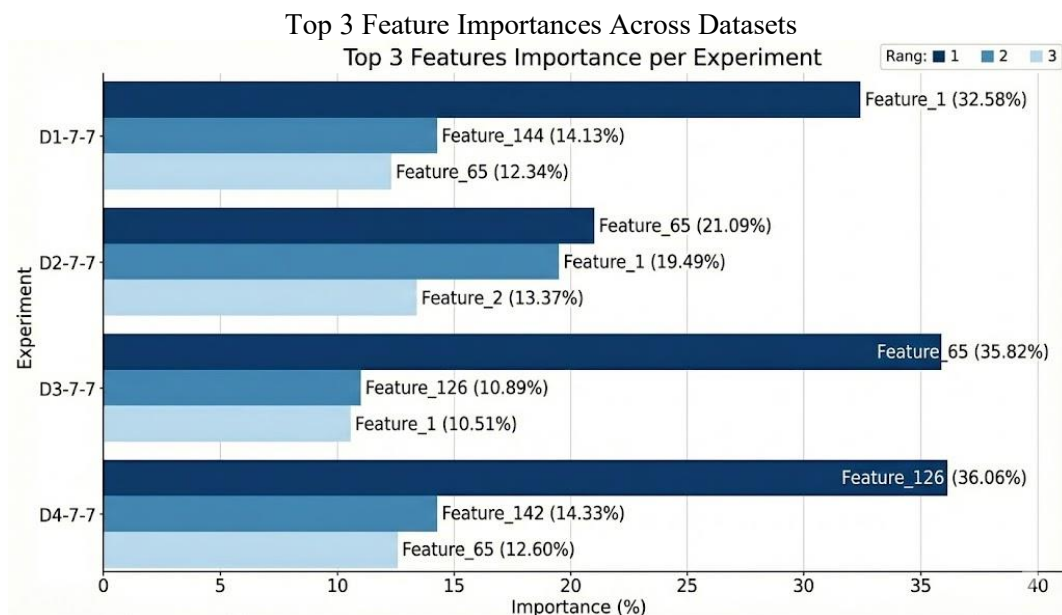
Evaluating computational cost is a critical factor for operational deployment, as it directly impacts resource allocation and pipeline scheduling. The following analysis breaks down the total execution time into its major constituent stages.



The cost breakdown clearly shows that the D2-7-7 experiment is by far the most computationally expensive, requiring **97.83 minutes** to complete. The significantly longer Balancing and Join times for D2 suggest that this dataset is either substantially larger after initial processing or has a more complex key distribution, making the join-shuffle operations more intensive for the Spark engine. In contrast, the bottleneck in the D4 ML Pipeline suggests its data, while perhaps smaller, is harder for the RandomForest algorithm to find optimal splits for, leading to longer tree-building times.

8.8.3 Cross-dataset feature importance analysis

To understand *why* predictive performance varies so dramatically between datasets, this subsection investigates what the model learns from each one. By analyzing the top-ranked features, we can infer how the underlying data distribution influences the model's decision-making process.



The graph reveals how each model prioritizes variables for its predictions.

Key Findings:

1. **Dominance of Feature_65 and Feature_1:** These two variables consistently appear in the top 3 across all experiments, indicating their strong universal predictive power.
2. **Shifting Priorities:** Rankings vary across configurations. Feature_1 dominates D1, Feature_65 takes precedence in D2-D3, while Feature_126 emerges as the leader in D4.
3. **Critical Factor - Balance vs. Concentration:**
 - a. D1, D3, D4: Excessive dependence on a single feature (>35% for the primary feature), creating vulnerability to noise or missing data
 - b. D2: Balanced distribution (21%, 19%, 13% for top 3), leveraging multiple information sources

Conclusion: D2's balanced feature importance likely explains its superior performance, reflecting a more robust and generalizable model compared to configurations that rely heavily on a single variable.

8.8.4 Strategic conclusion on dataset selection

The comprehensive analysis presented in this section leads to a clear and decisive strategic conclusion. Despite its significantly higher computational cost—nearly double that of the next-best configuration—the **D2 dataset provides a vastly superior predictive model**, establishing a performance gap of 10.63 percentage points on the F1-Score over the next-best dataset. The stability and feature analysis further confirm that the D2 dataset contains more robust and reliable predictive signals.

Therefore, it is strongly recommended that the D2 dataset be adopted as the standard for all future model development and deployment. Its proven ability to generate the most accurate and reliable predictions makes it the unequivocal choice for maximizing the operational value of the flight delay prediction system. The next section will build upon this finding by using the D2 dataset to explore the optimal integration of weather data in the Lamsade cluster environment.

8.9 Scalability experiments (Lamsade & Dataproc)

Scalability is a critical performance characteristic in distributed computing systems like Apache Spark. It measures how efficiently a system can handle increasing workloads by adding more computational resources. In the context of our flight delay prediction application, understanding scalability helps us determine the optimal resource allocation and predict how the system will perform as data volumes grow or computational demands increase.

This analysis examines the relationship between the number of executors deployed and the overall processing speedup achieved. Ideally, in a perfectly scalable system, doubling the number of executors would double the performance (linear scalability). However, real-world distributed systems face various bottlenecks that prevent perfect linear scaling, including network communication overhead, data shuffling costs, resource contention, and inherently sequential portions of the workload as described by Amdahl's Law.

This section outlines the strategic experimentation conducted to rigorously assess the flight delay prediction pipeline's performance, scalability, and stability. The experiments were executed across two distinct computational environments: Google Cloud Dataproc and the Lamsade Cluster.

To specifically evaluate the Spark application's scalability, we systematically varied the number of executors. The executor count ranged from 2 to 4 on Google Cloud Dataproc (restricted by cloud limitations) and from 2 to 12 on the Lamsade Cluster.

The combined results from these two clusters will provide crucial, data-driven insights. These findings are essential for formulating clear recommendations on the optimal resource allocation and environment selection for all future development and deployment phases.

8.9.1 Dataproc Horizontal scalability analysis

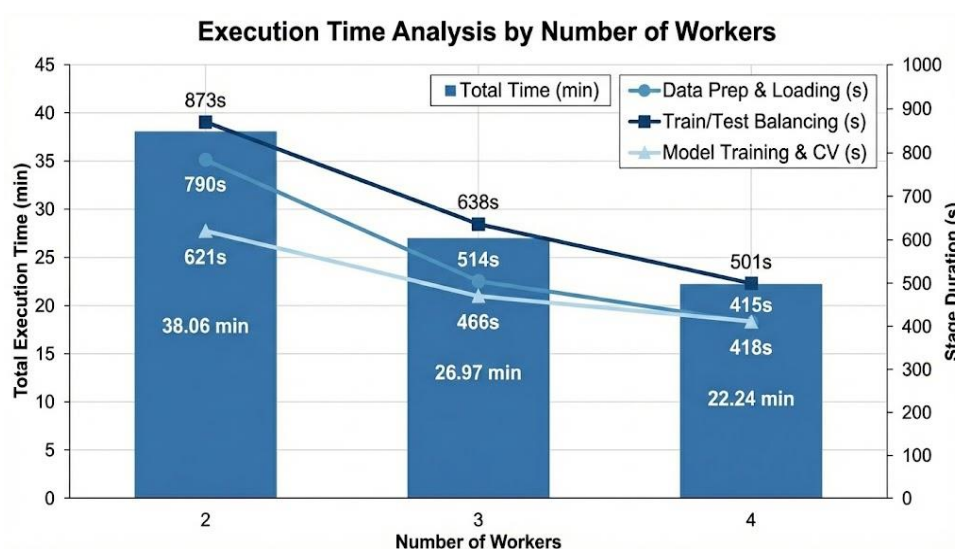
This analysis isolates the impact of horizontal scaling by systematically increasing the number of Spark workers within a controlled, managed cloud environment. The primary objective is to quantify the relationship between computational resources, pipeline execution time, and final model performance for the baseline workload. By observing how the system behaves under increasing parallelism, we can identify the optimal balance between infrastructure cost and processing speed.

Methodology and Controlled variables

To ensure a direct and unbiased comparison, the experimental setup was rigorously controlled. The baseline model (Experience-dataproc-d2-60-0-0), which does not integrate weather features, was executed on Google Dataproc clusters configured with 2, 3, and 4 worker nodes, respectively. All other parameters—including the D2 dataset, model architecture (RandomForest), and key hyperparameters—were held constant across all three runs. This approach guarantees that any observed differences in runtime or behavior can be directly attributed to the change in the number of available workers.

Execution time and performance scaling

The following figure presents the comprehensive results of this scalability study



The results demonstrate a clear, though non-linear, relationship between the number of workers and the total pipeline execution time.

Scaling from 2 to 4 workers yielded a substantial reduction in total execution time of 15.82 minutes, a speedup of approximately 41.6%. However, a critical evaluation of the scaling efficiency reveals a pattern of diminishing returns. Adding the third worker reduced the runtime by a significant 11.09 minutes. In contrast, adding the fourth worker saved only an additional 4.73 minutes. This indicates that while the workload benefits from increased parallelism, the marginal performance gain decreases with each added node. The speedup was systemic, with all major pipeline stages—Data Prep & Loading, Train/Test Balancing, and the Model Training & CV—showing significant reductions in duration as more computational resources were added.

Impact on model accuracy and stability

As expected, scaling the cluster horizontally had no effect on the final predictive accuracy of the model.

Metric	Value (All Configurations)
Hold-out F1-Score	73.02%
Hold-out AUC-ROC	0.7929

The identical hold-out test metrics across all three configurations confirm the deterministic nature of the data processing, feature engineering, and training logic. This consistency is a crucial indicator of a reliable and reproducible pipeline, as the final model output is independent of the degree of parallelism used during its creation. Furthermore, all three runs completed successfully without critical errors or failures, demonstrating that the Google Dataproc environment robustly and reliably handled the baseline workload at all tested scales.

Strategic conclusion for Dataproc

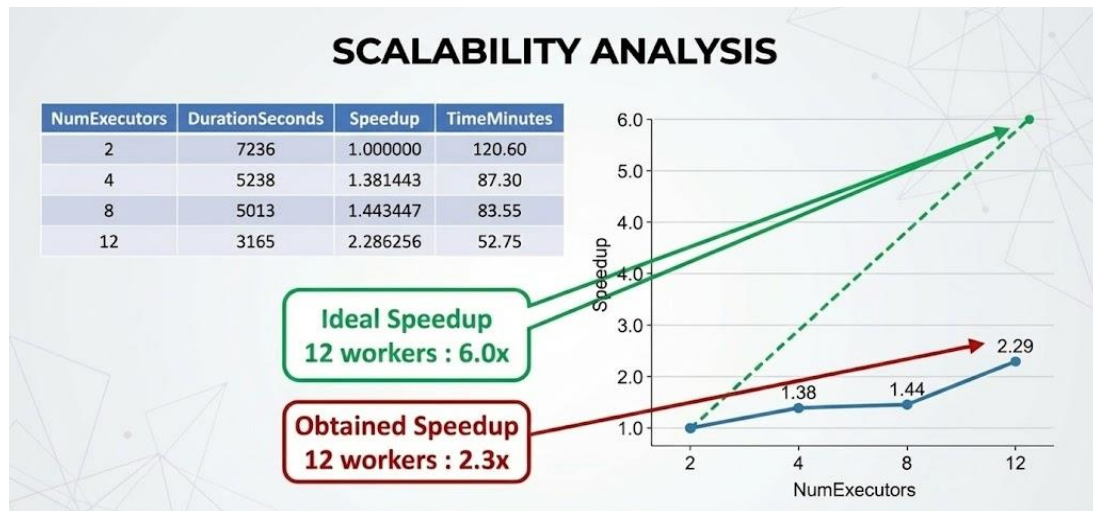
The Dataproc scalability analysis confirms that horizontal scaling provides a clear, but non-linear, reduction in pipeline execution time. The findings highlight a direct trade-off between infrastructure cost (i.e., the number of worker nodes) and processing speed. Based on the observed diminishing returns, the **3-worker configuration represents a compelling "sweet spot"**. It offers a substantial performance improvement of over 11 minutes compared to the 2-worker setup while avoiding the diminishing returns and lower cloud spend efficiency of the 4-worker configuration.

8.9.2 Lamsade cluster stability and performance analysis

The initial experiment focused on analyzing the scalability of a Spark application as more executors were added to handle the flight delay prediction workload. This study assessed how well the system scaled.

The chosen task, Experience-lamsade-D4-9-9, was designed to push the environment's limits by integrating a deep, symmetric 9-hour window of weather data from both the origin and destination airports. The main objective was to determine the stability and performance of the environment under the strain of extensive data joins and a massively expanded feature set.

To evaluate the scalability of our Spark application, we conducted a series of experiments varying the number of executors from 1 to 12 while processing the same flight delay prediction workload. The following figure presents the comprehensive results of this scalability study. The following figure presents the comprehensive results of this scalability study.



Performance metrics analysis

The experimental data reveals several key performance characteristics of our distributed system:

- **Baseline Performance:** With a single executor, the application required 7,236 seconds (approximately 120.6 minutes) to complete the entire workload. This establishes our performance baseline against which all improvements are measured.
- **Initial Scaling (1 to 4 executors):** Increasing to 4 executors reduced execution time to 5,238 seconds (87.3 minutes), yielding a speedup of 1.38x. While this represents a 28% improvement, it falls significantly short of the theoretical 4x speedup expected from quadrupling resources.
- **Moderate Scaling (4 to 8 executors):** Doubling the executors from 4 to 8 produced only marginal improvements, with execution time decreasing to 5,013 seconds (83.55 minutes) for a speedup of 1.44x. The minimal gain of 0.06x despite doubling resources indicates significant scalability barriers emerging at this level.
- **Higher Parallelism (12 executors):** The maximum configuration tested used 12 executors, achieving an execution time of 3,165 seconds (52.75 minutes) and a speedup of 2.29x. Although this represents the best performance observed, the gap between the achieved 2.29x and the ideal 6x speedup (indicated by the green dashed line) demonstrates substantial efficiency losses.

Scalability limitations

The divergence between ideal and actual performance curves reveals several fundamental limitations affecting our distributed application:

- **Communication Overhead:** As the number of executors increases, inter-executor communication becomes more complex and costly. Data must be exchanged during shuffle operations (particularly during joins between flight and weather datasets), and this communication overhead grows non-linearly with the number of participating executors.
- **Data Shuffling Bottleneck:** The join operations between large datasets require extensive data redistribution across the cluster. Network bandwidth becomes a limiting factor, and the time spent shuffling data can outweigh the benefits of additional parallelism. This is particularly evident in the diminishing returns observed between 4 and 8 executors.
- **Resource Contention:** Multiple executors compete for shared resources including network bandwidth, HDFS I/O capacity, and memory bandwidth on worker nodes. This contention increases coordination overhead and reduces the effective utilization of each additional executor.
- **Sequential Components (Amdahl's Law):** Certain portions of the workload cannot be parallelized effectively, such as driver-side operations, final aggregations, and data serialization/deserialization. According to Amdahl's Law, these sequential components impose a fundamental limit on maximum achievable speedup, regardless of the number of processors added.

- **Task Scheduling Overhead:** The Spark scheduler must coordinate an increasing number of tasks across more executors. This scheduling overhead, including task serialization, distribution, and result collection, becomes more significant as parallelism increases. Based on the comprehensive analysis of scalability experiments on both Dataproc and the Lamsade Cluster, this section concludes with critical findings and recommendations for the optimal resource strategy for the flight delay prediction pipeline.

Synthesis of Key Findings:

1. **Non-Linear Scaling and Diminishing Returns:** Both experimental environments—the restricted Dataproc setup (2 to 4 workers) and the expanded Lamsade setup (1 to 12 executors)—demonstrated a clear, but non-linear, reduction in execution time with increased parallelism. This highlights the practical limitations described by Amdahl's Law and the growing impact of distributed computing bottlenecks.
2. **Identification of the "Sweet Spot" (Dataproc):** In the Dataproc environment, the **3-worker configuration** was identified as the optimal trade-off. It provided a substantial performance boost (41.6% speedup from 2 to 4 workers, with the most significant gain realized by the third worker) while maintaining better infrastructure cost efficiency than the 4-worker setup, which showed noticeable diminishing returns.
3. **Scalability Barriers (Lamsade Cluster):** The Lamsade experiments, which pushed parallelism further, confirmed significant scalability barriers. The maximum observed speedup was only **2.29x with 12 executors**, far short of the ideal 6x speedup. This performance gap is attributed primarily to **data shuffling bottlenecks** during extensive joins of flight and weather data, as well as increasing **communication overhead** and **resource contention** across the cluster.
4. **Consistency in Model Accuracy:** Crucially, horizontal scaling had no impact on the final model's predictive performance (F1-Score and AUC-ROC remained constant), confirming the **reproducibility and reliability** of the Spark pipeline logic across varying degrees of parallelism.

Strategic Conclusion and Recommendations:

The results indicate that while increasing parallelism is effective up to a point, the flight delay prediction workload is increasingly limited by sequential components and data shuffling costs, especially with the complex feature engineering required for weather data integration.

For production or large-scale development deployments, the following is recommended:

- **Optimal Cloud Strategy (Cost/Performance):** Leverage a **3-worker configuration** on a managed environment like Dataproc for baseline workloads, as it offers the best balance between execution speed and cloud infrastructure cost.
- **Mitigation of Scaling Limits:** For workloads involving extensive data joins (e.g., integrating deep weather windows), future efforts should focus on **optimizing data partitioning, reducing shuffle operations**, and evaluating more efficient join strategies rather than simply scaling out the cluster horizontally. Addressing these inherent data bottlenecks is the key to unlocking further performance improvements beyond the current scalability limit.

8.9.3 Strategic conclusion and recommendations

The results indicate that while increasing parallelism is effective up to a point, the flight delay prediction workload is increasingly limited by sequential components and data shuffling costs, especially with the complex feature engineering required for weather data integration.

For production or large-scale development deployments, the following is recommended:

- **Optimal Cloud Strategy (Cost/Performance):** Leverage a **3-worker configuration** on a managed environment like Dataproc for baseline workloads, as it offers the best balance between execution speed and cloud infrastructure cost.
- **Mitigation of Scaling Limits:** For workloads involving extensive data joins (e.g., integrating deep weather windows), future efforts should focus on **optimizing data partitioning, reducing shuffle operations**, and evaluating more efficient join strategies rather than simply scaling out the cluster horizontally. Addressing these inherent data bottlenecks is the key to unlocking further performance improvements beyond the current scalability limit.

8.10 Synthesis and recommendations

This section serves as the capstone of the entire experimental analysis. Its purpose is to consolidate the key findings from over 100 experiments—spanning seven distinct families—into a unified set of actionable, data-driven recommendations for the flight delay prediction system. By synthesizing the complex interactions between data, model architecture, and computational environments, we can establish a clear, evidence-based roadmap for production deployment and future development.

8.10.1 Summary of key findings: from data to decisions

This subsection distills the most critical, high-impact findings from each major experimental thrust. These empirically derived conclusions serve as the evidentiary foundation for the strategic recommendations that follow, connecting raw experimental data to decisive operational strategy.

Finding Weather Integration is crucial, but nuanced

Integrating meteorological data provides a massive performance lift over the baseline model. However, the source of this data matters significantly. Origin weather (Section 3.4) proved to be a more potent predictor than destination weather (Section 2). Furthermore, standard symmetric models incorporating both sources (Section 4) offered only marginal performance gains for more than double the runtime cost, highlighting a clear point of diminishing returns.

Finding 2: The optimal data configuration

The exhaustive evaluation of datasets and weather integration strategies has revealed a definitive optimal configuration for the input data. The **D2 dataset consistently produced the most accurate and robust models**, achieving a top F1-Score of 76.70% in the formal dataset comparison (Section 7.2.2), which dramatically outperformed the D1, D3, and D4 configurations. Concurrently, the analysis of weather data's impact showed a clear evolution. While origin-only weather offered the best balance of performance and computational efficiency for standard models (Section 3.4.4), a **symmetric 9-hour window of both origin and destination weather** ultimately yielded the highest absolute performance when paired with a sufficiently capable model architecture (Section 4.1). This combination of the D2 dataset and a deep, symmetric weather profile forms the richest possible feature set for prediction.

Finding 3: The decisive impact of model capacity

The Optimized Model Study (Section 5) demonstrated conclusively that model capacity is a primary limiting factor for predictive performance. Increasing the model's complexity (from a baseline of 15 trees at depth 7 to an optimized configuration of 50 trees at depth 30) unlocked a significant performance improvement that was simply unattainable with the standard, lower-capacity model. This larger architecture was essential to fully capitalize on the high-dimensional feature set created by the optimal data configuration.

Model Configuration	Peak Hold-out F1-Score
Standard Symmetric Model (D2-60-9-9)	76.75%
Optimized Symmetric Model (D2-60-9-9)	82.13%

As shown in the table above, the optimized model configuration boosted the peak F1-Score by over 5 percentage points, a transformative improvement. This confirms that a larger model is essential to fully leverage the rich, high-dimensional feature set and capture the complex, non-linear relationships within the data.

The critical role of environment right-sizing

The scalability and stability experiments (Section 8) underscored a fundamental MLOps principle: the computational environment must be appropriately sized for the workload. The Dataproc analysis, based on corrected log file runtimes (410.36 min for 2 workers, 35.10 min for 3, and 210.05 min for 4), showed that horizontal scaling provides a clear but non-linear reduction in runtime. The addition of a third worker yielded a dramatic 14.26-minute speedup, while the fourth worker provided only an additional 6.05-minute improvement. This pattern of diminishing returns identifies the **3-worker configuration as the optimal balance of cost and performance** for baseline, CPU-bound jobs (Section 8.1.4).

In a textbook example of MLOps workload characterization, the Lamsade cluster demonstrated severe instability when subjected to the memory- and I/O-intensive D4-9-9 workload (Section 8.2.1). This workload's symmetric 9-hour weather join created massive intermediate dataframes, triggering intense shuffle operations that overwhelmed the fixed resources of the cluster. The execution log revealed a predictable failure cascade: executors ran out of memory, triggering exit code 137 OOM errors. This executor loss made their shuffle data unavailable to the rest of the cluster, causing a storm of `FetchFailed` exceptions and ultimately leading to `ExecutorLostFailure` messages as the job disintegrated. This stark difference proves that matching the computational environment's memory and I/O capacity to the workload's demands is paramount for achieving both stability and reliable performance.

These findings collectively point toward a single, best-in-class model configuration that harmonizes the optimal data, model, and environment characteristics.

8.10.2 The champion model: experience-optimized-local-D2-60-9-9

Based on the overwhelming evidence presented across all experimental families, the Experience-optimized-local-D2-60-9-9 model is the definitive champion. This configuration consistently outperformed all others, establishing a new benchmark for predictive accuracy. Its selection is justified by several key achievements:

- **Peak Predictive Performance:** It achieved the highest F1-Score (82.13%) and AUC-ROC (0.8925) of all experiments conducted, demonstrating superior accuracy and discriminatory power (Section 5.2.2).
- **Optimal Data Integration:** It utilizes the superior D2 dataset combined with the optimal 9-hour symmetric weather depth, a configuration that consistently marked the point of peak performance before the onset of diminishing returns (Sections 5.1 and 7.4).
- **Effective Use of Capacity:** Its higher-capacity architecture successfully leverages the high-dimensional feature space to capture more complex predictive signals. This is evidenced by the dilution of feature importance (Section 5.3), where the model spreads its decision-making across a wider array of features. This contrasts sharply with the standard model's reliance on a few dominant predictors, demonstrating that the champion's higher capacity is not merely redundant but essential for capturing the subtle, complex interactions present in the high-dimensional feature space.

While this champion model is the most computationally expensive configuration tested, its substantial performance lead makes it the best choice for maximizing the operational value delivered by the flight delay prediction system.

8.10.3 Strategic recommendations

Based on the preceding analysis, the following strategic recommendations are proposed to guide the future development, deployment, and operational management of the flight delay prediction system.

Recommendation 1: Standardize on the D2 dataset and Champion model

It is recommended that the project officially adopt the **D2 dataset** and the **Experience-optimized-local-D2-60-9-9 model configuration** as the production standard. This combination provides a proven, high-performing, and rigorously validated baseline for all future development, operational deployment, and performance monitoring. This decision is directly supported by the D2 dataset's definitive superiority in the dataset comparison (Section 7.4) and the champion model's top-tier performance across all key metrics (Section 5.4.3).

Recommendation 2: Implement a workload-aware infrastructure strategy

A "right-sizing" protocol for computational infrastructure must be adopted to ensure both stability and cost-efficiency. The scalability experiments (Section 8) provide clear evidence that a one-size-fits-all approach is suboptimal and can lead to catastrophic failure. The following tiered strategy is recommended:

1. **For baseline workloads (CPU-bound, low shuffle):** Utilize a **3-worker Dataproc configuration**. This was identified as the optimal balance of cost and performance for routine, less-demanding jobs like retraining on static feature sets.
2. **For complex workloads (memory/I/O-bound, high shuffle):** Acknowledge that the Lamsade cluster, as currently configured, is unsuitable. Mandate that these jobs be run on a Lamsade configuration with significantly increased executor memory (e.g., doubled to 16GB per executor) to accommodate large shuffle blocks, or be migrated to a scalable cloud environment to prevent the instability observed in Section 8.2.1.
3. **Integrate Profiling:** Mandate a "**workload profiling**" step before running new, complex experiments on fixed-resource clusters. This proactive measure will help identify potential memory, CPU, or I/O bottlenecks before they can compromise experimental results or cause cluster instability.

Recommendation 3/ Prioritize cost-benefit analysis for future enhancements

The experimental results repeatedly demonstrated the principle of diminishing returns, particularly when adding weather depth beyond the 9-hour mark (Section 5.2.2). To ensure development efforts remain focused and efficient, it is recommended that all future proposed enhancements—such as integrating new data sources or further increasing model capacity—must be accompanied by a **rigorous cost-benefit analysis**. This analysis should quantify the expected performance gain against the projected increase in computational runtime, infrastructure cost, and pipeline complexity, using the framework from Section 5.4.2 as a template.

8.10.4 Conclusion

This exhaustive experimental process has successfully moved the project beyond theoretical analysis to identify a specific, high-performing, and production-ready "champion" model. The key to this success was a systematic and multi-faceted evaluation of the core pillars of any machine learning system: the data it consumes, the architecture of the model itself, and the computational environment in which it operates. By isolating variables and rigorously testing hypotheses across these dimensions, we have developed a clear, data-driven roadmap. The project is now equipped to deploy a best-in-class flight delay prediction system built on a foundation of empirical evidence, ready to deliver significant operational value.

9. General Conclusion and perspectives

In this project, we had the opportunity to successfully conceptualize, design, develop, and validate a weather-based flight delay predictor leveraging the distributed Apache Spark architecture, balancing Big Data engineering with a rigorous scientific experimentation strategy.

We were aiming at replicating the reference paper methodology and use their result as a benchmark, which was also a success, as we tried to go beyond the strict scope of the paper, culminating in a **production-ready solution ready** characterized by architectural robustness, high predictive performance, and a thorough analysis of its operational costs.

9.1 Summary of Key accomplishments

Predictive Excellence: The Champion Model

Over 100 experiments were conducted, leading to the identification of an optimal model. The champion model, **Experience-optimized-local-D2-60-9-9** (a Random Forest with 60 trees and a depth of 9, using the D2 dataset and a 9-hour weather window), established the following performances:

- **F1-Score: 82.13%** (a critical measure of the balance between precision and recall).
- **AUC-ROC: 0.8925** (a measure of the model's discriminatory capacity).

This model represents a substantial gain of **+5.38 percentage points** on the F1-Score compared to the baseline approach (Belcastro et al., 2016), demonstrating the effectiveness of the feature engineering and hyperparameter choices.

Foundational Architectural Contributions

The second major achievement lies in the construction of a modular, distributed ML pipeline, driven by YAML configuration, that manages the model's entire lifecycle: ingestion, cleaning, joining, feature engineering, training, and evaluation. This approach is now reproducible and highly configurable, which is essential for MLOps environments.

Mastery of big data engineering: the spatio-temporal join

The complexity of the project resided primarily in integrating two massive and heterogeneous datasets: millions of flight records (AOTP) and hundreds of millions of hourly weather observations (QCLCD). The success of this fusion, known as the Spatio-Temporal Join, is one of the project's architectural pillars.

The Data Alignment Challenge - The join between the datasets required a simultaneous double match:

- **Spatial Correspondence:** Ensuring alignment between the flights' airport codes (IATA) and the nearby weather station codes (WBAN).
- **Temporal Correspondence:** Aligning the flight's local departure time (LLZ) with the Universal Coordinated Time (UTC) of the weather observations, considering the entire set of observations available within a predefined time window (the weather depth \$W\$).

Spark Optimizations to Overcome Latency - Initial tests revealed that a “naïve” join would have been a fatal bottleneck, requiring hours of execution and disproportionate resources. We implemented specific optimizations in Scala/Spark to mitigate **data skew** and the **volume of shuffled data**:

- **Pre-aggregation into Map Structures:** Instead of letting Spark directly join the millions of raw observations, the QCLCD data was **pre-aggregated** into **Map structures (wmap)** per join key (WBAN, Date). This technique converted a costly $O(N)$ lookup operation into a fast $O(1)$ access during the join phase, drastically reducing the volume of data transferred over the network.
- **Explicit Partitioning:** The use of `repartition(n)` with the join keys (ORIGIN_WBAN, UTC_FL_DATE) forced the co-location of flight and weather data onto the same cluster executors, minimizing inter-node communications.
- **Boundary Days Management:** The strategic choice to **duplicate the J/J+1 weather data** rather than the flight data minimized the impact on the overall volume of the AOTP dataset, keeping the cost of this operation optimal.

Thanks to these optimizations, the execution time for the join step was stabilized to approximately **12 to 21 minutes**, transforming the biggest technical challenge into a stable foundation for experimentation.

Tested three different implementations

A good achievement was to test the three implementations, starting by the local docker, but also the LAMSADE cluster and Google Cloud Dataproc.

9.2 Conclusions derived from experimentation – lessons learnt

The systematic approach to testing (over 100 configurations) provided solid empirical conclusions on the nature of the prediction problem, as follows for the most compelling ones:

Superiority of the D2 Feature Set

Experiments unequivocally demonstrated the superiority of the **D2 dataset**, which integrates a wider range of features than the D1, D3, and D4 datasets.

- **Result:** Models trained on D2 consistently outperformed the others, with the champion D2-9-9 model surpassing the D1, D3, and D4 models by **more than 10 percentage points** on the F1-Score.
- **Conclusion:** The predictive signal is not diluted by adding relevant flight variables (previous delays, carrier, etc.). On the contrary, providing the model with a complete multidimensional context is essential to distinguish the causality of delays

The Critical Role of the Weather Window

The feature engineering analysis showed that the temporal depth of the weather window (\$W\$) is crucial.

- **Result:** Performance peaked with a **9-hour symmetric window (\$W=9\$) before and after the departure time**
- **Conclusion:** Weather conditions beyond 9 hours bring no significant added value, suggesting that the critical impact of weather is absorbed by the model during this period. Adding more hours primarily introduces noise and increases computational cost without providing prediction, confirming the principle of feature parsimony

Importance of Origin Weather

The distinction between departure and arrival weather is significant:

- **Result:** The best model trained only with origin weather reached a **F1-Score of 75.85%**, surpassing the best model using only destination weather (F1-Score of 73.50%)
- **Conclusion:** This confirms that the immediate and most predictive cause of a delay is the aircraft's inability to take off on time. The model must focus on departure conditions, as arrival conditions (destination weather) are often better managed by air traffic control systems that adjust flights en route.

9.3 Cost-Benefit Analysis and operational efficiency

Experimentation went beyond simply seeking the best predictive performance to focus on operational efficiency and cost management (execution time and resources).

Algorithmic Trade-off: GBT vs. Random Forest

The comparison of algorithms revealed a decisive trade-off for deployment.

Algorithm	Max F1-Score	Average Execution Time	Operational Conclusion
Random Forest (RF)	81.30%	34.27 minutes	Maximum performance, but high cost.
Gradient Boosted Trees (GBT)	81.03%	16.90 minutes	Nearly identical performance for half the cost.

- **Conclusion:** GBT is the strategic choice for production. For a negligible marginal performance gain (+0.27 F1-Score point), Random Forest required **2.03 times more execution time**. Choosing GBT allows for substantial savings in Cloud resources for daily or weekly training, without compromising service quality.

Analysis of Bottlenecks and Diminishing Returns

Scalability tests identified inherent limitations in the design of Big Data tasks.

- **Bottlenecks:** The **Explode** operation (which transforms weather Map structures into feature rows) emerged as the major scalability bottleneck; its execution time increased non-linearly with added weather features. Similarly, large shuffles (data transfer between nodes) during massive joins remain a source of latency
- **Diminishing Returns:** Increasing the number of workers in the cluster (e.g., from 2 to 4) led to a decreasing rate of acceleration (**speedup**). The third worker often provided the best cost-

performance ratio, as the benefit of additional cores is quickly offset by network coordination overhead

- **Infrastructure Cost:** The local test environment (Docker) proved to be **36.8% faster** than the distributed Lamsade/HDFS cluster for small-to-medium scale tests. This is essential for the R&D phase: prototyping should favor optimized local environments to avoid the latency of distributed file systems

9.4 Limitations and future strategic perspectives

The project delivered a production-level solution for predicting individual delays, but it highlights areas for improvement in the next iteration.

Limitations of the Current Binary Model

The model has three major limitations that require a roadmap for future perspectives:

- **Absence of Network Effects:** The model treats each flight atomically. It ignores the impact of **delay propagation** (a plane delayed on flight A causes the delay of flights B, C, etc., that it subsequently operates).
- **Limited Binary Classification:** Binary prediction (delayed/on-time) does not account for delay severity. A 15-minute delay is treated the same as a 5-hour delay, which reduces operational utility.
- **Historical Weather Data:** Training is based on past weather observations. For real-time prediction, the system must integrate **weather forecasts**, which adds a layer of complexity related to data uncertainty.

Deployment and Research Roadmap

The success of the project leads to a clear action plan for industrialization and research:

- **Operational Deployment (MLOps):** The solution must be deployed with the **GBT** algorithm to maximize cost efficiency. Careful attention must be paid to the **right-sizing** of clusters to optimize the cost/speed ratio, particularly by increasing memory per executor to counteract shuffle errors during Explode operations on growing data volumes. Integrating monitoring tools (e.g., Prometheus/Grafana) to track model drift in production is essential.
- **Deep Learning Research:** Network effects modeling could be approached by exploring **Graph Neural Networks (GNNs)**, where airports and flights form the nodes and edges of a graph. **Recurrent Neural Networks (RNNs)** could improve the modeling of the temporal sequences of weather and delays.
- **Transition to Regression:** The next goal must be to evolve the problem from binary classification to a **regression problem**, in order to predict the exact duration of the delay.

In conclusion, the project not only delivered a highly performant flight delay predictor (F1-Score > 82%) but also established a robust and scientifically validated framework for handling distributed Big Data, paving the way for more sophisticated and economically efficient decision-making systems in the aviation sector.

APPENDIX

APPENDIX 1 – Spatio-Temporal Join

```
/**
 * Jointure spatio-temporelle entre vols et observations météo pour ORIGINE et DESTINATION
 *
 * @param flightDF DataFrame des vols
 * @param weatherDF DataFrame météo
 * @param weatherOriginDepthHours Nombre d'heures d'observations météo à récupérer avant le vol
 *      - Valeur négative : pas de jointure météo pour l'origine
 *      - 0 : jointure à l'heure exacte du départ
 *      - >0 : jointure avec historique (max 11)
 * @param weatherDestinationDepthHours Nombre d'heures d'observations météo à récupérer avant le vol
 *      - Valeur négative : pas de jointure météo pour la destination
 *      - 0 : jointure à l'heure exacte de l'arrivée
 *      - >0 : jointure avec historique (max 11)
 * @param removeLeakageColumns Si true, supprime automatiquement les colonnes qui causent du data leakage (par défaut: false)
 * @param selectedFlightColumns Colonnes du DataFrame vols à conserver (si None, toutes les colonnes)
 * @param selectedWeatherColumns Colonnes du DataFrame météo à inclure dans les observations (si None, toutes les colonnes)
 * @return DataFrame avec origin_weather_observations et dest_weather_observations
 */
def joinFlightsWithWeather(
    flightDF: DataFrame,
    weatherDF: DataFrame,
    weatherOriginDepthHours: Int = 0,
    weatherDestinationDepthHours: Int = 0,
    removeLeakageColumns: Boolean = true,
    selectedFlightColumns: Option[Seq[String]] = None,
    selectedWeatherColumns: Option[Seq[String]] = None
)(implicit spark: SparkSession, configuration: AppConfiguration): DataFrame = {

    // Validation : valeurs négatives acceptées (= pas de jointure), sinon max 11
    require(weatherOriginDepthHours <= 11,
        s"weatherOriginDepthHours doit être <= 11 (ou négatif pour désactiver), valeur fournie: $weatherOriginDepthHours")

    require(weatherDestinationDepthHours <= 11,
        s"weatherDestinationDepthHours doit être <= 11 (ou négatif pour désactiver), valeur fournie: $weatherDestinationDepthHours")

    // Si les deux sont négatifs, pas de jointure du tout
    if (weatherOriginDepthHours < 0 && weatherDestinationDepthHours < 0) {
        info("[FlightWeatherJoiner] Both weatherOriginDepthHours and weatherDestinationDepthHours are negative - NO weather join")
        return flightDF
    }

    // Paramètres de partitions "reducers"
    val cores = spark.sparkContext.defaultParallelism

    val numPartsOrigin = pickParts(3.3, 32, 128, cores) // ≈ 40
    val numPartsDest = pickParts(5.2, 48, 192, cores) // ≈ 64
    spark.conf.set("spark.sql.shuffle.partitions", numPartsDest) // borne haute DF
}
```

```

// Préparation de WeatherDF
val weather = weatherDF.select(
  col("Date").cast(DateType).as("WDATE"),
  col("Time").as("WTIME_HHMM"),
  col("*")
).where(col("WBAN").isNotNull && length(col("WBAN")) > 0 && col("WDATE").isNotNull)

// -----
// Préparation météo avec relHour + duplication J/J+1 (Map)
// -----
val weatherWithHour = weather
  .withColumn("hour", hhmmHourCol(col("WTIME_HHMM")))
  .na.fill(Map("hour" -> -1))

val meteoSameDay = weatherWithHour
  .withColumn("relHour", col("hour"))
  .withColumn("DATE", col("WDATE"))

val meteoNextDay = weatherWithHour
  .withColumn("relHour", col("hour") - lit(24))
  .withColumn("DATE", date_add(col("WDATE"), 1))

val weatherRel = meteoSameDay.unionByName(meteoNextDay)
  .filter(col("relHour").between(-24, 23))

// -----
// Reduce météo par clé → Map relHour -> struct (Reduce)
// -----

// 1) Récupérer les features météo depuis la config (Option[Map[...]])

// 2) Colonnes fixes dans la struct
val staticCols = Seq(
  col("relHour").as("hour"),
  col("WBAN"),
  col("WDATE"),
  col("WTIME_HHMM")
)

// 3) Colonnes dynamiques venant de la config
val staticNames = Set("relHour", "WBAN", "WDATE", "WTIME_HHMM")
val allWeatherCols: Seq[String] =
  selectedWeatherColumns.getOrElse(Seq.empty[String])
val dynamicFeatureCols =
  allWeatherCols
  .filterNot(staticNames.contains)
  .map(c => col(c).alias(c))

// 4) Struct finale : [hour, WBAN, WDATE, WTIME_HHMM, <features>...]
val weatherStruct =
  struct((staticCols ++ dynamicFeatureCols): _*)

// 5) Agrégation en map relHour -> struct(...)
val weatherByKey: DataFrame =
  weatherRel
  .groupBy(col("WBAN"), col("DATE"))
  .agg(
    map_from_entries(
      collect list(struct(col("relHour"), weatherStruct))
    )
  )

```

```

    ).as("wmap")
  )

// Première jointure : aéroport d'origine (seulement si >= 0)
val withOriginWeather = if (weatherOriginDepthHours >= 0) {
  // -----
  // JOIN #1 — ORIGIN (Partition = hash(ORIGIN_WBAN, UTC_FL_DATE))
  // -----
  val flightsDep = flightDF
    .withColumn("depHour", coalesce(hhmmHourCol(col("UTC_CRS_DEP_TIME")), lit(0)))

  val originPre = flightsDep
    .repartition(numPartsOrigin, col("ORIGIN_WBAN"), col("UTC_FL_DATE")) // <-- Hash partition explicite
    .join(
      weatherByKey.hint("shuffle_hash"),
      col("ORIGIN_WBAN") === weatherByKey("WBAN") &&
      col("UTC_FL_DATE") === weatherByKey("DATE"),
      "left"
    )
    .drop(weatherByKey("WBAN")).drop(weatherByKey("DATE"))

  val originWithWoArr = originPre
    .withColumn("Wo", expr(s"transform(sequence(1, ${weatherOriginDepthHours}), i -> element_at(wmap,
    depHour - i)))"))
    .drop("wmap")

  val woCols = (0 until weatherOriginDepthHours).map(i => col("Wo").getItem(i).as(s"Wo_h${i+1}"))
  val originDF = originWithWoArr
    .select(col("*") +: woCols: _*)
    .drop("Wo")
    .persist()

  originDF
} else {
  info(s"[FlightWeatherJoiner] Skipping origin weather join
(weatherOriginDepthHours=${weatherOriginDepthHours})")
  flightDF
}

// Deuxième jointure : aéroport de destination (seulement si >= 0)
val withBothWeather = if (weatherDestinationDepthHours >= 0) {
  val flightsArr = withOriginWeather
    .withColumn("arrHour", coalesce(hhmmHourCol(col("UTC_ARR_TIME")), lit(0)))

  val destPre = flightsArr
    .repartition(numPartsDest, col("DEST_WBAN"), col("UTC_ARR_DATE")) // <-- Hash partition explicite
    .join(
      weatherByKey.hint("shuffle_hash"),
      col("DEST_WBAN") === weatherByKey("WBAN") &&
      col("UTC_ARR_DATE") === weatherByKey("DATE"),
      "left"
    )
    .drop(weatherByKey("WBAN")).drop(weatherByKey("DATE"))

  val destWithWdArr = destPre
    .withColumn("Wd", expr(s"transform(sequence(1, ${weatherDestinationDepthHours}), i -> element_at(wmap,
    arrHour - i)))"))
    .drop("wmap")

  val wdCols = (0 until weatherDestinationDepthHours).map(i => col("Wd").getItem(i).as(s"Wd_h${i+1}"))

```

```

val baseCols: Seq[org.apache.spark.sql.Column] =
  withOriginWeather.columns.map(col).toSeq

val joinedDF = destWithWdArr
  .select( (baseCols ++ wdCols):_*)
  .drop("Wd")
  .persist()

joinedDF
} else {
  info(s"[FlightWeatherJoiner] Skipping destination weather join
(weatherDestinationDepthHours=$weatherDestinationDepthHours)")
  withOriginWeather
}

// Supprimer les colonnes météo non désirées basées sur les valeurs négatives
var finalResult = withBothWeather

// Si weatherOriginDepthHours < 0, supprimer origin_weather_observations si elle existe
if (weatherOriginDepthHours < 0 && finalResult.columns.contains("origin_weather_observations")) {
  info(s"[FlightWeatherJoiner] Removing origin_weather_observations column
(weatherOriginDepthHours=$weatherOriginDepthHours)")
  finalResult = finalResult.drop("origin_weather_observations")
}

// Si weatherDestinationDepthHours < 0, supprimer destination_weather_observations si elle existe
if (weatherDestinationDepthHours < 0 && finalResult.columns.contains("destination_weather_observations")) {
  info(s"[FlightWeatherJoiner] Removing destination_weather_observations column
(weatherDestinationDepthHours=$weatherDestinationDepthHours)")
  finalResult = finalResult.drop("destination_weather_observations")
}

// Supprimer les colonnes de leakage si demandé
if (removeLeakageColumns) {
  removeDataLeakageColumns(
    finalResult,
    keepRelativeTime = removeLeakageColumns, // ou true si tu veux garder hours_before_flight
    selectedWeatherColumns = selectedWeatherColumns
  )
} else {
  finalResult
}
}

```

APPENDIX 2 – Tracking and Managing Experiments with MLFlow

1.0 Introduction: Establishing a Foundation for Reproducible Machine Learning

In any complex machine learning endeavor, such as our flight delay prediction project, managing the lifecycle of experiments is a critical challenge. Without a systematic approach, our development process descends into a costly mire of disconnected scripts and untraceable model artifacts, wasting compute resources and making it impossible to answer simple questions like, "Which experiment produced our best-performing model?" The strategic implementation of a robust tracking system is therefore not just a convenience, but a necessity for building reliable and maintainable machine learning systems. Our goal is to move beyond ad-hoc experiment management to a systematic MLOps workflow that ensures complete traceability, comparability, and reliability for every model we build.

To achieve this, we have chosen MLFlow as our centralized solution for experiment tracking and management. Integrated directly into our Spark-based data pipeline, MLFlow provides the structured framework needed to log, query, and visualize the entire lifecycle of our machine learning experiments. The following sections detail our specific MLFlow integration architecture, our standardized tracking methodology, and the comprehensive set of data points we capture for each experimental run.

2.0 MLFlow Integration Architecture

We architected our MLFlow integration to provide a consistent and seamless tracking experience across different working environments, from local development on a single machine to large-scale jobs on a production cluster. This dual-environment design ensures that all experimental results, whether from rapid prototyping or official training runs, are captured in a standardized manner.

2.1 Docker-Based Environment

For local development and testing, we utilize a self-contained environment managed by docker-compose. Within this setup, MLFlow runs as a dedicated service container, ensuring isolation and portability.

Based on the `docker-compose.yml` configuration, the `mlflow-server` service uses the official `ghcr.io/mlflow/mlflow:v3.4.0` image. Its key configurations are:

- **Backend Store:** An SQLite database, located at `sqlite:///mlflow/mlflow.db`. This database file resides in the `../work/mlflow` directory on the host machine, which is volume-mapped into the container, ensuring metadata persistence across container restarts.
- **Artifact Store:** The local filesystem, mapped to `/mlflow/artifacts` within the container, serves as the artifact store. This is where all large outputs, such as trained model files, datasets, and generated plots, are saved.
- **Accessibility:** The MLFlow server is accessible to all other services within the Docker network at the address `http://mlflow-server:5000`.

This Dockerized approach provides an easily reproducible and self-contained environment, allowing any team member to spin up the entire Spark and MLFlow stack with a single command for local experimentation.

2.2 Spark Cluster Integration

To enable our distributed Spark jobs to communicate with the MLFlow server, we have built a custom Spark image. As defined in `Dockerfile.spark-mlflow`, this image extends the base Bitnami Spark image and installs the `mlflow==3.4.0` Python client library, along with visualization libraries like `matplotlib` and `seaborn`.

The strategic value of this custom image is to empower every Spark job with native MLFlow capabilities. By baking the client libraries directly into the execution environment, we eliminate boilerplate and ensure that any application or notebook session can seamlessly log parameters, metrics, and artifacts to a unified tracking server.

2.3 Deployment Environments

Our MLFlow architecture is deployed across two primary environments to support the full development lifecycle:

1. **Local Development:** As part of the docker-compose stack, this instance is ideal for rapid prototyping, debugging, and individual experimentation.
2. **Lamsade Cluster:** A shared, persistent installation of MLFlow serves as the central repository for larger, official experiments. This instance acts as the single source of truth for all significant model development and evaluation results.

This dual-setup provides the flexibility for developers to work independently in isolated environments while ensuring that all canonical experimental results are consolidated into a centralized and durable tracking server. This architecture is the foundation upon which our software abstraction layer is built to standardize all interactions with MLFlow.

3.0 The MLFlowTracker Abstraction Layer

To ensure consistency and enforce best practices across our codebase, we have developed a custom Scala object, `MLFlowTracker.scala`. This object serves as a centralized and standardized interface for all interactions with the MLFlow API. Instead of allowing scattered, ad-hoc calls to the MLFlow client throughout our applications, this abstraction layer provides a single, controlled entry point. This prevents technical debt and ensures that our tracking strategy can be evolved and maintained from a single location rather than being scattered across dozens of scripts.

The primary purpose of the `MLFlowTracker` is to enforce a consistent methodology for what is logged and how it is logged. It simplifies the process of starting runs, logging parameters, and saving artifacts, while preventing inconsistencies that can arise from direct API usage. The key public methods of this abstraction are summarized below.

Method	Purpose
<code>initialize</code>	Configures and initializes the MLFlow client with the tracking URI.
<code>startRun</code>	Begins a new MLFlow run under a specified experiment and returns the run ID.
<code>logParams</code>	Logs a map of key-value parameters for a given run (e.g., hyperparameters).
<code>logMetric / logMetrics</code>	Logs a single or multiple performance metrics (e.g., F1 score, accuracy).
<code>logModelMetrics</code>	A specialized method to log a map of model evaluation metrics for a specific stage (e.g., validation, test).
<code>logModel</code>	Saves a trained Spark ML model and logs it as an artifact.
<code>logPipelineModel</code>	Saves a complete Spark PipelineModel and logs it as an artifact.
<code>logArtifact</code>	Logs a local file or directory (e.g., a plot or configuration file) as an artifact.
<code>logDataset</code>	Logs metadata about a dataset used in the run, such as its path and size.
<code>logFeatureImportances</code>	Logs the feature importance scores from a trained model.
<code>endRun</code>	Terminates the current MLFlow run and sets its status (e.g., FINISHED).

This toolkit is the engine of our tracking strategy; methods like `logModelMetrics` and `logParams` directly populate the metrics and parameters sections of the MLFlow UI, while `logPipelineModel` and `logArtifact` are responsible for storing the crucial outputs that ensure end-to-end reproducibility, as detailed in the following section.

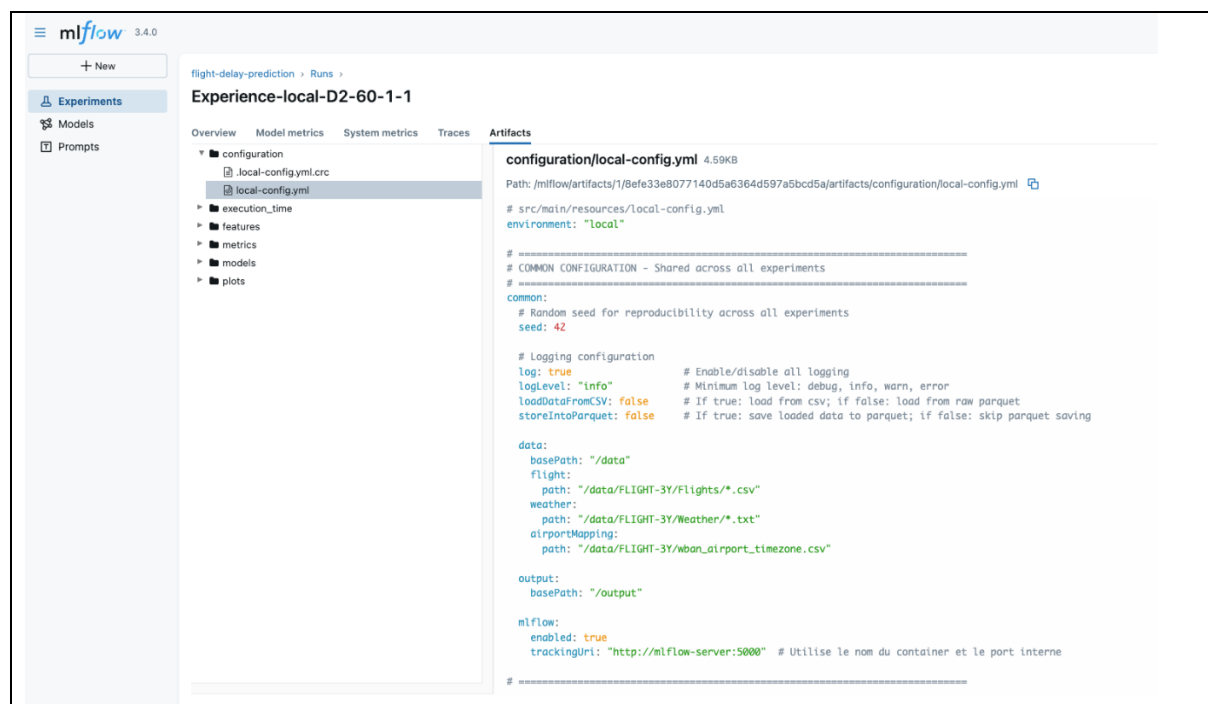
4.0 What We Track: A Comprehensive Overview of an Experiment's Lifecycle

Our MLFlow integration is designed to capture a complete, multi-faceted record of each experiment. This holistic view, which spans everything from high-level model performance down to the granular details of Spark job execution, is essential for debugging, optimization, and informed decision-making. By systematically logging parameters, metrics, and artifacts, we create a fully auditable and reproducible history of our model development process.

4.1 Experiment Parameters & Configuration

Every experiment run begins by logging the exact configuration that defined it. Using the `logParams` function from our `MLFlowTracker`, we capture all critical hyperparameters and configuration settings. For our Random Forest models, this includes parameters specified in our experiment configuration files, such as:

- `numTrees`
- `maxDepth`
- `maxBins`
- `featureSubsetStrategy`



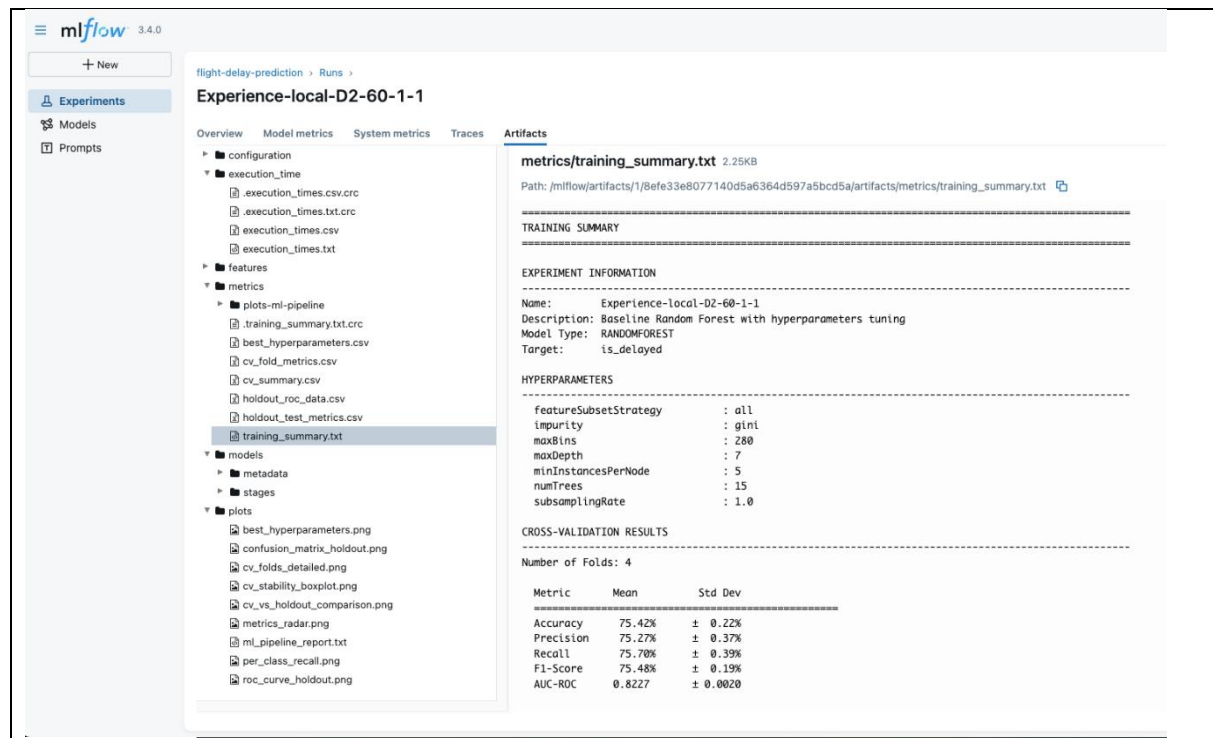
Logging these parameters is the first and most crucial step toward ensuring perfect reproducibility. With this information, we can trace any result back to the precise set of configurations that produced it, allowing us to recreate the experiment at any time.

4.2 Performance Metrics

We track two distinct but equally important categories of metrics: the predictive performance of our machine learning models and the computational performance of the underlying Spark pipeline.

4.2.1 Model Evaluation Metrics

Using the `logModelMetrics` and `logMetric` functions, we capture the final evaluation results of our trained models. This allows for direct, quantitative comparison between different models and hyperparameter sets.



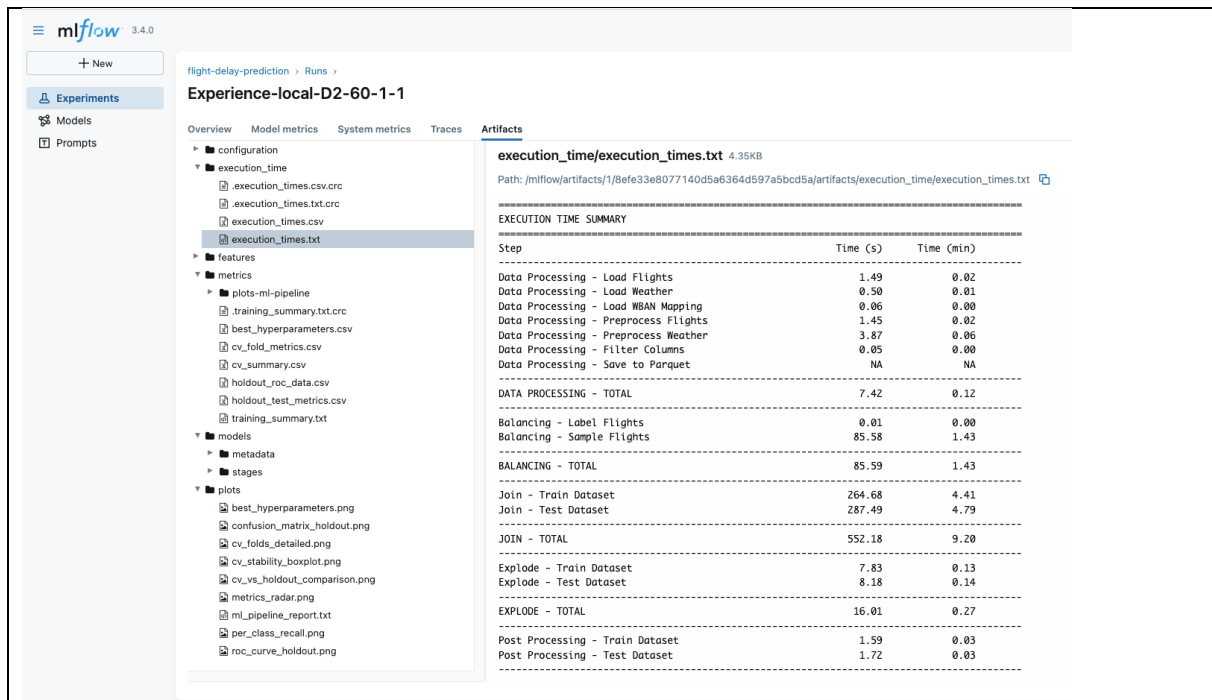
The key model performance metrics we log include:

- **f1 score:** The primary metric for our classification task, as defined in our experiment configuration.
- **accuracy:** A secondary metric providing an overall measure of correctness.

These metrics are logged for both the validation and test datasets, providing a clear picture of the model's ability to generalize to unseen data.

4.2.2 Spark Job Execution Metrics

Beyond model accuracy, we meticulously track the computational performance of our data processing and training pipelines. While these metrics are sourced directly from the Spark UI, MLFlow provides the essential run-level context that links a specific performance profile to a unique set of parameters, code, and data. This makes it straightforward to diagnose performance regressions and identify computational bottlenecks.



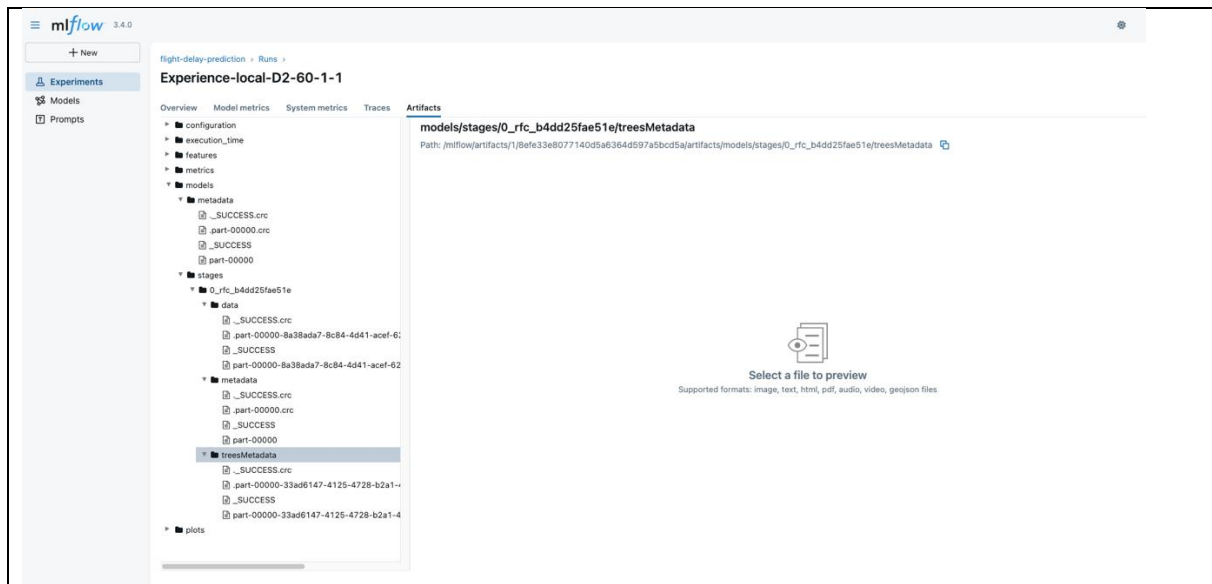
Key execution metrics we monitor include:

- **Job and Stage Durations:** Total time taken for key pipeline stages (e.g., Duration: 1.9 min for a feature engineering job like Job 93, up to 2.0 min for a complex join operation like Job 92).
- **Data I/O:** The volume of data read from and written to storage (e.g., Input: 161.1 MiB, Output: 17.2 MiB).
- **Shuffle Performance:** The size and record count of data shuffled between Spark executors, a critical indicator of distributed performance (e.g., Shuffle Read Size / Records: 639.7 MiB / 32631312).
- **Resource Utilization:** The memory consumed by Spark tasks during execution (e.g., Peak Execution Memory: 64.2 MiB).

4.3 Stored Artifacts

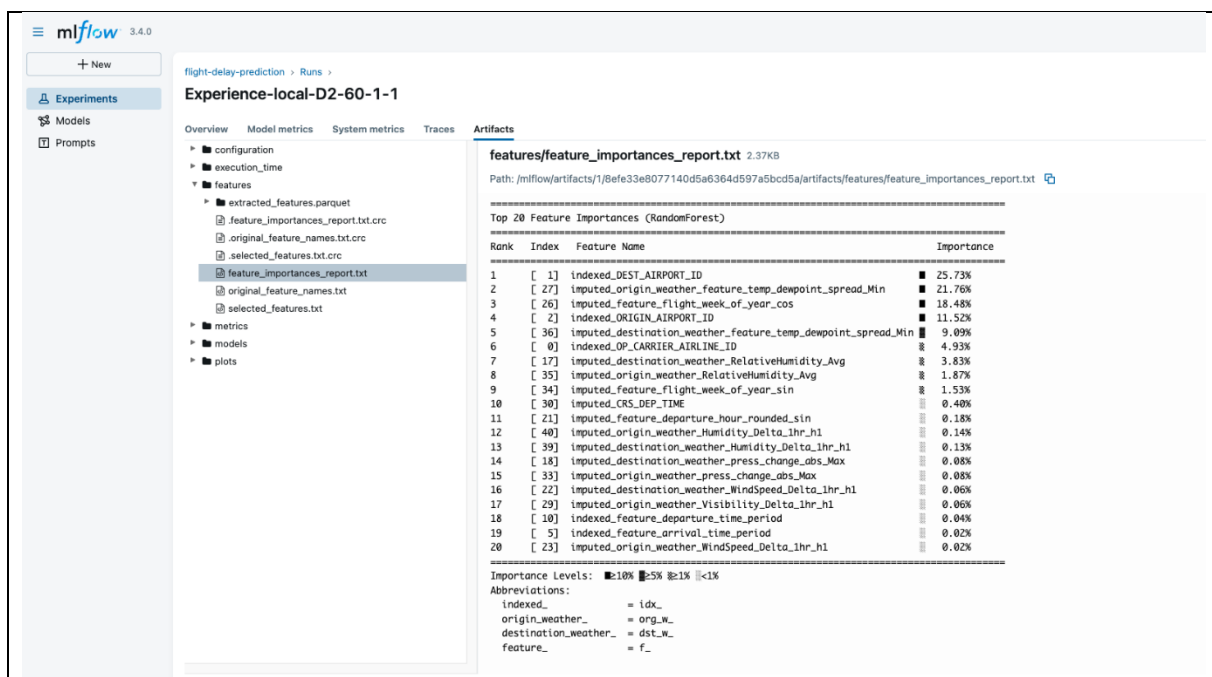
Artifacts are the tangible outputs of an experiment run. Using functions like `logArtifact`, `logModel`, and `logPipelineModel`, we store critical files such as trained models, datasets, and visualizations, making them permanent and easily accessible.

4.3.1 Trained Models and Pipelines



The most critical artifact we store is the fully trained Spark PipelineModel. This object encapsulates the entire workflow, including all preprocessing stages (e.g., string indexing, vector assembly) and the final trained classification model. By saving this complete pipeline to the MLFlow artifact store for each run, we ensure that we can retrieve and deploy the *exact* model that produced a given set of metrics, guaranteeing end-to-end integrity from training to production. Saving the PipelineModel instead of just the final classifier is a critical design choice. It captures not only the trained model but also the exact state of all feature transformers (e.g., StringIndexers, VectorAssembler) fitted on the training data. This guarantees that the exact same preprocessing logic is applied during inference, eliminating a common source of training-serving skew.

4.3.2 Datasets and Feature Importance



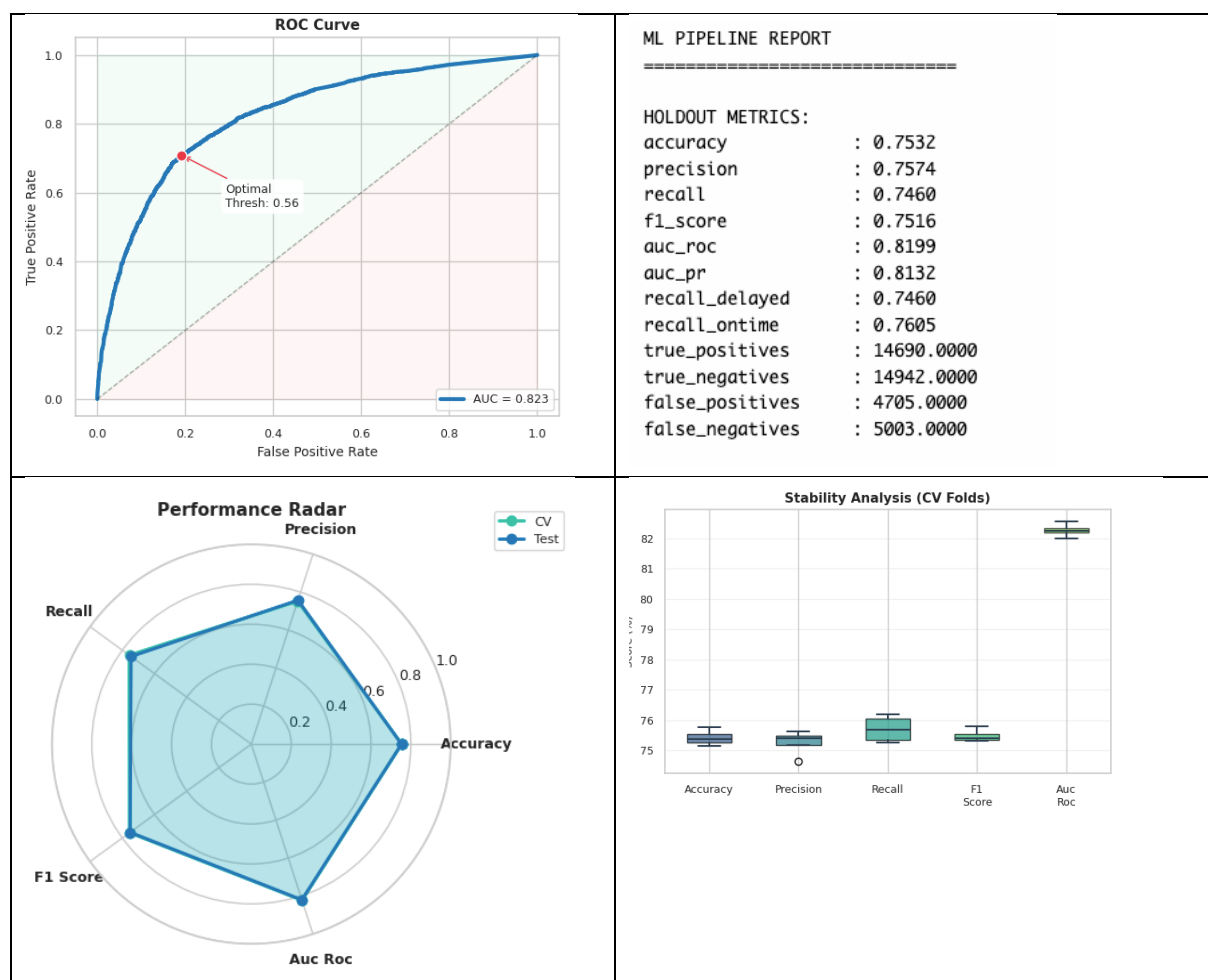
To maintain a clear audit trail and enhance model interpretability, we use our logDataset and logFeatureImportances functions. We log metadata about the datasets used in each run, including their storage paths and row counts, which provides a clear view of data lineage. Additionally, we calculate and log the feature

importance scores from our models, helping us understand which factors are most influential in predicting flight delays.

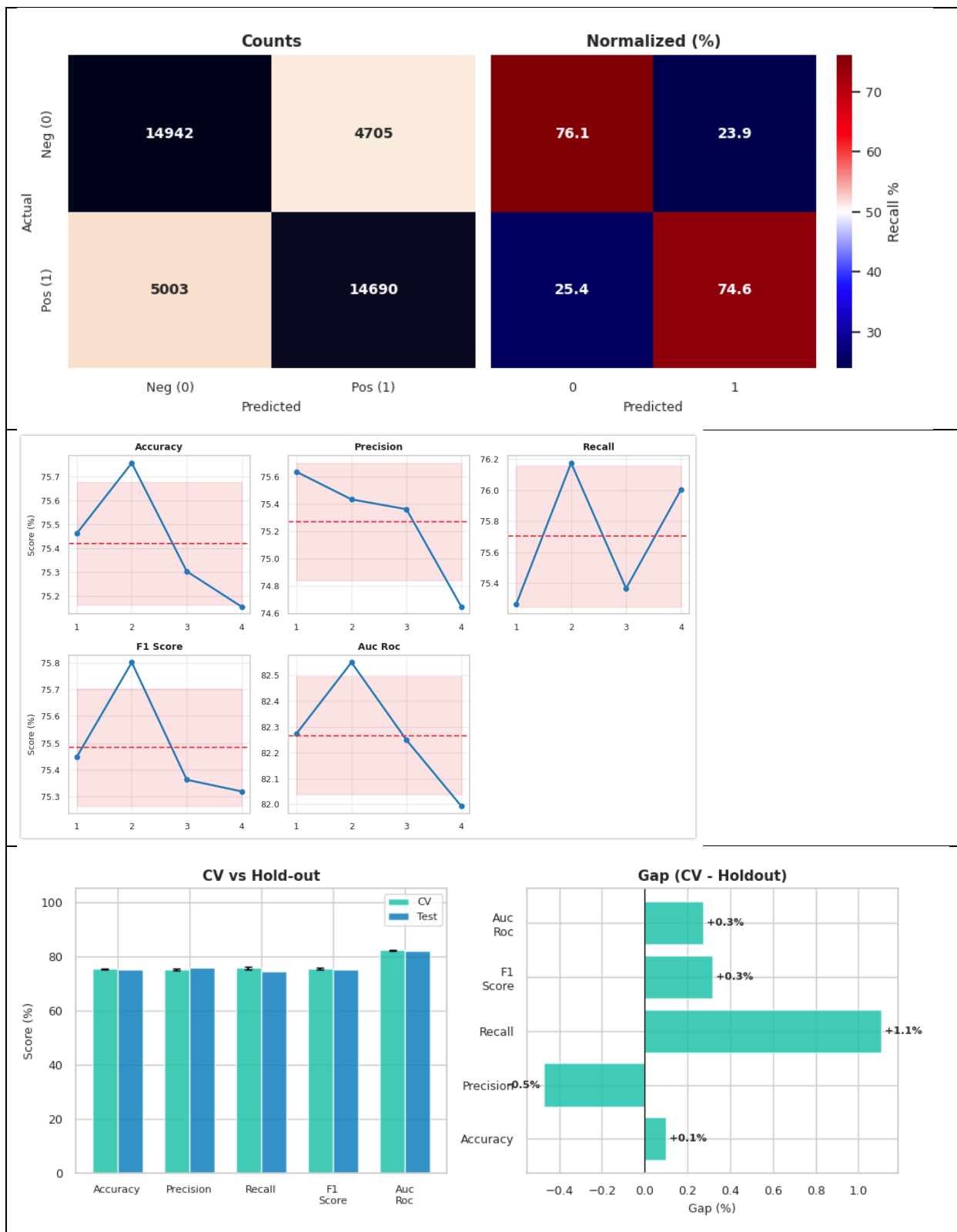
4.3.3 Generated Visualizations

Our custom Spark Docker image (Dockerfile.spark-mlflow) comes pre-installed with matplotlib and seaborn. This setup enables our Spark applications to generate and save critical visualizations as .png image artifacts directly to MLFlow. These artifacts provide quick, easily digestible summaries of model performance and behavior, such as:

- Feature importance plots
- Confusion matrices
- ROC curves



By enriching each run with these visual summaries, we make the experimental results more accessible and easier to interpret for the entire team. This comprehensive approach to tracking delivers significant benefits to our development lifecycle.



5.0 Conclusion: The Value of Systematic Experiment Tracking

By integrating MLflow into our Spark-based workflow and adopting a standardized tracking methodology, we have established a robust and scalable foundation for our machine learning development. This systematic approach transforms our process from a series of isolated tasks into a fully traceable and reproducible science, delivering significant strategic advantages. The value of this integration is distilled into several key benefits:

- **Full Reproducibility:** Every result is automatically linked to its exact code, data, and configuration. This eliminates ambiguity and allows any experiment to be perfectly recreated, which is essential for debugging, validation, and regulatory compliance.
- **Accelerated Development:** The centralized MLFlow UI enables rapid, side-by-side comparison of dozens of experiments. This allows our team to quickly identify the most promising models, feature sets, and hyperparameters, significantly shortening the development cycle.
- **Enhanced Collaboration:** MLFlow serves as a central, shared repository for all experimental results, providing a single source of truth for the entire team. This shared context eliminates silos and ensures that all team members are working with the same information when making decisions.
- **Foundation for CI/CD for ML:** This robust tracking system is the prerequisite for automation. The metadata logged for each run enables a CI/CD pipeline to programmatically query the MLFlow API for the run with the highest validation_f1 score, retrieve its associated PipelineModel artifact, and automatically trigger a deployment job, turning our research into a production-ready asset with minimal human intervention.

APPENDIX 3 – A Practical Guide to Testing and Debugging Spark Applications with Jupyter and Spark History Server

Modern data engineering demands a robust workflow that supports both interactive development for rapid iteration and deep performance analysis for building scalable solutions. An effective process must bridge the gap between writing code and understanding how it behaves under load, ultimately de-risking complex data projects and reducing their total cost of ownership. This guide details a unified workflow that leverages two critical tools in the Spark ecosystem: JupyterLab for iterative, component-level testing, and the Spark History Server for post-execution diagnostics. By integrating these tools, development teams can build, test, and troubleshoot Spark applications more effectively, ensuring both logical correctness and high performance.

1. Interactive Development and Unit Testing with JupyterLab

In the Spark ecosystem, an interactive development environment is a strategic asset, not just a convenience. JupyterLab, in particular, empowers developers to move faster by facilitating rapid prototyping, immediate logic validation, and isolated testing of application components. This approach significantly accelerates the development lifecycle by allowing for quick, iterative cycles of coding and testing before a full-scale deployment is ever attempted.

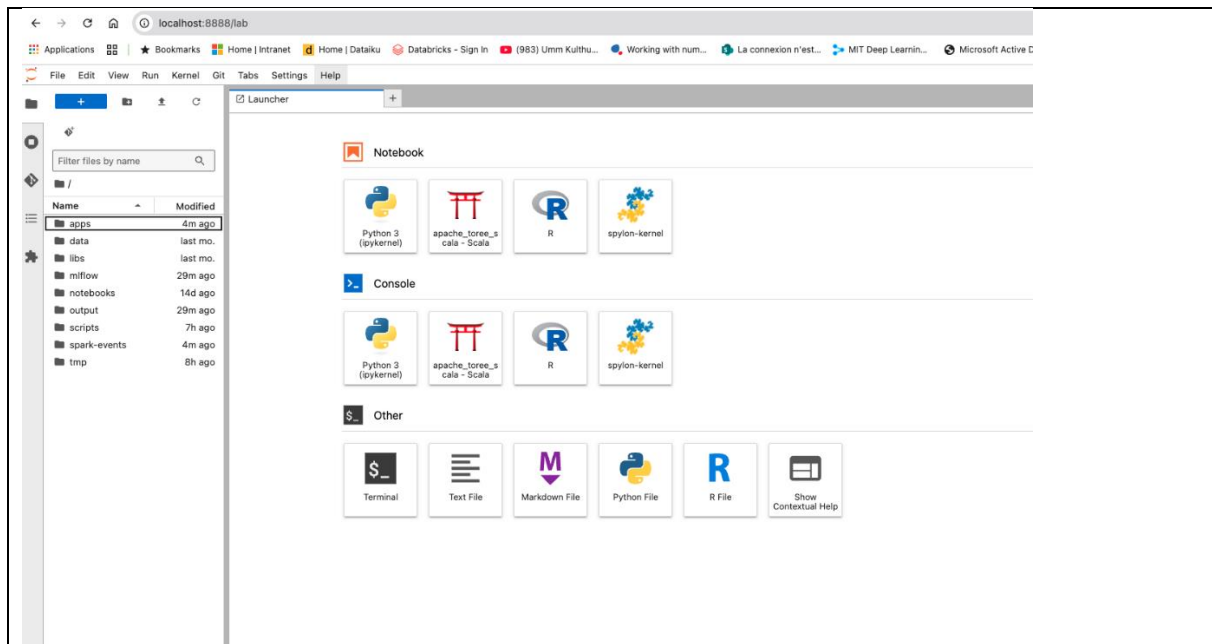
1.1. Structuring the Development Environment

A well-organized project structure is the foundation for any maintainable and scalable data application. A best-practice approach, as seen in the JupyterLab interface, involves a clear separation of concerns into distinct directories.

This structure typically includes:

- **apps:** For compiled application artifacts, such as JAR files.
- **data:** For raw and processed datasets.
- **notebooks:** For all interactive development and exploration work, often subdivided by function (e.g., DataExploration, FeatureEngineering).
- **scripts:** For shell scripts or automation helpers.

This separation of concerns is critical for professional data engineering. It enables automated CI/CD pipelines, simplifies dependency management, and allows different team members—such as data engineers and data scientists—to work in parallel without creating conflicts in the codebase.



1.2. Integrating Compiled Application Code

To ensure consistency between development and production, it is crucial to test the exact code that will be deployed. Jupyter notebooks with a Scala kernel support the `%AddJar` magic command, which provides a direct and efficient way to achieve this.

```

Weather Dataset Preprocessing and feature Engineering

%AddJar file:///home/jovyan/work/apps/Emiasd-Flight-Data-Analysis.jar

Starting download from file:///home/jovyan/work/apps/Emiasd-Flight-Data-Analysis.jar
Finished download of Emiasd-Flight-Data-Analysis.jar
Using cached version of Emiasd-Flight-Data-Analysis.jar

```

The command `%AddJar file:///.../Emiasd-Flight-Data-Analysis.jar` dynamically loads the compiled application JAR into the Spark session's classpath. This approach is non-negotiable for ensuring a single source of truth; it eradicates the entire class of "it worked on my machine" bugs that plague less disciplined teams. By importing and executing classes from the production codebase directly within the notebook, we guarantee that the logic validated interactively is the same logic that will run in the final spark-submit job.

1.3. Executing and Validating Code in Isolation

With the development environment and application code integrated, developers can test specific components of their application in a controlled, cell-by-cell manner. This process is fundamental for debugging complex transformations and validating business logic.

```
[2]: import org.apache.spark.sql.SparkSession
import com.flightdelay.config.{AppConfiguration, ConfigurationLoader}
import com.flightdelay.data.loaders.FlightDataLoader

//Env Configuration
val args: Array[String] = Array("jupyter")

val spark = SparkSession.builder()
  .config(sc.getConf)
  .getOrCreate()

// Rendre la session Spark implicite
implicit val session = spark
implicit val configuration = AppConfiguration = ConfigurationLoader.loadConfiguration(args)

val rawFlightsPath = s"${configuration.common.output.basePath}/common/data/raw_flights.parquet"
val flightsDF = spark.read.parquet(rawFlightsPath)

args = Array(jupyter)
spark = org.apache.spark.sql.SparkSession@68a9fced
session = org.apache.spark.sql.SparkSession@68a9fced
configuration = AppConfiguration(local,CommonConfig(42,true,debug,false,false,DataConfig(/home/jovyan/work/data,FileConfig(/home/jovyan/work
(/home/jovyan/work/data/FLIGHT-3Y/Weather/20101*.txt),FileConfig(/home/jovyan/work/data/FLIGHT-3Y/wban_airport_timezone.csv)),OutputConfig(/
n/work/o...

[2]: AppConfiguration(local,CommonConfig(42,true,debug,false,false,DataConfig(/home/jovyan/work/data,FileConfig(/home/jovyan/work/data/FLIGHT-3Y/
rk/data/FLIGHT-3Y/Weather/20101*.txt),FileConfig(/home/jovyan/work/data/FLIGHT-3Y/wban_airport_timezone.csv)),OutputConfig(/home/jovyan/work
```

```
[6]: import com.flightdelay.features.joiners.FlightWeatherDataJoiner

val joinedDF = FlightWeatherDataJoiner.joinFlightsWithWeather(
  labeledFlightData,
  weatherData,
  weatherOriginDepthHours,
  weatherDestinationDepthHours,
  true,
  flightFeaturesWithTarget,
  weatherFeatures
)

println(s"Rows after ORIGIN join: ${labeledFlightData.count()}, rows after DEST join: ${joinedDF.count()}")

[INFO] [Anti-Leakage] Colonnes supprimees : ARR_DELAY_NEW, NAS_DELAY, UTC_ARR_DATE, UTC_ARR_TIME, WEATHER_DELAY, feature_utc_arrival_hour_roun
Rows after ORIGIN join: 3908461, rows after DEST join: 3908461
joinedDF = [CRS_ARR_TIME: string, CRS_DEP_TIME: int ... 32 more fields]

[6]: [CRS_ARR_TIME: string, CRS_DEP_TIME: int ... 32 more fields]
```

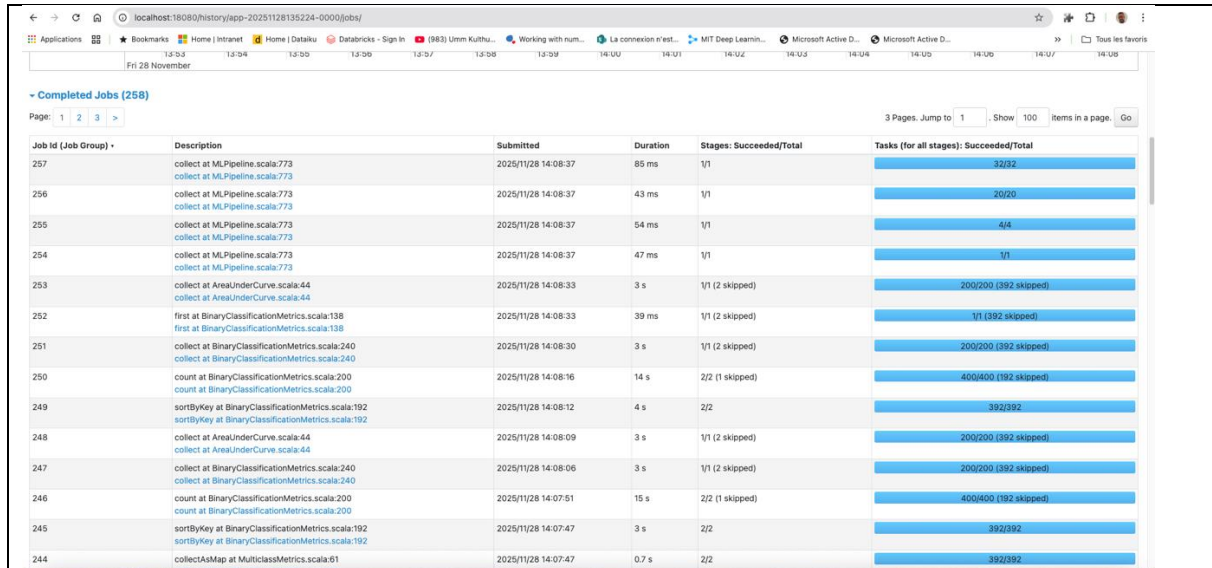
The workflow for testing a feature typically follows these steps:

1. **Spark Session Initialization:** The code first initializes a Spark session. Critically, it calls a shared `ConfigurationLoader.loadConfiguration(args)` method, passing a specific "jupyter" argument. This is a key design pattern for environment-agnostic code, as it allows developers to reuse production configuration logic while easily overriding specific parameters (like file paths) for local testing.
2. **Component Execution:** A specific component, such as the `FlightWeatherDataSourceJoiner`, is imported from the compiled JAR. The relevant method, `joinFlightsWithWeather`, is then executed with the necessary `DataFrame` inputs. This isolates the test to a single, manageable unit of work.
3. **Immediate Validation:** After execution, a simple but powerful validation step is performed. The `println` statement outputs the row counts before and after the join, revealing Rows after ORIGIN join: 3908461, rows after DEST join: 3908461. The identical row count is a critical piece of feedback. It confirms that the join is behaving as an enrichment (a 1-to-1 or N-to-1 lookup) and is not inadvertently filtering rows or causing a row explosion due to a faulty join key—a crucial sanity check that provides immediate confidence in the logic.

This interactive, cell-by-cell execution is fundamental for effective data exploration, unit testing feature engineering logic, and efficiently troubleshooting specific functions. With the logic now validated for correctness in an interactive environment, the focus must shift to ensuring it performs efficiently at scale. For this, we turn to the Spark History Server, the definitive record of the application's execution.

2. Deep Dive Analysis with the Spark History Server

The Spark History Server is the definitive tool for post-mortem analysis of an application's performance. It enables the crucial transition from asking, "Is the logic correct?" to "Will the logic scale?" Whether an application was run interactively or via spark-submit, the History Server provides the detailed execution record necessary to ensure its production viability by identifying and resolving performance bottlenecks.



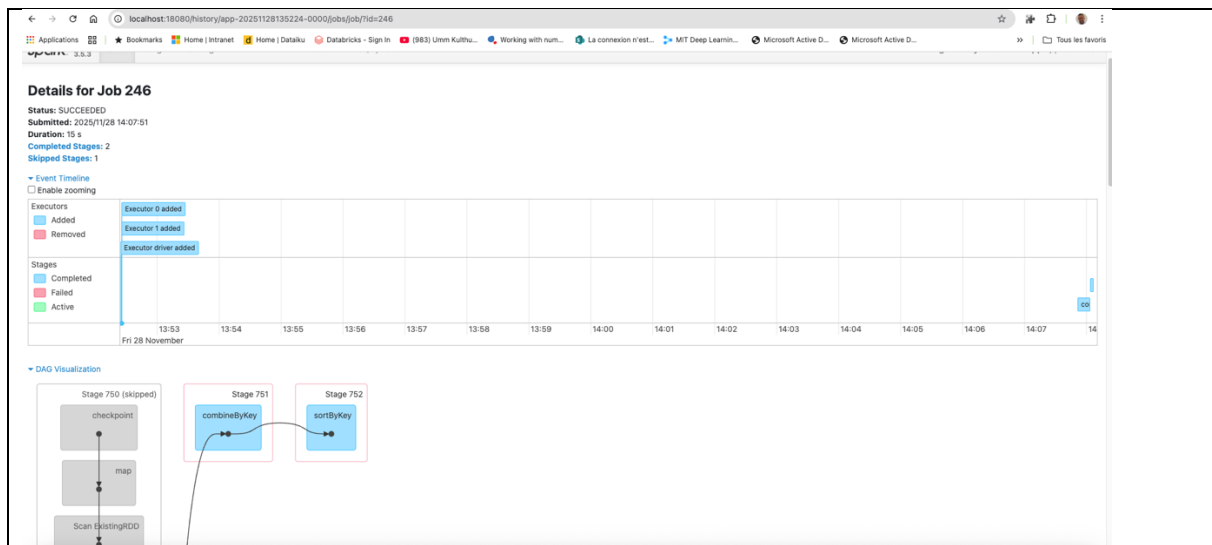
Job ID (Job Group)	Description	Submitted	Duration	Stages: Succeeded/Total	Tasks (for all stages): Succeeded/Total
257	collect at MLPipeline.scala:773	2025/11/28 14:08:37	85 ms	1/1	32/32
256	collect at MLPipeline.scala:773	2025/11/28 14:08:37	43 ms	1/1	20/20
255	collect at MLPipeline.scala:773	2025/11/28 14:08:37	54 ms	1/1	4/4
254	collect at MLPipeline.scala:773	2025/11/28 14:08:37	47 ms	1/1	1/1
253	collect at AreaUnderCurve.scala:44	2025/11/28 14:08:33	3 s	1/1 (2 skipped)	200/200 (392 skipped)
252	first at BinaryClassificationMetrics.scala:138	2025/11/28 14:08:33	39 ms	1/1 (2 skipped)	1/1 (392 skipped)
251	collect at BinaryClassificationMetrics.scala:240	2025/11/28 14:08:30	3 s	1/1 (2 skipped)	200/200 (392 skipped)
250	count at BinaryClassificationMetrics.scala:200	2025/11/28 14:08:16	14 s	2/2 (1 skipped)	400/400 (192 skipped)
249	sortByKey at BinaryClassificationMetrics.scala:192	2025/11/28 14:08:12	4 s	2/2	392/392
248	collect at AreaUnderCurve.scala:44	2025/11/28 14:08:09	3 s	1/1 (2 skipped)	200/200 (392 skipped)
247	collect at BinaryClassificationMetrics.scala:240	2025/11/28 14:08:06	3 s	1/1 (2 skipped)	200/200 (392 skipped)
246	count at BinaryClassificationMetrics.scala:200	2025/11/28 14:07:51	15 s	2/2 (1 skipped)	400/400 (192 skipped)
245	sortByKey at BinaryClassificationMetrics.scala:192	2025/11/28 14:07:47	3 s	2/2	392/392
244	collectAsMap at MulticlassMetrics.scala:61	2025/11/28 14:07:47	0.7 s	2/2	392/392

2.1. From Application Overview to Job-Level Details

Analysis begins at a high level. The main History Server page provides a summary of completed applications, including the "Flight Delay Prediction App," its duration, and submission time.

From here, we drill down into the "Completed Jobs" view, which breaks down the application into a sequence of Spark Jobs. This view is the starting point for a "top-down" debugging approach. The job descriptions, such as collect at ML.Pipeline.scala:773, are invaluable; they link execution directly to a line in the source code. An engineer can immediately identify the longest-running jobs and know precisely which action in the code triggered them, providing a clear path for investigation.

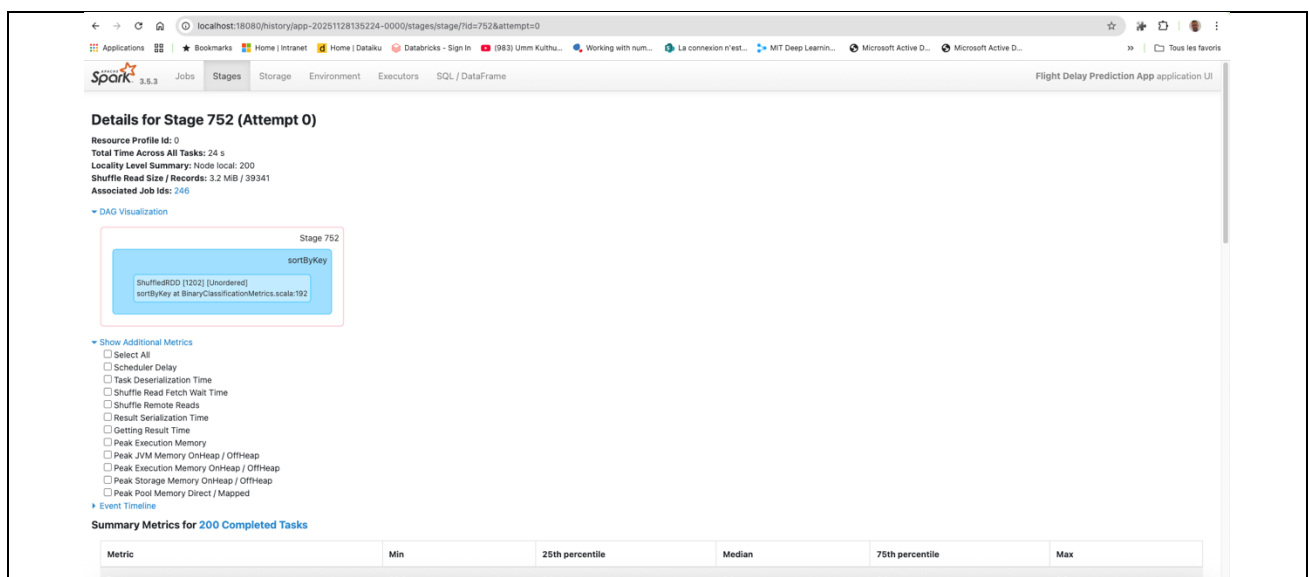
2.2. Visualizing Execution Flow with the DAG



For each job, the Spark History Server provides a Directed Acyclic Graph (DAG) visualization. The DAG is a map of the data flow, breaking the job into stages—groups of tasks that can be executed without data redistribution.

The lines connecting stages in the DAG, such as the one between Stage 751 and Stage 752, represent a **shuffle boundary**. The presence of operations like `combineByKey` and `sortByKey` are classic indicators of a shuffle, where Spark must redistribute data across the network. This is the most common and severe performance bottleneck in distributed processing, and the DAG makes these boundaries immediately obvious.

2.3. Diagnosing Bottlenecks with Stage and Task Metrics

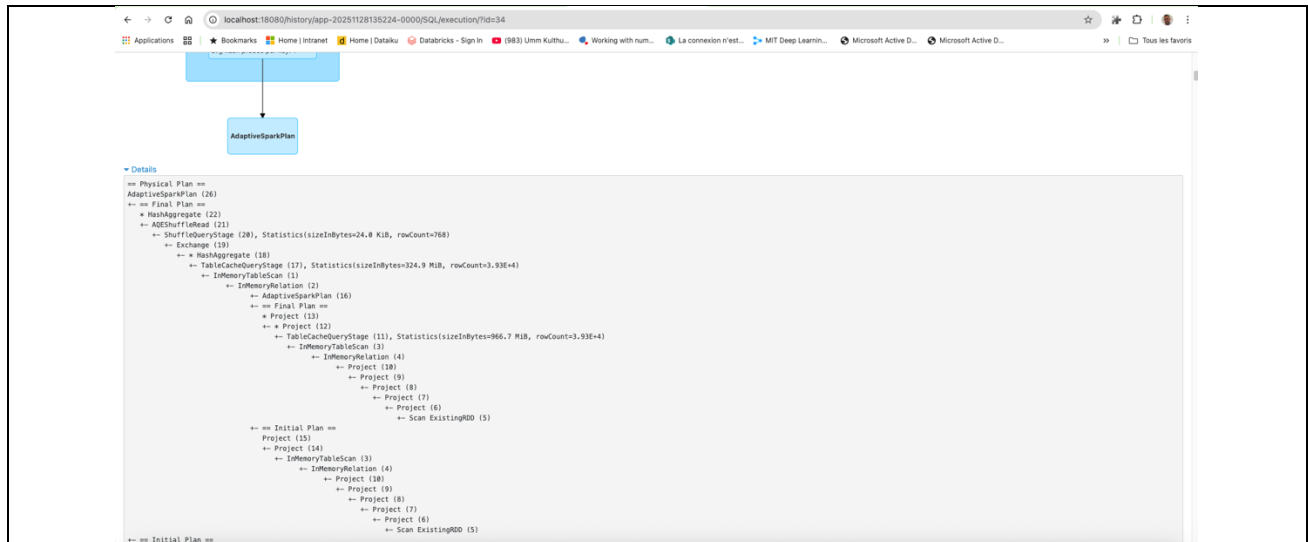


Drilling down into a specific stage, like Stage 752, reveals the granular metrics needed to diagnose the root cause of a slowdown.

Key metrics available on this page include:

- **Total Time:** The wall-clock time for the stage to complete.
- **Shuffle Read / Write:** The volume of data transferred between executors. High shuffle I/O is a primary indicator of a performance bottleneck.
- **Summary Metrics (e.g., Duration percentiles):** A statistical summary of task-level metrics. A large variance between the median and max duration is a classic sign of data skew, where a few tasks are straggling because they are processing a disproportionate amount of data.

2.4. Optimizing Queries via the Physical Plan



For jobs involving DataFrame or SQL operations, the History Server provides the query execution plans. The Physical Plan is the ultimate ground truth of Spark's execution, detailing the final sequence of operations that will run on the cluster.

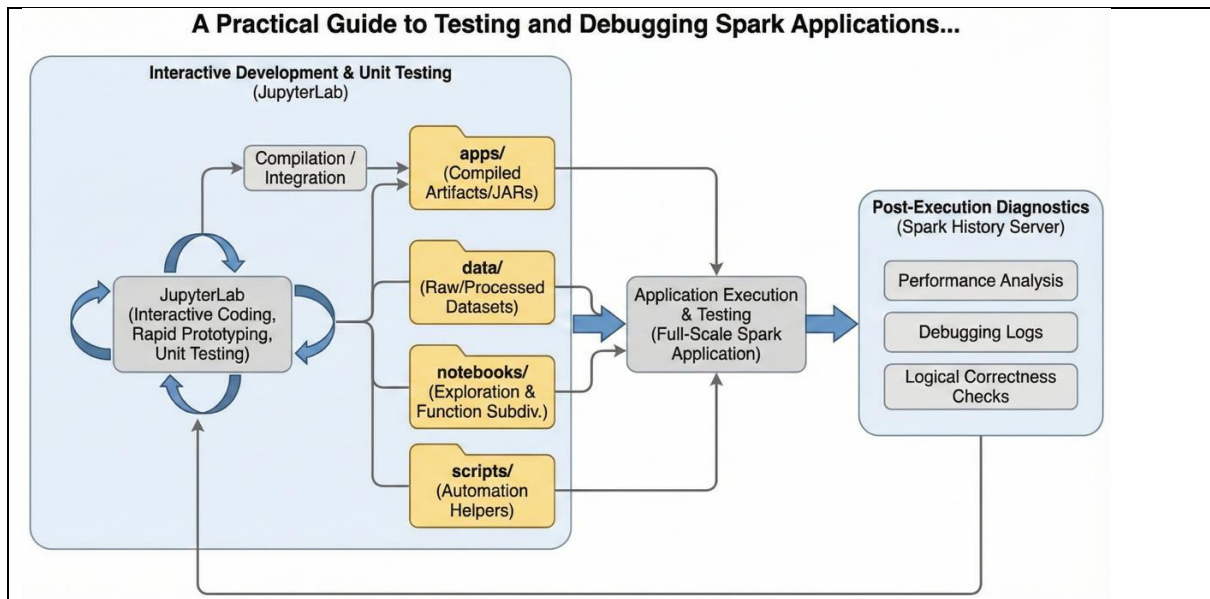
Senior engineers use this view to **verify their optimizations**. By examining the AdaptiveSparkPlan tree, they can confirm whether their intentions were realized by Spark's Catalyst optimizer. Did a broadcast hint result in a BroadcastHashJoin? Did filtering on a partitioned column result in partition pruning? The plan, showing operations like HashAggregate, Exchange, and TableScan, overrides any assumptions and reveals exactly how the query is being executed.

These multi-layered diagnostics, from application overview to the physical plan, are not merely informational. They are the critical inputs that power the second half of our robust development loop: refinement and re-validation.

3. The Unified Workflow for Robust Development

The true power of these tools is unlocked not by using them in isolation, but by integrating them into a cohesive development lifecycle. This unified workflow creates a continuous feedback loop where interactive testing in Jupyter informs performance analysis in the Spark History Server, and vice-versa, leading to more robust and efficient applications.

3.1. The Development and Debugging Lifecycle



This end-to-end workflow enables a structured and iterative approach to building high-quality Spark applications:

1. **Iterate and Validate in Jupyter:** A developer writes and tests discrete units of logic, leveraging compiled JARs for consistency. Immediate outputs (`println`, `printSchema`) provide rapid validation and debugging.
2. **Execute Full Run:** Once the logic is confirmed, the full application is run via a larger notebook execution or a `spark-submit` command to generate the necessary event logs.
3. **Analyze in Spark History Server:** The resulting application run is analyzed to identify performance bottlenecks, inspect query plans, and ensure the application runs efficiently at scale.
4. **Refine and Repeat:** Insights from the History Server (e.g., a slow join or an inefficient shuffle) feed back into the development process, where the specific code can be isolated and refined again in Jupyter. This cycle repeats until both correctness and performance goals are met.

3.2. Core Benefits of this Integrated Approach

Adopting this integrated workflow yields significant strategic advantages for any data engineering team:

- **Increased Development Velocity:** This approach slashes the feedback loop on logic errors from hours to seconds, which is a critical competitive advantage. Developers are no longer forced to wait for long-running batch jobs to fail before they can begin debugging.
- **Enhanced Code Quality and Reliability:** By testing production code directly and validating performance with the History Server, teams build defensible, auditable, and performant data pipelines that are more resilient in production.
- **Proactive Performance Optimization:** This workflow empowers developers to move from a reactive "fix-it-when-it-breaks" mindset to proactively designing for performance. It prevents costly post-deployment firefighting and emergency cluster resizing by ensuring that applications are built to be efficient from the start.

This combined workflow represents a best practice for professional Spark development, ensuring that applications are built to be both correct and efficient from the ground up.

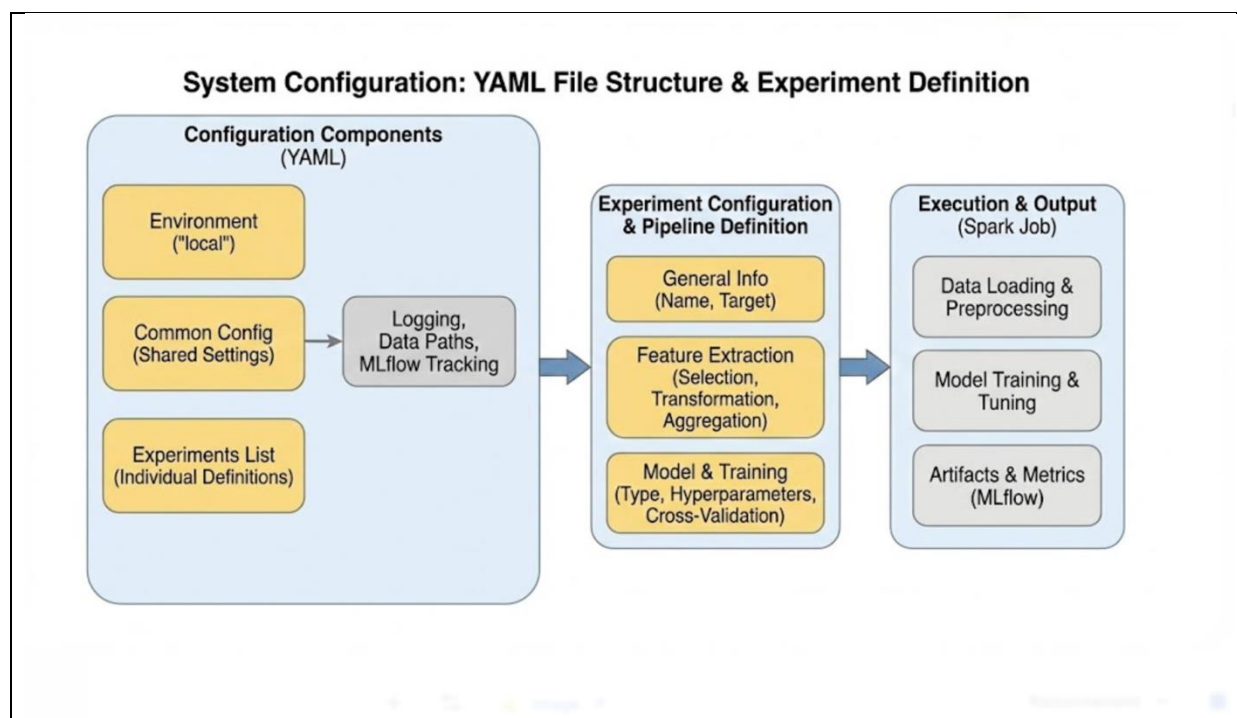
4. Conclusion

Building high-quality, scalable Spark applications requires a development methodology that embraces both iterative, logic-focused testing and deep, post-mortem performance analysis. By mastering the interplay between the interactive environment of JupyterLab and the powerful diagnostic capabilities of the Spark History Server, development teams can create a virtuous cycle of continuous improvement. This unified workflow significantly improves efficiency, reduces bugs, and empowers engineers to build sophisticated, high-performance data applications that can meet the demands of any modern data platform.

APPENDIX 4 – ML Configuration Experiments and Execution

1. Introduction: A Framework for Reproducible ML Experiments

This annexe outlines the configuration and execution framework for the flight delay prediction project. The strategic importance of this system lies in establishing a robust and standardized process for conducting machine learning experiments. By separating experiment configuration from the core application logic, this framework ensures that every experiment is reproducible, scalable, and easily manageable, allowing teams to define, modify, and run complex data pipelines and model training routines without altering the underlying source code.

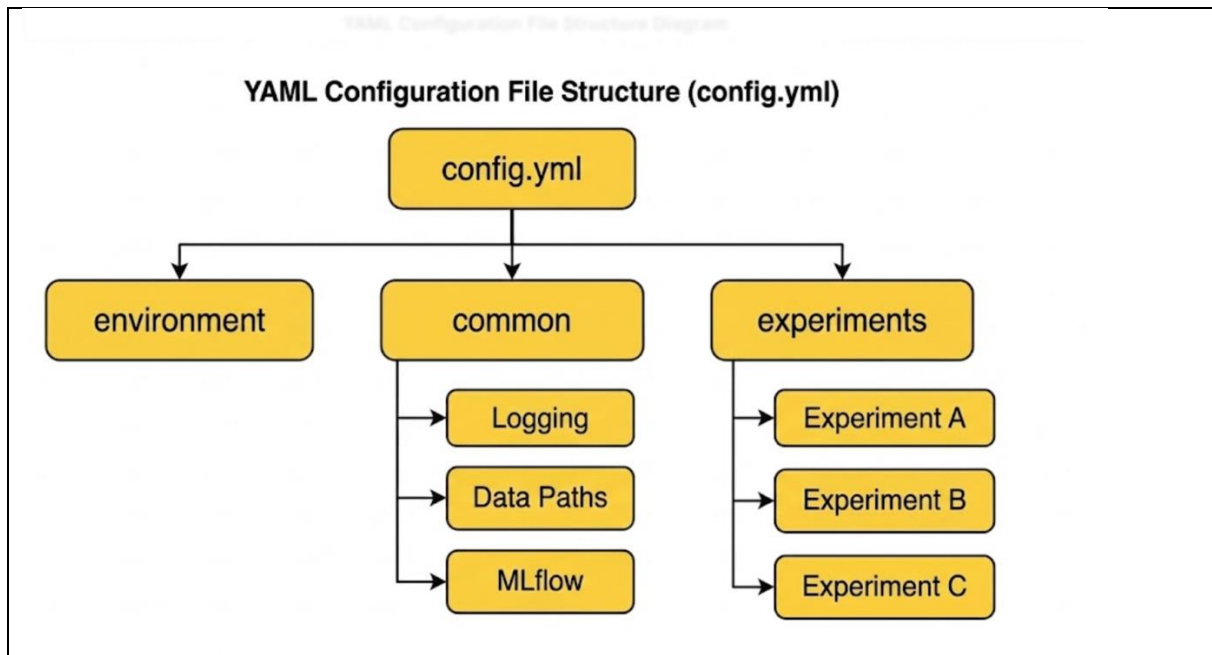


The system is built upon two core components. The first is a central YAML configuration file (`config.yml`), which serves as the declarative blueprint for an experiment, defining everything from data sources and feature engineering steps to model hyperparameters. The second component is a set of shell scripts (`local-submit.sh` and `spark-submit.sh`) that act as the operational engine, responsible for launching and managing the Spark execution environment based on the specified configuration.

This document will now proceed with a detailed examination of the configuration system, followed by the execution workflow that brings these declarative experiments to life.

2. The Configuration System: `config.yml`

The YAML configuration file is the central hub for defining all aspects of a data processing and modeling experiment. It provides a human-readable, declarative interface to control the entire ML pipeline, from initial data loading to model training and evaluation. This section deconstructs the file's structure and key parameters, demonstrating how users can define complex pipelines in a simple, organized manner.



2.2. Analyze the Top-Level Structure

The configuration file is organized into three primary top-level keys, each serving a distinct purpose:

- **environment:** Specifies the execution environment (e.g., "local"), allowing for environment-specific settings.
- **common:** Contains global parameters and settings that are shared across all experiments defined within the file.
- **experiments:** Holds a list of individual experiment definitions, each with its own specific feature extraction and modeling configuration.

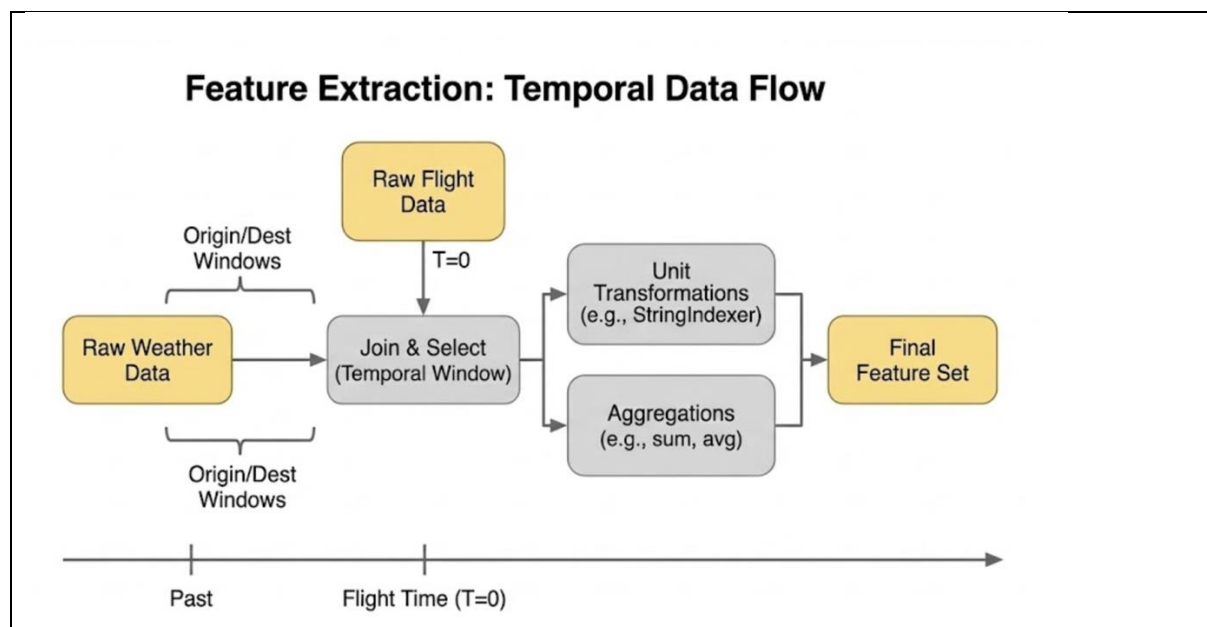
2.3. Deconstruct the common Configuration Block

The common block centralizes parameters that apply globally, ensuring consistency and reducing redundancy across different experiments. These settings control fundamental aspects like data paths, logging, and integration with tracking services.

Parameter	Description
seed	An integer used to initialize random number generators, ensuring reproducibility of results.
log	A boolean flag (true/false) to enable or disable all application logging.
logLevel	Sets the minimum logging level to display (e.g., info, debug, warn).
loadDataFromCSV	A boolean flag; if true, data is loaded from CSV files, otherwise from raw Parquet files.
storeIntoParquet	A boolean flag; if true, the initially loaded data is saved in the efficient Parquet format.
data	A nested object containing the base path and relative paths for input datasets like flights and weather.
output	A nested object specifying the base path where output artifacts should be stored.
mlflow	A nested object to configure MLflow integration, including an enabled flag and the trackingUri.

2.4. Deconstruct the experiments Configuration Block

The experiments key contains a list of one or more experiment definitions. This structure allows multiple experimental variations—such as different models, feature sets, or hyperparameters—to be managed within a single configuration file. For this analysis, we will focus on the example experiment named "Experience-md-rf-local-D2-60-7-7".



Each experiment begins with several descriptive keys:

- **name:** A unique identifier for the experiment (e.g., "Experience-md-rf-local-D2-60-7-7").
- **description:** A human-readable summary of the experiment's goal (e.g., "Baseline Random Forest with hyperparameters tuning").
- **enabled:** A boolean flag that determines whether this specific experiment will be run when the application is executed.
- **target:** The name of the target column for the prediction task (e.g., "is_delayed").

2.5. Deep Dive: featureExtraction Configuration

This section dictates all data preprocessing and feature engineering steps, defining how raw data is transformed into a feature set ready for model training.

Key parameters at the top level of the featureExtraction block include:

Parameter	Description
type	The type of feature extraction process to apply (e.g., "feature_selection").
dxCol	Specifies the column defining the delay event (e.g., "D2_60").
delayThresholdMin	The threshold in minutes to classify a flight as delayed (e.g., 60).
weatherOriginDepthHours	The time window in hours to look back for weather data at the origin airport.
weatherDestinationDepthHours	The time window in hours to look back for weather data at the destination airport.

handleInvalid	The strategy for handling invalid or null values in categorical features (e.g., "keep").
storeJoinData	A boolean flag to control whether the intermediate joined dataset is saved to disk.
storeExplodeJoinData	A boolean flag to control whether the intermediate exploded-join dataset is saved.
maxCategoricalCardinality	The maximum number of unique values a categorical feature can have before being handled.

flightSelectedFeatures

This block lists the specific features to be selected from the flight dataset. For each feature, a transformation is specified.

- CRS_DEP_TIME: transformation: "None"
- OP_CARRIER_AIRLINE_ID: transformation: "StringIndexer"
- ORIGIN_AIRPORT_ID: transformation: "StringIndexer"
- DEST_AIRPORT_ID: transformation: "StringIndexer"
- CRS_ELAPSED_TIME: transformation: "None"
- feature_departure_hour_rounded_sin: transformation: "None"
- feature_departure_hour_rounded_cos: transformation: "None"
- feature_flight_week_of_year_sin: transformation: "None"
- feature_flight_week_of_year_cos: transformation: "None"
- feature_departure_time_period: transformation: "StringIndexer"
- feature_arrival_time_period: transformation: "StringIndexer"

weatherSelectedFeatures

This block specifies the features to be selected from the weather dataset, along with their respective transformations.

- feature_weather_type: transformation: "StringIndexer"
- feature_weather_intensity: transformation: "StringIndexer"
- feature_flight_category_ordinal: transformation: "None"
- feature_visibility_category: transformation: "StringIndexer"
- feature_icing_risk_level: transformation: "None"
- feature_icing_risk_flag: transformation: "None"
- feature_is_pressure_dropping_fast: transformation: "None"
- HourlyPrecip: transformation: "None"
- RelativeHumidity: transformation: "None"
- press_change_abs: transformation: "None"
- feature_temp_dewpoint_spread: transformation: "None"
- Temp_Delta_1hr: transformation: "None"
- Visibility_Delta_1hr: transformation: "None"
- Humidity_Delta_1hr: transformation: "None"
- WindSpeed_Delta_1hr: transformation: "None"

aggregatedSelectedFeatures

This block defines features that are created by aggregating weather data over the specified time window. Each entry includes the feature name, an aggregation function, and a final transformation.

- HourlyPrecip: aggregation: "sum", transformation: "None"
- RelativeHumidity: aggregation: "avg", transformation: "None"
- press_change_abs: aggregation: "max", transformation: "None"
- feature_temp_dewpoint_spread: aggregation: "min", transformation: "None"

2.6. Deep Dive: model Configuration

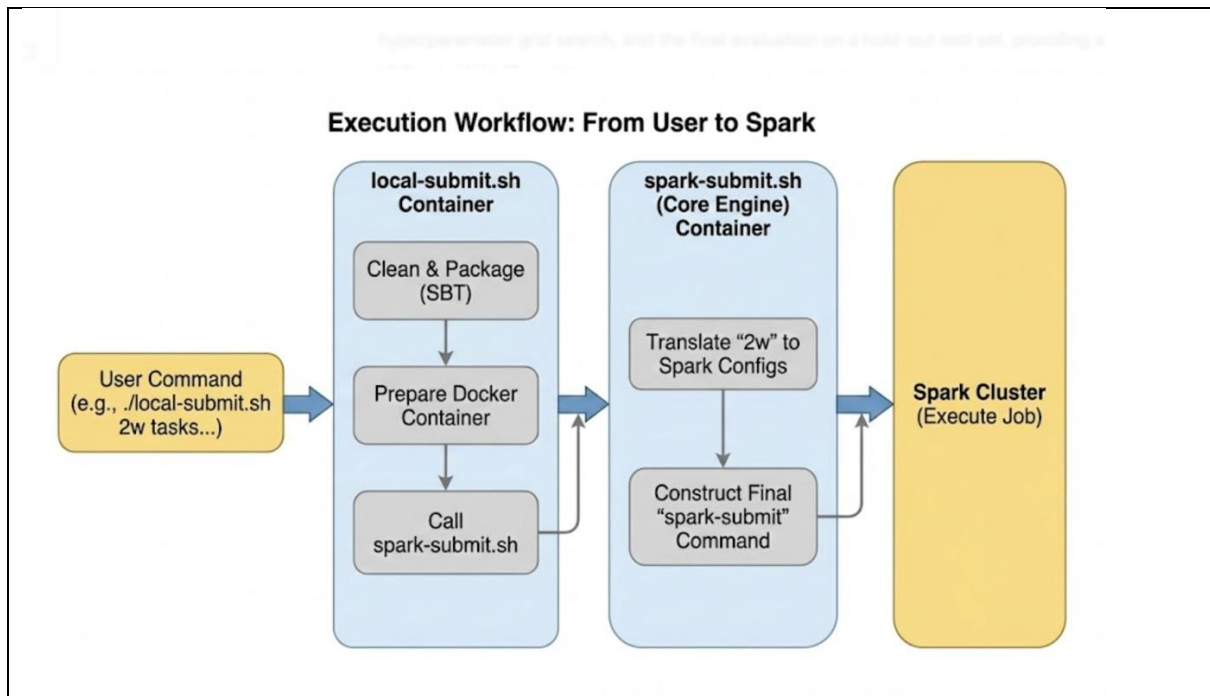
The model block defines the machine learning model to be trained, its specific hyperparameters, and the training procedure.

- **modelType**: Specifies the algorithm to use, in this case, "randomforest".
- **hyperparameters**: A nested object containing the parameters for the chosen model.
 - numTrees: The number of trees in the forest.
 - maxDepth: The maximum depth of each tree.
 - maxBins: The maximum number of bins used for discretizing continuous features.
 - minInstancesPerNode: The minimum number of instances each node must have.
 - subsamplingRate: The fraction of the training data to use for learning each tree.
 - featureSubsetStrategy: The number of features to consider when looking for the best split.
 - impurity: The criterion used for information gain calculation (e.g., "gini").
- **train**: A nested object that controls the training and evaluation process.
 - fast: A boolean flag; when true, it skips the computationally expensive cross-validation step.
 - trainRatio: The proportion of the dataset to allocate for training (e.g., 0.75).
 - crossValidation: Contains settings for cross-validation.
 - numFolds: The number of folds to use in cross-validation.
 - gridSearch: An object with an enabled flag and the evaluationMetric (e.g., "f1") used to select the best model from the hyperparameter grid.

The declarative nature of the YAML file provides a clear and version-controllable definition of each experiment, which is then actioned by the operational scripts that bring it to life.

3. The Execution Workflow: From Local Submission to Spark Job

This section details the operational scripts that parse the experiment configuration and launch it as a Spark job. The workflow is orchestrated by two scripts: local-submit.sh, which serves as the user-facing wrapper for local execution, and spark-submit.sh, the core engine that configures and launches the Spark application. This two-script design provides a powerful abstraction layer, allowing users to specify high-level resource needs (e.g., '2 workers') while the system automatically translates this into a finely-tuned Spark environment with specific memory, core, and parallelism settings.



3.2. User Entry Point: local-submit.sh

The `local-submit.sh` script is the primary user entry point for initiating an experiment locally. It simplifies the execution process by abstracting away the complexities of building the project and configuring the Spark submission command.

At the top of the script, three main environment variables are set to control the job:

- **WORKERS:** Specifies the number of workers to simulate for the Spark cluster (e.g., 1w, 2w, 4w).
- **TASKS:** A comma-separated list of tasks to execute (e.g., "data-pipeline,feature-extraction,train").
- **EXPERIENCE:** Defines the name of the experiment to run. **Note:** This variable must be manually updated to match the name of the target experiment in the configuration file.

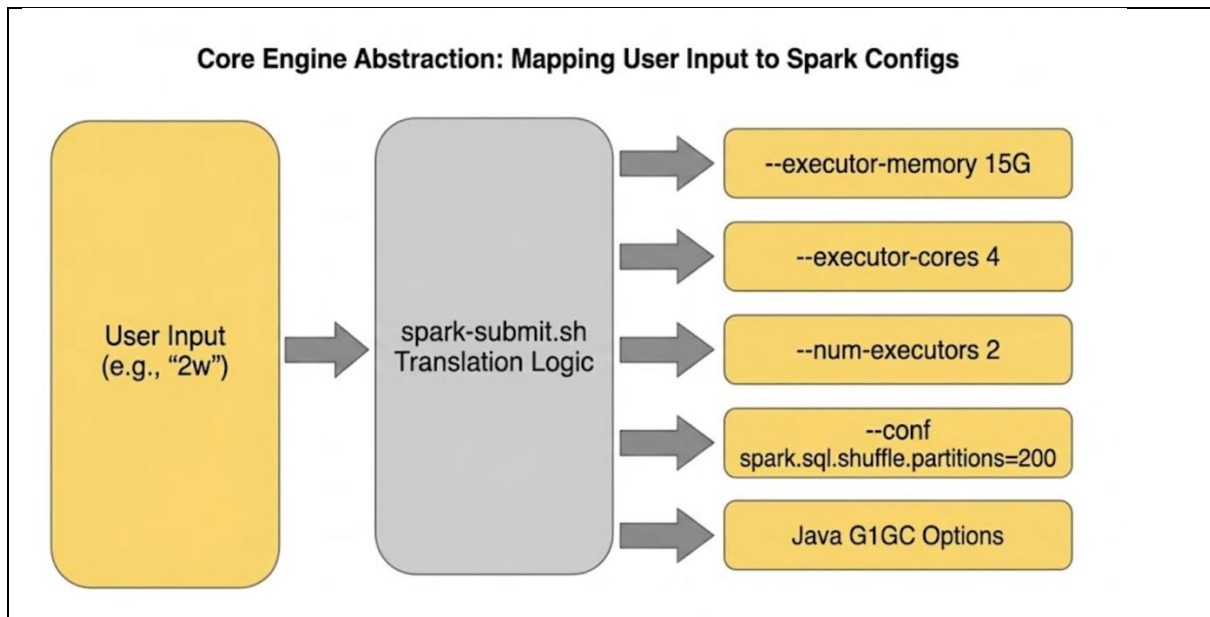
The script includes user-friendly conditional logic for the `$WORKERS` variable. If the variable is not set or invalid, it provides helpful echo statements with examples of valid usage and worker configurations, guiding the user on how to run the script correctly.

The script executes the following sequence of commands:

1. `sbt clean package`: Cleans previous builds and packages the application into a JAR file.
2. `echo "--> Spark job built."`: Prints a status message to the console.
3. `docker exec -it spark-submit chmod +x /scripts/spark-submit.sh`: Ensures the core submission script is executable within the container.
4. `docker exec -it spark-submit /scripts/spark-submit.sh "$WORKERS" "$TASKS" "$EXPERIENCE"`: Invokes the core `spark-submit.sh` script, passing the worker, task, and experiment settings as arguments.

This script effectively orchestrates the entire build and submission process, providing a simple and consistent interface for the user.

3.3. Core Engine: spark-submit.sh



The spark-submit.sh script is the powerful backend responsible for translating the simple worker count provided by the user into a finely-tuned Spark environment. It dynamically sets resource allocation parameters based on the \$WORKERS variable.

The script's conditional logic adjusts key Spark parameters to scale the job's resources appropriately. The configurations are summarized below:

Worker Setting	Executor Memory	Executor Cores	Num Executors	Shuffle Partitions
1w	8G	4	1	160
2w	15G	4	2	200
4w	15G	3	4	240

The core of the script is the final spark-submit command, which is meticulously constructed with numerous configuration flags organized into logical groups:

- **# MEMORY CONFIGURATION:** Sets the memory and cores for the driver and executors (--driver-memory, --executor-memory, --num-executors, etc.).
- **# MEMORY SPARK TUNING:** Fine-tunes Spark's internal memory management (--conf spark.driver.maxResultSize, --conf spark.memory.fraction).
- **# PARALLELISM SQL:** Controls the level of parallelism for Spark SQL operations (--conf spark.sql.shuffle.partitions, --conf spark.default.parallelism).
- **# SERIALIZATION & NETWORK:** Configures data serialization and network settings. The spark.serializer is set to KryoSerializer, a common performance optimization over the default Java serializer for complex data structures often found in ML pipelines.
- **# BROADCAST & COMPRESSION:** Manages settings for broadcast joins and data compression (--conf spark.broadcast.compress, --conf spark.sql.autoBroadcastJoinThreshold).
- **# GARBAGE COLLECTION (RF / GBT):** Provides specific Java garbage collection tuning via spark.executor.extraJavaOptions. These settings tune the G1GC garbage collector, which is crucial for preventing long GC pauses in memory-intensive algorithms like Random Forest (RF) and Gradient-Boosted Trees (GBT).
- **# EVENT LOGGING:** Enables and configures Spark's event logging for monitoring and debugging (--conf spark.eventLog.enabled).

Finally, the script passes the \$EXPERIENCE and \$TASKS variables as command-line arguments to the main application JAR (Emlaad-Flight-Data-Analysis.jar), instructing it which experiment to run and which stages of the pipeline to execute.

Together, these two scripts provide a powerful abstraction layer, translating a simple user command into a robust, optimized Spark job and enabling a user-friendly yet highly configurable execution workflow.

4. Practical Usage Guide: Running an Experiment

This section provides a practical, step-by-step guide to running an experiment, synthesizing the concepts from the configuration and execution sections into a clear workflow. The goal is to walk a user through the process of configuring and launching a specific experiment.

Step 1: Configure the Experiment

Begin by opening the my-experiment-config.yml file. This is where you can define or modify any aspect of your experiment. For example, you can alter a model hyperparameter in the model.hyperparameters block by changing the value for maxDepth or modify a feature transformation in the featureExtraction.flightSelectedFeatures list.

Step 2: Execute the Local Submission Script

Once the configuration is set, you can launch the experiment from your terminal using the local-submit.sh script.

With the script, you can execute it from the command line, providing the number of workers and the list of tasks as arguments.

Example to run the 'data-pipeline' and 'train' tasks with 2 workers:

```
./local-submit.sh 2w data-pipeline,train my-experiment
```

Example to run the full pipeline with 4 workers:

```
./local-submit.sh 4w data-pipeline,feature-extraction,train my-experiment
```

Executing the first command sets \$WORKERS to 2w and \$TASKS to data-pipeline,train. The script then initiates the SBT build process and triggers the Spark job inside the Docker container, using the EXPERIENCE name you configured within the script.

Step 3: Monitor the Output

After execution, monitor the console for output from SBT, Spark, and the application's internal logging system. This provides real-time feedback on the job's progress. If MLflow is enabled in the common configuration, the experiment's parameters, metrics, and artifacts will be automatically tracked and visible in the MLflow UI at the specified trackingUri.

By combining declarative configuration with a streamlined execution workflow, this system provides a robust framework for advanced machine learning development. The clear separation of concerns promotes reproducibility, while the scalable execution engine supports both rapid local prototyping and larger-scale jobs, establishing a solid foundation for reliable and efficient experimentation.

References

- [1] Belcastro, L., Marozzo, F., Talia, D., & Trunfio, P. (2016). Using Scalable Data Mining for Predicting Flight Delays. *ACM Transactions on Intelligent Systems and Technology*, 8(1), 1-20.
- [2] Bureau of Transportation Statistics. Airline On-Time Performance Data. <https://www.transtats.bts.gov/>
- [3] National Climatic Data Center. Quality Controlled Local Climatological Data. <https://www.ncdc.noaa.gov/>
- [4] Zaharia, M., et al. (2016). Apache Spark: A Unified Engine for Big Data Processing. *Communications of the ACM*, 59(11), 56-65.
- [5] Breiman, L. (2001). Random Forests. *Machine Learning*, 45(1), 5-32.
- [6] Federal Aviation Administration. (2021). FAA Aerospace Forecast.
- [7] International Civil Aviation Organization. (2020). Annex 3: Meteorological Service for International Air Navigation.